

Modern C++

The Language, and Everything the Industry Expects on Top of It

A complete working reference — C++11 through C++26

About this book

This is a book about C++ as it is actually written and shipped in 2026, by people who are paid to do it.

It has two halves, and the second half is the unusual one.

Part 1 (§1–18) is the language. Not a syntax tour — the language as a set of cost models and hazards. Why value semantics is the decision everything else follows from. What the compiler generates behind your back and when it silently stops. Where the abstractions really are free, where they aren't, and how to check rather than believe. Every claim about performance in these chapters is one you can verify yourself with the tools in Part 2, and most of them are verified in the text with real compiler output.

Part 2 (§19–29) is everything else you are expected to know. Toolchains, build systems, sanitizers, debuggers, testing, fuzzing, security hardening, performance engineering, interop, and the domain dialects — games, HFT, embedded, graphics, HPC, systems — that real C++ jobs are written in. This material is not in the standard and not in most C++ books, and it is roughly half of what separates someone who knows C++ from someone who can be trusted with a C++ codebase.

The two parts are not independent. Part 1 keeps making claims — *this is zero-overhead*, *this allocates*, *this is a cache miss* — and Part 2 is how you check them. In C++ more than in any other mainstream language, the verification is part of knowing the language.

Who it's for

- **You already program.** The book assumes you know what a pointer, a loop, and a hash table are. It does not assume you know C++.
 - **You're coming from C** and need to understand why C++ is not "C with classes" — start at §1.7, then §3 and §4, which are the whole difference.
 - **You're coming from a managed language** (Java, C#, Python, Go) — §3.1 on value semantics is the section that reorganizes everything else.
 - **You already write C++** and want the parts nobody taught you: §5, §10, §14, §15, and all of Part 2.
 - **You're preparing for interviews** — Appendix G is a self-assessment ladder, and Appendix D is the gotcha list those interviews are drawn from.
-

How to use it

The book is written to be read in order but built to be used as a reference. Sections are numbered *§chapter.section* and cross-referenced constantly, because in C++ nothing is genuinely independent of

anything else — move semantics can't be explained without value categories, and value categories don't matter until you see what moves do with them.

Three ways through:

Read it front to back if you're learning the language. §1–9 are foundational and sequential. §10–18 can be read in any order once you have them.

Read the priority order below if you're preparing for a job.

Use the index — Appendix C for the tables you'll look up weekly, Appendix D before code review, Appendix A whenever you need a flag.

Reading roadmap

Before your first job: §1–9, §11–13, §19, §20 (CMake), §21 (debugger + sanitizers), §22 basics

Makes you strong: §10 (templates + concepts), §12 (ranges), §14 (UB), §15, §23, §24, §25, §28

Then specialize: §16–17 (modern async and modules) · §24 + §26 (games / HFT / HPC) · §23 + §26 (embedded and automotive) · §25 (bindings and tooling)

Conventions

⚠ marks a hazard: a construct that compiles, looks correct, and is wrong — or is correct but costs far more than it appears to. These are the parts worth rereading. They are also, disproportionately, what interviews and production incidents are made of.

☑ marks the recommended modern practice where an older one is still widespread.

§x.y is a cross-reference. Follow them; the density of cross-references is a property of the language, not of the writing.

Code is C++20 unless a feature is labelled otherwise. Compiler output, sizes, and timings shown in the text were produced with **GCC 13.3 and Clang 18 on x86-64 Linux**, and where a number is quoted you should reproduce it rather than trust it — the number will differ on your hardware, and the *reason* it differs is usually the interesting part.

Every chapter ends with three things: a **gotcha checklist**, the **flags and tools** relevant to that material, and **exercises worth doing**. The exercises are not comprehension checks; most of them are "build the thing and measure it", because that is how this material actually sticks.

The honest summary

C++ is roughly **three languages layered on top of each other**: C-with-classes (legacy), modern value-semantic C++11–17, and the template/concept/ranges metalanguage. Most real codebases contain all three, often in the same file, and a large part of professional competence is recognizing which one you're currently reading.

The biggest differentiators in interviews and in real work:

1. **RAII and ownership expressed in types** — `unique_ptr` by default, `shared_ptr` only when the sharing is real
2. **Value semantics, moves, and copy elision** — knowing when copies happen, and why
3. **Lifetime discipline** — dangling references, `string_view`, iterators, lambda captures
4. **Knowing the STL cold**, including iterator invalidation and algorithmic complexity
5. **Sanitizers and Godbolt as daily tools**, not occasional ones
6. **Templates and concepts** enough to read and write library code without fear
7. **Knowing what you're allowed to use** — many shops ban exceptions, RTTI, or the heap, and each ban changes how you design

If you internalize nothing else from this book, internalize the first three. They are the difference between C++ that works and C++ that works until it's under load in production.

Table of Contents

Front matter · About this book · How to use it · Conventions · The honest summary

PART 1 — The C++ Language

Chapter 1 — Foundations

- 1.1 History: why C++ is shaped the way it is
- 1.2 Which standard is your compiler using?
- 1.3 Compilers: divergence and extensions
- 1.4 Translation phases, translation units, and the ODR
- 1.5 The zero-overhead principle ⚠
- 1.6 `main()`, exit codes, and static initialization order
- 1.7 Why C++ is not "C with classes"
- Your standing build command
- Module 1 rules
- Practice

Chapter 2 — Types, Values, Initialization

- 2.1 Fundamental types
- 2.2 Integer promotion and the usual arithmetic conversions
- 2.3 `using` over `typedef`
- 2.4 The initialization maze ⚠
- 2.5 `{}` vs `()` — the two traps ⚠
- 2.6 `auto` and type deduction
- 2.7 `decltype` and `decltype(auto)`
- 2.8 `const`, `constexpr`, `constexpr`, `constexpr`
- 2.9 Casts
- 2.10 `enum` vs `enum class`
- 2.11 Structured bindings (C++17)
- Module 2 rules
- Practice

Chapter 3 — Value Semantics, References, Pointers

- 3.1 Value semantics is the default
- 3.2 Value categories [⚠](#)
- 3.3 References: `&`, `&&`, `const&`
- 3.4 Lifetime extension [⚠](#)
- 3.5 Dangling references, reference members, reference collapsing
- 3.6 Raw pointers — now only for non-owning observation
- 3.7 `nullptr`, `std::optional`, `std::reference_wrapper`
- 3.8 Pointer arithmetic, arrays, decay, and `std::span`
- 3.9 Aliasing, `restrict`, strict aliasing, `std::launder`
- Gotcha checklist for this section
- Compiler flags that catch this section's bugs
- Exercises worth doing

Chapter 4 — RAII & Resource Management [⚠](#)

- 4.1 The idea
- 4.2 Constructors, destructors, deterministic destruction
- 4.3 Scope-based cleanup — why C++ has no `finally`
- 4.4 Smart pointers
- 4.5 `make_unique` / `make_shared` — and why
- 4.6 Custom deleters and `unique_ptr<T[]>`
- 4.7 [⚠](#) The cost of `shared_ptr`
- 4.8 Ownership as a design decision
- 4.9 The Rule of Zero / Three / Five
- 4.10 Scope guards (`scope_exit` / `gsl::finally`)
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 5 — Move Semantics [⚠](#)

- 5.1 Copy vs move — why moves exist
- 5.2 `std::move` is a cast, not a move [⚠](#)
- 5.3 Forwarding (universal) references vs rvalue references [⚠](#)
- 5.4 `std::forward` and perfect forwarding
- 5.5 Move constructor / assignment, and `noexcept`
- 5.6 Moved-from state: valid but unspecified [⚠](#)
- 5.7 Copy elision, RVO, NRVO
- 5.8 When moves don't happen [⚠](#)
- 5.9 `std::exchange` and the swap idiom
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 6 — Functions

- 6.1 Overloading and overload resolution ⚠
- 6.2 Default arguments
- 6.3 Trailing return types and `decltype(auto)`
- 6.4 `inline`, ODR, and what `inline` actually means now
- 6.5 `constexpr` and `constexpr` functions
- 6.6 `noexcept` — specifier and operator
- 6.7 Attributes
- 6.8 Lambdas
- 6.9 ⚠ Dangling captures — the #1 lambda bug
- 6.10 `std::function` vs templates vs `function_ref`
- 6.11 Function pointers, member pointers, `std::invoke`
- 6.12 Variadic templates, parameter packs, fold expressions
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 7 — Classes

- 7.1 Special member functions: generated, suppressed, deleted ⚠
- 7.2 Rule of Zero / Three / Five, `= default`, `= delete`
- 7.3 Member initializer lists and initialization order ⚠
- 7.4 `explicit` constructors and conversion operators
- 7.5 `const` member functions, `mutable`, logical vs bitwise `const`
- 7.6 Static members and inline variables
- 7.7 Nested classes and `friend`
- 7.8 Aggregates and designated initializers
- 7.9 `this`, returning `*this`, and chaining
- 7.10 Copy-and-swap
- 7.11 Empty base optimization and `[[no_unique_address]]`
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 8 — Inheritance & Polymorphism

- 8.1 Public, protected, private inheritance
- 8.2 `virtual` functions, vtables, and dispatch cost
- 8.3 ⚠ Virtual destructors
- 8.4 `override` and `final`
- 8.5 Abstract classes and pure virtual functions
- 8.6 Slicing ⚠
- 8.7 Multiple inheritance, virtual inheritance, the diamond
- 8.8 `dynamic_cast`, RTTI, `typeid`
- 8.9 Prefer composition; prefer static polymorphism
- 8.10 CRTP — static polymorphism
- 8.11 Type erasure
- Gotcha checklist for this section

Flags and tools

Exercises worth doing

Chapter 9 — Operator Overloading

9.1 What can be overloaded

9.2 Member vs free — the decision

9.3 Arithmetic operators — canonical forms

9.4 Subscript, call, and conversion

9.5 `operator<=>` and defaulted comparisons ⚠

9.6 Rewritten and reversed candidates ⚠

9.7 `operator*` and `operator->` for pointer-like types

9.8 User-defined literals

9.9 ⚠ Don't be clever

Gotcha checklist for this section

Flags and tools

Exercises worth doing

Chapter 10 — Templates ⚠

10.1 Function and class templates; parameter kinds

10.2 Instantiation, two-phase lookup, and disambiguators ⚠

10.3 Specialization

10.4 SFINAE — and why it's obsolete

10.5 Concepts and constraints (C++20)

10.6 Variadic templates and packs

10.7 CTAD — class template argument deduction

10.8 Metaprogramming: type traits and `if constexpr`

10.9 Compile-time computation

10.10 Code bloat, compile times, and `extern template`

10.11 `declval`, dependent names, and ADL

Gotcha checklist for this section

Flags and tools

Exercises worth doing

Chapter 11 — The STL — Containers

11.1 `std::vector` — the default

11.2 `array`, `deque`, `list`, `forward_list`

11.3 Ordered vs hashed associative containers

11.4 `multimap/multiset` and `flat_map/flat_set`

11.5 ⚠ Iterator invalidation — the table to memorize

11.6 `emplace` vs `insert` vs `push_back`

11.7 Custom hash and equality

11.8 Container adaptors

11.9 `std::string` and `std::string_view`

11.10 `span` and `mdspan`

11.11 ⚠ `std::vector<bool>` — the famous mistake

11.12 Allocators and PMR

Gotcha checklist for this section

Flags and tools

Exercises worth doing

Chapter 12 — The STL — Iterators, Algorithms, Ranges

12.1 Iterator categories and C++20 concepts

12.2 Iterator adaptors

12.3 The `<algorithm>` surface

12.4 The erase-remove idiom ⚠

12.5 `<numeric>`

12.6 Predicates, comparators, and strict weak ordering ⚠

12.7 Ranges (C++20)

12.8 ⚠ Dangling ranges, `borrowed_range`, `owning_view`

12.9 Parallel algorithms

Gotcha checklist for this section

Flags and tools

Exercises worth doing

Chapter 13 — Error Handling

13.1 Exceptions: mechanics

13.2 Exception safety guarantees

13.3 RAII makes exception safety tractable — the whole argument

13.4 `noexcept`, `std::terminate`, and destructors ⚠

13.5 The exception hierarchy and custom exceptions

13.6 `optional`, `expected`, and `variant` for errors

13.7 `std::error_code` and `system_error`

13.8 ⚠ The exception debate and `-fno-exceptions`

13.9 `assert`, `NDEBUG`, and contracts (C++26)

13.10 ⚠ Never let an exception escape

Gotcha checklist for this section

Flags and tools

Exercises worth doing

Chapter 14 — Undefined Behavior ⚠

14.1 The categories

14.2 The C++ catalogue

14.3 How optimizers exploit UB

14.4 Object lifetime, `launder`, `start_lifetime_as`

14.5 Sanitizers — mandatory ⚠

14.6 Avoiding UB by construction

Gotcha checklist for this section

Flags reference

Exercises worth doing

Chapter 15 — Concurrency

- 15.1 The C++11 memory model
- 15.2 `std::thread` and `std::jthread`
- 15.3 Mutexes
- 15.4 RAII locking
- 15.5 `condition_variable`
- 15.6 `std::atomic` and memory orders
- 15.7 Futures: `future`, `promise`, `packaged_task`, `async`
- 15.8 `latch`, `barrier`, `semaphore` (C++20)
- 15.9 `std::execution` — senders and receivers (C++26)
- 15.10 Thread pools, work stealing, false sharing
- 15.11 Lock-free programming, ABA, and why you should use a mutex
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 16 — Coroutines (C++20)

- 16.1 What a coroutine actually is
- 16.2 The three keywords
- 16.3 The machinery: `promise` type, `awaitable`, `awaiter`
- 16.4 Generators
- 16.5 Async tasks and the executor problem
- 16.6 Libraries you'll actually use
- 16.7  Lifetime traps
- 16.8 The cost model
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 17 — Modules (C++20)

- 17.1 Why modules were needed
- 17.2 The syntax
- 17.3 What you actually gain
- 17.4 `import std;` (C++23)
- 17.5  Build system and tooling status (as of mid-2026)
- 17.6 The practical migration story
- Gotcha checklist for this section
- Flags and tools
- Exercises worth doing

Chapter 18 — C++23 & C++26 Highlights

- 18.1 C++23 — the consolidation release
- 18.2 C++26 — status: **done**
- 18.3 Practical adoption
- Where Part 1 ends
- Exercises worth doing

PART 2 — What Industry Actually Expects

Chapter 19 — Toolchain & Compilation

- 19.1 The three compilers
- 19.2 Warnings
- 19.3 `-Werror` policy
- 19.4 Optimization levels
- 19.5 Reading generated assembly, and Godbolt
- 19.6 `-E`, `-S`, and dumps
- 19.7 LTO and section garbage collection
- 19.8 Compile-time management
- 19.9 Name mangling
- 19.10 Cross-compilation and sysroots
- Gotcha checklist for this section
- Exercises worth doing

Chapter 20 — Build Systems & Libraries

- 20.1 Modern CMake: targets, not variables
- 20.2 Generator expressions
- 20.3 Getting dependencies
- 20.4 Shipping an installable config package
- 20.5 Other build systems
- 20.6 Static vs shared libraries
- 20.7 `△` ABI stability in C++
- 20.8 `-fvisibility=hidden` and export macros
- 20.9 Header-only vs compiled
- 20.10 `compile_commands.json`
- Gotcha checklist for this section
- Exercises worth doing

Chapter 21 — Debugging & Analysis

- 21.1 GDB, LLDB, and the Visual Studio debugger
- 21.2 Core dumps and post-mortem
- 21.3 Sanitizers in practice
- 21.4 Valgrind
- 21.5 `rr` — record and replay
- 21.6 `perf`, VTune, flame graphs
- 21.7 Static analysis
- 21.8 `△` Deciphering template error messages
- Gotcha checklist for this section
- Exercises worth doing

Chapter 22 — Testing

- 22.1 Choosing a framework
- 22.2 GoogleTest

- 22.3 Catch2 and doctest
- 22.4 Mocking: interfaces vs templates
- 22.5 Property-based testing
- 22.6 Coverage
- 22.7 Fuzzing ★
- 22.8 Benchmarking
- 22.9 CTest and CI matrices
- Gotcha checklist for this section
- Exercises worth doing

Chapter 23 — Secure & Safe Coding

- 23.1 The bug classes
- 23.2 Integer issues
- 23.3 Where C++ is safer than C — and where it isn't
- 23.4 C++ Core Guidelines and the GSL
- 23.5 MISRA, AUTOSAR, HIC++, CERT
- 23.6 Hardening — what to turn on in production
- 23.7 Constant-time code and secure erase
- 23.8 ⚠ The memory-safety debate
- Gotcha checklist for this section
- Exercises worth doing

Chapter 24 — Performance Engineering

- 24.1 Profile first
- 24.2 Cache
- 24.3 Data-oriented design: AoS vs SoA
- 24.4 Branch prediction and branchless code
- 24.5 Allocation — the usual culprit
- 24.6 Moves, elision, and the cheap wins
- 24.7 `std::function` and virtual-call overhead
- 24.8 SIMD
- 24.9 Templates vs runtime polymorphism — the monomorphization trade-off
- 24.10 `constexpr` as a runtime optimization
- 24.11 Methodology reminders
- 24.12 PGO and BOLT
- Gotcha checklist for this section
- Exercises worth doing

Chapter 25 — Interoperability

- 25.1 `extern "C"` and the C ABI
- 25.2 Consuming C libraries: wrap them in RAII
- 25.3 Exposing C++ to C: opaque handle + free functions
- 25.4 ⚠ Never let an exception cross a C boundary
- 25.5 Python: pybind11 and nanobind
- 25.6 Rust interop

- 25.7 JNI
- 25.8 WebAssembly via Emscripten
- 25.9  ABI fragility
- Gotcha checklist for this section
- Exercises worth doing

Chapter 26 — Domain Tracks

- 26.0 The map
- 26.1 Game development
- 26.2 Finance / HFT
- 26.3 Embedded and automotive
- 26.4 Graphics
- 26.5 Scientific computing and HPC
- 26.6 Systems and infrastructure
- 26.7 Qt for desktop applications
- 26.8 Moving between domains
- Gotcha checklist for this section
- Exercises worth doing

Chapter 27 — Libraries Worth Knowing

- 27.1 Boost — the incubator
- 27.2 Formatting and logging
- 27.3 The "second standard library" tier: Abseil, folly, EASTL
- 27.4 Eigen — linear algebra
- 27.5 JSON
- 27.6 Networking and RPC
- 27.7 range-v3
- 27.8 Testing and benchmarking
- 27.9 Small, high-leverage utilities
- 27.10 Choosing a dependency
- Gotcha checklist for this section
- Exercises worth doing

Chapter 28 — Professional Practice

- 28.1 Git and `git bisect`
- 28.2 Code review culture in C++
- 28.3 clang-format, clang-tidy, editorconfig, hooks
- 28.4 Style guides
- 28.5 Documentation
- 28.6 API and ABI versioning
- 28.7 Licensing
- 28.8 Reading large C++ codebases
- 28.9 Production debugging, logging, crash reporting
- Gotcha checklist for this section
- Exercises worth doing

Chapter 29 — Study These Real Codebases

- 29.1 How to read someone else's C++
- 29.2 fmt — clean modern C++
- 29.3 spdlog — composition and policy
- 29.4 nlohmann/json — header-only design and API ergonomics
- 29.5 Catch2 — macros meeting templates
- 29.6 Eigen — expression templates
- 29.7 Abseil — industrial-grade utilities
- 29.8 folly — advanced performance techniques
- 29.9 libstdc++ / libc++ — the STL from the inside
- 29.10 LLVM — large-scale architecture
- 29.11 Chromium `base/` — cross-platform abstraction at scale
- 29.12 Godot and O3DE — game architecture
- 29.13 Techniques index — what to read for what
- 29.14 A reading plan
- Closing

Appendices

- Appendix A — The Flag Reference**
 - Appendix B — Feature Index by Standard**
 - Appendix C — Complexity, Invalidation, and Cost Models**
 - Appendix D — The Master Gotcha Checklist**
 - Appendix E — Glossary**
 - Appendix F — A Starter Project**
 - Appendix G — Self-Assessment**
 - Appendix H — Further Reading**
-

PART 1 — The C++ Language

Eighteen chapters on the language itself: what it does, what it costs, and where it will quietly do the wrong thing. The order is deliberate — value semantics (§3) explains RAII (§4), which explains moves (§5), which explain half of what the standard library does (§11–12). Read §1–9 in sequence; after that the chapters stand alone.

Chapter 1 — Foundations

What this chapter covers

- History: C with Classes → C++98 → C++11 (the reboot) → 14 → 17 → **20** → **23** → 26
- **Which standard your compiler uses:** `__cplusplus`, `-std=c++20`, feature-test macros (`__cpp_lib_*`, `<version>`)
- Compilers: GCC, Clang, MSVC, ICX — divergence and extensions
- Translation phases, translation units, ODR

- **The zero-overhead principle** — "you don't pay for what you don't use"
- `main()`, exit codes, static initialization order
- Why C++ is *not* "C with classes": value semantics, RAII, templates, overloading

Everything below is compiled and run on **GCC 13.3.0** and **Clang 18.1.3**, x86-64 Linux. Open a terminal and follow along — that matters more than reading.

1.1 History: why C++ is shaped the way it is

```
1979  Bjarne Stroustrup starts "C with Classes" at Bell Labs
      → classes, derived classes, public/private, constructors, destructors
1983  Renamed C++. Adds virtual functions, operator overloading, references
1985  The C++ Programming Language (1st ed) – the de-facto standard for 13 years
1989  C++2.0 – multiple inheritance, abstract classes, static members
1998  C++98 – the FIRST ISO standard. Templates, STL, exceptions, namespaces, RTTI
2003  C++03 – bug fixes only ("TC1")
2011  C++11 – THE REBOOT. auto, move semantics, lambdas, smart pointers, range-
      for,
      threads, constexpr, nullptr, variadic templates, =default/=delete, enum
      class
2014  C++14 – polish: generic lambdas, make_unique, return-type deduction, init-
      capture
2017  C++17 – structured bindings, if constexpr, optional/variant/any, filesystem,
      parallel algorithms, GUARANTEED copy elision, inline variables
2020  C++20 – concepts, ranges, coroutines, modules, operator<=>, std::format,
      jthread
2023  C++23 – expected, print, mdspan, flat_map, generator, deducing this, import
      std;
2026  C++26 – static reflection, contracts, std::execution, safety profiles (in
      progress)
```

The single most important date is 2011. C++11 changed the language so much that "modern C++" means C++11 and later. Pre-2011 code looks and behaves like a different language — and you will meet a lot of it.

The design principles that explain everything

Principle	Consequence
Zero-overhead abstraction	"What you don't use, you don't pay for. What you do use, you couldn't hand-code better."
Compatibility with C	You inherit <i>every</i> C footgun: UB, integer promotion, arrays, the preprocessor
Value semantics by default	Objects are values, not references — the deepest difference from Java/Python/C#
Deterministic destruction	No GC. Objects die at a <i>known point</i> . RAII depends entirely on this.
Multi-paradigm	Procedural, OO, generic, functional — none mandated

Principle	Consequence
Leave no room for a lower-level language	Systems programming must be possible without dropping to C

The tension that defines C++: it is simultaneously trying to be maximally fast, maximally compatible with 40 years of code, and maximally expressive. Every design decision compromises between those three — and that is why the language is large.

The three languages inside C++

Real codebases contain all three, often in one file:

```
// 1. C-with-classes (legacy, 1990s)
char *buf = (char*)malloc(n); strcpy(buf, s); free(buf);

// 2. Modern value-semantic C++ (C++11-17) — what you should write
std::string buf = s;

// 3. The template metalinguage (library code)
template <std::ranges::input_range R>
    requires std::integral<std::ranges::range_value_t<R>>
auto sum(R&& r) { return std::ranges::fold_left(r, 0, std::plus{}); }
```

Learning C++ means learning all three: writing (2), reading (1) and (3).

1.2 Which standard is your compiler using?

`__cplusplus` — and why it lies

=== GCC 13.3 ===	=== Clang 18.1 ===
-std=c++98 199711 -> C++98/03	-std=c++17 201703 -> C++17
-std=c++11 201103 -> C++11	-std=c++20 202002 -> C++20
-std=c++14 201402 -> C++14	-std=c++23 202302 -> C++23
(official)	
-std=c++17 201703 -> C++17	-std=c++26 202400
-std=c++20 202002 -> C++20	(default) 201703 -> C++17
-std=c++23 202100 -> C++23 (PLACEHOLDER!)	
(default) 201703 -> C++17	

Four things to extract:

1. **Both compilers default to C++17** (as `gnu++17`, with GNU extensions). Always pass `-std=` explicitly.
2. **GCC's `-std=c++23` reports 202100L, not the official 202302L** — a placeholder while support is incomplete. Clang reports the real value.
3. **`__STRICT_ANSI__` tells you whether GNU extensions are on** — `-std=c++20` defines it, `-std=gnu++20` does not.

4. ⚠ Therefore `__cplusplus` is useless for feature detection.

☑ Feature-test macros — the correct answer

```
#include <version>      // C++20: ALL library feature-test macros in one header
```

GCC 13.3, `-std=c++23`:

FEATURE	STD	VALUE	
<code>std::expected</code>	C++23	202211	☑
<code>std::print/println</code>	C++23	NOT AVAILABLE	
<code>std::generator</code>	C++23	NOT AVAILABLE	
<code>std::mdspan</code>	C++23	NOT AVAILABLE	
<code>std::flat_map</code>	C++23	NOT AVAILABLE	
<code>ranges::to<></code>	C++23	NOT AVAILABLE	
<code>std::format</code>	C++20	202110	☑
<code>ranges</code>	C++20	202202	☑
<code>std::jthread</code>	C++20	201911	☑
<code>concepts</code>	C++20	202002	☑
<code>modules</code>	C++20	NOT AVAILABLE	
<code>coroutines</code>	C++20	201902	☑
<code>deducing this</code>	C++23	NOT AVAILABLE	
<code>if constexpr</code>	C++23	202106	☑

`-std=c++23` on GCC 13 gives you roughly 60% of C++23. Partial implementation is the norm, not the exception. Compilers ship features individually as they're implemented.

⚠ And it gets worse — the same library, two answers

```
GCC 13 + libstdc++13 : std::expected AVAILABLE
Clang 18 + libstdc++13 : std::expected NOT AVAILABLE
```

Same standard flag. Same standard library. Different answer. Why?

```
clang: #define __cpp_concepts 201907L
gcc : #define __cpp_concepts 202002L
```

`libstdc++`'s `<expected>` is guarded on `__cpp_concepts >= 202002L`. Clang 18 implements an older **concepts revision**, so the header disables itself:

```
$ clang++ -std=c++23 exp.cpp
error: use of undeclared identifier 'std'    ← <expected> compiled to nothing
```

The lesson: feature availability depends on the *compiler*, the *standard library*, and the *version of each*. Only `__cpp_lib_*` tells you the truth.

```
#if __cpp_lib_expected >= 202202L
#   include <expected>
    template <class T, class E> using Result = std::expected<T, E>;
#else
#   include <tl/expected.hpp>
    template <class T, class E> using Result = tl::expected<T, E>;
#endif
```

Reference: en.cppreference.com/w/cpp/feature_test — bookmark it.

Which standard should you use professionally?

Standard	When
C++17	The safe floor. Universal support, including old embedded and enterprise toolchains.
C++20	☒ The default for new work in 2026. Concepts and ranges alone justify it.
C++23	Adopt selectively, guarded by <code>__cpp_lib_*</code> .
C++11/14	Legacy maintenance only.
C++98	You are doing archaeology.

1.3 Compilers: divergence and extensions

```
gcc 13.3.0                libstdc++ (__GLIBCXX__)
clang 18.1.3              libstdc++ (__GLIBCXX__)
...and it ALSO defines __GNUC__ = 4  <- check __clang__ FIRST
```

Compiler	Notes
GCC	Linux/embedded default. Excellent conformance, huge target coverage.
Clang/LLVM	Best-in-class tooling: <code>clang-tidy</code> , <code>clang-format</code> , sanitizers, <code>-ftime-trace</code> . Default on macOS/FreeBSD.
MSVC	Windows. Good C++ conformance — but you must pass <code>/permissive-</code> .
ICX	Intel, LLVM-based. HPC/vectorization focus.

⚠ **Clang defines `__GNUC__` (as 4!) for compatibility.** Always test `__clang__` first.

Divergence 1 — GNU extensions accepted silently




```
int n = 5;
int vla[n];           // VLA: a GNU extension, NOT standard C++
S s = { .a = 1, .b = 2 }; // designated init: C++20 only
```

```
-std=gnu++17 : 3 errors      ← with -pedantic-errors
-std=c++17   : 3 errors
-std=c++20   : 1 error      ← designated init is now legal
```

```
error: ISO C++ forbids variable length array 'vla' [-Wvla]
error: C++ designated initializers only available with '-std=c++20'
```

Without `-pedantic-errors`, `-std=gnu++17` (the default!) accepts VLAs silently. Your code compiles on GCC and fails on MSVC. **Use `-std=c++NN` and `-pedantic-errors`** if portability matters.

Divergence 2 — the big one: two-phase lookup

```
template <class T> struct Base { int value = 1; };
template <class T> struct Derived : Base<T> {
    int get() { return value; }           // `value` is a DEPENDENT name
};
```

```
GCC    : 1 error
Clang  : 2 errors
error: 'value' was not declared in this scope
```

Both reject it — correctly. MSVC historically accepted it, because MSVC did not implement two-phase lookup (Module 10). `/permissive-` makes MSVC conform.

This is the single biggest MSVC divergence. Codebases written MSVC-only routinely fail to compile with GCC/Clang for this reason. The fix is `this->value` or `Base<T>::value`.

Divergence 3 — diagnostic quality

```
GCC    error output: 78 lines
Clang  error output: 225 lines
```

Interestingly, **Clang was more verbose here** — for template errors it typically gives better *structure* (a clear "candidate template ignored because..." chain) even when longer. Neither is pleasant; **concepts (Module 10) are the real fix**.

The practical rule

Build with GCC *and* Clang. They warn about different things, exploit different UB, and produce different diagnostics. It is the cheapest quality improvement available in C++.

Prefer capability probes over version checks: `__has_include`, `__has_cpp_attribute`, `__has_builtin`.

1.4 Translation phases, translation units, and the ODR

The scale of C++ preprocessing

```
#include <vector>
#include <string>
// ... 23 lines of code total
```

```
phases.cpp source lines      : 23
after -E (preprocessed)     : 36,530 lines (1588x expansion)
a C program with <stdio.h>   : ~800 lines, for comparison
generated assembly lines    : 3,666
object file size            : 53,728 bytes
final executable            : 37,368 bytes
```

36,530 lines from 23. This is why C++ compiles slowly, and it is the entire motivation for modules (C++20), precompiled headers, `ccache`, and `include-what-you-use` (Module 19).

The nine phases

```
1-4. Preprocessing    : trigraphs, line splicing, tokenization, #include/#define
5-6. String handling  : character-set conversion, adjacent literal concatenation
7.  COMPILATION       : the actual C++ compiler. Templates instantiated here.
8.  INSTANTIATION     : remaining template instantiation (implementation-defined
placement)
9.  LINKING           : resolve symbols across translation units
```

A translation unit is one `.cpp` plus everything it `#includes`, after preprocessing. The compiler sees one TU at a time; only the linker sees them all.

Name mangling — the thing C doesn't have

```
int    add(int, int);
double add(double, double);           // overload
namespace app { int add(int, int); }  // namespace
template <typename T> T twice(T);      // template
struct Widget { virtual ~Widget(); }; // polymorphic
```

```

raw symbols:                                demangled (nm -C):
_Z3adddd                                    add(double, double)
_Z3addii                                    add(int, int)
_Z5twiceIdET_S0_                            double twice<double>(double)
_Z5twiceIiET_S0_                            int twice<int>(int)
_ZN3app3addEii                              app::add(int, int)
_ZN6Widget3setEi                            Widget::set(int)
_ZN6WidgetD0Ev                             Widget::~~Widget()      ← THREE destructors
_ZN6WidgetD1Ev                             Widget::~~Widget()
_ZN6WidgetD2Ev                             Widget::~~Widget()

```

And the polymorphic class brings extra symbols:

```

typeinfo for Widget
typeinfo name for Widget
vtable for Widget

```

This is the mechanism behind overloading, namespaces, and templates. The linker still sees flat, unique symbol names — the compiler encodes the whole signature into them.

Consequences you will meet:

```

extern "C" int c_fn(int, double);
raw: c_fn                                ← unmangled
raw: _Z6cpp_fnid                         ← mangled

```

- **extern "C" disables mangling** — required for C interop (Module 25). But only *one* overload can have it.
- **nm -C and c++filt** are essential. Raw linker errors are unreadable without them.
- **GCC and Clang agree** (both use the Itanium C++ ABI). **MSVC uses a different scheme** (? `cpp_fn@@YAHHN@Z`) — you cannot mix their object files.
- Changing a signature changes the symbol → **C++'s notorious ABI fragility** (Module 20).

Templates are *instantiated*, not compiled once

```

W double twice<double>(double)
W int twice<int>(int)

```

Two separate functions in the binary. This is **monomorphization**, and it's the central trade-off of C++ templates:

You get	You pay
No indirection, full inlining, full optimization per type	Slower compiles

You get	You pay
The <code>std::sort</code> speedup in §1.5	Larger binaries (code bloat)
Compile-time type checking	Errors surface at instantiation, not definition

Note the **W** — *weak* symbol. Template instantiations appear in every TU that uses them; the linker merges duplicates. That's the ODR exemption below.

The One Definition Rule

Every entity must have exactly one definition in the program. For entities that may appear in multiple TUs (inline functions, templates, class types), all definitions must be **token-for-token identical** and mean the same thing.

C's ODR is simple. C++'s has **three** distinct behaviors, and only two are safe.

Case 1 — the linker catches it. Good, loud failure.

```
/usr/bin/ld: multiple definition of `g'; first defined here
/usr/bin/ld: multiple definition of `f()'; first defined here
```

Case 2 — `inline` and templates are exempt, by design.

```
inline int h(){ return 42; }
template<class T> T t(T x){ return x; }
```

```
linked fine, exit=87
W h()
W int t<int>(int)
^ W = WEAK symbol. The linker merges duplicates. That is what 'inline' MEANS.
```

`inline` has almost nothing to do with inlining the call. It means "*this definition may legally appear in multiple translation units.*" That's why header-only libraries work, and it applies automatically to templates, `constexpr` functions, and in-class member function definitions.

C++17 added **`inline variables`**, which finally lets you put a global in a header: `inline constexpr int kMax = 1024;`

 **Case 3 — the silent violation.**

```
// odr5.cpp                                // odr6.cpp
struct Config { int a, b; };                 struct Config { int a,b,c,d; }; //
DIFFERENT
```

```
inline int config_size()           inline int config_size()
{ return sizeof(Config); }        { return sizeof(Config); }
```

```
TU1 thinks sizeof(Config) = 8
TU2 thinks sizeof(Config) = 8    ← should be 16!
```

The linker picked one definition of `config_size()` and used it for both. No error, no warning, wrong answer.

Can any tool catch it?

```
-flto -Wodr          : 0 diagnostics
-flto -Wall -Wextra   : 0 diagnostics
ASAN_OPTIONS=detect_odr_violation=2 : nothing
```

No. ASan's ODR checker only detects duplicate *global variables*, not mismatched type definitions used through weak functions.

But LTO *does* catch the variant where the type crosses a function boundary:

```
warning: type 'struct Config' violates the C++ One Definition Rule [-Wodr]
note: a different type is defined in another translation unit
note: the first difference of corresponding definitions is field 'c'
```

```
without LTO:  a=1 b=2 c=1574526720 d=-409308967    ← garbage
```

This is the most dangerous class of C++ bug. It's UB, it's silent, and it survives most tooling.

Real causes: a header included with different `-D` flags in different TUs; mixing debug and release objects; different `NDEBUG` or `_GLIBCXX_DEBUG` states; different `#pragma pack`; mixing standard-library versions.

Defenses: put every type definition in a header and never duplicate it; compile the whole project with identical flags; use `-flto -Wodr`; prefer **modules** (C++20), which make this structurally impossible.

1.5 The zero-overhead principle ⚠

"What you don't use, you don't pay for. And what you do use, you couldn't hand-code any better." — Bjarne Stroustrup

This is the claim that justifies C++'s existence. Three tests.

Test 1 — `std::array` vs a C array

```
int c_array() { int a[4] = {1,2,3,4}; return a[0]+a[3]; }
int std_array() { std::array<int,4> a{1,2,3,4}; return a[0]+a[3]; }
```

```
c_array():          std_array():
    mov eax, 5      mov eax, 5
    ret            ret
```

Byte-for-byte identical. `std::array` is the C array, plus a size that travels with it, plus `begin/end/size`, plus it can be copied and returned by value. All free.

Test 2 — manual `malloc/free` vs RAII

```
void manual() { W* p = (W*)malloc(sizeof(W)); if(!p) return; p->x=1; use(p);
free(p); }
void raii() { auto p = std::make_unique<W>(); p->x=1; use(p.get()); }
```

```
manual():          raii():
    push rbx       push rbx
    mov edi, 4     mov edi, 4
    call malloc@PLT call operator new
    test rax, rax  mov DWORD PTR [rax], 1
    je .L1        mov rbx, rax
    mov DWORD PTR [rax], 1 mov rdi, rax
    mov rbx, rax  call use(W*)
    mov rdi, rax  mov rdi, rbx
    call use(W*)  mov esi, 4
    mov rdi, rbx  pop rbx
    pop rbx       jmp operator delete
    jmp free@PLT
```

Structurally identical — allocate, use, tail-jump to the deallocator.

And `raii()` is exception-safe. If `use()` throws, `manual()` leaks. The C++ version cannot.

Test 3 — the abstraction is *faster*

SORTING 5,000,000 ints (best of 5)

C qsort (function pointer)	683.0 ms	
std::sort (template)	320.4 ms	← 2.13x FASTER
std::sort + lambda comparator	356.5 ms	

```
std::sort + std::function      622.3 ms    ← 1.75x slower than the
lambda
```

Why:

- `qsort` takes an `int (*)(const void*, const void*)`. Every comparison is an **indirect call through a function pointer** that cannot be inlined, plus two `void*` dereferences.
- `std::sort` is a **template**. The comparator type is known at compile time, so `operator<` is inlined into the sort loop — then vectorized and unrolled.
- `std::function` **re-introduces the indirection** and lands back at `qsort` speed. That's the cost of type erasure (Module 6).

This is zero-overhead in its strongest form. The higher-level, type-safe, harder-to-misuse interface generates *better* code than the C one, because monomorphization gives the optimizer more information.

⚠ Where the principle does *not* hold

Being honest about this matters:

Feature	Real cost
Exceptions	Zero cost when not thrown; ~1.2 μs when thrown (Module 13). Tables enlarge the binary.
RTTI / <code>dynamic_cast</code>	Type info in the binary; a runtime hierarchy walk
<code>virtual</code>	One pointer per object + an indirect call; blocks inlining — measured 2.4–11× (Module 8)
<code>shared_ptr</code>	Atomic refcount, separate control block — measured 2.79× vs a raw pointer (Module 4)
<code>std::function</code>	Type erasure: possible heap allocation + an indirect call — measured 1.75× above
<code>iostreams</code>	Notoriously heavy; <code>std::format</code> / <code>std::print</code> are the modern replacement
Templates	Compile time and binary size

Every one of those is opt-in. Don't use `virtual`, and you pay nothing for it. That's the principle working — and it's why knowing *what your team allows* matters (Module 26: games, embedded, and HFT commonly ban exceptions and RTTI).

1.6 `main()`, exit codes, and static initialization order

The signatures

```
int main()                                // note: NOT main(void) – () already means "no
params"
```

```
int main(int argc, char *argv[])
```

C++-specific rules that differ from C:

- You may **not** call `main` recursively, take its address, or mark it `inline`, `static`, or `constexpr`
- `main` cannot be overloaded
- **Falling off the end returns 0** — `main` is the only non-`void` function where that's defined

Deterministic destruction, observed

```
static Tracer g_static{"global static"};

int main() {
    std::atexit(on_exit_handler);
    Tracer local{"local in main"};
    { Tracer inner{"inner scope"}; }
    static Tracer fn_static{"function-local static"};
    return 0;
}
```

```
ctor   global static           ← BEFORE main
main() entered, argc=1
ctor   local in main
ctor   inner scope
dctor  inner scope             ← destroyed at the closing brace, exactly
ctor   function-local static   ← constructed on FIRST USE
main() returning 0
dctor  local in main           ← reverse order
dctor  function-local static
atexit handler
dctor  global static
```

Read that ordering carefully — it is the foundation of RAII (Module 4):

1. Globals constructed before `main`, destroyed after, in **reverse order of construction**
2. Locals destroyed at the **exact closing brace**, in reverse declaration order
3. Function-local statics constructed on **first use**, destroyed with the globals
4. `atexit` handlers and static destructors interleave in reverse registration order

There is no garbage collector, and that is a feature. You know *precisely* when a file is closed, a lock is released, or a socket is shut down.

Exit codes

```
return 42      -> $? = 42
return -1      -> $? = 255 (truncated to 8 bits)
```



```

return 256      -> $? = 0      (wraps)
fall off the end -> $? = 0      (main IS special)
abort()         -> $? = 134    (128 + SIGABRT)

```

Use `EXIT_SUCCESS/EXIT_FAILURE`, or values **0–125** (126/127/128+ are reserved by shells).

Which exits run destructors

EXIT	local	global	atexit	code
return	1	1	1	0
exit	0	1	1	0
quick_exit	0	0	0	0
_Exit	0	0	0	0
abort	0	0	0	134
throw	0	0	0	134

⚠ **An uncaught exception: whether the stack unwinds first is *implementation-defined*.** GCC does not unwind — `local`'s destructor did not run. **Never rely on cleanup happening for an uncaught exception.**

⚠ The static initialization order fiasco

```

// siof_a.cpp                                // siof_b.cpp
extern std::string g_config;                  std::string g_config = "the-real-
configured-value";
struct User { User(){ use(g_config); } };
static User g_user;

```

```

link order A B: User ctor sees g_config = "(null)" (len 0)
link order B A: User ctor sees g_config = "the-real-configured-value" (len 25)

```

Same source. Different link order. Different behavior.

The order of *dynamic* initialization across translation units is unspecified. Within one TU it's declaration order; across TUs, the compiler and linker decide.

The two fixes

1. Construct-on-first-use — the general answer:

```

static std::string& config() {
    static std::string s = "the-real-configured-value";    // C++11: thread-safe,
    lazy
    return s;
}

```

```
sees config() = "the-real-configured-value"    ← always correct
```

Function-local statics are initialized on first use, and since C++11 that initialization is thread-safe (the compiler emits a guard variable). This is also the correct way to write a singleton, if you must.

2. `constexpr` (C++20) — forces *static* initialization:

```
constexpr int g_limit = 64;           // OK: a constant expression
constexpr std::string s = "x";        // ERROR: needs dynamic init
```

```
g_limit = 64 (always correct - no dynamic init)
```

`constexpr` fails to compile if the initializer can't be evaluated at compile time — so it's a *guarantee*, not a hope. Use it for every global you can.

1.7 Why C++ is not "C with classes"

The same task, both ways

```
// ----- C style: 37 lines -----
typedef struct { char **items; size_t n, cap; } CList;
static int clist_push(CList *l, const char *s) {
    if (l->n == l->cap) {
        size_t nc = l->cap ? l->cap*2 : 4;
        char **p = (char**)realloc(l->items, nc * sizeof *p);
        if (!p) return -1;           /* check 1 */
        l->items = p; l->cap = nc;
    }
    char *copy = (char*)malloc(strlen(s)+1);
    if (!copy) return -1;           /* check 2 */
    memcpy(copy, s, strlen(s)+1);
    l->items[l->n++] = copy;
    return 0;
}
static void clist_free(CList *l) { /* must remember */
    for (size_t i = 0; i < l->n; i++) free(l->items[i]);
    free(l->items);
}
// ... plus a qsort comparator, plus error handling at every call site
```

```
// ----- C++ style: 6 statements -----
std::vector<std::string> v{"delta","alpha","charlie","bravo"};
std::sort(v.begin(), v.end());
for (const auto& s : v) std::printf("%s ", s.c_str());
// nothing to free
```

```
C    : alpha bravo charlie delta
C++  : alpha bravo charlie delta
ASan/UBSan exit=0

C version   : 37 lines, 3 manual error checks, 1 required free()
C++ version : 6 lines, 0 error checks, 0 frees
Both are correct. Only ONE can leak, double-free, or overflow.
```

The four mechanisms that make the difference

1. Value semantics — objects are values, not handles:

```
a = "hello"    b = "hello world"    <- INDEPENDENT
In Java/Python/C#, `b = a` makes b ALIAS a. This is the deepest difference.
```

2. RAII — a destructor releases the resource on every exit path:

```
file closed automatically
```

No **finally**, no **defer**, no **try/except** — the cleanup lives in the *type*, once, not at every call site.

3. Templates — one implementation, monomorphized per type:

```
largest<int>      = 9
largest<double>   = 1.5
largest<string>   = z
```

```
int largest<int>(std::vector<int> const&)
double largest<double>(std::vector<double> const&)
std::string largest<std::string>(std::vector<std::string> const&)
```

One source function → **three separate functions in the binary.**

4. Overloading and operators — your types behave like built-ins:

```
{1,2} + {10,20} = (11,22),  == works: 1
  an int: 42
  a string: text
  a point: (11,22)
ONE name, THREE functions, chosen by argument type at COMPILE time
```

Plus everything that follows from them: **exceptions** (which only work because RAII makes cleanup automatic), **the STL** (which only works because of templates), and **constexpr** (compile-time execution of ordinary-looking code).

"C with classes" is a real style, and it is the wrong one. If your C++ has raw **new/delete**, manual cleanup, **char*** strings, C casts, and no destructors doing work, you are paying C++'s compile times and complexity without collecting any of its benefits.

Your standing build command

From here on, use this for everything:

```
g++ -std=c++20 \
  -Wall -Wextra -Wpedantic \
  -Wshadow -Wconversion -Wsign-conversion \
  -Wnon-virtual-dtor -Wold-style-cast -Woverloaded-virtual -Wcast-align \
  -Wnull-dereference -Wdouble-promotion -Wformat=2 -Wimplicit-fallthrough \
  -O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer \
  prog.cpp -o prog
```

Three of those are C++-specific and catch real bugs you'll meet in Modules 2–8:

- **-Wnon-virtual-dtor** — deleting through a base pointer without a virtual destructor
- **-Wold-style-cast** — forces named casts, which are searchable and checked
- **-Woverloaded-virtual** — you meant to override but changed the signature

Module 1 rules

#	Rule
1	Always pass -std= explicitly — both GCC and Clang default to gnu++17
2	Use feature-test macros (<code>__cpp_lib_*</code>), never <code>__cplusplus</code>
3	⚠ Availability depends on compiler AND stdlib AND version — same flag, different answer
4	Build with GCC and Clang ; check <code>__clang__</code> before <code>__GNUC__</code>
5	-pedantic-errors if portability matters; MSVC needs /permissive-
6	nm -C / c++filt — raw C++ linker errors are unreadable

#	Rule
7	Mangling is compiler-specific — you cannot mix MSVC and GCC objects
8	inline means "definable in multiple TUs", not "please inline"
9	⚠ ODR violations are silent UB — compile every TU with identical flags
10	<code>-flto</code> <code>-Wodr</code> catches <i>some</i> ODR violations; nothing catches all
11	Zero-overhead is real: <code>std::array</code> is byte-identical, <code>std::sort</code> is 2.13× faster than <code>qsort</code>
12	Exceptions, RTTI, <code>virtual</code> , <code>shared_ptr</code> , <code>std::function</code> , iostreams do cost — all opt-in
13	Destruction is deterministic and in reverse order — this is what makes RAII work
14	⚠ Uncaught exceptions may not unwind — GCC does not
15	Never rely on cross-TU static init order; use construct-on-first-use or <code>constinit</code>
16	Value semantics + RAII + templates + overloading is what C++ is

Practice

- Standard archaeology.** Run the `__cplusplus` probe across every `-std=` your compilers support. Then write a program using `std::expected`, `std::print`, and a module, each guarded by feature-test macros with fallbacks. Determine exactly which your toolchain supports.
- The divergence hunt.** Compile the same file with GCC and Clang at `-std=c++23`. Diff the `__cpp_*` macro output (`-E -dM`). Find three features where they disagree and explain each.
- Mangling.** Write a file with an overloaded function, a namespaced function, a class member, a template, a virtual class, and an `extern "C"` function. Dump the symbols with `nm` and `nm -C`. Decode the Itanium encoding for each. Then find one in `libstdc++.so` and demangle it.
- Header cost.** Measure `wc -l` after `-E` for `<cstdio>`, `<iostream>`, `<vector>`, `<string>`, `<algorithm>`, `<ranges>`, `<regex>`, `<format>`. Then measure compile time for each. Which header costs the most per unit of benefit?
- Monomorphization.** Write `template<class T> T sum(const std::vector<T>&)` and instantiate it for 6 types. Count the symbols and measure `.text` growth per instantiation.
- Reproduce the silent ODR violation.** Build the §1.4 case-3 example and confirm the wrong answer. Then try to detect it with `-flto -Wodr, gold --detect-odr-violations, ASAN_OPTIONS=detect_odr_violation=2`, and LTO. Record which work and which don't. Then build the variant that LTO *does* catch.
- Zero-overhead audit.** For `std::array`, `std::pair`, `std::optional<int>`, `std::string_view`, `unique_ptr`, `shared_ptr`, and `std::function` — write a minimal function using each and its raw equivalent. Dump both at `-O2` on `Godbolt` and classify each as zero-overhead or not. Quantify the ones that aren't.

8. **The sort benchmark.** Reproduce §1.5. Then add: `std::stable_sort`, `std::ranges::sort`, and a hand-written quicksort. Explain the ordering you get and confirm it with `perf stat` branch-miss counts.
9. **Destruction order.** Extend the §1.6 tracer with: a global in a second TU, a `thread_local`, an object whose constructor throws, and each of the six exit mechanisms. Predict every line of output before running.
10. **Reproduce the fiasco.** Build the two-TU SIOF example and flip the behavior by changing the *link order* on the command line. Then fix it three ways: construct-on-first-use, `constinit`, and the Schwarz counter (Nifty Counter) idiom that `<iostream>` itself uses.
11. **Reasoning, no compiler.** Given `struct S { S(); ~S(); };` and:

```
S a; // global
int main() { S b; { S c; } static S d; throw 1; }
```

Which destructors run, in what order, if the exception is uncaught? What if it's caught in `main`? What changes with `-fno-exceptions`?

Chapter 2 — Types, Values, Initialization

What this chapter covers

- Fundamental types, fixed-width (`<cstdint>`), `char/char8_t/char16_t/wchar_t`
- Integer promotion and the usual arithmetic conversions (inherited from C — all the same traps)
- **Signed overflow is UB; unsigned wraps**
- **The initialization maze:** default, value, direct, copy, list, aggregate, zero
- **{ } vs ()** — most vexing parse, narrowing prevention, and the `initializer_list` hijack (`vector<int>{3,1}!`)
- `auto` — and `auto` vs `auto&` vs `const auto&` vs `auto&&`
- `decltype`, `decltype(auto)`
- `const` / `volatile` / `constexpr` / `constexpr` / `consteval` / `constinit`
- **constexpr everything** — compile-time computation as a first-class tool
- Type deduction rules: template, `auto`, and how they differ
- Narrowing, casts: `static_cast`, `const_cast`, `reinterpret_cast`, `dynamic_cast`, and **never C-style casts**
- Aliases: `using` over `typedef`
- `enum` vs `enum class`, underlying types
- Structured bindings

Real GCC 13.3 output throughout. **§2.4 (the initialization maze) is the single most confusing corner of C++** — seven forms that look interchangeable and aren't.

2.1 Fundamental types

TYPE	SIZE	ALIGN	NOTE
bool	1	1	sizeof is 1, NOT a bit
char	1	1	signedness is IMPLEMENTATION-DEFINED
signed char	1	1	a DISTINCT type from char
unsigned char	1	1	a DISTINCT type; may alias anything
char8_t	1	1	C++20, UTF-8 code unit, DISTINCT from char
char16_t	2	2	UTF-16 code unit
char32_t	4	4	UTF-32 code unit
wchar_t	4	4	4 here, 2 on Windows - avoid
short	2	2	
int	4	4	
long	8	8	8 here, 4 on Win64 (LLP64) - the LEAST
portable type			
long long	8	8	
float	4	4	IEEE-754 binary32
double	8	8	IEEE-754 binary64
long double	16	16	80-bit x87 padded to 16 here; 8 on MSVC
void*	8	8	
std::nullptr_t	8	8	
std::max_align_t	32	16	the strictest fundamental alignment

char is three distinct types

```
is_same<char, signed char>    = 0
is_same<char, unsigned char> = 0
is_same<char8_t, char>       = 0  <- C++20 made u8"" a DIFFERENT type
char is signed here?         1    (unsigned on ARM/PowerPC!)
```

Unlike C, where this only affects values, in C++ these are three separate types for overload resolution. `f(char)`, `f(signed char)`, and `f(unsigned char)` are three different functions.

⚠ **char8_t (C++20) broke a lot of code** — `u8"..."` changed from `const char[]` to `const char8_t[]`, so it no longer converts to `std::string`.

⚠ **unsigned char and std::byte may alias any object type** — that's the escape hatch for byte-wise inspection (§2.9).

Fixed-width types

```
int8/16/32/64    = 1/2/4/8
intptr_t=8 size_t=8 ptrdiff_t=8 intmax_t=8
int_fast32_t=8 int_least32_t=4
```

Family	Meaning
<code>intN_t</code>	Exactly N bits. ⚠ <i>Optional</i> — requires two's complement and no padding.

Family	Meaning
<code>int_leastN_t</code>	At least N bits, smallest such type. Always present.
<code>int_fastN_t</code>	At least N bits, fastest such type. Note <code>int_fast32_t</code> is 8 bytes here.
<code>intptr_t</code> / <code>uintptr_t</code>	Can hold a pointer. Never use <code>long</code> for this.
<code>size_t</code> / <code>ptrdiff_t</code>	Sizes and pointer differences

C++20 mandates two's complement, so the exact-width types exist everywhere in practice.

The portability rules

Need	Use
Pointer ↔ integer	<code>intptr_t</code> / <code>uintptr_t</code>
Sizes and indices	<code>size_t</code> (or <code>std::ssize()</code> , §2.2)
Wire formats, file formats	<code>int32_t</code> / <code>uint64_t</code> + <code>PRId32</code> / <code>PRId64</code>
⚠ Never	<code>long</code> — 8 bytes on Linux, 4 on Win64

`_Static_assert` your assumptions: `static_assert(sizeof(void*) == sizeof(uintptr_t));`

2.2 Integer promotion and the usual arithmetic conversions

C++ inherits all of C's integer conversion rules — and all of C's traps.

Promotion: anything smaller than `int` becomes `int`

```
decltype(c + c) is int? 1 (char + char -> INT)
decltype(s * s) is int? 1
decltype(b + b) is int? 1
decltype(uc+uc) is int? 1 <- unsigned char promotes to SIGNED int
uc + uc = 400 (400, NOT 144 - no wrapping, because it promoted)
(unsigned char)(uc+uc) = 144 (now it truncates)
```

unsigned char + unsigned char yields a signed int. This surprises people who expect modular arithmetic.

The usual arithmetic conversions: mixed signedness

```
int i = -1; unsigned u = 1;
(i < u)      -> FALSE   <- -1 converted to 4294967295
(i + u)      -> 0
(i < v.size()) -> FALSE <- the classic loop bug
```


The rule: if one operand is unsigned and at least as wide as the other, the signed operand converts to unsigned. `-1` becomes a huge positive number.

This is why `for (int i = 0; i < v.size(); ++i)` warns, and why `i >= 0 && i < v.size()` with a signed `i` can silently accept negatives.

☑ The C++20 fixes

```
std::cmp_less(i, u)           -> 1   <- CORRECT for any signedness
std::cmp_less(i, v.size())    -> 1
std::ssize(v)                 -> 3   (SIGNED size)
(i < std::ssize(v))           -> 1
std::in_range<int>(4e9)        -> 0
```

`<utility>` provides `cmp_equal`, `cmp_not_equal`, `cmp_less`, `cmp_greater`, `cmp_less_equal`, `cmp_greater_equal`, and `in_range<T>()`. They give mathematically correct answers regardless of signedness.

`std::ssize(c)` returns a signed size — use it and your loop index can be `int`.

Signed overflow is UB; unsigned wraps

```
unsigned 0 - 1      = 4294967295   (DEFINED: modular arithmetic)
UINT_MAX + 1       = 0             (DEFINED: wraps to 0)
INT_MAX + 1         -> UNDEFINED BEHAVIOUR
```

⚠ **C++20 mandates two's-complement *representation*, but signed overflow is still UB.**

Representation and arithmetic are different questions. Module 14 shows what the optimizer does with the assumption that overflow never happens.

Defenses: `-Wconversion` `-Wsign-conversion`, `-fsanitize=undefined`, and `__builtin_add_overflow` / C++26's `<stdckdint.h>`-equivalent for checked arithmetic.

2.3 using over typedef

```
typedef int (*Handler)(int, char);           // reads backwards
using Handler = int (*)(int, char);          // reads left to right

template <typename T> using Vec = std::vector<T, MyAlloc<T>>; // typedef CANNOT
do this
```

`typedef` cannot be templated. **Use `using` exclusively.**

2.4 The initialization maze ⚠

1. Default initialization — `T x;`

```
int x;           = 4607           <- INDETERMINATE. Reading it is UB.
Agg a;           = 32765, -815081454 <- members INDETERMINATE
int arr[3];      = 0, -809057472, 32597 <- INDETERMINATE
Ctor c;          = 7, 8           <- the default ctor RAN
string s;        = "" (len 0)     <- the default ctor RAN
static int       = 0              <- STATICS are always zero-initialized
```

Rule: run the default constructor if there is one; otherwise leave it uninitialized.

⚠ `std::string s;` is safe. `int x;` is not. They look identical.

2. Value initialization — `T x{} ☒`

```
int x{};         = 0              <- ZERO, guaranteed
Agg a{};         = 0, 0
int arr[3]{};    = 0, 0, 0
Ctor c{};        = 7, 8          <- the ctor still runs
Mixed m{};       = 0, ""        <- int zeroed, string default-constructed
```

Rule: zero-initialize, then run the default constructor if there is one.

☒ This is what you want almost always. It is safe for every type.

3–5. Direct, copy, and list

```
Ctor d(1,2)      = 1, 2          direct-init
Ctor l{3,4}      = 3, 4          direct-list-init
Agg a = {5,6}    = 5, 6          copy-list-init
Agg a{7,8}       = 7, 8          direct-list-init
int ci = 9       = 9             copy-init
```

Form	Name	Can call <code>explicit</code> ctors?
<code>T x(args)</code>	direct-init	☒
<code>T x = v</code>	copy-init	✗
<code>T x{args}</code>	direct-list-init	☒ (and no narrowing)
<code>T x = {args}</code>	copy-list-init	✗

6. Aggregate initialization

```
Agg{1}      = 1,0    <- missing members are VALUE-initialized
Mixed{5}    = 5,""
```

An **aggregate** has: no user-declared constructors, no private/protected non-static data members, no virtual functions, no virtual/private/protected bases. Default member initializers are allowed (C++14) and public inheritance is allowed (C++17).

7. Designated initializers (C++20)

```
Agg{.a=10,.b=20} = 10,20
Agg{.b=30}       = 0,30  <- unmentioned members ZEROED
```

⚠ **C++ requires declaration order; C does not.** `{.b=1, .a=2}` compiles in C and errors in C++. No nested or array designators either.

The summary table

You write	Built-in	Aggregate	Class with ctor
<code>T x;</code>	⚠ garbage	⚠ garbage	default ctor
<code>T x{};</code>	0	all zero	default ctor
<code>T x(a,b);</code>	—	⚠ error	ctor(a,b)
<code>T x{a,b};</code>	—	members a,b	init-list ctor if any , else ctor(a,b)
<code>T x = {a,b};</code>	—	members a,b	same, no explicit

Rules of thumb:

- 1. `T x{} or T x{args}` **by default** — safe, prevents narrowing, never declares a function
- 2. `T x(args)` **when {} would hijack the constructor** — chiefly `vector`, `string`, and anything with an `initializer_list` constructor

2.5 {} vs () — the two traps ⚠

Trap 1 — the `initializer_list` hijack

```
vector<int> v(3, 1) -> size=3 contents: 1 1 1
vector<int> v{3, 1} -> size=2 contents: 3 1
^^ COMPLETELY DIFFERENT MEANINGS.
```

```
struct Probe {
    Probe(int, int) { puts("(int,int) ctor"); }
```

```
Probe(std::initializer_list<int>) { puts("initializer_list ctor"); }
};
```

```
Probe p(1,2):      (int,int) ctor
Probe p{1,2}:      initializer_list ctor (2 elems)      ← NOT what you'd expect
NoList n{1,2}:     (int,int) ctor                        ← no init-list ctor ->
normal resolution
```

If a type has an `initializer_list` constructor, `{}` will pick it — even if another constructor is a better match, and even if it then requires a narrowing conversion that fails to compile.

The subtle case:

```
vector<string> v{10}      -> size=10   (no init-list<string> match -> size ctor)
vector<string> v{"a","b"} -> size=2    (init-list wins)
```

Same syntax, two different constructors, chosen by whether the arguments happen to match `initializer_list<T>`.

Trap 2 — the most vexing parse

```
warning: empty parentheses were disambiguated as a function declaration [-Wvexing-
parse]
warning: parentheses were disambiguated as a function declaration [-Wvexing-parse]
```

```
Timer t1;          // an object
Timer t2{};         // an object
Timer t3();         // ← a FUNCTION DECLARATION
Wrapped w1(Timer()); // ← a function taking a POINTER TO FUNCTION
Wrapped w2{Timer{}}; // an object
```

The compiler confirms it:

```
error: request for member 'tick' in 't3', which is of non-class type 'Timer()'
error: request for member 'go' in 'w1', which is of non-class type 'Wrapped(Timer
(*)())'
```

The rule: anything that *can* be parsed as a declaration is a declaration. The fix: use `{}` — it cannot be parsed as a function declaration. This is the single strongest argument for brace initialization.

`{}` prevents narrowing

```
int x = 3.99;    -> 3    (silent truncation)
int x(3.99);    -> 3    (silent truncation)
int x{3.99};    -> COMPILE DIAGNOSTIC
char c{long(300)} -> COMPILE DIAGNOSTIC
char c{100}     -> OK (a CONSTANT that fits is allowed)
```

```
warning: narrowing conversion of 'd' from 'double' to 'int' [-Wnarrowing]
warning: narrowing conversion of 'b' from 'long int' to 'char' [-Wnarrowing]
error:  narrowing conversion of '1.5e+0' from 'double' to 'int'  ← inside a
vector init-list
```

⚠ **GCC makes most of these warnings, not errors. Add `-Werror=narrowing`** — it turns three warnings into three errors here, and it catches real bugs: truncating `size_t` into `int`, `double` into `float`, `long` into `short`.

2.6 `auto` and type deduction

```
auto a = cx      (const int)      ref=no  const=no      ← strips const
auto a = rx      (const int&)     ref=no  const=no      ← strips reference AND
const
auto& a = cx      ref=yes  const=yes
const auto& a = x  ref=yes  const=yes
auto&& a = x      (lvalue)        ref=yes  rval=no       ← int&
auto&& a = 42     (rvalue)        ref=no   rval=yes       ← int&&
auto* p = &cx     ptr=yes
auto a = arr      (DECAYS)        ptr=yes  array=no       ← int*
auto& a = arr     (no decay)      ref=yes  array=no*
```

Bare `auto` strips references and top-level `const`. It always deduces a *value* type, so it always copies.

Form	Deduces	Use for
<code>auto x = e</code>	value (copy)	when you want a copy
<code>auto& x = e</code>	lvalue reference	modifying the original
<code>const auto& x = e</code>	const reference	read-only, no copy — the safe default
<code>auto&& x = e</code>	forwarding reference	generic code, proxy types
<code>auto* p = e</code>	pointer, keeps pointee <code>const</code>	documenting that it's a pointer

`auto` vs template deduction — one difference

```
auto il = {1,2,3}; -> initializer_list? 1
template<class T> f(T) with f({1,2,3}) -> COMPILE ERROR (cannot deduce)
auto single{5}; -> int? 1 (C++17 change)
```

auto follows template-parameter deduction rules in every case except one: a braced-init-list.

auto deduces `std::initializer_list<T>`; a template parameter deduces *nothing* and fails.

C++17 also changed `auto x{1}`; from `initializer_list<int>` to plain `int`.

Array decay differs by form

```
by_value(arr): T is ptr? 1    ← T = int*
by_ref(arr)  : T is array? 1  ← T = int[5]
```

By-value parameters decay; by-reference parameters do not. This is how `template<class T, size_t N> void f(T(&)[N])` deduces an array's size.

⚠ The range-for consequence, measured

```
for (auto s : v)          10.9 ms  <- COPIES every string (7x)
for (auto& s : v)         1.5 ms
for (const auto& s : v)   1.5 ms  <- the default you want
```

7x from one missing &. `for (auto x : vec_of_strings)` is one of the most common performance bugs in C++.

⚠ `std::vector<bool>` needs `auto&&`

```
is_same<decltype(vb[0]), bool&> = 0
for (auto& b : vb) -> COMPILE ERROR (operator[] returns a PROXY)
for (auto&& b : vb) b = true; -> works
```

`vector<bool>` is a bit-packed specialization whose `operator[]` returns a **proxy object**, not a `bool&` (Module 11).

When to use `auto`

Yes: iterators, lambdas (you can't spell the type), template-dependent types, obvious initializers (`auto p = std::make_unique<Widget>()`), range-for, structured bindings.

No: when the type is the point of the line, or when it hides a surprising conversion.

2.7 `decltype` and `decltype(auto)`

```
is_same<decltype(x), int> = 1    (declared type of the ENTITY)
is_same<decltype((x)), int&> = 1  ⚠ PARENTHESES change everything
```

decltype(x) gives the declared type of the entity. **decltype((x))** — with parentheses — gives the type of the *expression*, which for a named lvalue is **T&**.

The expression rules: **lvalue** → **T&**, **xvalue** → **T&&**, **prvalue** → **T**.

decltype(auto) (C++14) uses **decltype** rules instead of **auto** rules — it **preserves** references and **const**:

```
template <class C, class I>
decltype(auto) at(C&& c, I i) { return std::forward<C>(c)[i]; } // returns a
reference if [] does
```

Use it for perfect-forwarding return types. Use **auto** everywhere else.

2.8 const, constexpr, consteval, constexpr

Keyword	Meaning
const	Read-only through this name. Runtime value.
constexpr (variable)	Value known at compile time; implies const
constexpr (function)	May run at compile time if the arguments allow
consteval (C++20)	Must run at compile time — an "immediate function"
constexpr (C++20)	Must be <i>statically</i> initialized — kills the SIOF (Module 1)
volatile	Same as C: MMIO and signal handlers. Not thread safety.

Compile-time computation, measured

```
constexpr int count_vowels(std::string_view s) {           // loops and locals
    int n = 0; for (char c : s) if (is_vowel(c)) ++n; return n;
}
constexpr std::array<int,10> squares() { ... }           // containers
constexpr int sum_sorted_vector() {                       // C++20: ALLOCATION!
    std::vector<int> v{5,3,9,1,7};
    std::sort(v.begin(), v.end());
    return std::accumulate(v.begin(), v.end(), 0) + v.front()*100;
}
constexpr Version parse(std::string_view s) { ... }      // a compile-time
parser
```

```
factorial(10)           = 3628800
count_vowels("...fox") = 5
squares()[9]           = 81
sum_sorted_vector()     = 125    <- std::vector + std::sort AT COMPILE TIME
parse("1.4.2")          = 1.4.2
g_table_size (constinit) = 64

factorial(runtime_n)    = 120    <- SAME function, runtime call
```

And all of them pass **static_assert**. Proof it happened at compile time:

```
mov edx, 3628800          ← an immediate operand; factorial() was never called
```

constexpr functions have loops, local variables, and since C++20 containers, algorithms, and dynamic allocation. This is not a restricted sublanguage anymore — it's ordinary C++ the compiler can execute.

Compile-time computation is a runtime optimization. Lookup tables, parsed configuration, hashed strings, unit conversions, validated constants — computed once by the compiler, costing zero at runtime.

constexpr enforces it

```
error: the value of 'r' is not usable in a constant expression
```

Related: if **constexpr** (C++17) discards the untaken branch; **std::is_constant_evaluated()** / if **constexpr** (C++23) lets one function take different paths at compile time and runtime.

2.9 Casts

The four named casts

```
static_cast<int>(3.99)      = 3
static_cast up then down   = Derived (unchecked - you promise it is right)
dynamic_cast<Derived*>      = OK, extra=42
dynamic_cast<Other*>        = nullptr  <- CHECKED at runtime
dynamic_cast<Other&>        = THROWS bad_cast
const_cast<int*>(&ci)       = legal to FORM; WRITING through it is UB
reinterpret_cast<uintptr_t> = 0x7ffff7d4c6b0
```

Cast	Purpose	Checked?
------	---------	----------

Cast	Purpose	Checked?
<code>static_cast<T></code>	Related types: numeric conversions, up/downcasts, <code>void*→T*</code>	Compile time
<code>dynamic_cast<T></code>	Polymorphic downcast/crosscast	Runtime — <code>nullptr</code> for pointers, throws <code>bad_cast</code> for references
<code>const_cast<T></code>	Add or remove <code>const/volatile</code> only	—
<code>reinterpret_cast<T></code>	Bit reinterpretation	Nothing

⚠ Why C-style casts are banned

```
(T)x tries, IN ORDER:
  const_cast -> static_cast -> static_cast+const_cast
  -> reinterpret_cast -> reinterpret_cast+const_cast
```

A C-style cast silently escalates to the most dangerous cast that compiles. You wrote `(Foo*)p` intending a downcast and got a `reinterpret_cast` because the types were unrelated. And you cannot grep for it.

```
warning: use of old-style cast to 'int' [-Wold-style-cast]
```

GCC even suggests the replacement inline:

```
(std::size_t)reinterpret_cast<std::uintptr_t>(&dv));
static_cast<-      > (      )
```

Put `-Wold-style-cast` in your build. Named casts are verbose *on purpose* — a cast should be visible and searchable.

Type punning — the correct ways

```
memcpy                : 1.0f -> 0x3F800000
std::bit_cast (C++20)  : 1.0f -> 0x3F800000 (and it is CONSTEXPR)
*reinterpret_cast<uint32_t*>: UB (strict aliasing + alignment)
```

```
constexpr auto bc = std::bit_cast<std::uint32_t>(1.0f);
static_assert(bc == 0x3F800000);           // works at COMPILE time
```

`std::bit_cast` (C++20) is the answer. Safe, `constexpr`, compiles to nothing. Before C++20, `memcpy`.

2.10 enum vs enum class

```
=== UNSCOPED ===
int i = RED;           -> 0    (IMPLICIT conversion, no cast)
RED visible unqualified -> NAME COLLISIONS at namespace scope
sizeof(OldColor) = 4    (underlying type IMPLEMENTATION-DEFINED)
sizeof(Flags)      = 4    (C++11: you CAN fix it: `enum E : unsigned`)

=== SCOPED ===
Color::Red            -> must be qualified
int j = c;             -> COMPILE ERROR
static_cast<int>(c)    -> 0
to_underlying(c)       -> 0    (std::to_underlying in C++23)
sizeof(Color)=4  sizeof(Status)=1  <- Status is fixed to uint8_t
```

	unscoped enum	scoped enum class
Implicit conversion to <code>int</code>	yes	no
Names leak into enclosing scope	yes	no
Forward-declarable	only with a fixed type	yes
Underlying type controllable	C++11+	yes
Usable directly as a bitmask	yes	needs operator overloads

Use `enum class` by default. Add `enum class Status : std::uint8_t` when the size is part of a contract (wire formats, packed structs, ABI boundaries).

Scoped enums as bitmasks

```
constexpr Perm operator|(Perm a, Perm b){ return
Perm(to_underlying(a)|to_underlying(b)); }
constexpr bool has(Perm set, Perm f){ return (set & f) == f; }
```

```
Perm::R | Perm::W -> has(R)=1 has(W)=1 has(X)=0
```

Three operators buy you type safety that unscoped enums don't have.

Exhaustiveness checking is free

```
switch (c) {                                     // NO `default:` -> the compiler checks coverage
case Color::Red:   return "red";
case Color::Green: return "green";
```

```
case Color::Blue: return "blue";
}
```

```
warning: enumeration value 'D' not handled in switch [-Wswitch]
```

Omit **default**: on enum switches so **-Wswitch** (in **-Wall**) tells you every site that needs updating when you add a value. Combine with **-Werror=switch**.

2.11 Structured bindings (C++17)

```
auto [px,py] = Point{3,4}    -> 3,4
auto& [rx,ry]; rx=99         -> p.x is now 99
from tuple                  -> 1, 2.5, three
from C array                -> 7,8,9
from std::array             -> 1,2
```

The killer use — map iteration

```
for (const auto& [name, score] : m) std::printf("%-6s %d\n", name.c_str(), score);
```

```
alice 90 / bob 85 / carol 95
BEFORE C++17: for (const auto& kv : m) use kv.first / kv.second
```

With if-initializers (C++17)

```
if (auto [it, inserted] = m.try_emplace("dave", 70); inserted)
    printf("inserted %s = %d\n", it->first.c_str(), it->second);
```

```
inserted dave = 70
already present: dave = 70 (not overwritten)
```

The scope of **it** and **inserted** is exactly the **if/else**. Call, destructure, test, all in one scoped statement — idiomatic modern C++.

☒ Opting your own type in — the tuple protocol

```

class Range {
    int lo_, hi_;
public:
    template <std::size_t I> auto get() const {
        if constexpr (I == 0) return lo_; else return hi_;
    }
};
namespace std {
    template <> struct tuple_size<Range> : integral_constant<size_t,2> {};
    template <size_t I> struct tuple_element<I, Range> { using type = int; };
}

```

```
auto [lo,hi] = Range{10,20} -> 10,20
```

Three declarations — `tuple_size`, `tuple_element`, and `get<I>()` (member or free) — and your type destructures like a tuple.

The rules

- Works on **aggregates**, **C arrays**, and **tuple-protocol types**
- `auto [...]` **copies**; `auto& [...]` binds; `const auto& [...]` **is the default you want**
- The names are not variables — they name members of a hidden object. **You cannot skip one or declare its type.**
- C++20 allows `[[maybe_unused]]` and lambda capture of them

Module 2 rules

#	Rule
1	<code>char</code> , <code>signed char</code> , <code>unsigned char</code> , <code>char8_t</code> are four distinct types
2	⚠ long is the least portable type — 8 on Linux, 4 on Win64
3	Every C integer trap applies; use <code>std::ssize</code> and <code>std::cmp_*</code> (C++20)
4	Signed overflow is UB even though C++20 mandates two's complement
5	<code>using</code> , never <code>typedef</code>
6	⚠ T x; leaves built-ins and aggregates uninitialized . T x{}; does not.
7	Prefer {} — prevents narrowing, cannot be parsed as a function declaration
8	⚠ Except where an <code>initializer_list</code> ctor exists: <code>vector<int> v(3,1) ≠ v{3,1}</code>
9	<code>Timer t();</code> is a function declaration — <code>-Wvexing-parse</code>
10	<code>-Werror=narrowing</code> — GCC only warns by default
11	Bare <code>auto</code> strips references and top-level <code>const</code> , so it always copies

#	Rule
12	const auto& in range-for — measured 7× on strings
13	auto = template deduction, except for braced-init-lists
14	<code>decltype((x))</code> is <code>int&</code> ; <code>decltype(x)</code> is <code>int</code>
15	constexpr is ordinary C++ the compiler can run — including vector and sort in C++20
16	Never C-style casts; -Wold-style-cast
17	std::bit_cast (or memcpy) for type punning
18	enum class by default ; omit default: so -Wswitch checks exhaustiveness
19	Structured bindings + if-initializer is the idiomatic lookup pattern
20	Opt your types in with tuple_size / tuple_element / get<I>()

Practice

1. **The 36-cell initialization table.** Build a table with rows `int`, `int[3]`, an aggregate, a class with a default ctor, a class with an `initializer_list` ctor, and `std::string`; columns `T x`; `T x{}`; `T x{}`; `T x(1,2)`; `T x{1,2}`; `T x = {1,2}`; `T x = {1,2}`. Fill every cell with what actually happens (or "compile error"). Verify all 36.
2. **Uninitialized hunt.** Write a program reading uninitialized locals of `int`, `double`, `int[10]`, an aggregate, and a class with a defaulted default ctor. Run under UBSan, and MSan if you have Clang. Fix everything with `{}` and confirm the sanitizers go quiet.
3. **Both `{}` traps.** Reproduce the `vector` hijack and three most-vexing-parse cases. Confirm **-Wvexing-parse** catches each. Then find a case where `{}` picks the init-list constructor and *fails to compile* because of narrowing, where `()` would have worked.
4. **Narrowing audit.** Take a real C++ file and change every `=` and `()` initialization to `{}`. Record every narrowing diagnostic. How many were real bugs? Then add **-Werror=narrowing** permanently.
5. **auto deduction quiz.** Predict, then verify with `static_assert(std::is_same_v<...>):`

```
const int ci=1; const int& cr=ci; int arr[5]; const char* s="x"; int&& rr=1;
auto a=ci; auto b=cr; auto& c=ci; auto&& d=ci; auto&& e=5; auto f=arr;
auto& g=arr; auto h=s; const auto* i=s; auto j=rr; auto&& k=rr;
```

6. **Range-for cost.** Reproduce §2.6's benchmark with `std::string`, a 64-byte struct, `int`, and `std::vector<int>` elements. Plot copy vs reference cost against element size. Find where the difference stops mattering.
7. **decltype puzzle.** Write `decltype(auto) get(C&& c, size_t i)` that returns a reference when the container's `operator[]` does and a value when it doesn't. Test with `vector<int>`, `vector<bool>`, `map<int,int>`, and a `constexpr` array.

8. **Compile-time everything.** Write a `constexpr` function that parses a config string `"name=value;name2=value2"` into a `std::array` of pairs at compile time. `static_assert` the result. Confirm with `-S` that no parsing code was emitted.
9. **Cast audit.** Find a C++ file using C-style casts. Enable `-Wold-style-cast` and convert each. For each, determine which cast the C-style form was actually performing — were any of them `reinterpret_cast`?
10. **Enum migration.** Take an unscoped enum used as a bitmask. Convert to `enum class` with a fixed underlying type, add `|`, `&`, `~`, `|=`, `&=`, and `has()`. Confirm `-Wswitch` now checks your switches and that the old implicit conversions are compile errors.
11. **Tuple protocol.** Opt three of your own types into structured bindings: a fixed-size one, one with heterogeneous members (needing `tuple_element` specializations), and one returning references from `get<I>()` so `auto& [a,b] = obj;` modifies it.
12. **Reasoning, no compiler.** For each, say what it does and which are function declarations:

```
std::vector<int> a(3);          std::vector<int> b{3};
std::vector<int> c(3, 1);      std::vector<int> d{3, 1};
std::vector<std::string> e(3);  std::vector<std::string> f{3};
std::string g();              std::string h{};
std::string i(std::string());  std::string j{std::string{}};
```

Chapter 3 — Value Semantics, References, Pointers

What this chapter covers

- **Value semantics is the default** — this is the deepest difference from most languages
- Lvalues, rvalues, xvalues, prvalues, glvalues — **value categories**
- `&` lvalue references, `&&` rvalue references, `const&` binding to temporaries
- **Lifetime extension** of temporaries (and its limits)
- Dangling references, reference members, reference collapsing
- Raw pointers — now only for **non-owning observation**
- `nullptr`, `std::optional` for "maybe", `std::reference_wrapper`
- Pointer arithmetic, arrays, decay, `std::span` (C++20) as the fix
- Aliasing, `restrict` (not standard), strict aliasing, `std::launder`

The section where C++ stops being "C with classes" and stops being "Java with manual memory". Almost every C++ design decision downstream of here — RAII, move semantics, the Rule of Zero, why the STL looks the way it does — is a consequence of value semantics.

3.1 Value semantics is the default

The core distinction

In most modern languages, a variable is a **handle** to an object living somewhere else. In C++, a variable **is** the object.

```
struct Point { int x, y; };

Point a{1, 2};
Point b = a;    // b is a SEPARATE Point. Copy.
b.x = 99;
// a.x is still 1
```

Python / Java / C# / JavaScript equivalent:

```
a = Point(1, 2)
b = a          # b and a name the SAME object
b.x = 99
# a.x is now 99
```

This one difference cascades:

Property	Reference-semantics language	C++ (value semantics)
<code>b = a</code>	aliases	copies
Object lifetime	until GC decides	until scope ends (deterministic)
"Where does it live?"	heap, always	wherever you declared it — stack, static, inside another object
Null	every reference can be null	values are never null
Layout of <code>vector<Point></code>	array of pointers to scattered heap objects	contiguous <code>Point</code> objects, no indirection
Cleanup	<code>finally</code> / <code>using</code> / GC	destructor, automatic

Why this is the deepest difference

1. Composition is real. A `Point` member inside a `Rect` is *stored inside* the `Rect`. No indirection, no separate allocation, no null check.

```
struct Rect {
    Point tl, br;           // 16 bytes, all inline
    std::string label;      // owns its buffer, but the string object itself is
    inline
};
sizeof(Rect); // one object, one allocation for the whole thing (+ string's heap
              // buffer)
```

2. You get destruction for free, and it composes. Destroying a `Rect` destroys `label` which frees its buffer. Nobody wrote that code. This is the **Rule of Zero** (§7) — a class built out of well-behaved values needs *no* destructor, copy ctor, or assignment operator.

3. Cache locality. `std::vector<Point>` is one contiguous block. `ArrayList<Point>` in Java is a pointer array into scattered heap. This is a large constant factor in real programs (§24).

4. No aliasing surprises. If you take a parameter by value, nobody else can mutate it under you. Reasoning is local.

The cost, and the answer

Copies cost. The naive worry: "surely value semantics means copying `vector<string>` everywhere?"

The answer is **not** "use pointers". The answers are:

- **References / `const&`** for passing without copying (this section)
- **Move semantics** for transferring without copying (§5)
- **Copy elision / RVO** — returning by value is usually free (§5)

```
std::vector<std::string> load();           // returns by value
auto v = load();                          // no copy at all (guaranteed elision,
C++17)
void process(const std::vector<std::string>& v); // no copy
process(v);
auto w = std::move(v);                    // steals the buffer, no copy
```

Modern C++ style: **think in values, optimize with references and moves** — not "allocate everything with `new` and pass pointers".

Regular types

Stepanov's design rule the STL is built on: a type should behave like `int`. Copyable, assignable, equality-comparable, default-constructible, and `a = b` implies `a == b`. C++20 names this:

```
#include <concepts>
static_assert(std::regular<Point>);           // semiregular + equality_comparable
static_assert(std::regular<std::string>);
```

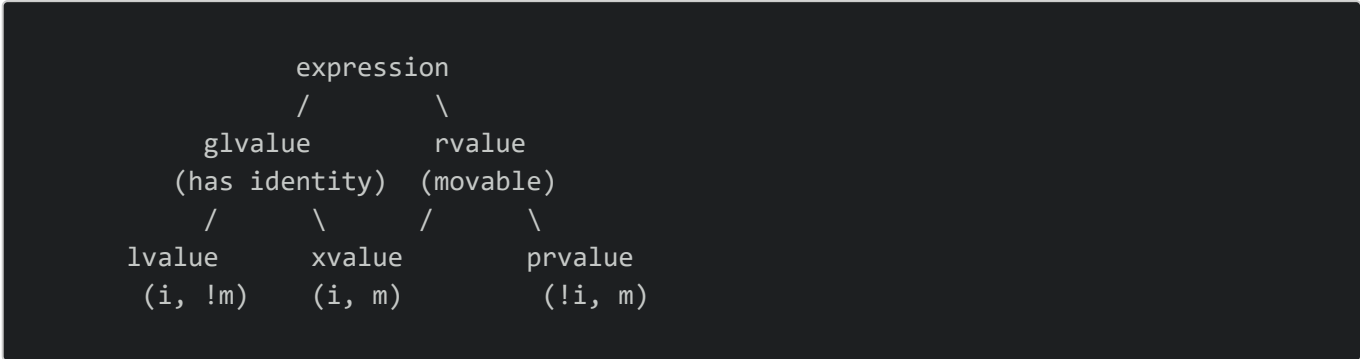
If your type is regular, every STL container and algorithm just works with it. If you write a type with reference members or a raw owning pointer, you break regularity and you will be fighting the language.

3.2 Value categories ⚠

Where the mess comes from

C had two categories, named for assignment: **lvalue** (can appear left of `=`) and **rvalue** (can't). C++11 needed to distinguish "temporary I can steal from" from "real object I must not damage", so it split them along **two independent axes**:

- **has identity (i)** — the object has an address you could take; you can tell it apart from another object
- **can be moved from (m)** — the expression is about to expire, so stealing its guts is safe



Category	Identity	Movable	Meaning
lvalue	yes	no	a named, persistent object
xvalue	yes	yes	an "expiring" object — has an address, but you're done with it
prvalue	no	yes	a "pure rvalue" — an initializer for an object that doesn't exist yet
glvalue	yes	—	lvalue or xvalue ("generalized")
rvalue	—	yes	xvalue or prvalue

The rule that trips up everyone

A named variable is always an lvalue — even if its type is an rvalue reference.

```
void sink(std::string&& s) {
    // s is DECLARED as string&&
    // but the EXPRESSION `s` is an LVALUE (it has a name, it has an address)
    other(s);                // calls other(const string&) – copies!
    other(std::move(s));      // calls other(string&&)      – moves. Correct.
}
```

Type ≠ value category. `s` has type `string&&`; the expression `s` has category lvalue. This is why `std::move` and `std::forward` exist at all (§5): they are casts that re-apply the category the type promised.

Classification cheat sheet

```
int  n = 0;
int  arr[4]{};
int* p = &n;
int& lref = n;
int&& rref = 42;
```

```
struct S { int m; };
S  makeS();           // returns by value
S& getRef();          // returns by reference
S&& getRRef();         // returns by rvalue reference
```

Expression	Category	Why
<code>n, lref, rref</code>	lvalue	named
<code>arr, arr[2]</code>	lvalue	named / subscript on lvalue array
<code>*p</code>	lvalue	dereference yields an object
<code>p->m, s.m</code> (s lvalue)	lvalue	member of lvalue
<code>"hello"</code>	lvalue	string literals are arrays with static storage — they have an address
<code>42, 3.14, true, nullptr</code>	prvalue	no identity
<code>n + 1, n < 2, -n</code>	prvalue	computed result
<code>makeS()</code>	prvalue	function returning by value
<code>getRef()</code>	lvalue	function returning T&
<code>getRRef(), std::move(n)</code>	xvalue	function returning T&&
<code>makeS().m</code>	xvalue	member of a prvalue — the temporary has identity now
<code>arr[0]</code> where <code>arr</code> is a temporary array member	xvalue	same reason
<code>n++</code>	prvalue	returns the old value
<code>++n</code>	lvalue	returns the object itself
<code>lambda []{} </code>	prvalue	it's a temporary closure object

The C++17 change: prvalues aren't objects anymore

Pre-C++17, `S s = makeS();` conceptually created a temporary and copied it (with the copy usually elided as an optimization). Since C++17, **a prvalue is not an object at all** — it's a recipe for initializing one. The object is only created when the prvalue is *used* to initialize something ("temporary materialization conversion").

Consequences:

```
struct NonMovable {
    NonMovable() = default;
    NonMovable(const NonMovable&) = delete;
    NonMovable(NonMovable&&) = delete;
};
```

```
NonMovable make() { return NonMovable{}; } // OK in C++17! No copy/move exists
to elide.
NonMovable x = make(); // OK - x is initialized directly
```

This is **guaranteed copy elision** (§5). Factory functions returning immovable types became legal.

Detecting the category (debugging trick)

`decltype` reports the category when given a parenthesized expression:

- `decltype(expr)` for a *name* → the declared type
- `decltype((expr))` → `T&` if lvalue, `T&&` if xvalue, `T` if prvalue

```
int n = 0;
static_assert(std::is_same_v<decltype(n), int >); // declared type
static_assert(std::is_same_v<decltype((n)), int& >); // lvalue
static_assert(std::is_same_v<decltype(std::move(n)), int&&>); // xvalue
static_assert(std::is_same_v<decltype(n + 0), int>); // prvalue
```

Handy compile-time probe:

```
template <class T>
constexpr const char* category() {
    if constexpr (std::is_lvalue_reference_v<T>) return "lvalue";
    else if constexpr (std::is_rvalue_reference_v<T>) return "xvalue";
    else return "prvalue";
}
#define CATEGORY_OF(e) category<decltype((e))>()
```

⚠ `decltype(auto)` inherits this behaviour, which is exactly why `decltype(auto) f() { return (local); }` returns a **dangling reference** (§6).

3.3 References: `&`, `&&`, `const&`

Lvalue references `T&`

An alias for an existing object. Not an object itself.

```
int n = 5;
int& r = n; // r IS n
r = 10; // n == 10
int& bad; // ERROR - must be initialized
```

Properties:

- **Must be initialized**, always
- **Cannot be rebound** — `r = other`; assigns *through* `r`, it doesn't re-point `r`
- **No null references** in well-formed code (you can produce one via `*(T*)nullptr`, but that's already UB)
- **Not an object**: no `int&*`, no `int&[10]`, no `sizeof` of the reference itself (`sizeof(r) == sizeof(int)`)
- **Zero storage in practice** — usually compiled to a pointer, often optimized away entirely

`const T&` — the universal parameter

```
void f(const std::string& s);

std::string a = "x";
f(a);           // binds to lvalue
f("literal");   // binds to a TEMPORARY string built for the call
f(std::string("tmp")); // binds to a prvalue
```

`const&` binds to lvalues, xvalues, prvalues, and anything **convertible**. The conversion case is a real trap:

```
void g(const long& x);
int i = 5;
g(i);           // creates a temporary `long`, binds x to THAT
                // x is not an alias for i — it's an alias for a hidden copy

const double& d = i; // same: d refers to a temporary double, not to i
i = 99;
// d is still 5.0
```

Same trap with `const std::string&` bound to a `const char*`, or `const Base&` bound to a `Derived` by-value copy in some conversion paths.

Rvalue references `T&&`

Binds only to rvalues (xvalues and prvalues). Its purpose is **detection**: "this object is expiring, I'm allowed to gut it."

```
void take(std::string&& s);

std::string a = "x";
take(a);           // ERROR — a is an lvalue
take(std::move(a)); // OK — std::move casts to xvalue
take("literal");   // OK — prvalue temporary
```

⚠ `T&&` in a **deduced** context (`template<class T> void f(T&&)`, or `auto&&`) is a **forwarding reference**, not an rvalue reference. Totally different rules. §5.

The overload set

```
void f(std::string&);           // 1: non-const lvalues only
void f(const std::string&);    // 2: everything else (fallback)
void f(std::string&&);         // 3: rvalues

std::string s = "a";
const std::string c = "b";

f(s);                          // 1
f(c);                          // 2
f(std::move(s));               // 3
f("tmp");                      // 3 (temporary is an rvalue)
```

Remove #3 and rvalues fall back to #2. Remove #1 and non-const lvalues fall back to #2. `const&` is the catch-all — which is why it's the safe default and why `&&` overloads are an *optimization*, never a requirement.

`auto&&` — bind to anything

```
auto&& r = anything; // works for lvalue, const lvalue, xvalue, prvalue
```

Because of reference collapsing (§3.5), `auto&&` deduces `T&` for lvalues and `T&&` for rvalues. It also **lifetime-extends prvalues**. This is the idiom used in range-for and in generic forwarding code:

```
for (auto&& elem : container) { ... } // works whether container yields T&,
const T&, or T
```

3.4 Lifetime extension ⚠

The rule

Binding a **temporary** directly to a reference extends the temporary's lifetime to the lifetime of the reference.

```
std::string make();

{
    const std::string& r = make(); // temporary lives until end of this block
    std::cout << r;                // fine
}                                  // temporary destroyed here
```

Without this rule, `const&` parameters and locals would be useless with temporaries. It applies to `const T&`, `T&&`, and `auto&&`. It does **not** apply to non-const `T&` (which can't bind to a temporary anyway).

It extends through member access and subscripting

```
struct S { std::string name; int arr[3]; };
S make();

const std::string& n = make().name;    // OK – the whole S temporary is extended
const int& i = make().arr[1];          // OK – same
```

The temporary being extended is the *complete* `S` object, not just the subobject.

The limits — where it does NOT apply ⚠

1. Never through a function return. This is the big one.

```
struct S { std::string name; const std::string& get() const { return name; } };
S make();

const std::string& bad = make().get();    // DANGLES
// get() returns a reference; a reference return is not a temporary,
// so there is nothing to extend. The S temporary dies at the semicolon.

std::string&& also_bad = std::move(make()); // DANGLES
// std::move is a function returning T&& – same problem
```

Rule of thumb: **if a function call produced the reference, no extension happens.**

2. Never for reference members in constructors.

```
struct Holder {
    const std::string& ref;
    explicit Holder(const std::string& s) : ref(s) {}
};

Holder h{std::string("temp")};    // temporary dies at end of the FULL EXPRESSION
h.ref;                            // DANGLING – use-after-free
```

The temporary lives until the semicolon, not for the lifetime of `h`. Member initialization does not extend. This bites hard with `std::string_view` and `std::span` members too.

3. Never through a function parameter.

```
const std::string& identity(const std::string& s) { return s; }
const std::string& r = identity("hello");    // DANGLES
```

The temporary is extended to the lifetime of the *parameter* `s`, which ends when `identity` returns.

4. Extension happens once, at the point of binding. Passing an already-extended reference around does not re-extend anything.

The range-for trap (fixed in C++23)

```
struct Bag { std::vector<int> items; const std::vector<int>& get() const { return items; } };
Bag makeBag();

for (int x : makeBag().get()) { ... }
// C++20 and earlier: the Bag temporary is NOT extended -> UB, reads freed memory
// C++23 (P2718R0): all temporaries in the range-initializer are extended -> fine
```

Until you're confidently on C++23 across all your compilers, the safe form is:

```
auto bag = makeBag();
for (int x : bag.get()) { ... }
```

Practical guidance

- Lifetime extension is a **convenience for locals**, not a design tool.
- If you're relying on it for anything that crosses a function boundary or lives in a class, you have a bug.
- **-Wdangling-reference** (GCC 13+), **-Wdangling-gsl** (Clang), and **ASan** catch most of these. Turn them on.

3.5 Dangling references, reference members, reference collapsing

Dangling: the four common shapes

```
// 1. Reference to a local
const std::string& f() {
    std::string s = "oops";
    return s;                // DANGLING
}

// 2. Reference into a container that reallocates
std::vector<int> v{1, 2, 3};
int& r = v[0];
v.push_back(4);             // may reallocate
r = 99;                     // DANGLING - iterator/reference invalidation (§11)

// 3. Reference into a destroyed temporary
const std::string& r2 = getObj().name(); // DANGLING (function return, §3.4)

// 4. Captured by reference in a lambda that outlives the scope
```

```
auto make() {
    std::string s = "x";
    return [&s]{ return s; };    // DANGLING capture (§6)
}
```

Shape 2 is the one people underestimate. **Every container has its own invalidation rules and you must know them** (§11).

Reference members: usually the wrong tool

```
struct Bad {
    Logger& log;                // reference member
};
```

Consequences:

- Copy assignment is **implicitly deleted** (you can't rebind a reference), so the class isn't assignable, so it can't go in a **vector** and be sorted, erased, etc.
- Must be initialized in every constructor's init list
- Silent dangling risk if the referent dies first
- Breaks the Rule of Zero regularity

Alternatives, in order of preference:

Want	Use
Non-owning, always present, rebindable	<code>Logger*</code> (document as non-owning) or <code>gsl::not_null<Logger*></code>
Non-owning, always present, want reference syntax + copyable	<code>std::reference_wrapper<Logger></code>
Non-owning, optional	<code>Logger*</code> (null = absent)
Owning	<code>std::unique_ptr<Logger></code> or a <code>Logger</code> by value

Reference collapsing

You can't *write* a reference to a reference, but templates, **auto**, **decltype**, and type aliases can *produce* one. The language collapses it:

Formed	Collapses to
<code>T& &</code>	<code>T&</code>
<code>T& &&</code>	<code>T&</code>
<code>T&& &</code>	<code>T&</code>
<code>T&& &&</code>	<code>T&&</code>

Mnemonic: lvalue reference wins. Only `&&` stays `&&`.

```
using Lref = int&;
using Rref = int&&;

using A = Lref&;    // int&
using B = Lref&&;   // int&
using C = Rref&;    // int&
using D = Rref&&;   // int&&
```

This is the entire machinery behind **forwarding references** — `template<class T> void f(T&&)` deduces `T = int&` for lvalues, making the parameter `int& && → int&`; and `T = int` for rvalues, making it `int&&`. That's why one signature handles both. Full treatment in §5.

3.6 Raw pointers — now only for non-owning observation

The modern ownership rule

Owning: `unique_ptr`, `shared_ptr`, containers, or plain values. **Non-owning:** raw pointers, references, `span`, `string_view`, iterators.

A raw `T*` in modern C++ says: *"I observe this. I did not allocate it. I will not free it. I do not control its lifetime."*

```
// BAD — 1998 style
Widget* w = new Widget();
use(w);
delete w;                                // leaks on exception, double-free on copy

// GOOD
auto w = std::make_unique<Widget>();
use(w.get());                            // pass the raw pointer — observation only
                                           // destructor handles cleanup, exception-safe
```

You should essentially never write `new` or `delete` outside of a resource-managing class's implementation (§4).

Marking intent when you must use raw pointers

```
#include <gsl/gsl>                        // Microsoft GSL

gsl::owner<Widget*> p;                     // documents ownership at a C boundary; static
analysis checks it
gsl::not_null<Widget*> q;                  // never null, enforced
```

`gsl::owner<T*>` doesn't do anything at runtime — it's a marker that clang-tidy's `cppcoreguidelines-owning-memory` check enforces.

Parameter passing cheat sheet

Signature	Meaning	Use when
<code>f(T v)</code>	I get my own copy	cheap types (<code>int</code> , <code>string_view</code> , <code>span</code>), or you'll copy anyway (then <code>std::move</code> it in)
<code>f(const T& v)</code>	I read it, I don't keep it	the default for expensive types
<code>f(T& v)</code>	I will modify your object	out/in-out params (prefer returning a value when you can)
<code>f(T&& v)</code>	I will consume it	move-only sinks; usually paired with <code>const T&</code>
<code>f(T* v)</code>	optional, non-owning, may be null	"absent" is meaningful
<code>f(std::unique_ptr<T> v)</code>	I take ownership	transfer of ownership, explicit at the call site
<code>f(const std::unique_ptr<T>&)</code>	⚠ almost always wrong	you wanted <code>const T&</code> or <code>T*</code>
<code>f(std::shared_ptr<T> v)</code>	I share ownership	genuinely shared lifetime only
<code>f(std::span<T>)</code>	contiguous range, non-owning	replaces <code>(T* ptr, size_t n)</code>

The last two rows are the most common review comments in real C++ codebases: **don't take smart pointers by parameter unless the function is about ownership.**

3.7 `nullptr`, `std::optional`, `std::reference_wrapper`

`nullptr`

`NULL` is `0` or `((void*)0)`; both break overloading.

```
void f(int);
void f(char*);

f(NULL);      // ambiguous or calls f(int) – depends on the macro definition
f(nullptr);   // unambiguously f(char*)
```

`nullptr` has its own type, `std::nullptr_t`, which converts to any pointer type but not to integers. Always use it. `-Wzero-as-null-pointer-constant` will flag the old style.

`std::optional<T>` — "maybe a value", with value semantics

```
#include <optional>

std::optional<int> parse(std::string_view s);

if (auto n = parse("42")) {
    use(*n);           // operator* / operator-> ; UB if empty
}

int v1 = parse("x").value_or(0);    // default
int v2 = parse("x").value();        // throws std::bad_optional_access if empty
```

Key points:

- The **T** is stored **inline** in the optional (no heap allocation) — `sizeof(optional<int>)` is typically 8: the int plus a bool plus padding
- Copy/move semantics follow **T**; it stays a regular type
- C++23 adds monadic operations: `and_then`, `transform`, `or_else`

```
auto result = parse(input)
    .transform([](int n) { return n * 2; })
    .value_or(-1);
```

⚠ **`std::optional<T*>` does not exist** (through C++23; P2988 adds it in C++26). For an optional reference today:

```
T* p;           // simplest, idiomatic
std::optional<std::reference_wrapper<T>> o; // verbose, needs .value().get()
```

⚠ `optional<bool>` and `optional<T*>` are legal but read badly — `if (opt)` tests *presence*, not the contained value. Be explicit: `if (opt && *opt)`.

`std::reference_wrapper<T>`

A copyable, assignable, **rebindable** object that behaves like a reference.

```
#include <functional>

int a = 1, b = 2;
std::reference_wrapper<int> r = a;    // or: auto r = std::ref(a);
r.get() = 10;                        // a == 10
r = b;                               // REBINDS to b (a plain int& couldn't do this)
```

Why it exists — three concrete uses:

1. References in containers (you can't have `vector<T&>`):

```
std::vector<std::reference_wrapper<Widget>> selected;
for (auto& w : allWidgets)
    if (w.isSelected()) selected.push_back(w);    // stores a reference, not a
                                                    copy

for (Widget& w : selected) w.draw();             // implicit conversion back to
T&
```

2. Forcing by-reference through APIs that copy. `std::thread`, `std::bind`, and `std::async` copy their arguments:

```
void worker(std::atomic<int>& counter);

std::atomic<int> c{0};
std::thread t1(worker, c);    // ERROR – thread tries to copy the atomic
std::thread t2(worker, std::ref(c)); // OK – passes the reference through
```

This is a classic bug when the argument is copyable: `std::thread(worker, bigVector)` silently copies and the worker mutates the wrong object.

3. `std::cref` for the const version.

Note `reference_wrapper` converts implicitly to `T&`, so it usually reads like a reference; you only need `.get()` where the conversion can't be deduced (e.g. `r.get() = x`, or in templates).

3.8 Pointer arithmetic, arrays, decay, and `std::span`

Arrays are objects; they decay

```
int arr[5] = {1,2,3,4,5};
sizeof(arr);    // 20 – arr is a real array object
int* p = arr;   // array-to-pointer DECAY: implicit conversion to &arr[0]
sizeof(p);      // 8 – the size is gone forever
```

Decay happens in almost every expression context. The exceptions: `sizeof`, `&`, and binding to a reference-to-array (`int (&r)[5]`).

The parameter lie

```
void f(int arr[10]);    // this is a LIE. The signature is really:
void f(int* arr);      // ...this. The 10 is ignored, sizeof(arr) is 8.
```

```
void g(int (&arr)[10]); // this one actually takes an array of exactly 10
```

C++ inherits this from C, and it's the root cause of most buffer overruns. **The fix is `std::span`.**

Pointer arithmetic legality ⚠

Pointer arithmetic is only defined **within a single array object** (a scalar counts as an array of 1):

```
int a[5];
int* p = a + 3;    // OK
int* q = a + 5;    // OK – one-past-the-end is a valid VALUE
*q;               // UB – but dereferencing it is not
int* r = a + 6;    // UB – even forming the pointer

int x, y;
&x + 1;           // OK (one-past-end of a 1-element array)
&x < &y;           // unspecified – comparing unrelated pointers
```

Comparing or subtracting pointers into different objects is not defined. Optimizers rely on this.

Modern array handling

```
#include <iterator>

int arr[5]{};
std::size(arr);    // 5, works on arrays and containers
std::ssize(arr);   // 5 as a signed type (avoids unsigned wrap bugs)
for (int x : arr) {} // range-for works – no decay
std::begin(arr); std::end(arr);
```

Or skip C arrays entirely: `std::array<int, 5>` is a real value type — copyable, assignable, knows its size, no decay.

`std::span<T>` (C++20) — the fix

A non-owning view over a contiguous range: pointer + length, in one cheap-to-copy object.

```
#include <span>

void process(std::span<int> data) {
    for (int& x : data) x *= 2;
    data.size();           // the size came along
    auto first3 = data.first(3); // subviews
    auto mid    = data.subspan(2, 4);
}
```

```
int carr[5]{};
std::array<int,5> arr{};
std::vector<int> vec(5);

process(carr);    // all three work
process(arr);
process(vec);
```

- `std::span<const T>` for read-only
- `std::span<int, 3>` — **fixed extent**, size known at compile time, `sizeof` is just the pointer
- Pass **by value**; it's two words
- `std::string_view` is the same idea for character data (§11)
- `std::mdspan` (C++23) generalizes it to multiple dimensions

⚠ **Spans dangle exactly like references.** A `span` into a `vector` is invalidated by `push_back`. Never store a `span` as a member unless you control the owner's lifetime, and never return a `span` into a local.

```
std::span<int> bad() {
    std::vector<int> v{1,2,3};
    return v;           // DANGLING
}
```

3.9 Aliasing, `restrict`, strict aliasing, `std::launder`

What aliasing costs you

Two pointers alias when they may refer to the same object. The compiler must then assume a write through one invalidates what it knows about the other:

```
void add(int* a, const int* b, int n) {
    for (int i = 0; i < n; ++i)
        a[i] += *b;    // must RELOAD *b every iteration – a[i] might BE *b
}
```

The compiler can't hoist `*b` out of the loop, and can't vectorize freely. This is a real, measurable cost in numeric code.

The strict aliasing rule

The standard's escape hatch: **accessing an object through a glvalue of an unrelated type is UB**, so the compiler is allowed to assume `int*` and `float*` never alias.

Permitted access types for an object of type `T`:

- `T`, `const T`, `volatile T`
- signed/unsigned variants of `T`

- a type similar to `T` (cv-qualification differences)
- a class/aggregate containing `T` as a member
- `char`, `unsigned char`, `std::byte` — these may alias anything

```
float f = 1.0f;
int i = *reinterpret_cast<int*>(&f);    // UB – strict aliasing violation
                                         // may "work" at -O0 and break at -O2
```

`-fno-strict-aliasing` disables the assumption (the Linux kernel builds this way). Don't rely on it in portable code.

Type punning, done correctly

```
// C++20 – the right answer
#include <bit>
int i = std::bit_cast<int>(f);    // constexpr, no UB, compiles to zero
instructions

// Pre-C++20 – also correct, optimizers see through it
int i;
std::memcpy(&i, &f, sizeof i);
```

⚠ **Union punning is UB in C++** (it is legal in C99+). Writing `u.f` and reading `u.i` is defined in C, not in C++, even though every mainstream compiler supports it as an extension.

restrict

Not standard C++ (it's C99). All major compilers support `__restrict` / `__restrict__` as an extension:

```
void add(int* __restrict a, const int* __restrict b, int n) {
    for (int i = 0; i < n; ++i) a[i] += *b;    // now hoistable and vectorizable
}
```

It's a **promise** you make: for the pointer's lifetime, the object it points to is accessed only through that pointer. Break the promise and you get silent UB. Use it sparingly, at leaf numeric functions, and verify on Godbolt that it actually changed the codegen.

std::launder and object lifetime

C++ tracks *objects*, not just bytes. If you destroy an object and construct a new one in the same storage, pointers to the old object are not automatically usable for the new one — the compiler may have cached facts (like `const` members or vtable pointers) that are no longer true.

```
#include <new>
```

```

struct S { const int x; };    // const member – compiler may assume it never
                             changes

S* p = new S{1};
p->~S();
S* q = new (p) S{2};         // placement-new a NEW object in the same storage

p->x;                        // UB – p still refers to the OLD object
q->x;                        // OK – 2
std::launder(p)->x;         // OK – 2; launder "re-derives" a valid pointer

```

You need `launder` only when the type has `const`/reference members or a vtable, and you're doing placement-new over existing storage. If you're not writing an allocator, a variant, or a small-buffer-optimized type, you'll never need it.

Related, C++23:

```

// Reading a byte buffer as objects – previously UB, now well-defined
std::byte buffer[1024];
auto* ints = std::start_lifetime_as_array<int>(buffer, 256);

```

Practical rules

1. Never `reinterpret_cast` to read one type as another. Use `bit_cast` or `memcpy`.
2. `char*` / `std::byte*` are the only legitimate "raw bytes" view types.
3. Union punning: fine in C, UB in C++ — use `bit_cast`.
4. `__restrict` is a performance tool with a sharp edge; profile before and after.
5. **UBSan catches some aliasing violations; `-Wstrict-aliasing=3` catches some more.** Neither catches all.

Gotcha checklist for this section

- ☐ A named `T&&` parameter is an **lvalue** inside the function — `std::move` it to pass it on
- ☐ `const long& r = someInt;` binds to a **temporary copy**, not to `someInt`
- ☐ Lifetime extension **never** works through a function return, a parameter, or a reference member
- ☐ `std::move(f())` where `f` returns by value → **dangling** if bound to a reference
- ☐ References/spans/iterators into a `vector` die on reallocation
- ☐ Reference members silently delete your copy assignment operator
- ☐ `f(int arr[10])` is `f(int* arr)` — the size is a comment
- ☐ Forming `a + n + 1` past one-past-the-end is UB even without dereferencing
- ☐ `optional<T&&>` doesn't exist yet; use `T*`
- ☐ `std::thread(f, obj) copies obj` — use `std::ref`
- ☐ Union type punning is UB in C++ (unlike C)
- ☐ `reinterpret_cast<int*>(&someFloat)` is a strict aliasing violation

Compiler flags that catch this section's bugs


```

-Wdangling-reference      # GCC 13+, returning/binding refs to temporaries
-Wdangling-gsl            # Clang, span/string_view lifetime
-Wreturn-local-addr
-Wzero-as-null-pointer-constant
-Wstrict-aliasing=3
-fsanitize=address         # use-after-free, use-after-scope, OOB
-fsanitize=undefined       # some pointer arithmetic and alignment UB
-fsanitize=address -fsanitize=address-use-after-scope

```

Exercises worth doing

1. Write a function template that prints whether its argument was an lvalue, xvalue, or prvalue. Test it against all rows of the cheat sheet in §3.2.
2. Take a `struct` with a `std::string_view` member. Construct it from a temporary `std::string`. Run under ASan. Understand exactly which line is the use-after-free.
3. Rewrite a `(T* data, size_t len)` API in a codebase you have as `std::span<T>`. Note how many bounds checks and length parameters disappear.
4. On Godbolt, compile the `add(int*, const int*)` loop above with and without `__restrict` at `-O3 -march=x86-64-v3`. Count the instructions.
5. Implement a tiny `not_null<T*>` wrapper. Decide what it should do on construction from `nullptr` and why.

Chapter 4 — RAII & Resource Management ⚠

What this chapter covers

- **RAII** — acquisition is initialization, release is destruction
- Constructors, destructors, deterministic destruction
- **Scope-based cleanup** — why C++ has no `finally` and doesn't need one
- **Smart pointers:** `unique_ptr` (default), `shared_ptr`, `weak_ptr`
- `make_unique` / `make_shared` and why
- Custom deleters, `unique_ptr<T[]>`
- ⚠ `shared_ptr` cost: atomic refcount, control block, cycles
- **Ownership as a design decision**, expressed in types
- The **Rule of Zero** / Rule of Three / **Rule of Five**
- `std::scope_exit`-style guards, `gsl::finally`

If you only take one idea out of C++, take this one. RAII is why C++ can be simultaneously manually memory-managed *and* leak-free; it's why C++ has no `finally`; it's the only reason exception safety is tractable; and it's the mechanism every other feature (move semantics, smart pointers, lock guards, the whole STL) is built on top of.

4.1 The idea

What counts as a resource

Anything you **acquire** and must **release**:

Resource	Acquire	Release
Heap memory	<code>new</code> / <code>malloc</code>	<code>delete</code> / <code>free</code>
File	<code>fopen</code> , <code>open</code>	<code>fclose</code> , <code>close</code>
Socket	<code>socket</code> , <code>accept</code>	<code>close</code>
Mutex	<code>lock()</code>	<code>unlock()</code>
DB transaction	<code>BEGIN</code>	<code>COMMIT</code> / <code>ROLLBACK</code>
GPU buffer	<code>vkCreateBuffer</code>	<code>vkDestroyBuffer</code>
RefCount	<code>AddRef</code>	<code>Release</code>
Temp file / directory	<code>create</code>	<code>unlink</code>
Terminal mode, signal mask, locale	<code>set</code>	<code>restore</code>

Note that **memory is only one row of this table**. This matters enormously: garbage collection solves exactly one row and leaves the rest to `try/finally`. RAII solves all of them uniformly.

The mechanism

A resource's lifetime is tied to the lifetime of an object. Constructor acquires. Destructor releases. The compiler guarantees the destructor runs.

```
class File {
    std::FILE* f_;
public:
    explicit File(const char* path, const char* mode)
        : f_(std::fopen(path, mode))
    {
        if (!f_) throw std::runtime_error("cannot open"); // never half-
constructed
    }
    ~File() { if (f_) std::fclose(f_); } // ALWAYS runs

    File(const File&) = delete; // not copyable
    File& operator=(const File&) = delete;
    File(File&& o) noexcept : f_(std::exchange(o.f_, nullptr)) {}
    File& operator=(File&& o) noexcept {
        if (this != &o) { if (f_) std::fclose(f_); f_ = std::exchange(o.f_,
        nullptr); }
        return *this;
    }

    std::FILE* get() const noexcept { return f_; }
};
```

Now:

```
void work() {
    File f("data.txt", "r");
    if (nothing_to_do()) return;           // closed
    if (bad()) throw Error{};             // closed
    for (...) { if (x) break; }           // closed
}
```

Every exit path is covered, and you wrote no cleanup code at any of them.

The name is bad

"Resource Acquisition Is Initialization" describes only half of it — and the half that matters less. Bjarne has said he'd choose differently now. Better mental names:

- **Constructor acquires, destructor releases**
- **Scope-Bound Resource Management** (SBRM)

The *destructor* half is the load-bearing part.

How other languages handle it

```
// C – the goto cleanup ladder
int work(void) {
    FILE* f = fopen("a", "r");
    if (!f) return -1;
    char* buf = malloc(1024);
    if (!buf) goto fail_buf;
    sqlite3* db;
    if (sqlite3_open("x.db", &db)) goto fail_db;
    /* ... work ... */
    sqlite3_close(db);
    free(buf);
    fclose(f);
    return 0;
fail_db:
    free(buf);
fail_buf:
    fclose(f);
    return -1;
}
```

```
// Java – explicit, per-resource, and only for AutoCloseable types
try (FileReader f = new FileReader("a"); Connection db = open()) { ... }
```

```
with open("a") as f, connect() as db: ...    # explicit `with`, per-use
```

```
f, _ := os.Open("a"); defer f.Close()        // explicit defer, function-scoped
```

```
// C++ – nothing to write. The types carry it.
File f("a", "r");
Db db("x.db");
std::vector<char> buf(1024);
```

The difference: in every other language the **user of the resource** must remember the cleanup construct at every use site. In C++ the **author of the type** handles it once, and every user gets it automatically, including users who forgot it exists.

4.2 Constructors, destructors, deterministic destruction

Storage duration determines when destructors run

Duration	Declared as	Destroyed
automatic	local variable	end of enclosing scope, in reverse declaration order
static	namespace-scope, <code>static</code> local, <code>static</code> member	at program exit, reverse order of construction (⚠ static destruction order fiasco)
thread_local	<code>thread_local</code>	at thread exit
dynamic	<code>new</code> / allocator	when you <code>delete</code> — i.e. when a smart pointer's destructor runs
temporary	prvalue materialized	end of full-expression (unless lifetime-extended, §3.4)

Construction and destruction order

```
struct Base { Base() { log("Base ctor"); } ~Base() { log("Base dtor"); }
};
struct Member { Member() { log("Member ctor"); } ~Member() { log("Member dtor"); }
};

struct Derived : Base {
    Member a, b;
    Derived() : Base(), a(), b() { log("Derived body"); }
    ~Derived() { log("Derived body dtor"); }
};
```

Construction:

1. Virtual bases
2. Direct bases, in declaration order
3. Non-static members, **in declaration order** (⚠ *not* the order of the member initializer list — §7)
4. Constructor body

Destruction is the **exact reverse**.

```
Base ctor -> Member ctor(a) -> Member ctor(b) -> Derived body
Derived body dtor -> Member dtor(b) -> Member dtor(a) -> Base dtor
```

Deterministic destruction is the whole point

You know **exactly when** cleanup happens. Compare a GC language: a `finalize/___del__` runs at an unspecified time, possibly on a different thread, possibly never (at program exit, many runtimes just don't bother). That is fine for memory and useless for file handles, locks, and database transactions — which is precisely why those languages had to invent `try-with-resources` / `using` / `with` as a *separate* mechanism.

C++ needs one mechanism because destruction is deterministic.

Exceptions during construction ⚠

If a constructor throws, the object's destructor is not called — because the object never existed. But every **fully-constructed base and member is destroyed**, in reverse order.

```
struct Widget {
    std::vector<int> data;    // constructed 1st
    File             file;   // constructed 2nd — suppose this throws
    Socket           sock;   // never constructed

    Widget() : data(1000), file("x", "r"), sock(...) {}
    ~Widget() { /* NEVER RUNS if the ctor throws */ }
};
```

If `File`'s constructor throws: `sock` was never started, `file` was never completed (so no `~File`), and `data` is destroyed properly. No leak — **provided every member manages its own resource**.

Corollary: a class holding one raw resource plus anything else is unsafe. A class holding only RAII members is automatically safe.

This is the real argument for `unique_ptr` over raw pointer members: it makes partially-constructed objects leak-free for free.

Destructors must not throw ⚠

Destructors are implicitly `noexcept`. If one throws during stack unwinding from another exception, `std::terminate` is called immediately. So:

```

~Transaction() {
    try { if (!committed_) rollback(); }
    catch (...) { log_error(); } // swallow – you have no other option
}

```

If failure-to-release is a real error the caller must know about, give the class an explicit `close()/commit()` that can throw, and make the destructor the silent fallback. `std::ofstream` and database transaction wrappers both do this. Full treatment in §13.

4.3 Scope-based cleanup — why C++ has no `finally`

The argument

Bjarne's answer to "why no `finally`?: because `finally` puts the cleanup burden at **every use site**, while a destructor puts it in **one place, in the type**.

```

// finally: N resources, N nested try blocks, cleanup written at every use
Lock l = acquire();
try {
    Conn c = connect();
    try { work(); }
    finally { c.close(); }
} finally { l.release(); }

```

```

// RAII: N resources, N lines, cleanup written zero times
std::lock_guard lock(mutex);
Connection conn(url);
work();

```

If you find yourself wanting `finally`, you want a **scope guard** (§4.10) or, better, a proper RAII type.

Stack unwinding

When an exception propagates, the runtime destroys every automatic object in every scope it exits, in reverse order, before entering a `catch`. This is *unwinding*, and it is what makes RAII exception-safe rather than merely convenient.

```

void f() {
    std::lock_guard lock(m); // acquired
    auto p = std::make_unique<T>();
    might_throw(); // throws
    // unwinding: ~unique_ptr (frees), then ~lock_guard (unlocks), then propagate
}

```

Note the ordering is right by construction — you cannot forget to unlock after freeing.

Multiple resources, linear cost

```
Config load(const std::string& path) {
    std::ifstream in(path);           // closed automatically
    std::lock_guard lock(cacheMutex); // unlocked automatically
    auto scratch = std::make_unique<Buffer>(); // freed automatically
    Transaction tx(db);               // rolled back unless committed

    auto cfg = parse(in, *scratch);   // may throw anywhere
    tx.commit();
    return cfg;                       // NRVO, no copy (§5)
}
```

Four resources, four lines, zero cleanup code, exception-safe. The C version of this function is 40 lines and has a bug.

4.4 Smart pointers

`std::unique_ptr<T>` — the default

Exclusive ownership. **Move-only**. Zero runtime overhead with a stateless deleter: `sizeof(unique_ptr<T>) == sizeof(T*)`, and the destructor call inlines to a `delete`.

```
#include <memory>

auto p = std::make_unique<Widget>(arg1, arg2);

p->method();
*p;
p.get();           // raw pointer – for passing to observers / C APIs
p.reset();         // destroy now, become null
p.reset(new Widget{}); // destroy old, take new
Widget* raw = p.release(); // give up ownership WITHOUT destroying (you're on the
hook now)
if (p) { ... }     // explicit operator bool

auto q = std::move(p); // transfer. p is now null.
auto r = p;           // ERROR – not copyable. This is a feature.
```

The four things it's for:

```
// 1. Factory returning a polymorphic object
std::unique_ptr<Shape> makeShape(Kind k) {
    switch (k) {
        case Kind::Circle: return std::make_unique<Circle>();
```

```

        case Kind::Square: return std::make_unique<Square>();
    }
}

// 2. Polymorphic ownership inside a container
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>());

// 3. Pimpl (§7) – hides the implementation, keeps the header clean
class Engine {
    struct Impl;
    std::unique_ptr<Impl> impl_;
public:
    Engine();
    ~Engine(); // ⚠ MUST be defined in the .cpp, see below
};

// 4. Optional heap member with clear ownership
std::unique_ptr<Cache> cache_; // null = no cache

```

⚠ **Pimpl destructor trap.** `~unique_ptr<Impl>` needs `Impl` to be complete. If you let the compiler generate `~Engine()` in the header, `Impl` is incomplete there and you get a confusing `static_assert`. Declare `~Engine();` in the header and `Engine::~Engine() = default;` in the .cpp, after `struct Impl` is defined. Same for the move operations.

`std::shared_ptr<T>` — shared ownership

Reference-counted. The object dies when the last `shared_ptr` dies.

```

auto a = std::make_shared<Widget>();
auto b = a; // refcount 2 – atomic increment
a.reset(); // refcount 1
b.use_count(); // 1 (⚠ debugging only; racy by nature)

```

`sizeof(shared_ptr<T>)` is **two pointers**: the object pointer, and a pointer to the **control block**, which holds:

- strong refcount (atomic)
- weak refcount (atomic)
- the deleter (type-erased)
- the allocator
- possibly the object itself (if `make_shared`)

Two specialist features worth knowing:

```

// Aliasing constructor – share ownership of A, but point at a subobject
struct Outer { Inner inner; };
auto outer = std::make_shared<Outer>();

```



```

std::shared_ptr<Inner> inner(outer, &outer->inner);
// inner keeps the whole Outer alive; inner.get() is &outer->inner

// enable_shared_from_this – get a shared_ptr to yourself, from inside yourself
class Session : public std::enable_shared_from_this<Session> {
    void start() {
        asio::async_read(sock_, buf_,
            [self = shared_from_this()](auto ec, auto n) { self->onRead(ec, n);
    });
    // ^ keeps the Session alive for the duration of the async op – the
    // canonical fix for "my object died while a callback was in flight"
}
};

```

⚠ `shared_from_this()` throws `std::bad_weak_ptr` if the object is not currently owned by a `shared_ptr` (e.g. it's on the stack, or you're calling it from the constructor).

`std::weak_ptr<T>` — non-owning observer

Does not affect the refcount. Can tell you whether the object is still alive, and give you a temporary `shared_ptr` if so.

```

std::weak_ptr<Widget> w = sharedWidget;

if (auto s = w.lock()) {    // atomic: returns shared_ptr, or null if expired
    s->use();                // safe – s keeps it alive for this block
}
w.expired();                // ⚠ racy in multithreaded code; prefer lock()

```

Two uses:

1. Breaking cycles.

```

struct Node {
    std::vector<std::shared_ptr<Node>> children; // owns downward
    std::weak_ptr<Node> parent;                // observes upward – MUST be
weak
};

```

With `shared_ptr parent`, parent and child keep each other alive forever. Reference counting cannot collect cycles; this is its fundamental limitation versus tracing GC.

2. Caches / observer lists where entries should not keep the observed object alive:

```

std::vector<std::weak_ptr<Listener>> listeners_;

void notify() {

```

```
std::erase_if(listeners_, [](auto& w) { return w.expired(); }); // prune
for (auto& w : listeners_)
    if (auto l = w.lock()) l->onEvent();
}
```

Choosing

Does something own this?

```
├ no → raw T* / T& / span / string_view (observe only, §3.6)
└ yes → is ownership shared?
    ├── no → unique_ptr<T> (or just a value member! prefer that)
    └ yes → is the lifetime genuinely unclear/concurrent?
        ├── no → restructure: one owner + observers
        └ yes → shared_ptr<T>, + weak_ptr for back-edges
```

Default to a plain value member. If it must be heap-allocated or polymorphic, `unique_ptr`. Reach for `shared_ptr` only when you can articulate why the lifetime is genuinely shared. `shared_ptr` is not "the easy one"; it's the one that costs.

4.5 `make_unique` / `make_shared` — and why

Reason 1: exception safety

```
void f(std::unique_ptr<A> a, int b);
int compute(); // may throw

f(std::unique_ptr<A>(new A), compute());
```

Before C++17, the compiler could interleave the argument evaluations: `new A`, then `compute()` (throws), then the `unique_ptr` constructor — which never runs. **The `A` leaks.** C++17 requires each argument to be fully evaluated before the next starts, which fixes this specific case, but the general principle holds and the habit is worth keeping:

```
f(std::make_unique<A>(), compute()); // no window where a raw pointer is unowned
```

Reason 2: no `new`, no repeated type

```
std::unique_ptr<VeryLongTypeName> p(new VeryLongTypeName(a, b)); // said twice
auto p = std::make_unique<VeryLongTypeName>(a, b); // said once
```

The broader rule: **new and delete should not appear in application code at all**. If your codebase greps clean for them outside of allocator/container internals, an entire class of bugs is gone.

Reason 3 (**make_shared** only): one allocation instead of two

```
std::shared_ptr<Widget> a(new Widget);           // 2 allocations: Widget, control
block
auto b = std::make_shared<Widget>();           // 1 allocation: both, contiguous
```

Faster to create, better locality on every dereference.

⚠ When *not* to use **make_shared**

Memory retention. **make_shared** puts the object *inside* the control block, so the memory can't be released until the **weak** count also hits zero:

```
auto big = std::make_shared<HugeObject>();      // 100 MB
std::weak_ptr<HugeObject> w = big;
big.reset();
// object is DESTROYED, but the 100 MB block is still held because w exists
```

With **shared_ptr<HugeObject>(new HugeObject)**, the object's memory is freed immediately and only the small control block lingers. Use the two-allocation form for large objects with long-lived **weak_ptrs**.

Custom deleters. **make_shared/make_unique** can't take one. Use the constructor form.

Private/protected constructors. **make_shared** isn't a friend. Workarounds: a passkey type, or an inheriting helper struct.

C++20 additions

```
auto p = std::make_unique_for_overwrite<int[]>(1'000'000);
// default-initializes instead of value-initializing – skips zeroing a million
ints
```

4.6 Custom deleters and **unique_ptr<T[]>**

Wrapping C APIs

This is the single most valuable everyday application of RAI: making someone else's **create/destroy** pair automatic.

```
// Stateless functor deleter – PREFERRED. sizeof stays one pointer (EBO, §7)
struct FileDeleter {
```

```
void operator()(std::FILE* f) const noexcept { if (f) std::fclose(f); }
};
using FilePtr = std::unique_ptr<std::FILE, FileDeleter>;

FilePtr f(std::fopen("data.txt", "r"));
static_assert(sizeof(FilePtr) == sizeof(std::FILE*)); // zero overhead
```

```
// Function-pointer deleter – works, but costs a word
using FilePtr2 = std::unique_ptr<std::FILE, decltype(&std::fclose)>;
FilePtr2 g(std::fopen("x", "r"), &std::fclose);
static_assert(sizeof(FilePtr2) == 2 * sizeof(void*)); // ⚠ stores the pointer
```

Real-world examples:

```
struct SqliteDeleter { void operator()(sqlite3* d) const { sqlite3_close(d); } };
using DbPtr = std::unique_ptr<sqlite3, SqliteDeleter>;

struct FreeDeleter { void operator()(void* p) const { std::free(p); } };
using CStr = std::unique_ptr<char, FreeDeleter>; // for strdup, asprintf,
etc.

struct XDeleter { void operator()(X509* x) const { X509_free(x); } };
using CertPtr = std::unique_ptr<X509, XDeleter>;
```

`shared_ptr` deleters are type-erased

```
std::shared_ptr<std::FILE> f(std::fopen("x", "r"), [](std::FILE* p){
std::fclose(p); });
// The deleter lives in the control block, NOT in the type.
// shared_ptr<FILE> and shared_ptr<FILE> with different deleters are the same
type.
```

That's convenient for heterogeneous storage, and it's part of why `shared_ptr` is fatter. It also allows the "null deleter" trick for wrapping objects you don't own — which is usually a sign you should be using a raw pointer instead.

`unique_ptr<T[]>`

```
auto arr = std::make_unique<int[]>(100); // value-initialized to 0
arr[0] = 1; // operator[] is available
// no operator*, no operator->
```

It calls `delete[]`, which matters — mismatching `new[]/delete` is UB.

⚠️ **You almost never want this.** `std::vector<int>` gives you the same thing plus size, growth, iterators, and bounds-checked `at()`, at the same cost. `unique_ptr<T[]>` is justified when you need a fixed-size buffer with no capacity/size overhead and no growth, or when interfacing with a C API that hands you an array.

4.7 ⚠️ The cost of `shared_ptr`

Cost	Detail
Size	2 pointers instead of 1 — doubles the size of anything holding many of them
Allocation	control block (avoidable with <code>make_shared</code>)
Atomic refcount	every copy/destroy is a <code>lock</code> -prefixed RMW; under contention the cache line ping-pongs between cores
Indirection	dereferencing touches two cache lines (object + control block on destruction)
Type-erased destruction	the deleter call is indirect and usually not inlinable
Cycles	reference counting cannot collect them — you leak silently
Retention	with <code>make_shared</code> , memory is held until the last <code>weak_ptr</code> dies

The atomic cost is not theoretical

```
void process(std::shared_ptr<Widget> w);    // ⚠️ by value: refcount++ / refcount-
- per call
void process(const std::shared_ptr<Widget>& w); // no refcount traffic
void process(const Widget& w);                // BEST — no ownership
involvement at all
```

In a tight loop across threads, passing `shared_ptr` by value has been measured as an order of magnitude slower than passing a reference. In HFT and game code this is a firing offense (§26).

Rule: a function should take a `shared_ptr` by value only if it stores a copy. Otherwise take `const T&` or `T*`. The function's signature should express what it does with ownership, and most functions do nothing with ownership.

Thread safety — three separate things ⚠️

1. **The control block is thread-safe.** Copying/destroying `shared_ptr`s to the same object from multiple threads is fine.
2. **The pointed-to object is not.** `shared_ptr` gives you no protection on `T`'s data; you still need a mutex.
3. **A single `shared_ptr` instance is not.** Two threads reassigning the *same* `shared_ptr` variable is a data race. For that you need `std::atomic<std::shared_ptr<T>>` (C++20) or the deprecated free-function `atomic_load/atomic_store` overloads.

```
std::atomic<std::shared_ptr<Config>> current;    // C++20
auto cfg = current.load();                      // safe concurrent read
current.store(std::make_shared<Config>(newCfg)); // safe concurrent swap
```

4.8 Ownership as a design decision

Express it in the type

The type of a member or parameter should answer "who frees this?" without a comment:

Type	Ownership statement
T member;	I own it, it lives inside me
std::unique_ptr<T>	I exclusively own it, on the heap
std::vector<T> / std::vector<std::unique_ptr<T>>	I own all of them
std::shared_ptr<T>	I share ownership
std::weak_ptr<T>	I observe; I may outlive it; I check before use
T*	I observe; someone else guarantees it outlives me; may be null
T&	same, but never null and never rebound
std::span<T> / std::string_view	I observe a range; caller guarantees lifetime
gsl::owner<T*>	ownership at a C boundary (marker only)

A codebase where these are used consistently needs very few lifetime comments, and clang-tidy can check most of it mechanically.

Shape the ownership graph deliberately

- **Tree** (the common case) — parent owns children via `unique_ptr` or by value; children point back with a raw pointer or `weak_ptr`. Destroy the root, everything goes.
- **DAG with genuine sharing** — `shared_ptr` for the shared nodes only.
- **Any cycle** — one direction must be `weak_ptr` or a raw pointer. Decide which direction *owns*.
- **Everything-lives-as-long-as-the-system** — an **arena/pool** owns everything; users hold indices or handles.

Handles and indices — the games/HPC answer

```
struct EntityId { std::uint32_t index; std::uint32_t generation; };

class World {
    std::vector<Entity>    entities_;    // contiguous, owned here
```

```

    std::vector<std::uint32_t> generations_;
public:
    Entity* get(EntityId id) {
        if (id.index >= entities_.size())          return nullptr;
        if (generations_[id.index] != id.generation) return nullptr; // stale
handle
        return &entities_[id.index];
    }
};

```

Why this wins over `shared_ptr` in performance code:

- 8 bytes, trivially copyable, no atomics
- **Detects stale references** instead of silently keeping dead objects alive
- Storage stays contiguous — cache-friendly (§24)
- Serializable; a pointer is not

This is the standard pattern in ECS architectures, and worth knowing even if you never write a game.

4.9 The Rule of Zero / Three / Five

The five special member functions

```

class T {
    ~T();                // destructor
    T(const T&);          // copy constructor
    T& operator=(const T&); // copy assignment
    T(T&&) noexcept;      // move constructor
    T& operator=(T&&) noexcept; // move assignment
};

```

(Plus the default constructor, which follows its own rules — declaring *any* constructor suppresses it.)

Rule of Three (C++98)

If you need one of {destructor, copy constructor, copy assignment}, you almost certainly need all three — because needing a destructor means you're managing a resource, and the compiler-generated copies will shallow-copy it into a double-free.

Rule of Five (C++11)

Add the move constructor and move assignment. Same reasoning; if you're managing a resource by hand, you should also say how to *transfer* it, or you silently get copies where moves were possible.

Rule of Zero (the one to actually follow)

Declare none of the five. Build your class out of members that manage themselves.

```
class Config {                                // Rule of Zero
    std::string      name_;
    std::vector<int>  values_;
    std::unique_ptr<Parser> parser_;
};
// copyable? no (unique_ptr member) – correct, and enforced automatically
// movable? yes, memberwise – correct
// destructor? correct, automatic
// exception-safe during construction? yes (§4.2)
```

Every hand-written special member is a place to make a mistake. The best number is zero. Write a resource-managing class **only** when you're actually wrapping a raw resource (a C handle), and then that class should manage exactly **one** thing and nothing else.

⚠ The suppression table — the trap that costs real performance

Declaring one special member suppresses others:

You declare	Default ctor	Copy ctor	Copy assign	Move ctor	Move assign	Destructor
nothing	✓	✓	✓	✓	✓	✓
any constructor	✗	✓	✓	✓	✓	✓
destructor	✓	✓ (deprecated)	✓ (deprecated)	✗	✗	—
copy ctor	✗	—	✓ (deprecated)	✗	✗	✓
copy assign	✓	✓ (deprecated)	—	✗	✗	✓
move ctor	✗	deleted	deleted	—	✗	✓
move assign	✓	deleted	deleted	✗	—	✓

The one that bites in production:

```
class Logger {
    std::vector<std::string> lines_;
public:
    ~Logger() { flush(); }           // "harmless" destructor...
};

std::vector<Logger> v;
v.push_back(std::move(logger));    // ...silently COPIES the whole vector<string>
```


The destructor suppressed the move operations, so every "move" became a deep copy. No warning, no error, just a performance cliff. `-Wdeprecated-copy` catches part of this family; the real fix is either the Rule of Zero or:

```
class Logger {
public:
    ~Logger() { flush(); }
    Logger(Logger&&) noexcept = default;
    Logger& operator=(Logger&&) noexcept = default;
    Logger(const Logger&) = default;
    Logger& operator=(const Logger&) = default;
};
```

If you declare one of the five, declare (or = delete) all five. Explicitly, together, next to each other. Reviewers can then check them as a set.

The virtual destructor interaction ⚠

```
class Base {
public:
    virtual ~Base() = default;    // required for polymorphic delete (§8)
    // ...which just suppressed Base's move operations!
};
```

Two accepted answers:

```
// A: declare all five
class Base {
public:
    virtual ~Base() = default;
    Base(const Base&) = default;
    Base& operator=(const Base&) = default;
    Base(Base&&) noexcept = default;
    Base& operator=(Base&&) noexcept = default;
protected:
    Base() = default;
};

// B: protected non-virtual destructor – for interfaces never deleted through Base*
class Interface {
public:
    virtual void doIt() = 0;
protected:
    ~Interface() = default;    // non-virtual, protected: can't delete via Interface*
};
```

Also note: base classes are usually a **slicing** hazard when copyable (§8), so many codebases delete copy/move on polymorphic bases entirely.

= default and **= delete**

```
struct S {
    S() = default; // "I want the implicit one,
explicitly"
    S(const S&) = delete; // "copying this is a compile error"
    S& operator=(const S&) = delete;
    S(S&&) noexcept = default;
    S& operator=(S&&) noexcept = default;
};
```

- **= delete** produces a clean error at the call site, unlike the old private-and-undeclared trick.
- **= delete** works on any function, not just special members — useful for banning implicit conversions:

```
void process(int);
void process(double) = delete; // process(3.14) is now an error, not a silent
truncation
```

- **⚠ = default in the class body** requires member types to be complete. For pimpl, put **= default** in the .cpp (§4.4).

4.10 Scope guards (**scope_exit** / **gsl::finally**)

When RAII types aren't worth it

Sometimes cleanup is a one-off and writing a whole class is silly:

```
void draw() {
    glPushMatrix();
    auto guard = gsl::finally([&]{ glPopMatrix(); }); // runs on every exit path

    if (culled()) return;
    renderStuff(); // may throw
}
```

Also useful for **rollback on failure**:

```
void transfer(Account& from, Account& to, Money m) {
    from.debit(m);
    auto rollback = gsl::finally([&]{ from.credit(m); }); // undo if we don't
```

```
get there

    to.credit(m);           // may throw
    rollback.release();     // success – cancel the undo (name varies by library)
}
```

The three flavours

Guard	Runs cleanup
<code>scope_exit</code>	always
<code>scope_fail</code>	only if leaving via an exception
<code>scope_success</code>	only if leaving normally

`scope_fail` is the precise tool for the rollback pattern above. It's implemented by comparing `std::uncaught_exceptions()` at construction and destruction.

What's available

Library	Name
GSL (Microsoft)	<code>gsl::finally</code>
Abseil	<code>absl::Cleanup</code> (with <code>std::move(c).Cancel()</code>)
folly	<code>folly::makeGuard</code> , <code>SCOPE_EXIT</code> / <code>SCOPE_FAIL</code> / <code>SCOPE_SUCCESS</code> macros
Boost	<code>BOOST_SCOPE_EXIT</code> , and <code>Boost.Scope</code> (<code>scope_exit</code> , <code>scope_fail</code> , <code>defer</code>)
Standard	<code>std::experimental::scope_exit</code> etc. — Library Fundamentals TS v3, targeted at C++26 , not yet in <code><scope></code> on most toolchains

Roll your own — it's 15 lines

```
template <class F>
class ScopeExit {
    F f_;
    bool active_ = true;
public:
    explicit ScopeExit(F f) noexcept : f_(std::move(f)) {}
    ~ScopeExit() noexcept { if (active_) f_(); }           // ⚠ must be noexcept
    void release() noexcept { active_ = false; }

    ScopeExit(ScopeExit&& o) noexcept
        : f_(std::move(o.f_)), active_(std::exchange(o.active_, false)) {}
    ScopeExit(const ScopeExit&) = delete;
    ScopeExit& operator=(const ScopeExit&) = delete;
};
// CTAD (§10) means you write: ScopeExit g{[] { ... } };
```

With a macro for anonymous guards:

```
#define CONCAT_(a, b) a##b
#define CONCAT(a, b)  CONCAT_(a, b)
#define SCOPE_EXIT(...) ScopeExit CONCAT(scopeExit_, __LINE__){ [&]{ __VA_ARGS__; }}

void f() {
    FILE* fp = fopen("x", "r");
    SCOPE_EXIT(if (fp) fclose(fp));
    ...
}
```

⚠ Cautions

- **The lambda must not throw.** The destructor is `noexcept`; an escaping exception calls `std::terminate`.
- **Watch the captures.** `[&]` capturing a local that's already destroyed is the standard dangling-capture bug (§6). For guards this is usually safe since the guard dies first — but be careful with guards that outlive their scope.
- **If you write the same guard three times, write a class.** Scope guards are for one-offs; a repeated pattern deserves a named RAII type with a tested interface.

Gotcha checklist for this section

- ☐ Declaring a **destructor silently kills your move operations** → deep copies everywhere
- ☐ `virtual ~Base() = default;` triggers exactly that — declare all five, or use a protected non-virtual dtor
- ☐ A constructor that throws means **no destructor runs** — so every raw resource must already be in an RAII member
- ☐ **Destructors must not throw**; provide an explicit `close()/commit()` for reportable failures
- ☐ `unique_ptr<Impl>` for pimpl requires the destructor **defined in the .cpp**
- ☐ `shared_ptr` cycles leak — every back-edge must be `weak_ptr` or raw
- ☐ Passing `shared_ptr` **by value** when you don't store it costs two atomic RMWs per call
- ☐ The control block is thread-safe; the **object and the `shared_ptr` variable are not**
- ☐ `make_shared` + long-lived `weak_ptr` keeps the object's memory alive after destruction
- ☐ `make_shared` can't take a custom deleter or call a private constructor
- ☐ Mismatching `new[]` with `delete` is UB — `unique_ptr<T[]>` exists for this, but prefer `vector`
- ☐ `unique_ptr` with a **function pointer** deleter is twice the size of one with a functor deleter
- ☐ `shared_from_this()` throws if the object isn't already owned by a `shared_ptr`
- ☐ A scope-guard lambda that throws calls `std::terminate`

Flags and tools

```
-Wdeprecated-copy           # implicit copy where you declared a dtor/copy op
-Wnon-virtual-dtor         # polymorphic base without a virtual destructor
-Weffc++                   # GCC; noisy, but catches Rule-of-Three omissions
-fsanitize=address         # leaks, double-free, use-after-free
-fsanitize=leak            # LeakSanitizer standalone
valgrind --leak-check=full
```

clang-tidy checks:

```
cppcoreguidelines-owning-memory
cppcoreguidelines-special-member-functions
modernize-make-unique / modernize-make-shared
modernize-avoid-c-arrays
performance-unnecessary-value-param # catches shared_ptr-by-value
bugprone-use-after-move
```

Exercises worth doing

1. Take a C API you use (sqlite3, libcurl, OpenSSL, a Vulkan handle) and write the full RAI wrapper: move-only, stateless deleter, `sizeof` equal to the raw handle. Verify the `static_assert`.
2. Write a class with a destructor and a `vector<string>` member. Put it in a `vector`, `reserve` then `push_back` a hundred of them, and time it. Now add `= default` move operations. Measure the difference.
3. Build a parent/child tree with `shared_ptr` in both directions. Run under LeakSanitizer and confirm the leak. Change the parent link to `weak_ptr` and confirm it's gone.
4. Implement `ScopeExit` yourself, then extend it to `ScopeFail` using `std::uncaught_exceptions()`.
5. Write a class where a member constructor throws, with `printf` in every constructor and destructor. Trace exactly which destructors run.
6. Compile `void f(std::shared_ptr<T>)` vs `void f(const std::shared_ptr<T>&)` on Godbolt at -O2. Find the `lock xadd` instructions.

Chapter 5 — Move Semantics ⚠

What this chapter covers

- Copy vs move; why moves exist
- `std::move` is a **cast, not a move**
- `std::forward` and **perfect forwarding**
- **Forwarding (universal) references** `T&&` vs rvalue references
- Move constructor / move assignment; `noexcept` and why it matters for `vector`
- **Moved-from state**: valid but unspecified
- **Copy elision, RVO, NRVO**, guaranteed elision (C++17)
- When moves don't happen (`const`, returning a member, `return std::move(x)` pessimization)
- `std::exchange`, swap idiom

The C++11 feature that made value semantics affordable. Before moves, "return by value" meant "copy an expensive object", so codebases were full of output parameters, `swap` tricks, and reference-counted

string hacks. After moves, returning by value is the *fast* option. This section is also where the largest silent performance bugs in modern C++ live.

5.1 Copy vs move — why moves exist

The problem

```
std::vector<std::string> load();           // returns 10 MB of data

auto data = load();                       // pre-C++11: potentially a full deep copy
```

A copy of `vector<string>` allocates a new buffer, then allocates and copies every string. $O(n)$ work and $O(n)$ allocations, to produce a duplicate that's discarded a nanosecond later.

The pre-C++11 workarounds were all bad:

```
void load(std::vector<std::string>& out); // output params – ugly, no const, no
chaining
v1.swap(v2);                             // swap tricks everywhere
// Copy-on-write std::string – which C++11 BANNED because it can't be made
// thread-safe without atomics on every access
std::auto_ptr<T>                          // "copy that secretly steals" –
removed in C++17
```

`auto_ptr` is the instructive failure: it wanted move semantics but only had copy syntax, so a copy silently nulled the source. That's exactly what moves do — but with a *different* syntax so it can't happen by accident.

The mechanism

A move transfers ownership of the internals instead of duplicating them.

```
COPY  vector<string> b = a;

      [a.ptr] → [data] | [b.ptr] → [COPY of data]   allocate + copy N elements

MOVE  vector<string> b = std::move(a);

      [a.ptr] = null | [b.ptr] → [data]             3 pointer writes. O(1).
```

A `vector` move is: copy three pointers out, null three pointers in the source. It does not touch the elements at all. $O(n)$ becomes $O(1)$.

When moves help — and when they don't

Type	Move is
<code>vector</code> , <code>string</code> , <code>map</code> , <code>unique_ptr</code> , <code>thread</code> , <code>fstream</code>	much faster — steals a heap buffer or handle
<code>int</code> , <code>double</code> , <code>Point{int,int}</code> , any trivially copyable type	identical to a copy — there's nothing to steal
<code>std::array<int, 1000></code>	identical to a copy — the data is inline, it must be copied
A struct of one <code>string</code> + three <code>ints</code>	faster, but only by the string's worth

⚠ Moving does not make copying cheap in general. **It makes *ownership transfer* cheap.** If a type's data lives inside the object, move and copy cost the same.

Moves enable move-only types

The second, arguably more important, consequence: types that *cannot* be copied but *can* be transferred.

```
std::unique_ptr<T>    // exclusive ownership – copying would break the invariant
std::thread          // one thread handle, one owner
std::jthread
std::future<T>
std::fstream
std::unique_lock
```

Before C++11 these types were impossible to express safely. That's why `unique_ptr` couldn't exist and we had `auto_ptr` instead.

5.2 `std::move` is a cast, not a move ⚠

What it actually is

```
template <class T>
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

That's the whole implementation. `std::move` **generates zero instructions**. It does not move anything, does not modify its argument, does not null anything out.

What it does: change the expression's **value category** from lvalue to xvalue, so that overload resolution picks a `T&&` overload — which is usually a move constructor or move assignment operator. *That* function does the actual work.

```
std::string a = "hello";
std::move(a);           // NOTHING HAPPENS. a is untouched. Statement is
                        // a no-op.
std::string b = std::move(a); // NOW something happens: move ctor selected, a
                        // is gutted
```

It should have been called `rvalue_cast`. Read every `std::move(x)` as "**I promise I'm done with x**", not as "move x now".

The `const` trap ⚠

```
const std::string s = "expensive";
std::string t = std::move(s); // COPIES. Silently.
```

`std::move(s)` yields `const std::string&&`. The move constructor takes `std::string&&` — no match. The copy constructor takes `const std::string&` — `const&` binds to rvalues, so it wins. You get a copy with no warning, no error, and no indication that your `std::move` did nothing.

Consequences you'll hit in real code:

```
const auto data = compute();
sink(std::move(data)); // copies

struct S { const std::string name; }; // const MEMBER
std::vector<S> v;
v.push_back(std::move(s)); // the `name` member is copied, always
```

Don't make members `const` if the type should be movable. Use private + a getter, or just accept mutability.

Moving a trivially copyable type is a no-op

```
int x = 5;
int y = std::move(x); // just a copy. x is still 5.
std::array<int,4> a{}, b = std::move(a); // element-wise copy, a unchanged
```

No harm, but also no benefit — and writing `std::move` here misleads readers into thinking `x` is now dead.

5.3 Forwarding (universal) references vs rvalue references ⚠

These look identical and behave completely differently.


```
void f(std::string&& s);           // RVALUE reference – binds rvalues only

template <class T>
void g(T&& t);                     // FORWARDING reference – binds ANYTHING

auto&& r = anything;              // FORWARDING reference
```

The rule

`T&&` is a **forwarding reference** if and only if `T` is a template parameter being **deduced at that exact spot**. Otherwise it's an rvalue reference.

Not forwarding references, despite appearances:

```
template <class T> void a(std::vector<T>&& v); // rvalue ref – T is deduced, but
not "T&&"
template <class T> void b(const T&& t);       // rvalue ref – const kills it
template <class T> void c(T&& t) requires ...; // still forwarding (constraints
don't matter)

template <class T>
class Vec {
    void push(T&& x); // RVALUE ref! T is fixed by the class, not deduced
    here
    template <class U>
    void emplace(U&& x); // forwarding – U is deduced here
};

auto&& r = x; // forwarding
const auto&& r2 = x; // rvalue ref
```

How the deduction works

For `template <class T> void g(T&& t):`

Argument	T deduced as	Parameter type after collapsing
int lvalue	int&	int& && → int&
const int lvalue	const int&	const int&
int rvalue	int	int&&
const int rvalue	const int	const int&&

Reference collapsing (§3.5 — "lvalue reference wins") is what makes one signature cover all four cases. That's the whole trick.

⚠ Inside `g`, the expression `t` is an **lvalue** regardless (§3.2). The type remembers the category; the expression does not.

5.4 `std::forward` and perfect forwarding

The problem it solves

A wrapper function must pass its argument along **without changing its value category or constness**:

```
template <class T>
void wrapper(T&& arg) {
    target(arg);           // ⚠ ALWAYS an lvalue – target's move overload
    never fires
}
```

Even if the caller passed a temporary, `arg` inside the function is a named lvalue, so `target` copies. Perfect forwarding restores the original category:

```
template <class T>
void wrapper(T&& arg) {
    target(std::forward<T>(arg)); // lvalue in -> lvalue out; rvalue in ->
    rvalue out
}
```

What it is

```
template <class T>
constexpr T&& forward(std::remove_reference_t<T>& t) noexcept {
    return static_cast<T&&>(t);
}
```

A **conditional** cast, driven by the deduced `T`:

- `T = int& → static_cast<int& &&> → static_cast<int&> →` stays an lvalue
- `T = int → static_cast<int&&> →` becomes an xvalue

The usage rules

`std::move` on rvalue references. `std::forward<T>` on forwarding references. Never the other way around.

`std::move` never takes an explicit template argument. `std::forward` **always** does.

```
void a(std::string&& s) { sink(std::move(s)); } // ✓
template <class T>
void b(T&& t)          { sink(std::forward<T>(t)); } // ✓
```

```
void c(std::string&& s) { sink(std::forward<std::string>(s)); } // works, but
obfuscated
template <class T>
void d(T&& t)           { sink(std::move(t));           } // ⚠️ BUG – steals
from lvalues!
```

`d` is a genuine disaster: pass it a named variable and it gets silently gutted.

Canonical uses

```
// make_unique, essentially
template <class T, class... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// emplace_back, essentially
template <class... Args>
T& emplace_back(Args&&... args) {
    ::new (end_) T(std::forward<Args>(args)...); // construct in place
    return *end_++;
}

// A generic timing/logging decorator
template <class F, class... Args>
decltype(auto) timed(F&& f, Args&&... args) {
    Timer t;
    return std::invoke(std::forward<F>(f), std::forward<Args>(args)...);
}
```

Note `decltype(auto)` on the return: it preserves whether `f` returned a value or a reference (§6).

⚠️ Forward exactly once

```
template <class T>
void bad(T&& t) {
    use(std::forward<T>(t));
    use(std::forward<T>(t)); // BUG – the first call may have gutted it
}

template <class T>
void alsoBad(T&& t) {
    for (int i = 0; i < 10; ++i)
        consume(std::forward<T>(t)); // BUG – moved on every iteration
}
```

Forwarding is a one-shot promise. If you need the value twice, forward on the **last** use only.

⚠ The perfect-forwarding constructor problem

A forwarding-reference constructor is a greedy overload that hijacks calls you meant for the copy constructor:

```
class Person {
    std::string name_;
public:
    template <class T>
    explicit Person(T&& name) : name_(std::forward<T>(name)) {}
    Person(const Person& p);      // copy ctor
};

Person a{"Alice"};
Person b{a};                    // ⚠ calls the TEMPLATE (exact match on Person&),
                                // not the copy ctor (which needs a const conversion)
                                // -> tries to construct a string from a Person -> compile
                                error
```

The template is a better match for non-const lvalues. Constrain it:

```
// C++20
template <class T>
    requires (!std::same_as<std::remove_cvref_t<T>, Person>)
    && std::constructible_from<std::string, T>
explicit Person(T&& name) : name_(std::forward<T>(name)) {}

// Pre-C++20
template <class T,
    class = std::enable_if_t<!std::is_same_v<std::decay_t<T>, Person>>>
explicit Person(T&& name);
```

The same problem hits derived classes (the template beats the base's copy ctor). Honestly, for a two-overload case, just write `const std::string&` and `std::string&&` and skip the template. Perfect forwarding earns its complexity in variadic factories, not in ordinary constructors.

5.5 Move constructor / assignment, and `noexcept`

Writing them

```
class Buffer {
    char*      data_ = nullptr;
    std::size_t size_ = 0;
public:
    Buffer(Buffer&& o) noexcept
        : data_(std::exchange(o.data_, nullptr)),      // take, and null the
          source
          size_(std::exchange(o.size_, 0)) {}
```

```

    Buffer& operator=(Buffer&& o) noexcept {
        if (this != &o) {                                // self-move guard
            delete[] data_;                               // release what we hold
            data_ = std::exchange(o.data_, nullptr);
            size_ = std::exchange(o.size_, 0);
        }
        return *this;
    }

    ~Buffer() { delete[] data_; }
    Buffer(const Buffer&);
    Buffer& operator=(const Buffer&);
};

```

Three obligations for move assignment: **release your current resource, take theirs, leave the source destructible**. And handle self-move — `v[i] = std::move(v[j])` with `i == j` happens inside `std::sort` and friends.

Of course, the Rule of Zero version is better when it applies (§4.9):

```

class Buffer {
    std::vector<char> data_;    // all five special members correct, for free
};

```

`noexcept` and why `vector` cares ⚠

This is the highest-value detail in the section.

`std::vector::push_back` offers the **strong exception guarantee**: if it throws, the vector is unchanged. During reallocation it must move *N* elements to a new buffer. If element move #5 throws, the old buffer is already half-gutted — the vector can't be restored. So:

If the element's move constructor is not `noexcept`, `vector` copies instead of moving during reallocation.

The mechanism is `std::move_if_noexcept`, which returns an rvalue only when the move constructor is `noexcept` (or when the type isn't copyable at all — then there's no choice).

```

struct Slow {
    std::vector<std::string> data;
    Slow(Slow&&) { /* no noexcept */ }
};

struct Fast {
    std::vector<std::string> data;
    Fast(Fast&&) noexcept = default;
};

```

```
std::vector<Slow> a; // every reallocation DEEP COPIES everything
std::vector<Fast> b; // every reallocation moves – often 100x faster
```

No warning. No error. Just a silent order-of-magnitude difference in a hot path.

The habit

```
// 1. Always mark move operations noexcept
MyType(MyType&&) noexcept = default;
MyType& operator=(MyType&&) noexcept = default;

// 2. Assert it, next to the class
static_assert(std::is_nothrow_move_constructible_v<MyType>);
static_assert(std::is_nothrow_move_assignable_v<MyType>);
```

If your move *can't* be `noexcept`, that's a design smell: a move should be pointer shuffling, and pointer shuffling doesn't throw. The usual culprit is a member that allocates on move (e.g. `std::list` with a non-propagating allocator, or a member whose move is itself not `noexcept`).

⚠️ = `default` gives you `noexcept` **only if every member's move is `noexcept`**. One bad member poisons the whole class, which is why the `static_assert` is worth writing.

`noexcept` on moves also affects `std::swap`, `std::move_if_noexcept`, `vector::resize`, and several algorithms. Full `noexcept` treatment in §6/§13.

5.6 Moved-from state: valid but unspecified ⚠️

The contract

After `T b = std::move(a);`, the standard says `a` is in a **valid but unspecified state**. Concretely:

You may: destroy it, assign a new value to it, and call any member function with **no preconditions** (`clear()`, `size()`, `empty()`, `operator=`).

You may not: assume what's in it. `a.front()` has a precondition (non-empty), so it's UB.

```
std::string a = "hello";
std::string b = std::move(a);

a.size(); // legal – returns SOMETHING. Don't rely on the value.
a.clear(); // legal
a = "new"; // legal – the standard, safe thing to do
std::cout << a; // legal, prints something unspecified
a[0]; // ⚠️ UB if a is empty
```

Standard-library guarantees

Some types *do* specify the moved-from state:

Type	After move
<code>unique_ptr</code>	guaranteed null
<code>shared_ptr</code> / <code>weak_ptr</code>	guaranteed empty
<code>unique_lock</code> / <code>thread</code> / <code>jthread</code>	guaranteed to own nothing / not joinable
<code>optional</code>	still engaged (<code>△</code> contains a moved-from value — <code>has_value()</code> is still true!)
<code>string</code> , <code>vector</code> , <code>map</code> , ...	unspecified (in practice empty on libstdc++/libc++/MSVC — do not rely on it)

The `optional` row surprises people: `std::move(opt)` moves the *contained value*, it does not disengage the optional. Use `opt.reset()` if you want it empty.

The practical rule

After you move from something, the only things you should do to it are assign to it or let it die.

Static analysis catches the rest:

```
clang-tidy: bugprone-use-after-move
```

Self-move is the sharp edge:

```
v = std::move(v);           // legal, leaves v valid-but-unspecified. Probably not what
                             // you meant.
std::swap(v[i], v[j]); // fine — std::swap handles i == j
```

Designing your own moved-from state

Aim for the state that is **cheap to produce, cheap to destroy, and safe to assign over**. For a pointer-holding class: null. For a class with an invariant like "always non-empty", you have two choices — weaken the invariant, or make the type non-movable. Don't try to keep a strong invariant in a moved-from object; that usually means allocating, which means the move isn't `noexcept`, which triggers §5.5.

5.7 Copy elision, RVO, NRVO

RVO — guaranteed since C++17

Returning a **prvalue** performs no copy and no move. This is not an optimization; C++17 made it a language rule (§3.2: a prvalue isn't an object until it's materialized, and here it's materialized directly into the caller's storage).

```
std::vector<int> make() {
    return std::vector<int>(1000);    // constructed DIRECTLY in the caller's
    variable
}
auto v = make();                    // zero copies, zero moves. Guaranteed.
```

Proof that it's a language rule and not an optimization:

```
struct Immovable {
    Immovable() = default;
    Immovable(const Immovable&) = delete;
    Immovable(Immovable&&)      = delete;
};
Immovable make() { return Immovable{}; } // OK in C++17 – nothing to elide
Immovable x = make();                  // OK
```

NRVO — allowed, not guaranteed

Returning a **named local** is a different case:

```
std::vector<int> make() {
    std::vector<int> result;
    result.reserve(1000);
    fill(result);
    return result;    // NRVO: constructed in place, if the compiler
manages it
}
```

All major compilers do this at **-O1+** in straightforward cases. It is *not* guaranteed, but there's a safety net: **if NRVO doesn't apply, the return is treated as an rvalue** and the move constructor is used (this "implicit move" rule has been in place since C++11, and C++20/C++23 widened it further). So the fallback is a move, not a copy.

What blocks NRVO

```
// 1. Returning different objects on different paths
T f(bool b) { T a, c; return b ? a : c; }    // no NRVO (still implicitly moved)

// 2. Returning a parameter
T g(T param) { return param; }              // no NRVO ever (implicitly moved –
fine)

// 3. Returning a member or a global
T h() { return this->member_; }             // COPIES – member_ isn't a local
```



```
// 4. Returning through a reference
T i() { T x; T& r = x; return r; }           // no NRVO, and NO implicit move
either – copies

// 5. volatile
```

Row 3 and row 5 are where you actually pay.

Experiment with it

```
-fno-elide-constructors      # GCC/Clang: disables NRVO (and pre-17 RV0) so you
can see the calls
```

Put `printf` in every constructor, compile with and without, and read the difference. It's the fastest way to internalize the rules.

The upshot

Return by value. Don't return by output parameter for performance reasons. Don't `std::move` the return.

```
std::vector<int> good();           // ✓ idiomatic, fast, composable
void bad(std::vector<int>& out);    // ✗ 1998 style, no chaining, forces
default-construction
```

5.8 When moves don't happen ⚠

A consolidated list of every place a move silently degrades to a copy.

1. `const` objects and `const` members

```
const std::string s = "x";
auto t = std::move(s);           // COPIES (§5.2)

struct S { const std::string name; };
std::vector<S> v;
v.push_back(std::move(s2));      // the const member is copied
```

2. Returning a member

```
class Builder {
    std::string data_;
public:
```

```
std::string build()      { return data_; }           // COPIES
std::string build2()     { return std::move(data_); } // moves (destroys the
builder's state)
std::string build3() && { return std::move(data_); } // moves – only on
rvalue Builders
};
```

`return std::move(member_)` is **correct** — the member is not a local, so there's no NRVO to block. The ref-qualified `build3` is the clean design: it only allows the steal when the builder itself is expiring.

3. `return std::move(local)` — the pessimization

```
std::vector<int> bad() {
    std::vector<int> v(1000);
    return std::move(v); // ⚠ BLOCKS NRVO. Forces an actual move that
wasn't needed.
}
std::vector<int> good() {
    std::vector<int> v(1000);
    return v;           // NRVO: zero moves. Falls back to an implicit move
if not.
}
```

`return std::move(x)` turns a prvalue into an xvalue, which disqualifies elision. You converted "free" into "one move". GCC and Clang warn with `-Wpessimizing-move` / `-Wredundant-move`.

Summary of **return**:

Returning	Write
a local	<code>return x;</code>
a by-value parameter	<code>return x;</code> (implicit move; <code>std::move</code> is redundant)
a member	<code>return std::move(member_);</code>
a prvalue	<code>return T{...};</code>

4. Missing move operations

Declaring a destructor or a copy operation **suppresses** the implicit moves (§4.9 table). Everything then falls back to copies, silently. The single most common real-world cause of "why is this slow".

5. Non-`noexcept` moves in a `vector`

Covered in §5.5. Reallocation copies.

6. Passing an lvalue where you meant to give it up



```
std::string s = compute();
v.push_back(s);           // COPIES – s is an lvalue
v.push_back(std::move(s)); // moves
```

7. Lambda captures

```
auto p = std::make_unique<T>();
auto f = [p]{ };           // ERROR – copy capture of a move-only
                             type
auto g = [p = std::move(p)]{ }; // ✓ C++14 init-capture
```

And capturing a `vector` by value copies it every time the lambda is copied — including when it's stored in a `std::function`.

8. Moving into a `const&` parameter

```
void f(const std::vector<int>& v);
f(std::move(bigVec));           // no move – f never takes ownership. The move is a
no-op.
```

Harmless, but it signals an intent the callee can't honor. If `f` should take ownership, its signature should say so (by value, or `&&`).

5.9 `std::exchange` and the swap idiom

`std::exchange`

```
template <class T, class U = T>
constexpr T exchange(T& obj, U&& new_val) {
    T old = std::move(obj);
    obj = std::forward<U>(new_val);
    return old;           // "set to new, give me the old"
}
```

Purpose-built for move constructors:

```
Buffer(Buffer&& o) noexcept
: data_(std::exchange(o.data_, nullptr)),
  size_(std::exchange(o.size_, 0)),
  cap_ (std::exchange(o.cap_, 0)) {}
```

One expression per member, in the initializer list, source correctly reset. Compare the alternative (copy in the body, then null out each field) — same code, more lines, more chances to forget one.

Also useful generally:

```
auto old = std::exchange(state_, State::Running);
if (auto head = std::exchange(head_, head_->next)) { ... } // list pop
bool first = std::exchange(isFirst_, false);                // one-shot flag
```

The ADL two-step for `swap`

`std::swap`'s default implementation is three moves:

```
template <class T> void swap(T& a, T& b) {
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}
```

That's fine for most types. But some types have a cheaper member `swap` (containers swap pointers directly, and node-based containers must preserve iterator validity). To pick up a user's specialized `swap` and fall back to the standard one, use the two-step:

```
template <class T>
void generic(T& a, T& b) {
    using std::swap; // bring the default into scope
    swap(a, b);      // UNQUALIFIED – ADL finds a better swap if one exists
}
```

`std::swap(a, b)` (qualified) would never find a user's `swap` in their own namespace. C++20 gives you a shortcut that does the dance for you:

```
std::ranges::swap(a, b); // customization-point object, handles ADL correctly
```

Providing `swap` for your own type:

```
class Widget {
public:
    void swap(Widget& o) noexcept { /* member-wise swap */ }
};
inline void swap(Widget& a, Widget& b) noexcept { a.swap(b); } // same namespace
as Widget
```

Copy-and-swap

A single assignment operator that handles both copy and move, with the **strong exception guarantee**, and correct self-assignment for free:

```
class Buffer {
public:
    Buffer(const Buffer&);           // copy ctor
    Buffer(Buffer&&) noexcept;       // move ctor

    Buffer& operator=(Buffer other) { // ⚠ by VALUE — the copy/move happens
here                                // here
        swap(*this, other);
        return *this;              // `other` (holding our old guts) dies
here                                // here
    }
};
```

If the caller passes an lvalue, the parameter is copy-constructed; if an rvalue, move-constructed. Either way, the assignment body can't throw. If the copy throws, it throws *before* `*this` is touched — strong guarantee, no `if (this != &other)` needed.

⚠ **The cost:** move-assignment now always goes through a construct-then-swap, and you can't reuse existing capacity. For a `vector`-like type, `v = std::move(w)` under copy-and-swap does a full move-construct plus three swaps, and leaves the old buffer to be freed in `other`'s destructor — versus a hand-written move assignment that just frees and steals. It's elegant and it's the right default for correctness-critical code; performance-critical containers write the two operators separately.

Gotcha checklist for this section

- ☐ `std::move` **does nothing** by itself — it's a cast; the move happens in the selected overload
- ☐ `std::move` on a `const` object silently copies
- ☐ `const` **members** make a type effectively non-movable
- ☐ `T&&` is only a forwarding reference when **T is deduced right there** — `vector<T>&&` and `const T&&` are not
- ☐ Inside a function, a `T&&` parameter is an **lvalue**
- ☐ Use `std::move` on rvalue refs, `std::forward<T>` on forwarding refs — swapping them steals from lvalues
- ☐ **Forward exactly once**; never in a loop
- ☐ Forwarding-reference constructors **hijack the copy constructor** — constrain them
- ☐ A move that isn't `noexcept` makes `vector` reallocation **copy** — mark all moves `noexcept` and `static_assert` it
- ☐ `= default` moves are only `noexcept` if every member's move is
- ☐ Moved-from = **valid but unspecified**; only assign or destroy
- ☐ `std::move(optional)` leaves it **engaged**, holding a moved-from value
- ☐ `return std::move(local)` **blocks NRVO** — just `return local`;
- ☐ `return member_;` **copies** — that one needs `std::move`

- ☐ Declaring a destructor kills your moves (§4.9)
- ☐ `push_back(s)` with a named `s` copies

Flags and tools

```
-Wpessimizing-move      # return std::move(local)
-Wredundant-move        # std::move where implicit move already applies
-Wdeprecated-copy       # implicit copy ops where you declared a dtor
-fno-elide-constructors # turn off NRVO to see what's really happening
-Wmove                  # MSVC: /w14996 and friends; check /analyze too

clang-tidy:
  bugprone-use-after-move
  performance-move-const-arg      # std::move on a const or trivially-copyable
arg
  performance-no-automatic-move   # missed implicit moves on return
  performance-unnecessary-value-param
  modernize-pass-by-value

static_assert(std::is_nothrow_move_constructible_v<T>);
static_assert(std::is_nothrow_move_assignable_v<T>);
```

Exercises worth doing

1. Write a `Tracker` class that prints on every construct/copy/move/destroy. Put it in a `vector`, `push_back` ten of them without `reserve`, and read the output. Then add `reserve`. Then remove `noexcept` from the move constructor and read it again — count the copies.
2. Write `myMove` and `myForward` from scratch using `static_cast` and `remove_reference_t`. Test both against all four argument categories.
3. Write a wrapper `template <class F, class... A> auto call(F&&, A&&...)` that perfectly forwards and preserves the return category. Verify with `decltype` that it returns `T&` when the target does.
4. Write the perfect-forwarding-constructor bug (§5.4), get the error, then fix it with a `requires` clause.
5. Compile `return v;` vs `return std::move(v);` on Godbolt at `-O2` and diff the assembly.
6. Take a class in your codebase with a destructor and check `std::is_move_constructible_v / is_nothrow_move_constructible_v`. Count how many fail.

Chapter 6 — Functions

What this chapter covers

- Overloading and **overload resolution** (the ranking rules)
- Default arguments, and their interaction with overriding
- **Trailing return types**, `decltype(auto)`
- `inline`, ODR, and what `inline` actually means now
- `constexpr/consteval` functions

- `noexcept` — as specifier and operator; when it matters
- `[[nodiscard]]`, `[[maybe_unused]]`, `[[deprecated]]`, `[[likely]]`
- **Lambdas**: captures (`=`, `&`, init-capture, `this` vs `*this`), `mutable`, generic lambdas, `constexpr` lambdas
- ⚠ **Dangling captures** — the #1 lambda bug
- `std::function` vs templates vs `function_ref` — the type-erasure cost
- Function pointers, member function pointers, `std::invoke`
- Variadic templates and parameter packs; fold expressions

Functions in C++ carry more machinery than in almost any other language: overload resolution with formal ranking rules, template deduction, ADL, value-category-aware returns, compile-time evaluation, exception specifications, and four different ways to pass around "something callable". Most of the surprises in this section come from the compiler picking a *different* function than you expected — silently, and legally.

6.1 Overloading and overload resolution ⚠

Overload vs override vs hide

Three different things people conflate:

```
struct Base {
    void f(int);
    void f(double);           // OVERLOAD — same scope, different params
    virtual void g(int);
};

struct Derived : Base {
    void g(int) override;     // OVERRIDE — same signature, virtual dispatch
    void f(std::string);      // HIDES both Base::f! ⚠
};

Derived d;
d.f(42);                     // ERROR — Base::f is hidden, and int doesn't convert
                             // to string
```

Name hiding: a name in a derived class hides *every* overload of that name in the base. The fix:

```
struct Derived : Base {
    using Base::f;             // bring them all back into scope
    void f(std::string);
};
```

This bites hardest with `operator=` and with base classes that gain overloads later.

What can and can't differ between overloads

Can differ	Cannot differ (not an overload)
Number of parameters	Return type alone
Parameter types	Top-level const on a by-value param (<code>f(int) ≡ f(const int)</code>)
const / volatile on member functions	Default argument values
Ref-qualifiers & / && on member functions	Parameter <i>names</i>
T vs const T& (but this often creates ambiguity)	T[] vs T* (they're the same after decay)

The three phases

1. **Name lookup** — build the candidate set. Ordinary unqualified lookup, plus **ADL**, plus template candidates after deduction.
2. **Viability filter** — drop candidates whose parameters can't be initialized from the arguments (wrong arity, no conversion available, failed constraints).
3. **Ranking** — order the viable candidates by conversion quality. The best must be **strictly better than all others on at least one argument and no worse on any**, or the call is ambiguous.

ADL (argument-dependent lookup)

For an *unqualified* call, the compiler also searches the namespaces of the argument types:

```
namespace lib {
    struct Widget {};
    void print(const Widget&);
}

lib::Widget w;
print(w);           // FOUND — ADL searched namespace lib because w is a
lib::Widget
```

This is why `std::cout << x` works without qualifying `operator<<`, and why the `using std::swap; swap(a,b);` two-step (§5.9) is written the way it is. It's also a common surprise: adding a function to a namespace can change which overload an existing call resolves to.

The ranking table

From best to worst. Any conversion in a higher tier beats any conversion in a lower one.

Rank	Category	Examples
1	Exact match	identity; lvalue→rvalue; array→pointer decay; function→pointer; qualification conversion (<code>T* → const T*</code>)

Rank	Category	Examples
2	Promotion	<code>char/short/bool</code> → <code>int</code> ; <code>float</code> → <code>double</code> ; unscoped enum → its underlying/promoted type
3	Standard conversion	<code>int</code> → <code>double</code> ; <code>int</code> → <code>long</code> ; <code>Derived*</code> → <code>Base*</code> ; anything → <code>bool</code> ; <code>T*</code> → <code>void*</code>
4	User-defined conversion	converting constructor, or <code>operator T()</code> — at most one in a chain
5	Ellipsis ...	last resort

Tie-breakers after conversion rank:

- A **non-template** function beats a template specialization that's equally good
- A **more specialized** template beats a less specialized one (partial ordering)
- With concepts, a **more constrained** overload beats a less constrained one (subsumption, §10)

The classic traps

Literal `0` and `NULL`:

```
void f(int);
void f(char*);
f(0);           // f(int) - 0 is an int literal
f(NULL);        // f(int) or ambiguous, depending on how NULL is defined
f(nullptr);     // f(char*) - the reason nullptr exists
```

Promotion beats conversion:

```
void f(int);
void f(long);
void f(double);

f('a');        // f(int) - char→int is a PROMOTION (rank 2)
f(1.0f);        // f(double) - float→double is a PROMOTION
f(1u);          // AMBIGUOUS - unsigned→int, →long, →double are all rank-3
                // conversions
```

The `bool` hijack — the nastiest one:

```
void f(bool);
void f(const std::string&);

f("hello");    // calls f(bool)!!
```

`const char* → bool` is a **standard conversion** (rank 3). `const char* → std::string` requires the **std::string converting constructor** (rank 4). Rank 3 beats rank 4. Your string literal silently becomes `true`.

The fix is to make the bad overload unavailable:

```
void f(bool);
void f(const std::string&);
void f(const char* s) { f(std::string(s)); } // add the exact-match overload
// or:
void f(bool) = delete; // if you never wanted bool
```

Deleted functions still participate in overload resolution — they're chosen and *then* the program is ill-formed. That's the point: it produces a clear error at the call site rather than a silent conversion.

Ref-qualified member functions

```
class Builder {
    std::string data_;
public:
    const std::string& get() const& { return data_; } // on lvalues:
observe
    std::string      get() &&      { return std::move(data_); } // on rvalues:
steal
};

Builder b;
auto x = b.get(); // copies from the lvalue
auto y = Builder{}.get(); // moves out of the temporary
```

Also useful to *ban* calls on temporaries:

```
class Handle {
public:
    T* get() & { return p_; }
    T* get() && = delete; // Handle{}.get() would dangle – forbid it
};
```

6.2 Default arguments

Basics and rules

```
void draw(int x, int y, Color c = Color::Black, int width = 1);
draw(0, 0); // Black, 1
draw(0, 0, Color::Red); // Red, 1
```

- Only **trailing** parameters can have defaults
- The default expression is **evaluated at each call**, in the **caller's** context:

```
int counter();
void f(int n = counter()); // counter() runs on every call that omits n
void g(std::vector<int> v = {}); // ⚠ allocation-free here, but a non-empty
default allocates each call
```

- Defaults are attached to a **declaration**, and can be added in later declarations in the same TU:

```
// header
void f(int a, int b);
// .cpp
void f(int a, int b = 10); // ⚠ default visible only in this TU – confusing,
avoid
```

Put defaults in the header declaration, once.

⚠ Default arguments + virtual functions

This is the bug worth memorizing:

```
struct Base {
    virtual void draw(Color c = Color::Red) { print("Base", c); }
};
struct Derived : Base {
    void draw(Color c = Color::Blue) override { print("Derived", c); }
};

Base* p = new Derived;
p->draw(); // prints "Derived RED"
```

Which function runs is dynamic. Which default argument is used is static — taken from the type of the expression (`Base*`). You get the derived implementation with the base's default. Nothing warns you.

Core Guidelines rule: **never give a virtual function a default argument**. If you need one, use the non-virtual interface idiom:

```
struct Base {
    void draw(Color c = Color::Red) { doDraw(c); } // non-virtual, owns the
default
private:
    virtual void doDraw(Color c) = 0; // virtual, no default
};
```

Default arguments vs overloads

Prefer defaults when	Prefer overloads when
The extra parameter genuinely has a natural default	The behaviour differs, not just a value
One implementation	Different implementations
ABI stability isn't a concern	You need to add options later without breaking callers

⚠ Changing a default argument is an **ABI-visible change** — callers compiled against the old header keep passing the old default (§20).

6.3 Trailing return types and `decltype(auto)`

Trailing return types

```
template <class T, class U>
auto add(T a, U b) -> decltype(a + b) { return a + b; }
```

Needed pre-C++14 whenever the return type depends on the parameters — the parameter names aren't in scope until after the `->`. Also useful for readability inside classes:

```
auto Container::begin() -> iterator;           // vs
Container::iterator Container::begin();       // must qualify the
nested type
```

`auto` return deduction (C++14)

```
auto f() { return 42; }           // int
auto g() { return expr; }        // uses template deduction rules: STRIPS
ref and cv
```

```
const std::string& getRef();
auto x = getRef();               // std::string – a COPY. The & and const are gone.
auto& y = getRef();              // const std::string& – reference preserved
```

Same for return types: `auto f() { return getRef(); }` returns `std::string` **by value**. Usually what you want; occasionally a silent copy in a hot path.

Constraints: a function with deduced return type must be **defined before use** (no forward declaration usable), and all **return** statements must deduce the same type.

decltype(auto)

Deduces using **decltype** rules instead, so it **preserves** references and cv-qualification:

```
const std::string& getRef();

auto          a() { return getRef(); } // std::string (copy)
decltype(auto) b() { return getRef(); } // const std::string& (reference preserved)
```

The main use is generic forwarding wrappers, where you don't know whether the target returns by value or by reference:

```
template <class F, class... Args>
decltype(auto) logged(F&& f, Args&&... args) {
    log("calling");
    return std::invoke(std::forward<F>(f), std::forward<Args>(args)...);
}
```

⚠ The parenthesis trap

```
decltype(auto) f() {
    int x = 42;
    return x;      // int – fine
}
decltype(auto) g() {
    int x = 42;
    return (x);    // int& – DANGLING REFERENCE to a destroyed local
}
```

decltype((x)) is **int&** (§3.2). One pair of parentheses turns a value return into a dangling reference. GCC/Clang warn with **-Wreturn-local-addr**, but don't rely on it.

6.4 inline, ODR, and what inline actually means now

The One Definition Rule

- Every **entity** used must be defined exactly **once in the whole program** (functions, variables)
- **Classes, templates, inline functions, enums** may be defined once **per translation unit**, but every definition must be **token-for-token identical** and mean the same thing

⚠ **ODR violations are IFNDR** — *ill-formed, no diagnostic required*. The linker just picks one definition and everything else silently uses it. This produces some of the worst bugs in C++.

Real causes of ODR violations:

```
// a.cpp                                // b.cpp
#define NDEBUG                            // (no NDEBUG)
#include "widget.h"                       #include "widget.h"
// -> Widget has a different size/layout in each TU. Linker picks one. Heap
corruption.
```

- Different `-D` flags, `-std=`, or optimization-dependent `#ifdefs` across TUs
- Different struct packing (`#pragma pack`) in different TUs
- A type defined in an anonymous namespace in a **header** (each TU gets a distinct type)
- Mixing debug and release standard libraries (`_GLIBCXX_DEBUG`, MSVC `/MDd` vs `/MD`)

Tools: the `-flto` **link step** and `-Wodr` catch some; **gold/ld with `--detect-odr-violations`**; ASan reports "odr-violation" for global variables.

What **inline** means

inline does not mean "inline this function". It means "this entity may be defined in multiple translation units; the linker should merge them and keep one."

Whether the call is actually inlined is entirely the optimizer's decision, made from cost heuristics, and it will inline non-**inline** functions freely (especially with LTO) and refuse to inline **inline** ones.

To actually force it:

```
[[gnu::always_inline]] inline void f(); // GCC/Clang
__forceinline void g();                // MSVC
[[gnu::noinline]] void h();             // the opposite, useful for profiling
```

Implicitly inline

- Member functions **defined inside the class body**
- **constexpr** and **constexpr** functions
- Function templates and members of class templates (they have their own linkage rules)
- **inline** variables and **constexpr** static data members (C++17)

Inline variables (C++17)

Before C++17, a header-only library couldn't have a global object without the **static-in-a-function** trick or a **.cpp** file:

```
// header, C++17
inline constexpr double kPi = 3.14159265358979;
```

```
inline std::string gAppName = "app";           // one shared instance across all TUs

struct Config {
    static inline int instances = 0;           // in-class definition of a static
    member
};
```

⚠ `constexpr` variables at namespace scope are implicitly `const` and have **internal** linkage, so each TU gets its own copy (harmless for values, but taking the address gives different pointers per TU). `inline constexpr` fixes that.

Linkage cheat sheet

Declaration	Linkage	ODR risk
<code>void f() {}</code> in a header	external, one definition allowed	✗ multiple-definition link error
<code>inline void f() {}</code> in a header	external, merged	✓ (if identical)
<code>static void f() {}</code> in a header	internal, one per TU	✓ but code bloat, and <code>static</code> locals are per-TU ⚠
anonymous namespace in a header	internal, distinct type per TU	⚠ subtle ODR bugs with templates
<code>constexpr</code> variable in a header	internal	✓ values fine, addresses differ
<code>inline constexpr</code> in a header	external, merged	✓ preferred

6.5 `constexpr` and `constexpr` functions

`constexpr` — *may* run at compile time

```
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

constexpr int a = factorial(5); // compile time, guaranteed
int n = readInput();
int b = factorial(n);           // runtime – the SAME function
```

`constexpr` on a function means "this function is *eligible* for constant evaluation", not "this always runs at compile time". Whether it does depends on whether the context requires a constant.

Forcing compile-time evaluation:

```
constexpr auto v = factorial(5);           // constexpr variable – must be a constant
std::array<int, factorial(5)> arr;         // template argument – must be a constant
static_assert(factorial(5) == 120);
```

What's allowed inside, by standard

Standard	New capability
C++11	essentially a single <code>return</code> statement
C++14	loops, local variables, multiple statements, mutation of locals
C++17	<code>if constexpr</code> , lambdas
C++20	transient allocation (<code>new/delete</code> , so <code>vector/string</code> work at compile time), <code>try/catch</code> , virtual calls , <code>constexpr</code> destructors, <code>std::is_constant_evaluated</code>
C++23	non-literal variables, <code>static/thread_local</code> in <code>constexpr</code> functions (unevaluated), relaxed <code>goto</code> , <code>constexpr std::unique_ptr</code>
C++26	<code>constexpr</code> exceptions, <code>constexpr</code> placement-new, static reflection

"Transient" in C++20 means memory allocated during constant evaluation must also be **freed** during it — you can build a `std::vector` at compile time and compute from it, but you can't have one survive into the runtime program.

```
constexpr int sumOfPrimes(int limit) {           // C++20 – legal
    std::vector<bool> sieve(limit, true);
    int sum = 0;
    for (int i = 2; i < limit; ++i)
        if (sieve[i]) { sum += i; for (int j = i*i; j < limit; j += i) sieve[j] =
false; }
    return sum;                                // vector is destroyed before we
return
}
static_assert(sumOfPrimes(100) == 1060);
```

`constexpr` — *must* run at compile time

```
constexpr int square(int n) { return n * n; }

constexpr int a = square(5);    // OK
int n = readInput();
int b = square(n);              // ERROR – not a constant expression
```

Called an **immediate function**. Use it for things that are meaningless at runtime: compile-time string parsing, format-string validation (`std::format` uses this), ID generation, type registration.

constexpr

Not a function specifier, but it belongs to the same family: it guarantees **static initialization** (no dynamic init at startup), which kills the static initialization order fiasco for that variable — while still allowing mutation.

```
constexpr int counter = 0;           // guaranteed initialized at compile
time
constexpr Logger gLog = makeLogger(); // ERROR if makeLogger() isn't
constant-evaluable
```

constexpr = const + compile-time-initialized. **constexpr** = compile-time-initialized but mutable.

std::is_constant_evaluated() and if constexpr

For functions that want a different implementation at compile time (e.g. a slow-but-constexpr version vs an SIMD runtime version):

```
constexpr double power(double b, int e) {
    if (std::is_constant_evaluated())
        return naiveLoop(b, e);    // constexpr-friendly
    else
        return std::pow(b, e);      // runtime, uses the FPU intrinsic
}
```

⚠ The trap:

```
if constexpr (std::is_constant_evaluated()) // ALWAYS TRUE – the condition of
    ...                                     // `if constexpr` is itself a
constant                                   // -evaluated context
```

Use plain **if**, not **if constexpr**. C++23 added dedicated syntax that can't be misused:

```
if constexpr { ... } else { ... }
```

6.6 noexcept — specifier and operator

As a specifier

```
void f() noexcept;           // promise: nothing escapes
void g() noexcept(true);     // same
void h() noexcept(false);    // default – may throw
```

If an exception *does* try to escape a `noexcept` function, `std::terminate` is called immediately. Stack unwinding is **not guaranteed** to have happened, so destructors may not run.

Conditional `noexcept`

```
template <class T>
void wrapper(T& t) noexcept(noexcept(t.doThing())) { t.doThing(); }
//                ^specifier ^operator
```

The inner `noexcept(expr)` is the **operator**: a compile-time `bool` saying whether `expr` is declared not to throw. The outer is the specifier. The doubled keyword is ugly but standard.

```
template <class T>
void mySwap(T& a, T& b) noexcept(std::is_nothrow_move_constructible_v<T> &&
                                std::is_nothrow_move_assignable_v<T>);
```

Where it actually matters

Context	Effect
Move constructor / move assignment	<code>vector</code> reallocation moves instead of copying (§5.5) — the big one
<code>swap</code>	enables <code>noexcept</code> on containers holding the type
Destructors	implicitly <code>noexcept</code> ; throwing calls <code>terminate</code>
Deallocation functions	implicitly <code>noexcept</code>
Callbacks crossing a C ABI boundary	mandatory — an exception through C frames is UB (§25)
Hot leaf functions	occasionally lets the optimizer skip unwinding tables

⚠ The performance benefit of `noexcept` on ordinary functions is usually **zero** — the zero-cost exception model already costs nothing on the non-throwing path. Don't sprinkle it everywhere hoping for speed.

It's part of the type (C++17)

```
void f() noexcept;
void (*p)() noexcept = f;    // OK
void (*q)()           = f;    // OK — noexcept converts to may-throw
void (*r)() noexcept = g;    // ERROR if g may throw — the conversion isn't
allowed backwards
```

Interface consideration

`noexcept` is a **promise to your callers**. Adding it later is safe; removing it breaks code that relied on it (including `static_asserts` and `vector` performance). So: put it where it's semantically true and stable — move operations, swap, simple accessors, destructors — and leave it off where a future implementation might need to throw.

`throw(A, B)` dynamic exception specifications were deprecated in C++11 and **removed in C++17**. `throw()` was an alias for `noexcept` and is also gone in C++20.

6.7 Attributes

```
[[nodiscard]] int compute();
[[nodiscard("check the error code")]] Status doThing(); // C++20: with a reason
class [[nodiscard]] Error {};                          // on a TYPE: every
function returning it

compute(); // warning: ignoring return value
```

`[[nodiscard]]` is the highest-value attribute in practice. Put it on: anything returning an error status, anything pure (`empty()` — the classic bug is calling `v.empty()` meaning `v.clear()`), factory functions, and RAI guard types (`std::lock_guard` unnamed temporary = instantly unlocked bug).

```
void f(int x, [[maybe_unused]] int debugOnly); // suppress unused warnings

[[deprecated("use newApi() instead")]]
void oldApi();

[[noreturn]] void fatal(const char* msg); // never returns; helps flow
analysis

switch (x) {
    case 1: doA(); [[fallthrough]]; // intentional; suppresses -
Wimplicit-fallthrough
    case 2: doB(); break;
}

if (ptr) [[likely]] { fast(); } // C++20 branch hints
else     [[unlikely]] { slow(); }
```

⚠ `[[likely]]`/`[[unlikely]]`: **measure**. Modern CPUs predict branches well; the compiler's PGO data (§24) is far better than your guess. These are for cases you genuinely know are rare (error paths) and where profiling showed layout mattered.

⚠ `[[assume(expr)]]` (C++23): tells the optimizer `expr` is true, without evaluating it. If it isn't true, you have UB. Extremely sharp — treat like `__builtin_unreachable`.

`[[no_unique_address]]` (C++20) covered in §7.

6.8 Lambdas

Anatomy

```
[captures] <template-params> (params) specifiers -> ReturnType { body }
```

A lambda is syntax sugar for an **unnamed class with an operator()**:

```
int n = 5;
auto f = [n](int x) { return x + n; };

// approximately:
struct __Lambda {
    int n;
    int operator()(int x) const { return x + n; }    // note: const!
};
__Lambda f{n};
```

Every lambda has a **distinct type**, so two identical-looking lambdas are unrelated types. That's why you need **auto**, or type erasure (§6.10) to store them uniformly.

Captures

```
int a = 1, b = 2;

[]{};           // captures nothing
[a]{};          // a by value (copy)
[&a]{};         // a by reference
[a, &b]{};       // mixed
[=]{};          // everything used, by value
[&]{};          // everything used, by reference
[=, &b]{};       // by value, except b
[&, a]{};       // by reference, except a

[p = std::move(ptr)]{}; // C++14 INIT-CAPTURE – move-only types, renaming,
computed values
[n = a + b]{};        // arbitrary expression
[&r = a]{};           // reference init-capture

[this]{};            // captures the `this` POINTER – members accessible
[*this]{};          // C++17: captures a COPY of *this
[...args = std::move(args)]{}; // C++20: capturing a parameter pack
```

⚠ **[=] inside a member function captures **this** by pointer, not the object.** So a [=] lambda that touches a member holds a raw pointer to the enclosing object, and dangles if the object dies — despite "capture by

value" reading as safe. C++20 **deprecated** implicit `this` capture via `[=]`; write `[=, this]` or `[*this]` explicitly.

mutable

`operator()` is `const` by default, so by-value captures can't be modified:

```
int n = 0;
auto f = [n] { ++n; }; // ERROR
auto g = [n]() mutable { ++n; }; // OK – modifies the lambda's own copy, not the
original
```

Generic and template lambdas

```
auto f = [](auto x, auto y) { return x + y; }; // C++14 generic

auto g = [<class T>(const std::vector<T>& v) { // C++20 template lambda
    return v.size();                       // T is nameable
};

auto h = [<class... Ts>(Ts&&... ts) { // variadic, with proper
    forwarding
    return (... + std::forward<Ts>(ts));
};
```

A generic lambda's `operator()` is a template — which means it participates in overload resolution and constraints like any template.

constexpr lambdas

Implicitly `constexpr` since C++17 if the body qualifies:

```
auto square = [](int n) { return n * n; };
static_assert(square(4) == 16); // works, no keyword needed
auto f = [](int n) constexpr { return n * n; }; // explicit, if you want the
error
constexpr auto g = ...; // C++20 constexpr lambdas exist
too
```

Conversion to a function pointer

A **captureless** lambda converts implicitly to a plain function pointer — the bridge to C APIs:

```
auto cmp = [](const void* a, const void* b) -> int { ... };
std::qsort(arr, n, sizeof(T), cmp); // works: converts to int (*)(const
```

```
void*, const void*)

extern "C" void register_cb(void (*cb)(int));
register_cb([](int x) { handle(x); });           // fine – captureless
register_cb([&](int x) { handle(x, ctx); });     // ERROR – has captures
```

The standard C-callback workaround: pass state through the API's `void* userdata` parameter.

Useful idioms

```
// IIFE for complex const initialization
const auto config = []{
    Config c;
    c.load();
    c.validate();
    return c;
}();                                     // note the trailing ()

// Recursion (pre-C++23)
auto fib = [](this auto self, int n) -> int {           // C++23 "deducing this"
    return n < 2 ? n : self(n-1) + self(n-2);
};
// pre-C++23:
std::function<int(int)> fib2 = [&](int n) { return n < 2 ? n : fib2(n-1) + fib2(n-2); };
// (works, but allocates and prevents inlining – see §6.10)

// Overload set from lambdas
template <class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template <class... Ts> overloaded(Ts...) -> overloaded<Ts...>; // CTAD guide

std::visit(overloaded{
    [](int i) { print("int", i); },
    [](const std::string& s) { print("string", s); },
}, myVariant);
```

C++20 also made captureless lambdas **default-constructible and assignable**, and usable in unevaluated contexts — so they work as comparators in template arguments:

```
std::set<Point, decltype([](auto& a, auto& b){ return a.x < b.x; })> s; // C++20
```

6.9 ⚠ Dangling captures — the #1 lambda bug

The four shapes

```

// 1. Returning a lambda that captured locals by reference
auto makeAdder(int n) {
    return [&n](int x) { return x + n; };    // n dies when makeAdder returns –
DANGLING
}
// fix: [n] (by value)

// 2. Storing a [&] lambda for later
class Button {
    std::function<void()> onClick_;
public:
    void setup() {
        std::string label = "Click me";
        onClick_ = [&]{ print(label); };    // label dies at end of setup() –
DANGLING
    }
};
// fix: [label] or [label = std::move(label)]

// 3. Async / detached execution
void start() {
    LocalData data;
    std::thread([&]{ process(data); }).detach();    // data dies immediately –
DANGLING
}
// fix: capture by value, or use std::jthread and join

// 4. Implicit `this` capture
class Session {
    std::string id_;
public:
    void schedule() {
        timer_.async_wait([=](auto) { print(id_); });    // captured `this`, NOT id_
                                                         // Session may die first –
DANGLING
    }
};
// fix: [*this] (copies the whole object), or [id = id_], or
//       [self = shared_from_this()] (§4.4)

```

Shape 4 is the one that reaches production, because `[=]` looks like a safe by-value capture.

The rule

`[&]` is only safe for lambdas that are consumed before the current scope ends. That means: STL algorithm callbacks, immediately-invoked lambdas, and `std::for_each`-style code.

Any lambda that is stored, returned, queued, or passed to a thread must capture explicitly and by value — including `[*this]` or a `shared_ptr` to self.

```
std::sort(v.begin(), v.end(), [&](auto& a, auto& b) { ... }); // ✓ consumed
immediately
callbacks_.push_back([&]{ ... }); // ✗ stored –
audit every capture
```

Detection

```
-Wdangling-gsl # Clang
clang-tidy: cppcoreguidelines-avoid-capturing-lambda-coroutines
clang-tidy: bugprone-lambda-function-name
-fsanitize=address -fsanitize-address-use-after-scope # catches most of these at
runtime
```

ASan with `use-after-scope` is the reliable answer. Static analysis catches the obvious cases only.

6.10 `std::function` vs templates vs `function_ref`

Four ways to accept "something callable", with very different costs.

1. Template parameter — zero overhead

```
template <class F>
void forEach(const std::vector<int>& v, F f) {
    for (int x : v) f(x);
}
```

The lambda's `operator()` is **inlined into the loop**. No indirection, no allocation. This is the default choice and it's why `std::sort` beats `qsort` by 2-3x.

Costs: the definition must be in a header; compile times grow; every distinct callable instantiates a new copy of the function (code bloat, §10); error messages are template errors.

2. `std::function` — type erasure with ownership

```
void forEach(const std::vector<int>& v, std::function<void(int)> f);
```

Owns a copy of the callable, erased behind a uniform type. Costs:

Cost	Detail
Heap allocation	if the callable exceeds the small-buffer size (~16 bytes on libstdc++, 2-3 pointers) — so a lambda capturing a <code>string</code> allocates

Cost	Detail
Indirect call	virtual-dispatch-equivalent; never inlined
Size	~32 bytes on libstdc++
Copyability requirement	the callable must be copy-constructible — so <code>std::function</code> can't hold a lambda capturing a <code>unique_ptr</code>

The last one is a frequent surprise, and the reason C++23 added:

```
std::move_only_function<void()> f = [p = std::make_unique<T>()]{ ... }; // C++23
std::copyable_function<void()> g; // C++26
- fixed const-correctness
```

3. `function_ref` — non-owning, for parameters

```
void forEach(const std::vector<int>& v, tl::function_ref<void(int)> f);
```

Two pointers (object + type-erased invoker). **No allocation, no copy, no ownership.** Trivially copyable. Perfect for a parameter that's used and discarded, which is the overwhelmingly common case.

Available as `absl::FunctionRef`, `llvm::function_ref`, `tl::function_ref`; `std::function_ref` is proposed for C++26.

⚠ It's a **reference** — it dangles like any other (§3). Never store one.

4. Plain function pointer

```
void forEach(const std::vector<int>& v, void (*f)(int));
```

One pointer, no state, C-compatible. Only captureless lambdas convert.

Comparison

	Template F	<code>std::function</code>	<code>function_ref</code>	<code>void(*)()</code>
Allocation	none	possible	none	none
Inlinable	✓	✗	✗	usually ✗
Owens the callable	✓ (by value)	✓	✗	n/a
Stores state	✓	✓	✗	✗

	Template F	std::function	function_ref	void(*)()
Heterogeneous storage	X	✓	X	✓ (no state)
Definition in header	required	no	no	no
Move-only callables	✓	X (until <code>move_only_function</code>)	✓	X

The guideline

- **Hot path, or a header-only generic algorithm** → template parameter
- **A callback parameter you use and discard** → `function_ref` (or a template)
- **You must store it** (event handlers, task queues, `std::vector` of callbacks) → `std::function` / `move_only_function`
- **C API boundary** → function pointer + `void* userdata`

The most common real-world mistake is `std::function` in a parameter position, where `function_ref` or a template would have cost nothing.

6.11 Function pointers, member pointers, `std::invoke`

Function pointers

```
int add(int, int);
int (*p)(int, int) = add;           // & is optional
using BinOp = int (*)(int, int);    // alias – much more readable
BinOp q = add;
p(1, 2);                           // or (*p)(1, 2)
```

Member function pointers

A member function pointer is **not** an address — it's an offset/index plus (for virtual or multiple-inheritance cases) adjustment data. `sizeof` can be 8, 16, or more.

```
struct S { int f(int); int data; virtual void g(); };

int (S::*pf)(int) = &S::f;           // & is REQUIRED here, and the class must be
named
int S::*pd = &S::data;              // pointer to data member

S s; S* ps = &s;
(s.*pf)(42);                        // note the parentheses – .* binds looser than
()
(ps->*pf)(42);
s.*pd = 7;
```

The syntax is awful, which is why `std::invoke` exists.

`std::invoke` (C++17)

Unifies every callable form under one call:

```
#include <functional>

std::invoke(freeFunction, 1, 2);
std::invoke(lambda, 1, 2);
std::invoke(&S::f, s, 42);      // member function on an object
std::invoke(&S::f, ps, 42);     // ...on a pointer
std::invoke(&S::f, sharedPtr, 42); // ...on a smart pointer
std::invoke(&S::data, s);       // member DATA access reads as a call
```

This is why generic code writes `std::invoke(f, args...)` rather than `f(args...)` — the latter doesn't handle member pointers. Companions:

```
std::invoke_result_t<F, Args...>      // the return type
std::is_invocable_v<F, Args...>       // does this call compile?
std::is_nothrow_invocable_v<F, Args...>
```

`std::mem_fn`, `bind_front`, and why not `std::bind`

```
std::mem_fn(&S::f);                  // wraps a member pointer as a callable
std::transform(v.begin(), v.end(), out, std::mem_fn(&Widget::getName));

auto g = std::bind_front(add, 10);   // C++20 – partial application, g(5) == add(10,5)
auto h = std::bind_back(divide, 2);  // C++23
```

⚠ **`std::bind` is discouraged.** It has surprising placeholder semantics, nested-bind behaviour, it copies arguments by default (requiring `std::ref`), it doesn't compose with overloads, and it generates worse code. A lambda does everything more clearly:

```
auto g = std::bind(add, 10, std::placeholders::_1); // X
auto g = [](int x) { return add(10, x); };         // ✓
```

6.12 Variadic templates, parameter packs, fold expressions

Basics

```
template <class... Ts>           // Ts is a template parameter PACK
void f(Ts... args) {             // args is a function parameter pack
    sizeof...(Ts);               // number of types
    sizeof...(args);             // same number
}
```

A pack must be **expanded** with `...`; the pattern to the left is repeated:

```
g(args...);                      // g(a1, a2, a3)
g(std::forward<Ts>(args)...);     // g(fwd(a1), fwd(a2), fwd(a3))
g(h(args)...);                   // g(h(a1), h(a2), h(a3))
std::tuple<Ts...> t;              // tuple<T1, T2, T3>
std::tuple<std::vector<Ts>...> u; // tuple<vector<T1>, vector<T2>,
vector<T3>>
```

The old way: recursion

```
void print() {}                  // base case
template <class T, class... Rest>
void print(const T& first, const Rest&... rest) {
    std::cout << first << ' ';
    print(rest...);              // peel one off
}
```

Works, but each pack size instantiates a new function — slow to compile, code bloat.

Fold expressions (C++17)

Form	Expands to
<code>(pack op ...)</code>	unary right fold: <code>a1 op (a2 op (a3 op a4))</code>
<code>(... op pack)</code>	unary left fold: <code>((a1 op a2) op a3) op a4</code>
<code>(init op ... op pack)</code>	binary left fold: <code>((init op a1) op a2) op a3</code>
<code>(pack op ... op init)</code>	binary right fold: <code>a1 op (a2 op (a3 op init))</code>

```
template <class... Ts>
auto sum(Ts... xs) { return (... + xs); } // left fold – prefer this
for arithmetic

template <class... Ts>
void print(const Ts&... xs) { ((std::cout << xs << ' '), ...); } // comma fold

template <class... Ts>
bool allTrue(Ts... xs) { return (... && xs); }
```

```
template <class... Ts>
void pushAll(std::vector<int>& v, Ts&&... xs) {
    (v.push_back(std::forward<Ts>(xs)), ...);    // comma fold with side
    effects
}
```

⚠ **Empty packs:** only three operators have a defined identity for an empty fold — `&&` → `true`, `||` → `false`, → `void()`. Everything else is ill-formed on an empty pack, so use a binary fold with an explicit init:

```
auto sum(Ts... xs) { return (0 + ... + xs); }    // works for zero arguments
```

✓ **Comma folds guarantee left-to-right evaluation with sequencing** — unlike ordinary comma-separated function arguments. That makes them safe for side effects.

Idioms worth knowing

```
// Apply a function to each element, with the index
template <class Tuple, class F, std::size_t... Is>
void forEachImpl(Tuple&& t, F&& f, std::index_sequence<Is...>) {
    (f(std::get<Is>(std::forward<Tuple>(t))), ...);
}
template <class Tuple, class F>
void forEach(Tuple&& t, F&& f) {
    forEachImpl(std::forward<Tuple>(t), std::forward<F>(f),
        std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple>>>{});
}
// (or just use std::apply, which does this for you)

std::apply([](auto&&... args) { print(args...); }, myTuple);

// Constrain a pack (C++20)
template <std::integral... Ts>
auto sum(Ts... xs) { return (0 + ... + xs); }

// Type-level checks
static_assert((std::is_trivially_copyable_v<Ts> && ...));
```

C++26 adds **pack indexing**, which removes a lot of `index_sequence` boilerplate:

```
template <class... Ts>
using First = Ts...[0];    // C++26
template <class... Ts>
auto firstArg(Ts... args) { return args...[0]; }
```

Gotcha checklist for this section

- ☐ A derived-class name **hides every base overload** — using `Base::f;`
- ☐ `f("literal")` picks `f(bool)` over `f(const std::string&)` — standard conversion beats user-defined
- ☐ Overloads can't differ by **return type** or top-level `const` on a by-value parameter
- ☐ **Never put a default argument on a virtual function** — dynamic dispatch, static default
- ☐ Default arguments are re-evaluated at every call, in the caller's context
- ☐ `auto` return **strips** references and `const`; `decltype(auto)` preserves them
- ☐ `return (x);` with `decltype(auto)` returns a **dangling reference**
- ☐ `inline` means "mergeable definition", **not** "inline this call"
- ☐ ODR violations are **IFNDR** — mismatched `-D` flags or debug/release STL across TUs silently corrupt
- ☐ `if constexpr (std::is_constant_evaluated())` is **always true** — use plain `if`, or `if constexpr`
- ☐ `noexcept` matters for **move ops**; elsewhere it's mostly interface documentation
- ☐ `noexcept` is part of the function type since C++17 and can't be removed compatibly
- ☐ `[=]` in a member function captures **this by pointer** — deprecated in C++20, dangles freely
- ☐ `[&]` is safe **only** for immediately-consumed lambdas
- ☐ `std::function` requires the callable to be **copy-constructible** and may allocate
- ☐ `std::function` in a parameter is usually the wrong choice — use a template or `function_ref`
- ☐ Member function pointers need `&Class::f` and the `.*` / `->*` operators (with parentheses)
- ☐ `std::bind` is discouraged — write a lambda
- ☐ Empty-pack folds only work for `&&`, `||`, `,` — otherwise supply an init value

Flags and tools

```
-Woverloaded-virtual      # a derived function hides a virtual base overload
-Wshadow
-Wodr                     # requires -flto
-Wunused-result           # enforces [[nodiscard]]
-Wreturn-local-addr
-Wdangling-gsl            # Clang
-Wnoexcept                # GCC: functions that could be noexcept
-fsanitize=address -fsanitize-address-use-after-scope # dangling captures

clang-tidy:
  bugprone-forwarding-reference-overload # the perfect-forwarding ctor problem
  modernize-avoid-bind
  modernize-use-trailing-return-type
  performance-unnecessary-value-param
  cppcoreguidelines-missing-std-forward
  readability-function-cognitive-complexity
```

Exercises worth doing

1. Write `void f(bool)`, `void f(const std::string&)`, `void f(int)` and call `f("x")`, `f(0)`, `f('c')`, `f(1.0f)`, `f(nullptr)`. Predict each before compiling. Then explain each with the ranking table.

2. Reproduce the virtual-function-default-argument bug, then fix it with the NVI idiom.
3. Write the same algorithm four ways — template `F`, `std::function`, `function_ref`, function pointer — and benchmark all four with Google Benchmark. Look at the assembly for the template version and confirm the callback was inlined.
4. Write a lambda that dangles via implicit `this` capture. Run it under ASan with `-fsanitize-address-use-after-scope`.
5. Implement `overloaded` from scratch (including the deduction guide) and use it with `std::visit`.
6. Write `template <class... Ts> void printTypes()` that prints each type name using a fold expression. Handle the empty pack.
7. Write a `constexpr` function that builds a `std::vector` at compile time (C++20) and `static_assert` a property of it. Then try to return the vector and read the error.

Chapter 7 — Classes

What this chapter covers

- Special member functions and when they're implicitly generated/deleted
- **Rule of Zero / Three / Five**, `= default`, `= delete`
- Member initializer lists, initialization order (⚠ declaration order, not list order)
- `explicit` constructors and conversion operators
- `const` member functions, `mutable`, logical vs bitwise const
- Static members, inline variables (C++17)
- Nested classes, friend
- **Aggregates**, designated initializers (C++20)
- `this`, `*this` return for chaining
- Copy-and-swap idiom
- **Empty base optimization**, `[[no_unique_address]]`

§4 covered *why* classes manage resources; this section covers *how the compiler builds one*. Most of the surprises here are about what the compiler silently generates, silently deletes, or silently reorders — and about conversions you didn't ask for.

7.1 Special member functions: generated, suppressed, deleted ⚠

The six

```
class T {
    T();                // default constructor
    ~T();               // destructor
    T(const T&);         // copy constructor
    T& operator=(const T&); // copy assignment
    T(T&&) noexcept;     // move constructor
    T& operator=(T&&) noexcept; // move assignment
};
```

Three separate questions the compiler answers for each: **is it declared? is it defined as deleted? is it trivial?**

When is it implicitly declared?

Member	Implicitly declared unless...
Default ctor	you declared any constructor
Destructor	you declared one
Copy ctor	you declared a move ctor or move assignment
Copy assign	you declared a move ctor or move assignment
Move ctor	you declared any of : destructor, copy ctor, copy assign, move assign
Move assign	you declared any of : destructor, copy ctor, copy assign, move ctor

The move row is the one that costs performance (§4.9, §5.8): declare a destructor and your moves quietly vanish, so every "move" becomes a copy.

Note the difference between the two failure modes:

- **Not declared** (destructor suppresses moves) → the *copy* overload is selected instead. Compiles, runs, slow.
- **Declared but deleted** (a move ctor is declared but its members can't be moved) → special rule: *a defaulted move operation that is defined as deleted is **ignored by overload resolution***, so the copy is selected instead. Also compiles, also slow.

Either way there is no diagnostic. The `static_assert(std::is_nothrow_move_constructible_v<T>)` habit from §5.5 is your only reliable defence.

When is it implicitly deleted?

The copy/move operations are deleted if any base or non-static member:

- has the corresponding operation deleted, inaccessible, or ambiguous
- has a destructor that's deleted or inaccessible

Plus these member-specific killers:

Member kind	Deletes
reference member (<code>T& r;</code>)	copy assignment, move assignment (can't rebind)
const member (<code>const T x;</code>)	copy assignment, move assignment; also makes moves degrade to copies (§5.8)
non-copyable member (<code>unique_ptr</code> , <code>mutex</code> , <code>atomic</code>)	copy ctor, copy assignment
a union with a non-trivial member	the corresponding op, unless you write it
base with a private/deleted destructor	everything

Two of these are the reason to avoid `const` and reference members in classes intended to live in containers.

Triviality and layout categories

These determine whether the compiler can `memcpy` your type, and how it lays out in memory across an ABI boundary.

```
#include <type_traits>
std::is_trivially_copyable_v<T>      // can be memcpy'd; required by
std::bit_cast, atomic<T>
std::is_trivially_destructible_v<T>  // destroying is a no-op; vector can skip
the loop
std::is_standard_layout_v<T>         // C-compatible layout; offsetof is valid
std::is_trivial_v<T>                 // trivially copyable + trivially default
constructible
std::is_aggregate_v<T>               // §7.8
```

Why it matters in practice:

- `vector<T>` can relocate a trivially copyable `T` with `memmove` instead of element-wise moves
- `std::bit_cast` and `std::atomic<T>` **require** trivially copyable
- `is_standard_layout` is what lets you share a struct with C or serialize it (§25)
- `is_trivially_destructible` lets `vector::clear()` compile to nothing

⚠ **A subtle triviality rule:** a special member is *user-provided* if it's declared and **not** explicitly defaulted or deleted **on its first declaration**. So:

```
struct A { A(const A&) = default; };           // trivial – defaulted on first
declaration

struct B { B(const B&); };                     // not trivial
B::B(const B&) = default;                      // defaulting OUT OF LINE makes
it user-provided
```

This is why the pimpl idiom's out-of-line `Engine::~~Engine() = default;` (§4.4) makes the type non-trivially-destructible — which is fine there, but worth knowing when you're chasing triviality for a performance reason.

("POD" is the old umbrella term for trivial + standard layout. It was deprecated in C++20; use the specific traits.)

7.2 Rule of Zero / Three / Five, = `default`, = `delete`

The rules themselves are in §4.9. Here are the class-mechanics details that belong with them.

= `default`

```

struct S {
    S() = default;           // in-class: trivial-eligible
    S(const S&) = default;
    ~S();                   // declared here...
};
S::~~S() = default;        // ...defaulted here: user-provided,
NOT trivial

```

- `= default` in the class body requires all member types to be **complete** — the pimpl problem (§4.4)
- You can't `= default` a function whose signature doesn't match what the compiler would have generated:

```

struct S { S& operator=(const S&) const = default; }; // ERROR – wrong signature

```

`= delete`

More useful than most people use it for:

```

// 1. Ban a conversion at the call site
void process(int);
void process(double) = delete;           // process(3.14) is now an error, not
a truncation
void process(char) = delete;

// 2. Ban heap allocation (embedded, arena-only types)
struct StackOnly {
    static void* operator new(std::size_t) = delete;
    static void* operator new[](std::size_t) = delete;
};

// 3. Ban calls on temporaries (§6.1)
class Handle {
public:
    T* get() & { return p_; }
    T* get() && = delete;                 // Handle{}.get() would dangle
};

// 4. Ban a whole template instantiation
template <class T> void f(T) = delete;
template <> void f(int) {}                // only int is allowed

```

C++26 adds a reason string: `void f(double) = delete("use the int overload");`

7.3 Member initializer lists and initialization order ⚠

Initialization vs assignment

```

class Widget {
    std::string name_;
    std::vector<int> data_;
public:
    Widget(std::string n) {           // ✗ default-constructs both, THEN
assigns
        name_ = std::move(n);
    }
    Widget(std::string n)             // ✓ constructs directly
        : name_(std::move(n)) {}
};

```

For `std::string` the difference is small; for a type with an expensive default constructor, or one that allocates, it's real. And for two member kinds it's not optional:

```

class C {
    const int x_;           // MUST be in the init list – can't be assigned
    Logger& log_;           // MUST be in the init list – can't be rebound
    NoDefault nd_;         // MUST be in the init list – no default constructor
exists
public:
    C(int x, Logger& l) : x_(x), log_(l), nd_(args) {}
};

```

⚠ Initialization order is declaration order

```

class Bad {
    std::size_t size_;
    std::vector<int> data_;
public:
    Bad(std::size_t n)
        : data_(n), size_(data_.size()) {} // reads like: data_ first, then
size_
};                                         // ACTUALLY: size_ first (declared
first),
                                         // reading data_.size() on an
UNCONSTRUCTED vector

```

Members initialize in **declaration order in the class**, always, regardless of the order you wrote in the initializer list. Bases first (in declaration order), then members, then the constructor body.

-Wreorder (in **-Wall**) catches the common case, so keep it on. The habit that makes it a non-issue: **write the initializer list in the same order as the declarations, and don't initialize one member from another.**

Delegating constructors

```
class Widget {
    std::string name_;
    int size_;
public:
    Widget(std::string n, int s) : name_(std::move(n)), size_(s) { validate(); }
    Widget(std::string n) : Widget(std::move(n), kDefaultSize) {} // delegates
    Widget() : Widget("unnamed") {}
};
```

The delegated-to constructor runs to completion first; then the delegating constructor's body. ⚠ If the delegating constructor's body throws, the object *was* fully constructed, so **its destructor runs** — unlike a throw from the initializer list.

Default member initializers (NSDMI)

```
class Config {
    int      retries_ = 3; // used by any ctor that doesn't
    override it
    std::string host_ = "localhost";
    bool      verbose_ = false;
public:
    Config() = default;
    Config(std::string h) : host_(std::move(h)) {} // retries_ and verbose_ use
    the NSDMI
};
```

This is the cleanest way to guarantee no member is ever left uninitialized. **Use it by default** — the cost is zero and it eliminates a whole UB category (§14).

⚠ NSDMI and aggregates: allowed since C++14, and a class with NSDMIs can still be an aggregate (§7.8).

Inheriting constructors

```
struct Base { Base(int); Base(int, int); };
struct Derived : Base {
    using Base::Base; // inherits both constructors
    int extra_ = 0; // ⚠ NSDMI is essential – inherited ctors don't
    initialize it
};
Derived d{1, 2};
```

Function-try-block

For catching an exception from a member's constructor (rare, and rarely useful):

```
Widget::Widget(int n) try : data_(n) {
    // body
} catch (const std::bad_alloc&) {
    // ⚠️ you CANNOT recover – the exception is automatically rethrown at the
    end.
    // The object was never constructed. All you can do is log or translate the
    exception.
}
```

7.4 **explicit** constructors and conversion operators

The problem

```
class Buffer {
public:
    Buffer(std::size_t n);    // reserve n bytes
};

void process(const Buffer& b);
process(1024);              // compiles! Implicitly builds a 1024-byte Buffer.
Buffer b = 42;             // also compiles
```

Single-argument constructors are **converting constructors** by default. Sometimes that's what you want (`std::string s = "hello"` is a feature); usually it isn't.

```
class Buffer {
public:
    explicit Buffer(std::size_t n);
};

process(1024);              // ERROR – good
process(Buffer{1024});     // explicit, clear
```

explicit on multi-argument constructors

Since C++11, list-initialization can call multi-arg constructors implicitly, so **explicit** matters there too:

```
struct Point { Point(int, int); };
Point p = {1, 2};          // implicit – allowed
void f(Point);
f({1, 2});                 // implicit

struct EPoint { explicit EPoint(int, int); };
EPoint q = {1, 2};         // ERROR
EPoint r{1, 2};           // OK
```

explicit conversion operators

```
class Handle {
    void* p_;
public:
    operator bool() const { return p_; }           // ⚠ implicit – Handle can now
do arithmetic
    explicit operator bool() const { return p_; } // ✓ the right form
};

Handle h;
if (h) { }           // ✓ works – `if`, `while`, `!`, `&&`, `||`, `?:`
do a                 // CONTEXTUAL conversion, which allows explicit

operator bool
int n = h + 1;       // ✗ with explicit: error. Without: h converts to
bool, then int. Yikes.
```

Always make `operator bool` explicit. This is what `unique_ptr`, `optional`, `shared_ptr`, and `istream` do.

explicit(bool) — C++20

Conditional explicitness, used heavily inside the standard library:

```
template <class T>
class Wrapper {
public:
    template <class U>
    explicit(!std::is_convertible_v<U, T>) // explicit only when U->T isn't
implicit
    Wrapper(U&& u);
};
```

The guideline

Make every single-argument constructor **explicit** **unless** the implicit conversion is a deliberate, documented part of the design. Make every `operator bool` explicit. Make other conversion operators explicit unless you have a specific reason.

`clang-tidy: google-explicit-constructor` enforces this.

7.5 `const` member functions, `mutable`, logical vs bitwise const

Mechanics

```
class Widget {
    int value_ = 0;
public:
    int get() const { return value_; } // `this` is `const Widget*`
    void set(int v) { value_ = v; }   // `this` is `Widget*`
};
const Widget w;
w.get();    // OK
w.set(1);   // ERROR
```

const participates in overload resolution, giving the standard accessor pair:

```
class Container {
    std::vector<T> data_;
public:
    T& at(std::size_t i) { check(i); return data_[i]; }
    const T& at(std::size_t i) const { check(i); return data_[i]; }
};
```

De-duplicating the const/non-const pair

The traditional trick (ugly but correct — **const** is added, then cast away only on an object we know is non-const):

```
T& at(std::size_t i) {
    return const_cast<T&>(std::as_const(*this).at(i));
}
```

C++23 **deducing this** removes the duplication entirely:

```
template <class Self>
auto&& at(this Self&& self, std::size_t i) {
    self.check(i);
    return std::forward<Self>(self).data_[i];
}
// One definition. Returns T&, const T&, or T&& depending on how it was called.
```

mutable and logical const

Bitwise const: no bits of the object change. **Logical const**: the object's observable value doesn't change. **mutable** lets you have the second without the first.

```
class Document {
    std::string text_;
```

```

    mutable std::optional<std::size_t> wordCount_; // cache
    mutable std::mutex mtx_;                      // synchronization
public:
    std::size_t wordCount() const {
        std::lock_guard lock(mtx_);              // locking a mutex in a
const method
        if (!wordCount_) wordCount_ = compute(); // memoization
        return *wordCount_;
    }
};

```

Those are the two legitimate uses: **caches** and **synchronization primitives**. `mutable` on anything else usually means the method shouldn't be `const`.

⚠ `const` implies thread-safe-for-reading

The standard library contract is: **it must be safe for multiple threads to call `const` member functions on the same object concurrently**. All standard types honour this, and all standard algorithms assume it.

So the moment you add a `mutable` cache, you have taken on a synchronization obligation:

```

std::size_t wordCount() const {
    if (!wordCount_) wordCount_ = compute(); // ⚠ DATA RACE if called from two
threads
    return *wordCount_;                      // even though the method is
const
}

```

`const` is not a thread-safety mechanism; it's a *promise you must keep*.

⚠ `const` doesn't propagate through pointers

```

class Widget {
    Impl* impl_;
    std::unique_ptr<Impl> up_;
public:
    void f() const {
        impl_->mutate(); // ⚠ legal – const makes `impl_` a `Impl* const`,
        up_->mutate();   // not a `const Impl*`. Same for unique_ptr.
    }
};

```

This surprises people constantly, and it's why pimpl classes accidentally leak mutability. Options:

```

std::experimental::propagate_const<std::unique_ptr<Impl>> impl_; // LFTS v2,
GCC/Clang have it
// or just: provide const and non-const accessors that return const Impl& / Impl&

```


7.6 Static members and inline variables

Static data members

```
// Pre-C++17
class Widget {
    static int count_;           // DECLARATION
};
int Widget::count_ = 0;         // DEFINITION – must be in exactly one .cpp

// C++17
class Widget {
    static inline int count_ = 0;           // declaration + definition,
header-only
    static constexpr int kMax = 100;       // constexpr statics are implicitly
inline
};
```

static inline is one of the highest-value small features of C++17: header-only libraries no longer need a **.cpp** file just to hold their globals (§6.4).

Static member functions

No **this**, callable without an object, can't be **const** or **virtual**:

```
class Widget {
public:
    static Widget fromJson(std::string_view); // named constructor idiom
};
auto w = Widget::fromJson(json);
```

⚠ Static initialization order fiasco

Non-local static objects in **different translation units** have unspecified relative initialization order. If one's constructor uses another, you get a use-before-init.

```
// a.cpp
Logger gLogger;
// b.cpp
Registry gRegistry;           // ctor calls gLogger.log() – may run FIRST. UB.
```

The standard fix — **the Meyers singleton**, a function-local static, which is initialized on first use and (since C++11) **thread-safe**:

```

Logger& logger() {
    static Logger instance;           // constructed on first call, exactly once,
    thread-safely
    return instance;
}

```

The remaining hazard is *destruction* order at exit (the reverse of construction), which can still bite in a threaded shutdown. Common mitigations: leak the singleton deliberately (`static Logger* p = new Logger;`), or use `std::atexit` ordering carefully.

`constexpr` (§6.5) is the other tool: it forces compile-time initialization, so no ordering problem exists for that variable.

7.7 Nested classes and `friend`

Nested classes

```

class Outer {
    int secret_ = 42;
    class Inner {
        void f(Outer& o) { o.secret_; }    // ✓ Inner has access to Outer's
    private
    };
    void g(Inner& i) { i.hidden_; }        // ✗ Outer does NOT have access to
    Inner's privates
};

```

The relationship is one-directional. The most common use is the pimpl forward declaration:

```

class Engine {
    struct Impl;                               // declared, not defined
    std::unique_ptr<Impl> impl_;
};
// in the .cpp:
struct Engine::Impl { /* ... */ };

```

Also: iterator types (`Container::iterator`), and small implementation-detail helpers you don't want at namespace scope.

`friend`

```

class Widget {
    int value_;
    friend class Inspector;                    // whole class
}

```

```
friend void debugPrint(const Widget&);           // one function
friend std::ostream& operator<<(std::ostream&, const Widget&);
};
```

Friendship is not inherited, not transitive, and not reciprocal.

Contrary to instinct, **friend doesn't weaken encapsulation** when the friend is logically part of the class's interface — a non-member `operator<<` or `operator==` needs access, and making it a friend is more encapsulating than adding public getters that everyone can use.

Hidden friends — the idiom worth adopting

Define the friend **inside** the class body. It is then only findable by **ADL** (§6.1), not by ordinary lookup:

```
class Widget {
    int value_;
public:
    friend bool operator==(const Widget& a, const Widget& b) {    // "hidden
friend"
        return a.value_ == b.value_;
    }
    friend std::ostream& operator<<(std::ostream& os, const Widget& w) {
        return os << w.value_;
    }
};
```

Why this is better than a namespace-scope non-member:

- **Not in the overload set** for unrelated calls → smaller candidate sets → **faster compiles** and clearer error messages
- Can't be found by accident through a stray conversion
- Lives next to the class, so it can't get out of sync
- No need for a separate declaration in the header

Both libstdc++ and the C++ Core Guidelines push this style. Note it works only where ADL applies — which covers operators and functions called unqualified with the class as an argument.

7.8 Aggregates and designated initializers

What's an aggregate (C++20 rules)

An array, or a class with:

- no user-**declared** or inherited constructors
- no private or protected **non-static data members**
- no virtual functions
- no virtual, private, or protected base classes

Allowed and still an aggregate: public base classes, default member initializers (since C++14), static members, member functions, private *static* members.

```
struct Config {           // aggregate
    std::string host = "localhost";
    int         port = 8080;
    bool        tls  = true;
};
```

Aggregate initialization

```
Config c1{};                // all NSDMIs / value-init
Config c2{"example.com"};   // rest use NSDMIs
Config c3{"example.com", 443, false};
Config c4 = {"example.com", 443};

int arr[3]{1, 2, 3};
struct Nested { Config inner; int flag; };
Nested n{"h", 1, true, 5};
Nested m{"h", 1, true, 5};   // brace elision – legal, and
                             // confusing
```

⚠ **Narrowing is prohibited** inside braces — one of the main reasons to prefer {}:

```
struct S { int x; };
S s{3.14};           // ERROR – narrowing double->int
S t(3.14);           // (if it had a ctor) silently truncates
```

Designated initializers (C++20)

```
Config c{
    .host = "example.com",
    .tls  = false,           // port keeps its NSDMI
};
```

This is the closest C++ gets to named parameters, and it's the reason "options struct" APIs became pleasant:

```
auto w = createWindow({.width = 1920, .height = 1080, .fullscreen = true});
```

⚠ **C++'s rules are stricter than C's:**

	C	C++20
Out-of-order designators	✓	✗ must be declaration order
Skipping members	✓	✓
Mixing designated and positional	✓	✗
Array designators (<code>[2] = x</code>)	✓	✗
Nested designators (<code>.a.b = x</code>)	✓	✗ (write <code>.a = {.b = x}</code>)

So C code using designated initializers often doesn't compile as C++.

CTAD for aggregates (C++20)

```
template <class T> struct Pair { T first, second; };
Pair p{1, 2};           // deduces Pair<int> – no deduction guide needed in C++20
```

7.9 `this`, returning `*this`, and chaining

```
class Builder {
    std::string name_;
    int size_ = 0;
public:
    Builder& setName(std::string n) { name_ = std::move(n); return *this; }
    Builder& setSize(int s)         { size_ = s;           return *this; }
    Widget build() const           { return Widget{name_, size_}; }
};

auto w = Builder{}.setName("x").setSize(10).build();
```

Return `*this` **by reference** (`Builder&`), not by value — returning by value copies the whole builder on every call.

Ref-qualified builders

The above still works on lvalues, which is usually fine. If you want a builder that can only be used as a temporary, and that moves its state out at the end:

```
class Builder {
    std::string name_;
public:
    Builder&& setName(std::string n) && { name_ = std::move(n); return
std::move(*this); }
    Widget build() && { return Widget{std::move(name_)}; }
};
```

```
auto w = Builder{}.setName("x").build();    // fine
Builder b; b.setName("x");                  // ERROR – can't call && methods on
an lvalue
```

C++23: deducing **this**

Removes the const/non-const and lvalue/rvalue duplication across the board:

```
class Widget {
    std::vector<int> data_;
public:
    template <class Self>
    auto&& data(this Self&& self) {
        return std::forward<Self>(self).data_;
        // Widget&      -> vector<int>&
        // const Widget& -> const vector<int>&
        // Widget&&      -> vector<int>&&
    }
};
```

It also gives lambdas recursion without the Y-combinator (§6.8), and gives CRTP-free static polymorphism (§8).

7.10 Copy-and-swap

Mechanics and cost analysis are in **§5.9**. The class-side pieces:

```
class Buffer {
    char*      data_ = nullptr;
    std::size_t size_ = 0;
public:
    Buffer(const Buffer& o) : data_(new char[o.size_]), size_(o.size_) {
        std::copy_n(o.data_, size_, data_);
    }
    Buffer(Buffer&& o) noexcept
        : data_(std::exchange(o.data_, nullptr)), size_(std::exchange(o.size_, 0))
    {}

    Buffer& operator=(Buffer other) noexcept {    // by VALUE – copy or move
happens here
        swap(*this, other);
        return *this;
    }
    ~Buffer() { delete[] data_; }

    friend void swap(Buffer& a, Buffer& b) noexcept {    // hidden friend (§7.7)
        using std::swap;
        swap(a.data_, b.data_);
        swap(a.size_, b.size_);
    }
```

```
    }
};
```

What you get: one assignment operator instead of two, correct self-assignment for free, and the strong exception guarantee (the allocation happens in the parameter's construction, before `*this` is touched).

⚠ What it costs: move-assignment can no longer reuse existing capacity. `v = std::move(w)` becomes construct-then-swap rather than free-then-steal. For a container in a hot loop, write the two operators separately.

7.11 Empty base optimization and `[[no_unique_address]]`

The problem

An empty class still has `sizeof >= 1`, because distinct objects must have distinct addresses:

```
struct Empty {};
static_assert(sizeof(Empty) == 1);

struct Naive {
    Empty e;           // 1 byte...
    int n;             // ...+ 3 bytes of padding + 4
};
static_assert(sizeof(Naive) == 8);    // 8, for 4 bytes of data
```

EBO — the base-class workaround

An empty **base** class is allowed to contribute zero bytes:

```
struct Better : Empty {
    int n;
};
static_assert(sizeof(Better) == 4);    // Empty vanished
```

This is why so much of the standard library uses private inheritance from policy types: `unique_ptr` inherits from (or otherwise compresses) its deleter, allocator-aware containers compress the allocator, and `std::tuple` implementations inherit from each element type. Boost's `compressed_pair` exists solely to automate it.

```
struct StatelessDeleter { void operator()(T* p) const { delete p; } };
static_assert(sizeof(std::unique_ptr<T, StatelessDeleter>) == sizeof(T*));    //
EBO at work
static_assert(sizeof(std::unique_ptr<T, void(*)>) == 2*sizeof(T*));    //
stateful: no
```

[[no_unique_address]] (C++20)

The member-level version, so you don't have to distort your inheritance hierarchy:

```
template <class T, class Allocator = std::allocator<T>>
class MyVector {
    T* data_ = nullptr;
    std::size_t size_ = 0, cap_ = 0;
    [[no_unique_address]] Allocator alloc_;    // costs zero bytes when
    Allocator is empty
};
static_assert(sizeof(MyVector<int>) == 24);
```

```
struct S {
    [[no_unique_address]] Empty e;
    int n;
};
static_assert(sizeof(S) == 4);
```

⚠ **MSVC:** plain [[no_unique_address]] is accepted but ignored for ABI-compatibility reasons; you need [[msvc::no_unique_address]]. Portable code writes both, or uses a macro.

⚠ Two [[no_unique_address]] members of the *same* empty type can't share an address, so the second one still takes a byte.

Gotcha checklist for this section

- ☐ Declaring a **destructor suppresses the move operations** — copies everywhere, no diagnostic
- ☐ A defaulted move that would be deleted is **ignored by overload resolution** → silently copies
- ☐ **const** and **reference members** delete copy/move assignment and block moves
- ☐ Members initialize in **declaration order**, not initializer-list order (**-Wreorder**)
- ☐ **= default out of line** makes the function user-provided → non-trivial
- ☐ **= default** in the class body needs **complete** member types (pimpl)
- ☐ Single-argument constructors are implicit converting constructors unless marked **explicit**
- ☐ **operator bool** must be **explicit**, or your type does arithmetic
- ☐ **const** methods must be **safe to call concurrently** — a **mutable** cache needs a mutex
- ☐ **const** does **not** propagate through pointer/**unique_ptr** members
- ☐ Non-local statics in different TUs have unspecified init order — use a function-local static
- ☐ C++ designated initializers must be in **declaration order**, can't be mixed with positional, and don't support arrays or nesting (unlike C)
- ☐ Brace elision in nested aggregates compiles but reads badly
- ☐ Return ***this by reference** when chaining
- ☐ A function-try-block on a constructor **cannot swallow** the exception
- ☐ [[no_unique_address]] is ignored by MSVC — use [[msvc::no_unique_address]] too

Flags and tools

```
-Wreorder                # initializer list order != declaration order (in
-Wall)
-Wuninitialized -Wmaybe-uninitialized
-Wnon-virtual-dtor
-Wdeprecated-copy        # implicit copy where a dtor/copy op is declared
-Weffc++                 # GCC: members not in the init list, Rule-of-Three
-Wshadow                 # constructor params shadowing members
-Wextra-semi

clang-tidy:
  cppcoreguidelines-special-member-functions
  cppcoreguidelines-pro-type-member-init      # missing NSDMI / init-list entries
  google-explicit-constructor
  modernize-use-default-member-init
  modernize-use-equals-default / -equals-delete
  performance-move-const-arg
  readability-convert-member-functions-to-static
```

Exercises worth doing

1. Write a class with a `const` member and try to put it in a `std::vector`, then `std::sort` it. Read the error. Now make it non-const and try again.
2. Reproduce the declaration-order bug (`: data_(n), size_(data_.size())`) and run it under ASan/UBSan. Then turn on `-Wreorder`.
3. For five types in your codebase, print `is_trivially_copyable_v`, `is_standard_layout_v`, `is_nothrow_move_constructible_v`. Explain each result.
4. Write a `Matrix` class with `operator()` in both const and non-const forms. Then rewrite it once with the `const_cast` trick and once with C++23 deducing `this`.
5. Measure `sizeof` for: an empty struct; a struct with an empty member; a struct inheriting from empty; a struct with `[[no_unique_address]]`. Repeat with your compiler on both Linux and MSVC if you can.
6. Convert a class's `operator==` and `operator<<` to hidden friends. Measure the compile time before and after on a TU that includes it heavily.
7. Write a `Config` aggregate with NSDMIs and construct it three ways: `{}`, positional, designated. Then try to designate out of order and read the error.

Chapter 8 — Inheritance & Polymorphism

What this chapter covers

- Public/protected/private inheritance (and why non-public is rare)
- **virtual functions, vtables, dynamic dispatch** — and its cost
- **Virtual destructors** — required for polymorphic deletion
- `override`, `final`
- Abstract classes, pure virtual

- **Slicing** — the classic C++ bug
- Multiple inheritance, **virtual inheritance**, the diamond
- `dynamic_cast`, RTTI, `typeid` — and why to avoid them
- **Prefer composition; prefer templates/concepts to runtime polymorphism** where you can
- CRTP (static polymorphism)
- **Type erasure** — how `std::function` and `std::any` work

The part of C++ that most languages copied, and the part modern C++ uses least. Inheritance is still the right tool for open sets of types behind a stable interface — plugins, ABI boundaries, heterogeneous containers. For everything else, C++ gives you three cheaper alternatives (templates, `std::variant`, type erasure), and knowing when *not* to reach for `virtual` is most of the skill.

8.1 Public, protected, private inheritance

The three meanings

Form	Means	Frequency
<code>class D : public B</code>	is-a — a <code>D</code> can be used anywhere a <code>B</code> is expected (LSP)	~99% of real uses
<code>class D : private B</code>	implemented-in-terms-of — <code>B</code> is an implementation detail	rare
<code>class D : protected B</code>	same, but derived classes also see it	essentially never

`struct` defaults to `public` inheritance; `class` defaults to `private`. Another reason to write the keyword explicitly.

What happens to members

Member in Base	<code>public</code> inheritance	<code>protected</code>	<code>private</code>
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	inaccessible	inaccessible	inaccessible

Private members of the base are *never* accessible to a derived class, under any inheritance kind. (`friend` is the only way in.)

Why non-public inheritance is rare

Private inheritance and composition express the same thing, and composition is clearer:

```
class Timer { public: virtual void onTick(); };

class Widget : private Timer { void onTick() override; }; // private inheritance
```

```
class Widget { Timer timer_; }; // composition – usually better
```

Composition wins on almost every axis: no name collisions, no accidental conversions, you can hold several, you can swap the implementation at runtime, and the relationship is visible in the member list.

Private inheritance earns its keep in exactly three situations:

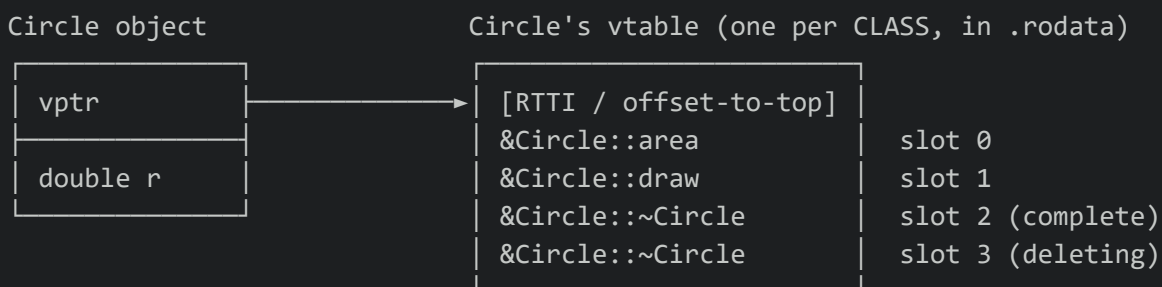
1. **You must override a virtual function** of the base (composition can't do this)
2. **You need access to protected members**
3. **Empty base optimization** (§7.11) — an empty policy/allocator/deleter class costs zero bytes as a base, one byte as a member (though `[[no_unique_address]]` mostly retired this reason in C++20)

8.2 virtual functions, vtables, and dispatch cost

The mechanism

```
struct Shape {
    virtual double area() const = 0;
    virtual void draw() const = 0;
    virtual ~Shape() = default;
};
struct Circle : Shape {
    double r;
    double area() const override { return 3.14159 * r * r; }
    void draw() const override;
};
```

Layout, on every mainstream ABI:



The **slot index is fixed at compile time** — `s->area()` compiles to "load vptr, load slot 0, call". The lookup is not a search.

```
mov rax, [rdi] ; load vptr
call [rax] ; indirect call through slot 0
```

What it actually costs

Cost	Detail
+8 bytes per object	one vptr, regardless of how many virtual functions
Two dependent loads + an indirect call	a handful of cycles when the branch predictor gets it right; a full pipeline flush (~15-20 cycles) when it doesn't
⚠ No inlining	the real cost — and with it, no constant propagation, no vectorization across the call, no dead-code elimination
Not trivially copyable	breaks <code>memcpy</code> relocation, <code>std::bit_cast</code> , <code>std::atomic<T></code>
Not standard layout	breaks <code>offsetof</code> and C interop (§25)
Cache pressure	the vptr occupies space in every object; the vtable is another cache line

The indirect call is cheap. **The lost inlining is expensive.** A one-line virtual `getValue()` called in a tight loop can be 10-50x slower than the non-virtual version, not because of the call but because the optimizer can't see through it.

Devirtualization

Compilers will remove the indirection when they can prove the dynamic type:

```
Circle c;
c.area(); // devirtualized – static type is known exactly

struct Circle final : Shape { ... };
Shape* s = getCircle();
static_cast<Circle*>(s)->area(); // `final` proves nothing derives further ->
direct call
```

Levers, in order of effectiveness: **final on the class or function**, **LTO** (whole-program visibility), **PGO** (speculative devirtualization — the compiler emits `if (vptr == &Circle_vtable) direct_call(); else indirect();`), and `-fwhole-program`.

⚠ Virtual calls in constructors and destructors

During **Base**'s constructor, the object **is** a **Base** — the vptr hasn't been updated to **Derived** yet. So:

```
struct Base {
    Base() { init(); } // ⚠ calls Base::init, NOT
Derived::init
    virtual void init() { print("Base"); }
};
struct Derived : Base {
    std::string data_ = "ready";
    void init() override { print(data_); } // never runs from the ctor –
```

```
};                                     // and data_ isn't even constructed
yet

Derived d;    // prints "Base"
```

This is *correct* and *deliberate* — calling `Derived::init` would touch uninitialized members. It's still a bug magnet. If the base is abstract and the constructor calls a **pure** virtual, you get UB (in practice, a link error or `__cxa_pure_virtual` → `terminate`).

The same applies in destructors, in reverse: by the time `~Base` runs, the derived part is already gone.

Rule: never call a virtual function from a constructor or destructor. If you need two-phase init, use a factory function that constructs then calls `init()`.

Other virtual gotchas

```
Base* p = &derived;
p->Base::f();           // explicitly call the base version – suppresses dispatch

// Default arguments on virtuals are statically bound (§6.2) – never do it
virtual void draw(Color c = Color::Red);
```

8.3 ⚠ Virtual destructors

The rule

```
struct Base { ~Base(); };               // NOT virtual
struct Derived : Base { std::vector<int> big_; };

Base* p = new Derived;
delete p;                               // UNDEFINED BEHAVIOUR
                                         // in practice: ~Derived never runs,
big_ leaks
```

If a class will be deleted through a pointer to it, its destructor must be virtual.

The two correct forms:

```
// A: public virtual – polymorphic deletion allowed
struct Base {
    virtual ~Base() = default;
    // ⚠ this just suppressed the move operations – see §4.9
};

// B: protected non-virtual – polymorphic deletion FORBIDDEN, no vtable cost for
the dtor
```

```
struct Interface {
    virtual void doIt() = 0;
protected:
    ~Interface() = default;           // can't `delete interfacePtr` at all
};
```

Form B is right for pure interfaces that are never owned through the base pointer — common in embedded and in performance code where the extra vtable slot matters.

The `shared_ptr` exception (worth knowing)

```
struct Base { ~Base(); };           // non-virtual!
struct Derived : Base { ... };

std::unique_ptr<Base> u = std::make_unique<Derived>(); // ⚠ UB on destruction
std::shared_ptr<Base> s = std::make_shared<Derived>(); // ✓ CORRECT — ~Derived
runs
```

`shared_ptr` **type-erases the deleter** at construction (§4.4), so the control block remembers it was built from a `Derived` and calls the right destructor. `unique_ptr`'s deleter is part of its type and only knows `Base`.

This is a genuine difference, not a technicality — but don't rely on it. Give polymorphic bases a virtual destructor and stop thinking about it.

Detection

```
-Wnon-virtual-dtor      # any polymorphic class with a non-virtual public dtor
-Wdelete-non-virtual-dtor # the actual delete site (in -Wall on Clang)
```

8.4 `override` and `final`

`override`

```
struct Base { virtual void f(int) const; };

struct Derived : Base {
    void f(int) const override; // ✓ verified
    void f(long) const override; // ✗ ERROR — caught. Without `override`:
    silent new function
    void f(int) override; // ✗ ERROR — const mismatch. Caught.
};
```

Without `override`, all three compile, two of them create new non-overriding functions, and calls through `Base*` silently go to the base version. **Always write `override`**. It costs nothing and catches signature drift

when the base changes.

Style: write **override** **without** **virtual** — the two together are redundant, and `-Wsuggest-override / modernize-use-override` will tell you.

final

```
struct Derived : Base {
    void f(int) const final;           // no further overriding
};
struct Leaf final : Derived { };      // nothing can derive from Leaf
```

Two reasons to use it: expressing design intent, and **enabling devirtualization** (§8.2). Marking leaf classes **final** is a cheap, safe optimization in hot code.

⚠ It's also a compatibility constraint — **final** on a widely used base breaks anyone extending it, including test doubles.

Covariant return types

An override may return a *more derived* pointer or reference:

```
struct Shape { virtual Shape* clone() const = 0; };
struct Circle : Shape { Circle* clone() const override; }; // ✓ covariant
```

⚠ Covariance works for raw pointers and references only — **not** for `unique_ptr<Derived>` vs `unique_ptr<Base>`. The usual workaround is the NVI pattern:

```
struct Shape {
    std::unique_ptr<Shape> clone() const { return std::unique_ptr<Shape>
(doClone()); }
private:
    virtual Shape* doClone() const = 0;
};
```

8.5 Abstract classes and pure virtual functions

```
struct Shape {
    virtual double area() const = 0;           // pure virtual -> Shape is abstract
    virtual ~Shape() = default;
};
Shape s;                                     // ERROR - cannot instantiate
Shape* p = new Circle; // fine
```

Pure virtual functions can have a definition

Surprising but occasionally useful — a default implementation derived classes must *opt into* explicitly:

```
struct Base {
    virtual void log() const = 0;
};
void Base::log() const { print("default"); } // legal!

struct Derived : Base {
    void log() const override { Base::log(); print("+ extra"); }
};
```

⚠ And it's **mandatory** for a pure virtual destructor:

```
struct AbstractBase {
    virtual ~AbstractBase() = 0; // makes the class abstract with no other pure
    virtuals
};
AbstractBase::~~AbstractBase() {} // MUST be defined – derived destructors call
it
```

Interface classes

```
class Serializer {
public:
    virtual std::string serialize(const Data&) const = 0;
    virtual Data deserialize(std::string_view) const = 0;
    virtual ~Serializer() = default;
};
```

No state, no non-virtual functions, all pure virtual. This is the shape you use for dependency injection, plugin boundaries, and mocking (§22).

The NVI idiom (non-virtual interface)

Public functions are non-virtual; the customization points are private virtuals:

```
class Task {
public:
    void run() { // public, non-virtual: owns the invariants
        log("starting");
        auto t = Timer{};
        doRun(); // the actual customization point
        log("done in", t.elapsed());
    }
```



```

    virtual ~Task() = default;
private:
    virtual void doRun() = 0;           // private virtual – derived classes CAN
override                               // even though they can't call it
};

```

What this buys you: pre/post conditions the derived class can't skip, a stable public signature, a place to put default arguments (§6.2), and the ability to add logging/locking/instrumentation in one spot. It's how the standard library's `std::streambuf` is designed.

8.6 Slicing ⚠

The bug

```

struct Base    { int a = 1; virtual void f(); };
struct Derived : Base { int b = 2; void f() override; };

Derived d;
Base    b = d;      // SLICED – only the Base subobject is copied. b.f() calls
Base::f.

```

The copy constructor being invoked is `Base::Base(const Base&)`. It knows nothing about `b` the member, and the result's vptr is `Base`'s. There is no error and no warning by default.

Where it shows up

```

void process(Base b);           // ⚠ by value – every call slices
process(derived);

std::vector<Base> shapes;       // ⚠ can only ever hold Base objects
shapes.push_back(circle);       // sliced

base = derived;                 // ⚠ assignment slices too

Base& r = derived;
Base copy = r;                  // ⚠ sliced – even though r refers to a Derived

```

Fixes

```

// 1. Never pass polymorphic types by value
void process(const Base& b);
void process(const Base* b);

// 2. Store pointers in containers
std::vector<std::unique_ptr<Base>> shapes;

```

```
// 3. Make the base abstract – an abstract class can't be instantiated, so it
// can't be sliced
struct Base { virtual void f() = 0; virtual ~Base() = default; };
Base b = d; // now a compile error

// 4. Delete copy/move on the base (the strongest form)
struct Base {
    virtual ~Base() = default;
    Base(const Base&) = delete;
    Base& operator=(const Base&) = delete;
protected:
    Base() = default;
};

// 5. If you genuinely need polymorphic copying, use clone()
struct Base {
    virtual std::unique_ptr<Base> clone() const = 0;
};
```

Options 3 and 4 are the ones to reach for by default: **make slicing impossible rather than remembering not to do it.**

```
-Wextra # some slicing warnings
clang-tidy: cppcoreguidelines-slicing
```

8.7 Multiple inheritance, virtual inheritance, the diamond

Multiple inheritance of interfaces — fine

```
class Serializable { public: virtual std::string toJson() const = 0; virtual
~Serializable() = default; };
class Drawable { public: virtual void draw() const = 0; virtual
~Drawable() = default; };

class Widget : public Serializable, public Drawable { ... };
```

This is uncontroversial and is what Java/C# allow. Each base contributes a vptr, so `sizeof(Widget)` has two of them, and converting `Widget*` to `Drawable*` **adjusts the pointer** by a fixed offset:

```
Widget* w = &widget;
Drawable* d = w; // pointer value CHANGES (adjusted by 8 or so)
assert((void*)w != (void*)d); // this assertion PASSES – surprising the first
time
```

The compiler also emits **thunks**: small stubs that fix up **this** before jumping to the real function. This is why **reinterpret_cast** between base and derived pointers in a multiple-inheritance hierarchy is UB while **static_cast** is fine.

The diamond

```
struct Animal { std::string name; };
struct Mammal : Animal {};
struct Bird   : Animal {};
struct Bat    : Mammal, Bird {};      // TWO Animal subobjects

Bat b;
b.name;                             // ERROR – ambiguous
b.Mammal::name;                     // disambiguated, but there really are two
names
```

Virtual inheritance

```
struct Animal { std::string name; };
struct Mammal : virtual Animal {};
struct Bird   : virtual Animal {};
struct Bat    : Mammal, Bird {};      // ONE shared Animal subobject

Bat b;
b.name;                             // ✓ unambiguous
```

⚠ **The rule that trips everyone up: the most-derived class initializes the virtual base**, bypassing the intermediate classes' initializer lists.

```
struct Animal { Animal(std::string n); };
struct Mammal : virtual Animal { Mammal() : Animal("mammal") {} };
struct Bird   : virtual Animal { Bird()   : Animal("bird")   {} };

struct Bat : Mammal, Bird {
    Bat() : Animal("bat"), Mammal(), Bird() {} // Bat MUST initialize Animal
    itself
};                                           // Mammal's and Bird's
Animal(...) calls are IGNORED
```

If **Animal** has no default constructor and **Bat** forgets to list it, you get a compile error — often in code where the author never wrote **virtual** and doesn't know why they're being asked.

The costs

- An extra pointer/offset per virtual base, consulted on **every** member access through it
- **static_cast** from a virtual base to a derived class is **illegal** (you must use **dynamic_cast**)

- Layout is no longer a simple concatenation → worse for interop, serialization, and constexpr
- Construction is slower and more complex

Guidance

Multiple inheritance from pure-virtual interfaces: fine, common, use it. Multiple inheritance from classes with state: avoid. Virtual inheritance: avoid unless you're implementing something like `istream` (`basic_istream` really does need the diamond).

If you find yourself wanting stateful MI, you almost always want composition plus delegation.

8.8 `dynamic_cast`, RTTI, `typeid`

`dynamic_cast`

```
Base* b = getSomething();

if (auto* d = dynamic_cast<Derived*>(b)) { d->derivedOnly(); } // null on
failure
Derived& r = dynamic_cast<Derived&>(*b);                      // throws
std::bad_cast

void* most = dynamic_cast<void*>(b);                          // address of the most-derived object
```

Requires a **polymorphic** type (at least one virtual function) — that's where the type information lives. Can also cross-cast between sibling bases in a multiple-inheritance hierarchy, which `static_cast` cannot.

`typeid` and `type_info`

```
#include <typeinfo>

const std::type_info& t = typeid(*b);    // DYNAMIC type (b must be polymorphic)
const std::type_info& u = typeid(Base);  // static

t.name();          // implementation-defined, MANGLED on GCC/Clang – pipe through
c++filt
t.hash_code();
t == u;

std::type_index idx{t};                  // hashable/comparable wrapper for use
in maps
std::unordered_map<std::type_index, Handler> handlers;
```

⚠ `typeid(*p)` where `p` is null throws `std::bad_typeid`. And for a polymorphic operand, `typeid` **evaluates** its argument (unlike `sizeof`), so `typeid(*fn())` calls `fn()`.

Cost

`dynamic_cast` is not a pointer comparison. On the Itanium ABI it walks the inheritance graph, and in the general cross-DSO case it may compare type-name **strings**. Measured cost is typically tens of nanoseconds — hundreds of times a virtual call. In a hot loop it's a disaster.

`-fno-rtti`

Games, embedded, and some HFT codebases disable RTTI entirely (`-fno-rtti`, MSVC `/GR-`). What breaks:

- `dynamic_cast` (except the `void*` form on some compilers)
- `typeid`
- `std::any` (needs `type_info` to retrieve)
- `std::function::target()` / `target_type()`
- Some exception matching corner cases in mixed-RTTI builds

Codebases that do this substitute a hand-rolled type ID: a static `int` or string per class, returned by a virtual `typeId()`. That's LLVM's approach (`isa<>` / `dyn_cast<>` / `cast<>` on top of a manual kind enum) and it's both faster and greppable.

Why to avoid it

```
void render(Shape* s) {
    if (auto* c = dynamic_cast<Circle*>(s)) { drawCircle(*c); }
    else if (auto* r = dynamic_cast<Rect*>(s)) { drawRect(*r); }
    else if (auto* t = dynamic_cast<Tri*>(s)) { drawTri(*t); }
}
```

A `dynamic_cast` chain is a switch on type, which means one of three things:

1. **You're missing a virtual function.** Add `draw()` to `Shape` and the chain disappears.
2. **The set of types is closed**, so it should be a `std::variant` + `std::visit` — which gets you exhaustiveness checking and no allocation.
3. **You're at a boundary** (deserialization, a plugin API, a visitor over a foreign hierarchy) — this is the legitimate case.

8.9 Prefer composition; prefer static polymorphism

Is-a, really?

Public inheritance is a **substitutability** claim (Liskov): every place that works with a `Base` must work with your `Derived`. The classic counterexample:

```
struct Rectangle { virtual void setWidth(int); virtual void setHeight(int); };
struct Square : Rectangle { ... }; // setWidth must also change the height ->
breaks
// any code that assumed the two are
independent
```

A **Square** **is** a rectangle mathematically and **is not** one behaviourally. Inheritance encodes behaviour, not taxonomy.

The four ways to be polymorphic

Approach	Set of types	Dispatch	Cost	Value semantics
Virtual functions	open (anyone can derive)	runtime, vtable	vp _{tr} + indirect call + no inlining	X (pointers)
Templates / concepts	open, compile-time	static	zero ; code bloat, header-only	✓
std::variant + visit	closed (fixed list)	runtime, jump table	no allocation; size = largest member	✓
Type erasure (§8.11)	open	runtime, vtable	allocation + indirect call	✓

std::variant — the underused one

When the set of types is fixed and known, this is almost always better than inheritance:

```
using Shape = std::variant<Circle, Rectangle, Triangle>;

double area(const Shape& s) {
    return std::visit([](const auto& x) { return x.area(); }, s);
}

std::vector<Shape> shapes; // ✓ contiguous storage, no heap, no vptr
shapes.push_back(Circle{1.0});
```

What you gain: value semantics (copyable, comparable, no **clone()**), contiguous storage, no allocation, **exhaustiveness checking** (add a type and every non-generic **visit** fails to compile), and inlinable dispatch.

What you give up: the type list is closed and lives in one header, so users can't add types, and every TU recompiles when it changes.

Combine with the **overloaded** helper from §6.8 for per-type handling.

Composition + strategy

```
// Instead of: class Widget : private CompressionPolicy { ... };
class Widget {
    std::function<Data(Data)> compress_; // runtime-swappable
};

// or, zero-cost, compile-time:
```

```
template <class Compressor>
class Widget {
    [[no_unique_address]] Compressor compress_;
};
```

When runtime polymorphism *is* right

- **Plugins / dynamically loaded modules** — the type list genuinely isn't known
- **Stable ABI boundaries** — a virtual interface is the only reliably stable C++ ABI shape (§20)
- **Heterogeneous containers with an open type set**
- **Mocking in tests** where you need to substitute at link/run time (§22)
- **Reducing compile times** — one virtual interface beats instantiating a template for 40 types

Don't let "prefer composition" become dogma. Inheritance is a tool with a specific shape; use it where the shape fits.

8.10 CRTP — static polymorphism

Curiously Recurring Template Pattern: the base is templated on the derived class, so it can call into it without a virtual function.

```
template <class Derived>
class ShapeBase {
public:
    double area() const {
        return static_cast<const Derived*>(*this).areaImpl(); // resolved at
        compile time
    }
    void describe() const { print("area is", area()); } // shared code,
    fully inlined
};

class Circle : public ShapeBase<Circle> {
    friend ShapeBase<Circle>;
    double r_;
    double areaImpl() const { return 3.14159 * r_ * r_; }
};

Circle c;
c.describe(); // ZERO indirection – the whole thing inlines to a multiply
```

What it's used for

```
// 1. Mixins that need the derived type
class Session : public std::enable_shared_from_this<Session> { }; // the STL's
own CRTP
```

```
// 2. Generating boilerplate operators
template <class D> struct Comparable {
    friend bool operator!=(const D& a, const D& b) { return !(a == b); }
    friend bool operator> (const D& a, const D& b) { return b < a; }
    // ... (Boost.Operators did this; C++20's operator<= made it obsolete, §9)
};

// 3. Static instance counting
template <class D> struct Counted {
    static inline int count = 0;
    Counted() { ++count; }
    ~Counted() { --count; }
};

// 4. Expression templates (Eigen) – the heavyweight use
```

Limits

- **No common base type.** `ShapeBase<Circle>` and `ShapeBase<Square>` are unrelated, so you cannot store them in one container or pass them through one function signature. If you need that, you need real inheritance or type erasure.
- **Code bloat** — one instantiation per derived type (§10)
- **Ugly diagnostics**
- The `static_cast<Derived>(*this)` is UB if someone writes `class Bad : ShapeBase<SomethingElse>` — constrain it if you care

C++23 makes much of it unnecessary

Deducing `this` (§7.9) gives you the derived type without the template parameter:

```
struct ShapeBase {
    template <class Self>
    double area(this const Self& self) { return self.areaImpl(); }
    template <class Self>
    void describe(this const Self& self) { print("area is", self.area()); }
};
struct Circle : ShapeBase { double areaImpl() const; };    // no <Circle>!
```

Same zero cost, none of the recursion. Where the compiler support exists, prefer it.

8.11 Type erasure

What it is

Runtime polymorphism without inheritance, with value semantics. The user's type doesn't derive from anything; the wrapper synthesizes the dispatch. `std::function`, `std::any`, `shared_ptr`'s deleter, and `std::pmr` all work this way.


```

struct Circle { void draw(std::ostream& os) const; }; // derives from NOTHING
struct Text   { void draw(std::ostream& os) const; };

std::vector<Drawable> things; // holds both, by value, copyable
things.push_back(Circle{});
things.push_back(Text{});

```

How it works (the concept/model idiom)

```

class Drawable {
    // 1. The internal interface – the "concept"
    struct Concept {
        virtual ~Concept() = default;
        virtual void draw(std::ostream& os) const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };

    // 2. The adapter – the "model", one instantiation per stored type
    template <class T>
    struct Model final : Concept {
        T obj_;
        explicit Model(T o) : obj_(std::move(o)) {}
        void draw(std::ostream& os) const override { obj_.draw(os); }
        std::unique_ptr<Concept> clone() const override {
            return std::make_unique<Model>(*this);
        }
    };

    std::unique_ptr<Concept> self_;

public:
    // 3. The templated constructor is the only thing the user touches
    template <class T>
    Drawable(T obj) : self_(std::make_unique<Model<std::decay_t<T>>>
(std::move(obj))) {}

    Drawable(const Drawable& o) : self_(o.self_->clone()) {} // value semantics
    Drawable(Drawable&&) noexcept = default;
    Drawable& operator=(Drawable o) noexcept { self_ = std::move(o.self_); return
*this; }

    void draw(std::ostream& os) const { self_->draw(os); }
};

```

The inheritance is **hidden inside the wrapper**. Users see a regular, copyable value type.

(Sean Parent's "Inheritance Is The Base Class of Evil" talk is the canonical presentation of this, and it's worth the 25 minutes.)

How `std::function` differs

Same structure, plus a **small buffer optimization**:

```
// conceptually
class function {
    alignas(max_align_t) char buffer_[16]; // small callables live HERE — no
    allocation
    Concept* self_; // points into buffer_, or to the
    heap
};
```

A lambda capturing one pointer fits inline; one capturing a `std::string` (32 bytes) allocates. That's the cost model from §6.10.

Modern high-performance implementations (folly, LLVM) skip the vtable entirely and store a **manual function-pointer table**, which gives more control over layout and avoids RTTI:

```
struct VTable {
    void (*draw)(const void* obj, std::ostream&);
    void (*destroy)(void* obj);
    void (*copy)(const void* src, void* dst);
};
template <class T>
inline constexpr VTable vtableFor{
    [](const void* o, std::ostream& os) { static_cast<const T*>(o)->draw(os); },
    [](void* o) { static_cast<T*>(o)->~T(); },
    [](const void* s, void* d) { ::new (d) T(*static_cast<const T*>(s)); },
};
```

`std::any`

Type erasure with **no** interface at all — it stores anything copy-constructible and gives it back only if you name the exact type:

```
#include <any>

std::any a = 42;
a = std::string("hello");

if (auto* p = std::any_cast<std::string>(&a)) { use(*p); } // pointer form:
null on mismatch
auto s = std::any_cast<std::string>(a); // value form:
throws std::bad_any_cast
a.type() == typeid(std::string); // ⚠ requires
RTTI
a.has_value();
a.reset();
```

⚠ `any_cast` requires an **exact** type match — no derived-to-base, no conversions. `std::any_cast<int>` on an `any` holding a `long` throws.

Use `std::any` sparingly: it's for genuinely heterogeneous storage where no common interface exists (config values, plugin data blobs). If you know the type set, use `std::variant`; if you know the interface, use type erasure or a virtual base.

Cost summary

	Allocation	Indirect call	Inlinable	Value semantics
Virtual base	user's (<code>make_unique</code>)	yes	no	✗
Type erasure	yes (unless SBO fits)	yes	no	✓
<code>std::variant</code>	none	jump table	often	✓
Template	none	none	yes	✓

Gotcha checklist for this section

- ☐ **Non-virtual destructor + `delete` through a base pointer = UB**; `shared_ptr` is the one exception
- ☐ `virtual ~Base() = default`; suppresses the move operations (§4.9)
- ☐ **Virtual calls in constructors/destructors dispatch to the current class**, not the derived one
- ☐ Pure virtual call from a constructor = UB / `terminate`
- ☐ Default arguments on virtual functions are **statically** bound (§6.2)
- ☐ Without `override`, a signature typo silently creates a new non-overriding function
- ☐ Covariant returns work for `T*/T&`, **not** for `unique_ptr<T>`
- ☐ Passing a polymorphic type **by value slices** it — silently
- ☐ `std::vector<Base>` can never hold derived objects
- ☐ A pure virtual destructor still needs a **definition**
- ☐ With multiple inheritance, `Base*` and `Derived*` have **different addresses** — never `reinterpret_cast`
- ☐ With virtual inheritance, the **most-derived class initializes the virtual base**
- ☐ `static_cast` from a virtual base to derived is **illegal** — you need `dynamic_cast`
- ☐ `typeid(*p)` with null `p` throws `bad_typeid`; `typeid` **evaluates** polymorphic operands
- ☐ A `dynamic_cast` chain means a missing virtual function or a missing `std::variant`
- ☐ CRTP bases of different derived types share **no common type** — can't be stored together

Flags and tools

```
-Wnon-virtual-dtor
-Wdelete-non-virtual-dtor
-Woverloaded-virtual           # a derived function hides a virtual base overload (§6.1)
-Wsuggest-override / -Winconsistent-missing-override
-Wextra                       # some slicing diagnostics
```

```

-fno-rtti                # if your domain forbids RTTI
-flto                    # enables cross-TU devirtualization

clang-tidy:
  cppcoreguidelines-virtual-class-destructor
  cppcoreguidelines-slicing
  modernize-use-override
  bugprone-virtual-near-miss          # "you probably meant to override this"
  cppcoreguidelines-pro-type-static-cast-downcast

```

Exercises worth doing

1. Build a **Shape** hierarchy and print **sizeof** for each class. Add a second virtual base and watch the size grow. Confirm **(void*)widgetPtr != (void*)drawablePtr**.
2. Write a virtual **getValue()** and a non-virtual one, call each 10 million times in a loop, and benchmark. Then mark the class **final** and re-measure. Look at the assembly at each step.
3. Reproduce the constructor-calls-virtual bug and trace it with print statements.
4. Delete a **Derived** through a **Base*** with a non-virtual destructor, under ASan. Then do the same through **shared_ptr<Base>** and confirm it's correct.
5. Take a **dynamic_cast** chain and rewrite it twice: once by adding a virtual function, once with **std::variant + std::visit**. Compare the assembly and the ergonomics.
6. Implement the **Drawable** type-erasure wrapper above from scratch. Then add a small-buffer optimization for types under 32 bytes.
7. Write a CRTP mixin that adds **operator!=**, **operator>**, **operator<=**, **operator>=** from **==** and **<**. Then delete all of it and replace it with **operator<=>** (§9).

Chapter 9 — Operator Overloading

What this chapter covers

- Which operators, and the member-vs-free decision
- Arithmetic, comparison, subscript, call, conversion
- **operator<=>** (**spaceship, C++20**) and **= default** comparisons
- Rewritten and reversed candidates
- **operator*/->** for smart pointer-like types
- User-defined literals
- ⚠ Don't be clever — surprise is worse than verbosity

A short section with one big feature in it. Most of operator overloading is mechanical — learn the canonical forms and follow them. The interesting parts are C++20's **operator<=>**, which deleted about 80% of the boilerplate, and the **rewritten candidate** rules that came with it, which quietly broke some existing code.

9.1 What can be overloaded

Overloadable

```
+ - * / % ^ & | ~ ! = < > += -= *= /= %= ^= &= |=
<< >> >>= <<= == != <= >= <=> && || ++ -- , ->* -> () []
new new[] delete delete[] co_await
```

Not overloadable

Operator	Why
::	scope resolution, not a runtime operation
. and .*	member access — proposals exist, none accepted
?:	ternary
sizeof, alignof, typeid	compile-time queries
#, ##	preprocessor, not operators

Hard rules

- **You cannot invent new operators** (** for power, <> for anything)
- **You cannot change arity, precedence, or associativity.** `a + b * c` always groups as `a + (b * c)`, whatever your `operator*` does
- **At least one operand must be a user-defined type** — you can't redefine `int + int`
- Overloads for `=`, `[]`, `()`, `->`, `->*` and conversion operators **must be members**
- `new/delete` overloads are implicitly `static` members

9.2 Member vs free — the decision

The mechanical difference

A member operator's **left operand is *this**, so no implicit conversion is applied to it. A free operator applies conversions symmetrically to both sides.

```
class Money {
    long cents_;
public:
    Money(long cents);           // implicit, for the sake of
the example                     // the example
    Money operator*(int n) const; // MEMBER
};

Money m{100};
m * 2;      // ✓ works
2 * m;      // ✗ ERROR – int has no operator*(Money)
```

With a free function:

```
Money operator*(const Money& a, int n);
Money operator*(int n, const Money& a);           // both directions available
```

The rule

Symmetric binary operators → **free functions** (arithmetic, comparison, `<<`, `>>`). **Asymmetric or mutating operators** → **members** (`=`, `+=`, `[]`, `()`, `->`, `++`, unary `-`).

And when a free operator needs private access, make it a **hidden friend** (§7.7) — defined inside the class body, findable only by ADL:

```
class Money {
    long cents_;
public:
    Money& operator+=(const Money& o) { cents_ += o.cents_; return *this; }

    friend Money operator+(Money a, const Money& b) { a += b; return a; }
    friend bool operator==(const Money&, const Money&) = default;
    friend std::ostream& operator<<(std::ostream& os, const Money& m);
};
```

Faster to compile, smaller overload sets, better error messages, and it lives next to the class.

9.3 Arithmetic operators — canonical forms

The `+=` / `+` pairing

Implement the mutating version, then define the non-mutating one in terms of it:

```
class Vec {
    double x_, y_;
public:
    Vec& operator+=(const Vec& o) { x_ += o.x_; y_ += o.y_; return *this; }
    Vec& operator-=(const Vec& o) { x_ -= o.x_; y_ -= o.y_; return *this; }
    Vec& operator*=(double s)      { x_ *= s; y_ *= s; return *this; }

    // note: lhs BY VALUE — the copy is elided when the caller passes a temporary,
    //          so `a + b + c` doesn't allocate an extra temporary at each step
    friend Vec operator+(Vec a, const Vec& b) { a += b; return a; }
    friend Vec operator-(Vec a, const Vec& b) { a -= b; return a; }
    friend Vec operator*(Vec a, double s)     { a *= s; return a; }
    friend Vec operator*(double s, Vec a)     { a *= s; return a; }

    Vec operator-() const { return {-x_, -y_}; } // unary negate
};
```

Operator	Returns	Takes
<code>a += b</code>	<code>T&(*this)</code>	<code>const T&</code>
<code>a + b</code>	<code>T</code> by value	lhs by value , rhs <code>const T&</code>
<code>-a</code> (unary)	<code>T</code> by value	member, <code>const</code>

Increment and decrement

```
class Iter {
public:
    Iter& operator++()    { ++p_; return *this; }           // PREFIX
    Iter operator++(int) { Iter tmp = *this; ++*this; return tmp; } // POSTFIX
    (dummy int)
};
```

The `int` parameter is an unused tag that distinguishes the two. Postfix must copy, so **prefer prefix** in loops on non-trivial types — and always implement postfix in terms of prefix.

Bitwise operators for flag enums

`enum class` gives you type safety but loses the bitwise operators. Restore them explicitly:

```
enum class Perm : unsigned { None = 0, Read = 1, Write = 2, Exec = 4 };

constexpr Perm operator|(Perm a, Perm b) {
    return static_cast<Perm>(std::to_underlying(a) | std::to_underlying(b)); //
C++23
}

constexpr Perm operator&(Perm a, Perm b) { /* ... */ }
constexpr Perm& operator|=(Perm& a, Perm b) { return a = a | b; }
constexpr bool hasFlag(Perm set, Perm f) { return (set & f) == f; }

auto p = Perm::Read | Perm::Write;
```

(Pre-C++23, `static_cast<std::underlying_type_t<Perm>>(a)` instead of `std::to_underlying(a)`.)

9.4 Subscript, call, and conversion

`operator[]`

```
class Buffer {
    std::vector<std::byte> data_;
public:
    std::byte& operator[](std::size_t i) { return data_[i]; }
```

```
const std::byte& operator[](std::size_t i) const { return data_[i]; }
};
```

Convention, inherited from the standard library: **operator[]** **does not bounds-check; at() throws**. Follow it, or you'll surprise every reader. (Hardened STL modes — §23 — add checks to **operator[]** in debug builds; that's the sanctioned way to get both.)

C++23 additions:

```
// Multidimensional subscript
class Matrix {
public:
    double& operator[](std::size_t r, std::size_t c) { return data_[r * cols_ +
c]; }
};
m[2, 3]; // C++23 – replaces the operator()(r,c) workaround

// Static operator[] – no `this` needed
struct Table { static constexpr int operator[](int i) { return kData[i]; } };
```

⚠ **Proxy return types.** When **operator[]** can't return a real reference, it returns a proxy — and proxies interact badly with **auto**:

```
std::vector<bool> v(10);
auto x = v[0]; // x is std::vector<bool>::reference, NOT bool
v[0] = true;
// x may now be true – it's a reference into the bit buffer, not a copy
bool y = v[0]; // ✓ explicit type forces the conversion
```

This is **std::vector<bool>**'s original sin (§11), and the reason to be cautious writing proxy-returning subscripts of your own.

operator()

Makes a **function object**. Unlike a plain function it can carry state, and unlike **std::function** it inlines (§6.10).

```
struct Scaler {
    double factor;
    double operator()(double x) const { return x * factor; }
};
std::transform(v.begin(), v.end(), out.begin(), Scaler{2.5});
```

A lambda is exactly this, generated for you (§6.8). Write the class by hand when you need a named type, several overloads, or templates on the class rather than the call. C++23 also allows **static operator()**,

which drops the unused `this`.

Conversion operators

```
class Handle {
public:
    explicit operator bool() const { return p_ != nullptr; } // ✓ always
    explicit
        operator void*() const; // ⚠ almost
    always a mistake
};
```

`explicit operator bool` still works in **contextual conversion** positions — `if`, `while`, `!`, `&&`, `||`, `?:` — which is exactly the set you want and nothing more (§7.4).

Every conversion operator except `explicit operator bool` should be viewed with suspicion. Named functions (`.get()`, `.value()`, `.toString()`) are clearer and don't participate in overload resolution behind your back (§6.1).

9.5 `operator<=>` and defaulted comparisons ⚠

The problem it solved

Pre-C++20, a comparable type needed six operators, all by hand, all as free functions to get the conversions right, all trivially derivable from one another, and all easy to get subtly inconsistent.

```
// C++20
struct Point {
    int x, y;
    auto operator<=>(const Point&) const = default;
};
// You now have: ==, !=, <, >, <=, >=
```

What `= default` generates

Defaulting `operator<=>` compares **members in declaration order**, lexicographically — first `x`, and only if equal, `y`. Bases first, then members.

⚠ **A defaulted `operator<=>` also implicitly declares a defaulted `operator==`.** But this only happens when `<=>` is *defaulted*:

```
struct A {
    int x, y;
    auto operator<=>(const A&) const = default; // ✓ get ==, !=, <, >, <=, >=
};
```

```
struct B {
    int x, y;
    std::strong_ordering operator<=>(const B& o) const { /* custom */ }
    // X NO operator== – you must write it yourself
    bool operator==(const B& o) const = default;    // add this
};
```

Why the split is deliberate: for many types `==` can be **much faster** than a full ordering.
`std::string::operator==` compares lengths first and bails; `<=>` must compare character by character.
Keeping them separate lets each be optimal.

The three comparison categories

Category	Meaning	Use for
<code>std::strong_ordering</code>	equal values are substitutable — indistinguishable in every observable way	<code>int</code> , <code>std::string</code> , most value types
<code>std::weak_ordering</code>	equivalent but distinguishable (case-insensitive strings, "sort by last name")	types with an equivalence coarser than equality
<code>std::partial_ordering</code>	some pairs are unordered	floating point (NaN), set inclusion, version ranges

```
auto r = a <=> b;
if (r < 0) { /* a < b */ }
if (r == 0) { /* equal / equivalent */ }
if (r > 0) { /* a > b */ }

// Named values
std::strong_ordering::less / ::equal / ::greater
std::weak_ordering::less / ::equivalent / ::greater
std::partial_ordering::less / ::equivalent / ::greater / ::unordered
```

Deducing `auto` gives you the **weakest** category among the members, which is usually right. Anything containing a `double` gets `partial_ordering`, because `NaN <=> anything` is `unordered`:

```
struct P { double x, y; auto operator<=>(const P&) const = default; };
// -> std::partial_ordering
```

If you want a total order on floats anyway (e.g. to put them in a `std::map`), say so explicitly:

```
struct P {
    double x, y;
    std::strong_ordering operator<=>(const P& o) const {
        if (auto c = std::strong_order(x, o.x); c != 0) return c;    // total order
```

```

on floats
    return std::strong_order(y, o.y);
}
bool operator==(const P&) const = default;
};

```

Writing a custom `<=>`

The idiom is a chain of early returns:

```

class Version {
    int major_, minor_, patch_;
    std::string tag_; // NOT part of ordering
public:
    std::strong_ordering operator<=>(const Version& o) const {
        if (auto c = major_ <=> o.major_; c != 0) return c;
        if (auto c = minor_ <=> o.minor_; c != 0) return c;
        return patch_ <=> o.patch_;
    }
    bool operator==(const Version& o) const {
        return major_ == o.major_ && minor_ == o.minor_ && patch_ == o.patch_;
    }
};

```

⚠ Traps with defaulted comparisons

```

struct Bad {
    std::string name;
    Cache* cache; // ⚠ compares POINTERS
    auto operator<=>(const Bad&) const = default; // two logically equal
objects
}; // with different caches
compare unequal

struct AlsoBad {
    int id;
    std::chrono::time_point<...> lastAccess; // ⚠ incidental state in
the comparison
    auto operator<=>(const AlsoBad&) const = default;
};

```

Defaulting compares **everything**, including members that are caches, back-pointers, timestamps, or IDs you didn't mean to include. Check the member list before defaulting.

Also: defaulting requires every member to be comparable. If one isn't, you get a deleted operator rather than a clear error — the failure shows up at the call site.

9.6 Rewritten and reversed candidates ⚠️

What C++20 added

For any relational expression, the compiler now synthesizes extra candidates:

You write	Compiler also considers
<code>a != b</code>	<code>!(a == b)</code> — rewritten
<code>b == a</code>	<code>a == b</code> — reversed
<code>a < b</code>	<code>(a <=> b) < 0</code> — rewritten
<code>a < b</code>	<code>0 < (b <=> a)</code> — reversed + rewritten

This is why writing `==` and `<=>` gives you all six operators in both argument orders, including for mixed types:

```
struct Meters { double v; };
bool operator==(const Meters& m, double d) { return m.v == d; }

Meters m{5.0};
m == 5.0;           // ✓ direct
5.0 == m;           // ✓ REVERSED candidate – no second overload needed
5.0 != m;           // ✓ rewritten + reversed
```

Pre-C++20 you had to write four functions for this. Now you write one.

⚠️ The C++17 → C++20 migration break

A **non-const member operator==** becomes ambiguous, because the reversed candidate is equally good:

```
struct S {
    bool operator==(const S&) { return true; }    // ⚠️ note: NOT const
};
S a, b;
a == b;     // C++17: fine.
             // C++20: ERROR – `a.operator==(b)` and the reversed `b.operator==(a)`
             //           are both viable and neither is better. Ambiguous.
```

Fix: make it **const**. This is a real and common breakage when moving a codebase to C++20.

The related case — a member `==` plus a namespace-scope `==` with swapped parameter types — hits the same ambiguity.

The safe form: a `const` hidden friend taking two `const&` parameters, or a defaulted member.

```
class S {
public:
```

```
friend bool operator==(const S&, const S&) = default;
};
```

9.7 `operator*` and `operator->` for pointer-like types

```
template <class T>
class MyPtr {
    T* p_;
public:
    T& operator*() const { return *p_; }
    T* operator->() const { return p_; } // note: returns a POINTER, not a
reference
    explicit operator bool() const { return p_ != nullptr; }
};
```

The `->` drill-down rule

`operator->` is special: if it returns something that isn't a raw pointer, the compiler **calls `operator->` again on the result**, repeating until it reaches a raw pointer. That's what makes `a->b` work through arbitrarily many wrapper layers.

This enables the **locking proxy** idiom — a temporary that holds a lock for exactly the duration of the expression:

```
template <class T>
class Guarded {
    T obj_;
    std::mutex mtx_;

    class Proxy {
        Guarded* g_;
        std::unique_lock<std::mutex> lock_;
    public:
        explicit Proxy(Guarded* g) : g_(g), lock_(g->mtx_) {}
        T* operator->() { return &g_->obj_; } // drill-down happens here
    };
public:
    Proxy operator->() { return Proxy{this}; }
};

Guarded<std::vector<int>> v;
v->push_back(42); // locks, calls push_back, unlocks at the end of the full-
expression
```

⚠ And the trap that comes with it: the lock lives only as long as the temporary, i.e. to the semicolon. `auto& r = *v.operator->();` escapes the protection. Two separate `v->` statements are two separate critical

sections — which is correct per-call but not atomic across calls.

`operator->` exists for member-pointer access through a smart pointer; it's rarely worth implementing.

9.8 User-defined literals

```
constexpr long double operator""_km(long double v) { return v * 1000.0; }
constexpr long double operator""_m (long double v) { return v; }

auto distance = 5.0_km + 300.0_m;      // 5300.0
```

Rules

- **The suffix must start with an underscore.** Suffixes without one are reserved for the standard library — using them is UB (and usually a warning).
- Allowed parameter lists:

```
T operator""_x(unsigned long long);      // integer literals
T operator""_x(long double);            // floating literals
T operator""_x(char);                   // character literals
T operator""_x(const char*, std::size_t); // string literals – the useful one
template <char...> T operator""_x();     // raw/templated form, for compile-
time parsing
```

- Mark them `constexpr` so they cost nothing.
- Put them in a namespace (ideally an `inline namespace`) so users opt in with a `using`.

The standard ones worth knowing

```
using namespace std::literals;

auto s  = "hello"s;           // std::string
auto sv = "hello"sv;          // std::string_view
auto d  = 100ms;               // std::chrono::milliseconds
auto t  = 2h + 30min;          //
auto c  = 3.0 + 4.0i;          // std::complex
auto ym = 2026y / std::chrono::August / 4d; // C++20 calendar
```

Chrono's are the ones you'll use daily — `sleep_for(500ms)` beats `sleep_for(std::chrono::milliseconds(500))`.

Where UDLs earn their keep

Strong types for units, which turn dimensional errors into compile errors:

```
struct Meters { double v; };
struct Seconds { double v; };
constexpr Meters operator""_m (long double v) { return {static_cast<double>(v)}; }
constexpr Seconds operator""_s (long double v) { return {static_cast<double>(v)}; }

auto speed = 100.0_m / 9.58_s;    // with the right operator/, returns
MetersPerSecond
```

(For anything serious, use a library — `mp-units` or `boost::units` — rather than hand-rolling the dimensional analysis.)

9.9 ⚠ Don't be clever

Operator overloading fails when it violates the reader's expectations. The costs of a surprising overload are paid at every call site by everyone, forever.

Preserve the expected semantics

Operator	Contract you must not break
<code>+, -, *</code>	do not mutate the operands; return a new value
<code>+=</code>	mutate and return <i>*this</i>
<code>==</code>	reflexive, symmetric, transitive (an equivalence relation)
<code><</code>	strict weak ordering — the STL's sorted containers and <code>std::sort</code> have UB otherwise (§12)
<code>[]</code>	fast, unchecked element access
<code><<</code>	stream output, not left-shift, unless you're actually doing bit manipulation

Never overload these

```
bool operator&&(const T&, const T&);    // X LOSES SHORT-CIRCUITING and
sequencing –
bool operator||(const T&, const T&);    // both operands are always evaluated,
T operator,(const T&, const T&);        // in unspecified order. Silently
changes behaviour.
```

An overloaded `&&` is an ordinary function call, so `p && p->f()` stops being safe. (C++17 sequenced `&&/||` for *built-in* types only.)

```
T* operator&() const;                  // X breaks address-of for everyone
```

This is why `std::addressof(x)` exists — generic code can't trust `&x`.

The judgement call

If a named function would be clearer, use a named function.

```
matrix1 * matrix2;           // ✓ universally understood
path / "file.txt";          // ✓ std::filesystem – odd at first, learned once,
reads well
str1 + str2;                 // ✓ concatenation is conventional
logger << "msg";             // ~ conventional by iostream precedent
config["key"];               // ✓ conventional
tree < node;                 // ✗ insert? compare? nobody knows
user + permission;          // ✗ grant()? name it.
socket >> data >> more;      // ✗ what happens on a partial read?
```

Expression-template libraries (Eigen, Boost.Spirit, range-v3's `|`) overload aggressively and get away with it because the domain notation is well-established and the library is large enough to be worth learning. That is not a licence for application code.

Gotcha checklist for this section

- ☐ A **member** binary operator doesn't allow conversions on the left operand — `2 * money` fails
- ☐ `operator+` should take its **left operand by value** and be built on `operator+=`
- ☐ Postfix `++` needs the dummy `int` and returns by value — prefer prefix
- ☐ `operator[]` conventionally does **not** bounds-check
- ☐ Proxy-returning `operator[]` breaks `auto` (`vector<bool>`)
- ☐ `operator bool` must be **explicit**
- ☐ A **defaulted** `operator<=>` gives you `==` too; a **custom** one does not
- ☐ Defaulted comparisons compare **every member**, including pointers, caches, and timestamps
- ☐ Anything containing a `double` deduces `partial_ordering` (NaN)
- ☐ Members are compared in **declaration order**
- ☐ A **non-const member** `operator==` is ambiguous in C++20 — reversed candidates
- ☐ `operator->` must return a raw pointer, or something with its own `operator->`
- ☐ A locking `operator->` proxy releases the lock at the **semicolon**
- ☐ UDL suffixes without a leading underscore are reserved
- ☐ Overloading `&&`, `||`, or `,` destroys short-circuiting and sequencing
- ☐ Overloading unary `&` breaks generic code — hence `std::addressof`

Flags and tools

```
-Wall -Wextra
-Wnon-template-friend
clang-tidy:
  modernize-use-default-member-init
  bugprone-unhandled-self-assignment      # operator= that forgets self-assignment
```



```
misc-unconventional-assign-operator    # operator= not returning T&
cert-oop54-cpp
google-explicit-constructor           # conversion operators too
```

Exercises worth doing

1. Write a `Money` class with a member `operator*` and confirm `2 * m` fails. Convert to a hidden friend and confirm both directions work.
2. Write a `Version` struct with three `ints` and default `<=>`. Sort a vector of them. Now add a `std::string tag_` member you didn't want compared, and observe the ordering change.
3. Take a C++17 class with six hand-written comparison operators and replace them all with `= default`. Diff the line count.
4. Write a class with a non-`const` member `operator==` and compile it as `-std=c++17` then `-std=c++20`. Read the C++20 error carefully.
5. Implement the `Guarded<T>` locking proxy. Then write code that escapes the lock (bind a reference to `*ptr`) and catch it with TSan.
6. Write `_m`, `_s`, and a `Meters / Seconds` operator that returns a `MetersPerSecond` type. Make a dimensional mistake and confirm it's a compile error.
7. Write `struct P { double x, y; auto operator<=>(const P&) const = default; };` and `static_assert` the category. Then compare two `Ps` where one has a NaN.

Chapter 10 — Templates ⚠

What this chapter covers

- Function and class templates; template parameters (type, non-type, template-template)
- **Instantiation**, two-phase lookup, `typename/template` disambiguators
- Specialization: full and partial; why you can't partially specialize function templates
- **SFINAE**, `enable_if`, `void_t`, detection idiom — and why **concepts replace all of it**
- **Concepts & constraints (C++20)**: `concept`, `requires` clauses and expressions, subsumption
- Variadic templates, parameter packs, fold expressions
- **CTAD** (class template argument deduction), deduction guides
- Template metaprogramming: `<type_traits>`, `if constexpr`, `constexpr` if vs SFINAE
- Compile-time computation, `constexpr` containers/algorithms (C++20)
- **Code bloat**, compile times, and the extern-template escape hatch
- `std::declval`, dependent names, ADL interaction

The largest section in Part 1, and the feature that separates C++ from everything else. Templates are not generics: they are a **compile-time code generator** with its own name lookup rules, its own overload resolution, and — since C++20 — its own type system in the form of concepts. Almost every hard C++ error message you will ever read comes from this section.

10.1 Function and class templates; parameter kinds

The four template forms

```
// Function template
template <class T>
T maxOf(T a, T b) { return a > b ? a : b; }

// Class template
template <class T, std::size_t N>
class Array { T data_[N]; };

// Variable template (C++14)
template <class T>
inline constexpr bool isPointer_v = std::is_pointer_v<T>;

// Alias template (C++11)
template <class T>
using Vec = std::vector<T, MyAllocator<T>>;
```

`class` and `typename` are interchangeable in a template parameter list. (`typename` reads better; `class` is historical. Only `typename` works as a *disambiguator* — §10.2.)

Parameter kinds

1. Type parameters — the common case.

2. Non-type template parameters (NTPs) — values, not types:

```
template <std::size_t N>          class Array { };
template <bool Checked>          class Buffer { };
template <auto V>                struct Constant { static constexpr auto value =
V; }; // C++17

Constant<42>      a;              // auto deduces int
Constant<'x'>     b;              // char
```

Allowed NTP types: integrals, enums, pointers/references to objects with linkage, `std::nullptr_t`, and — **new in C++20** — floating point and **literal class types** (any structural type: all public non-static members, no user-provided destructor or copy ops):

```
struct FixedString {                                // C++20
    char data[32]{};
    constexpr FixedString(const char (&s)[32]) { std::copy_n(s, 32, data); }
};
template <FixedString Name> struct Named { };        // string-ish template
parameters
```

3. Template-template parameters — a template as the argument:

```
template <class T, template <class...> class Container>
class Stack {
    Container<T> data_;
};
Stack<int, std::vector> s;           // note: std::vector, not std::vector<int>
```

Use `template <class...> class` rather than `template <class> class` — `std::vector` has an allocator parameter, and the variadic form matches both.

4. Defaults, which may reference earlier parameters:

```
template <class T, class Alloc = std::allocator<T>, std::size_t Chunk = 64>
class Pool;
```

⚠ Alias templates don't specialize

```
template <class T> using Ptr = T*;
template <> using Ptr<void> = void;    // X ERROR – alias templates can't be
specialized
```

Alias templates are transparent substitutions and are never deduced through. That's why the standard pairs `std::remove_reference` (a class template, specializable) with `std::remove_reference_t` (an alias for convenience).

10.2 Instantiation, two-phase lookup, and disambiguators ⚠

Instantiation

A template is a **recipe**. Nothing is compiled until you use it with concrete arguments — then the compiler generates a real class or function.

```
template <class T> void f(T) { /* ... */ }
f(42);           // implicit instantiation of f<int>
f(3.14);         // implicit instantiation of f<double> – a SEPARATE function in
the binary

template void f<char>(char);    // EXPLICIT instantiation definition – forces it
now
```

Only the **members you actually use** of a class template are instantiated:

```
template <class T>
struct S {
    void ok() { }
    void bad() { T::nonexistent(); } // never checked unless you call it
};
S<int> s;
s.ok(); // fine – bad() was never instantiated
```

This is why a class template can compile with a type that only satisfies *part* of its requirements.

Two-phase lookup

Templates are checked **twice**:

Phase 1 — at the point of definition. Syntax is checked, and every name that does **not** depend on a template parameter is looked up and bound *right there*.

Phase 2 — at the point of instantiation. Dependent names are looked up, using ordinary lookup from the definition context **plus ADL from the instantiation context**.

```
void helper(int); // #1

template <class T>
void f(T t) {
    helper(t); // DEPENDENT – resolved at instantiation
    helper(42); // NON-dependent – bound to #1 at definition time,
    permanently
}

void helper(double); // #2 – declared later
f(3.14); // calls helper(double)? NO – #2 wasn't visible at
definition, // and double doesn't trigger ADL. Calls helper(int).
Surprising.
```

⚠ **MSVC historically did not implement two-phase lookup.** Code written against MSVC often fails on GCC/Clang. Build with **/permissive-** (and ideally **/Zc:twoPhase-** off) to get standard behaviour.

The **typename** disambiguator

When a name depends on a template parameter, the compiler cannot tell whether **T::X** is a type or a value. It assumes **value** unless you say otherwise:

```
template <class T>
void f() {
    typename T::value_type x; // "T::value_type is a TYPE"
    T::value_type * p; // X without typename: parsed as
```

```
MULTIPLICATION
```

```
}
```

C++20 relaxed this in contexts where only a type is possible (return types, member declarations, trailing returns, `static_cast`, etc.), so you'll see less `typename` in new code. Writing it is always safe.

The `template` disambiguator

Similarly for **member templates**:

```
template <class T>
void f(T t) {
    t.template get<int>();           // "get is a TEMPLATE"
    // without `template`: `<` is parsed as less-than
}
```

Needed after `., ->`, or `::` when the member is a template and the object is dependent.

⚠ Dependent base classes hide their members

```
template <class T>
struct Base { int value; void helper(); };

template <class T>
struct Derived : Base<T> {
    void f() {
        value;           // X ERROR – not found! Base<T> is a DEPENDENT
    base
        helper();       // X ERROR
    }
};
```

Because `Base<T>` might be specialized later to not have those members, the compiler refuses to look in it during phase 1. Three fixes:

```
this->value;           // ✓ makes it dependent
Base<T>::value;        // ✓ explicit qualification (⚠ disables virtual
dispatch)
using Base<T>::value;  // ✓ pull it into scope once, at class level –
usually cleanest
```

This is one of the most common "why doesn't my CRTP compile" errors.

Where templates live

Because instantiation needs the definition, templates normally live **entirely in headers**. The exceptions are explicit instantiation (§10.10) and modules (§17).

10.3 Specialization

Full specialization

Works for class, function, and variable templates:

```
template <class T> struct Serializer { static std::string to(const T&); };

template <>                // FULL specialization
struct Serializer<bool> { static std::string to(bool b) { return b ? "true" :
"false"; } };
```

Partial specialization — classes only

```
template <class T>          struct S          { static constexpr int kind = 0;
};
template <class T>          struct S<T*>      { static constexpr int kind = 1;
};
template <class T>          struct S<T[]>      { static constexpr int kind = 2;
};
template <class T, class U> struct S<std::pair<T,U>> { static constexpr int kind =
3; };
template <class T>          struct S<const T>  { static constexpr int kind = 4;
};
```

The compiler picks the **most specialized** match. This is the workhorse of template metaprogramming — nearly every type trait is written this way:

```
template <class T> struct remove_reference      { using type = T; };
template <class T> struct remove_reference<T&> { using type = T; };
template <class T> struct remove_reference<T&&> { using type = T; };
```

Variable templates can be partially specialized too (C++14).

⚠ Why function templates can't be partially specialized

Because **overloading already does the job**, and mixing the two would be ambiguous:

```
template <class T> void f(T);           // primary
template <class T> void f(T*);          // NOT a specialization – a separate OVERLOAD
```

Overload resolution plus *partial ordering of function templates* picks the more specialized one. The language designers chose not to add a second, parallel mechanism.

⚠ The specialization-vs-overload trap (Dimov/Abrahams)

Full specializations of function templates are **not** considered during overload resolution. Overload resolution picks a *primary template* first; only then is the best specialization of **that** template selected.

```
template <class T> void f(T);           // #1
template <>         void f(int*);       // specialization of #1
template <class T> void f(T*);          // #2 – added later

int* p;
f(p);           // overload resolution picks #2 (more specialized primary).
                // Your specialization of #1 is NEVER CONSIDERED. Silently.
```

Guideline: don't specialize function templates. Overload them instead — or, if you need to specialize on something overloading can't express, delegate to a class template:

```
template <class T> struct Impl { static void run(T); };
template <class T> void f(T t) { Impl<T>::run(t); }      // specialize Impl, not f
```

Specializing standard-library templates

Legal, and sometimes necessary — you may specialize a standard template for **your own** types:

```
template <>
struct std::hash<MyType> {
    std::size_t operator()(const MyType& m) const noexcept {
        return std::hash<int>{}(m.id) ^ (std::hash<std::string>{}(m.name) << 1);
    }
};
```

You may **not** add new overloads to `namespace std`, only specialize existing templates for user-defined types. (`std::formatter<T>` and `std::hash<T>` are the two you'll actually do.)

10.4 SFINAE — and why it's obsolete

Substitution Failure Is Not An Error: if substituting template arguments produces an invalid type or expression **in the immediate context** of the declaration, that candidate is silently removed from the overload set instead of causing a compile error.

```
template <class T>
auto size(const T& c) -> decltype(c.size()) { return c.size(); } // removed if
```

```
no .size()

template <class T>
std::size_t size(const T& c) { return std::distance(std::begin(c), std::end(c)); }
```

std::enable_if

The pre-C++20 workhorse, in three placements:

```
// 1. Return type
template <class T>
std::enable_if_t<std::is_integral_v<T>, T> half(T x) { return x / 2; }

// 2. Extra defaulted template parameter – usually the least intrusive
template <class T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
T half(T x) { return x / 2; }

// 3. Extra function parameter
template <class T>
T half(T x, std::enable_if_t<std::is_integral_v<T>, int> = 0);
```

Note that **two overloads must have mutually exclusive conditions** — `enable_if` gives you no ordering, only on/off. That alone makes real overload hierarchies painful.

void_t and the detection idiom

```
template <class...> using void_t = void; // C++17: std::void_t

template <class T, class = void>
struct hasSize : std::false_type {};

template <class T>
struct hasSize<T, std::void_t<decltype(std::declval<T>().size())>> :
    std::true_type {};

static_assert(hasSize<std::vector<int>>::value);
static_assert(!hasSize<int>::value);
```

Clever, and completely opaque to anyone who hasn't seen it before.

⚠ Only the immediate context counts

```
template <class T>
auto f(T t) -> decltype(t.foo()) { // SFINAE-able – in the signature
    t.bar();                       // ⚠ HARD ERROR if T has no bar().
}                                  // The body is not the immediate
context.
```


SFINAE works on declarations, not bodies. An error inside the function is a real error, not a removal.

Why concepts replaced all of this

	SFINAE	Concepts
Readability	terrible	reads like a type system
Error messages	pages of substitution failures	"constraint not satisfied: <code>T</code> does not model <code>Sortable</code> "
Ordering of overloads	none — conditions must be disjoint	subsumption ranks more-constrained over less
Expressing intent	encoded in negative space	named, reusable, documented
Compile speed	slow (instantiates trait classes)	faster

In C++20 code, `enable_if` and `void_t` should appear only in library code that must support older standards. Everything below replaces them.

10.5 Concepts and constraints (C++20)

Defining

```
#include <concepts>

template <class T>
concept Numeric = std::integral<T> || std::floating_point<T>;

template <class T>
concept Container = requires(T c) {
    typename T::value_type;           // TYPE requirement
    c.size();                         // SIMPLE requirement: this
must compile
    { c.begin() } -> std::input_iterator; // COMPOUND: compiles AND
result models it
    { c.empty() } noexcept -> std::same_as<bool>; // ...and is noexcept, and
returns exactly bool
    requires std::copyable<typename T::value_type>; // NESTED requirement: a
boolean constraint
};
```

The four requirement kinds inside a `requires`-expression are exactly those four. Note a `requires`-expression only checks that things **compile** — it never evaluates them.

Using — four syntaxes for the same thing

```
// 1. Constrained template parameter
template <Numeric T> T twice(T x) { return x + x; }

// 2. requires clause after the parameter list
template <class T> requires Numeric<T>
T twice(T x) { return x + x; }

// 3. Trailing requires clause – can reference the parameters
template <class T>
T twice(T x) requires Numeric<T> { return x + x; }

// 4. Abbreviated function template – no `template` at all
Numeric auto twice(Numeric auto x) { return x + x; }
```

Form 4 is the most compact; forms 2 and 3 handle complex compound conditions. Note that in form 4, two `Numeric auto` parameters are **independent** types — it's `template <Numeric T, Numeric U>`, not one `T`.

⚠ requires requires

```
template <class T>
    requires requires(T t) { t.size(); }    // requires-CLAUSE + requires-
EXPRESSION
void f(T t);
```

Legal, ugly, and a sign you should have named the concept.

Standard concepts worth knowing

```
// <concepts>
std::same_as, std::derived_from, std::convertible_to, std::common_with
std::integral, std::signed_integral, std::floating_point
std::destructible, std::constructible_from, std::default_initializable
std::move_constructible, std::copy_constructible, std::assignable_from,
std::swappable
std::equality_comparable, std::totally_ordered
std::movable, std::copyable, std::semiregular, std::regular           // §3.1
std::invocable, std::regular_invocable, std::predicate, std::relation,
std::strict_weak_order

// <iterator>
std::input_iterator, std::forward_iterator, std::random_access_iterator,
std::sentinel_for

// <ranges>
std::ranges::range, sized_range, view, borrowed_range, viewable_range
```

Subsumption — how constrained overloads are ranked

The feature `enable_if` never had: a **more-constrained** overload wins over a less-constrained one.

```
template <std::integral T> void f(T) { print("integral"); }
template <std::signed_integral T> void f(T) { print("signed"); }

f(42); // "signed" – signed_integral SUBSUMES integral (it's defined in terms
of it)
f(42u); // "integral"
```

⚠ **Subsumption works on *atomic constraint identity*, and traits don't decompose.** Two textually identical `requires` expressions written in different places are *different* atomic constraints:

```
template <class T> requires std::is_integral_v<T> void
g(T); // #1
template <class T> requires std::is_integral_v<T> && std::is_signed_v<T> void
g(T); // #2
g(42); // ⚠ AMBIGUOUS – #1's atom and #2's first atom are distinct
expressions

template <class T> concept Int = std::is_integral_v<T>;
// named
template <class T> requires Int<T> void h(T);
template <class T> requires Int<T> && std::is_signed_v<T> void h(T);
h(42); // ✓ unambiguous – the SAME atomic constraint, so subsumption
applies
```

Always define named concepts. Never write raw type-trait expressions in `requires` clauses, or you lose subsumption and get ambiguity errors instead of ordering.

What concepts don't do

There is **no definition checking**. A concept constrains the *caller*, not the body:

```
template <std::integral T>
void f(T x) { x.someMethod(); } // compiles fine until instantiated – the
concept
// says nothing about what the body may use
```

So a template can still fail deep inside on a type that satisfies its concept. Concepts improve error messages and overload selection; they don't make templates checked-once like ML generics or Rust traits.

10.6 Variadic templates and packs

Full treatment including fold expressions is in **§6.12**. Template-specific additions:

Packs in class templates

```
template <class... Ts> struct Tuple;           // primary, undefined

template <> struct Tuple<> {};                 // base case
template <class T, class... Rest>
struct Tuple<T, Rest...> : Tuple<Rest...> {    // peel one, recurse
    T value;
};
```

Partial specialization is how you decompose a pack in a class template — there's no fold-expression equivalent for type-level recursion (until reflection, §18).

Pack expansion locations

```
template <class... Bases>
struct Mixin : Bases... {                    // in the base list
    using Bases::operator()...;              // in a using-declaration (C++17)
    Mixin(Bases... bs) : Bases(std::move(bs))... {} // in the mem-init list
};
```

That's the `overloaded` helper from §6.8 in full.

Constraining packs

```
template <std::integral... Ts>
auto sum(Ts... xs) { return (0 + ... + xs); }

template <class... Ts>
    requires (std::copyable<Ts> && ...) // fold over a concept
void store(Ts... xs);
```

10.7 CTAD — class template argument deduction

The basics (C++17)

```
std::vector v{1, 2, 3};           // vector<int>
std::pair p{1, 'c'};              // pair<int, char>
std::lock_guard lk{mtx};          // lock_guard<std::mutex> – a genuinely
nice win
std::array a{1, 2, 3};            // array<int, 3>
std::tuple t{1, 2.0, "x"};        // tuple<int, double, const char*>
```

The compiler synthesizes **implicit deduction guides** from the constructors of the primary template.

Explicit deduction guides

When the implicit ones aren't enough:

```
template <class T>
class Container {
    std::vector<T> data_;
public:
    template <class Iter>
    Container(Iter first, Iter last) : data_(first, last) {}
};

// Without a guide, Container(begin, end) can't deduce T
template <class Iter>
Container(Iter, Iter) -> Container<typename
std::iterator_traits<Iter>::value_type>;

std::vector<int> v;
Container c{v.begin(), v.end()};           // Container<int>
```

Syntax: `TemplateName(params) -> TemplateName<Args>;` at namespace scope, after the class.

Real examples from the standard library:

```
template <class... Ts> overloaded(Ts...) -> overloaded<Ts...>;           //
§6.8
template <class T> vector(std::size_t, T) -> vector<T>;
```

C++20 additions

```
// Aggregate CTAD
template <class T> struct Point { T x, y; };
Point p{1, 2};                               // Point<int> – no guide needed

// Alias template CTAD
template <class T> using IntPair = std::pair<int, T>;
IntPair ip{1, 'c'};                          // pair<int, char>
```

⚠ CTAD traps

```
std::vector<int> v1{1,2,3};
std::vector v2{v1};                          // vector<int> – the COPY deduction candidate
wins.                                         // NOT vector<vector<int>>, which is what the
```

```
braces suggest.
std::vector v3{v1, v1};           // vector<vector<int>> – two elements, so no copy
candidate

std::pair p{"a", "b"};           // pair<const char*, const char*> – NOT
std::string.                      // CTAD decays; it never applies conversions.

std::vector<std::string> v4{"a", "b"}; // ✓ explicit args when you want a
conversion
```

And CTAD is **all or nothing** — you can't deduce some parameters and specify others:

```
std::vector<int> v;               // ✓ fully specified
std::vector      v{1,2};         // ✓ fully deduced
// there is no middle ground for the allocator
```

10.8 Metaprogramming: type traits and `if constexpr`

`<type_traits>` by category

```
// Primary categories
is_integral_v, is_floating_point_v, is_pointer_v, is_reference_v, is_class_v,
is_enum_v, is_function_v, is_array_v, is_void_v

// Composite / properties
is_arithmetic_v, is_object_v, is_const_v, is_polymorphic_v, is_abstract_v,
is_final_v,
is_empty_v, is_aggregate_v, is_standard_layout_v, is_trivially_copyable_v

// Relationships
is_same_v<A,B>, is_base_of_v<B,D>, is_convertible_v<From,To>, is_invocable_v<F,
Args...>

// Special member properties (§7.1)
is_default_constructible_v, is_copy_constructible_v,
is_nothrow_move_constructible_v,
is_trivially_destructible_v

// Transformations (use the _t alias)
remove_reference_t, remove_cv_t, remove_cvref_t (C++20), decay_t, add_pointer_t,
underlying_type_t, common_type_t, conditional_t<B, T, F>, invoke_result_t<F,
Args...>,
make_signed_t, void_t
```

Convention: `X<T>::value` / `X<T>::type` are the class templates; `X_v<T>` / `X_t<T>` are the convenience aliases. **Always use the aliases.**

`std::remove_cvref_t<T>` (C++20) is the one you want when normalizing a forwarding reference — it's `remove_cv_t<remove_reference_t<T>>` in one step, and it's what `decay_t` should have been (`decay_t` also does array→pointer and function→pointer decay, which is usually not what you want).

`if constexpr` (C++17)

The discarded branch is **not instantiated** — so it may contain code that would be invalid for the current `T`:

```
template <class T>
void print(const T& x) {
    if constexpr (std::is_pointer_v<T>) {
        std::cout << *x;           // only instantiated when T IS a pointer
    } else if constexpr (requires { x.toString(); }) {
        std::cout << x.toString();
    } else {
        std::cout << x;
    }
}
```

This replaces most tag dispatch and a great deal of SFINAE. Note the `requires`-expression used inline as a boolean — a very handy C++20 idiom.

⚠ Two limits:

1. The discarded branch is still *parsed*, and its non-dependent parts are still checked.

```
template <class T>
void f(T t) {
    if constexpr (std::is_integral_v<T>) { ... }
    else { this_name_does_not_exist(); } // ⚠ error even for integral T – not
    dependent on T
}
```

2. In a non-template function, both branches are fully checked. `if constexpr` only has its magic property inside templates.

⚠ `static_assert(false)` in the `else`

```
template <class T>
void f(T) {
    if constexpr (...) { ... }
    else static_assert(false, "unsupported type"); // ⚠ fires even when never
    taken,                                         // on compilers pre-P2593
}
```

The condition isn't dependent, so the compiler can reject it immediately. The traditional workaround:

```
template <class> inline constexpr bool dependentFalse = false;
...
else static_assert(dependentFalse<T>, "unsupported type");    // ✓ dependent,
fires only on use
```

P2593 (C++26) makes the plain form work; until your toolchain has it, use `dependentFalse`.

Tag dispatch — still occasionally useful

```
template <class Iter>
void advanceImpl(Iter& it, int n, std::random_access_iterator_tag) { it += n; }
template <class Iter>
void advanceImpl(Iter& it, int n, std::input_iterator_tag) { while (n--) ++it; }

template <class Iter>
void advance(Iter& it, int n) {
    advanceImpl(it, n, typename std::iterator_traits<Iter>::iterator_category{});
}
```

Mostly superseded by `if constexpr` and concepts, but you'll read it constantly in the standard library's implementation.

10.9 Compile-time computation

Function-level detail is in §6.5. The template-relevant parts:

`constexpr` algorithms and containers (C++20)

Nearly all of `<algorithm>` is `constexpr` in C++20, and `std::vector` / `std::string` work at compile time (transiently — they must be destroyed before the evaluation ends):

```
constexpr auto buildTable() {
    std::array<int, 256> table{};
    for (std::size_t i = 0; i < table.size(); ++i)
        table[i] = static_cast<int>(i * i % 251);
    return table;                                // arrays CAN escape; vectors
cannot
}
constexpr auto kTable = buildTable();           // baked into .rodata, zero runtime
cost
```

```
constexpr bool isSorted() {
    std::vector<int> v{3, 1, 2};                // C++20 – transient allocation
    std::sort(v.begin(), v.end());              // constexpr algorithm
```



```
    return std::is_sorted(v.begin(), v.end());
}
static_assert(isSorted());
```

The standard pattern for returning compile-time-computed data: build into a `std::vector` internally, then copy into a fixed-size `std::array` whose size is itself computed by a first `constexpr` pass.

⚠ Compile-time computation is not free — it runs in the compiler's interpreter, which is orders of magnitude slower than native code and consumes real memory. Watch `-ftime-trace` if you go heavy on it.

10.10 Code bloat, compile times, and `extern template`

Monomorphization

Every distinct set of template arguments generates a **separate copy** of the code.

```
std::vector<int>           // one full set of instantiated member functions
std::vector<long>         // another – identical machine code on LP64, still
                          duplicated
std::vector<Widget*>      // another
std::vector<Gadget*>      // another
```

Symptoms: large binaries, instruction-cache pressure, slow linking, slow compiles, enormous debug info.

Mitigations

1. The thin-template / fat-base idiom — put the type-independent work in a non-template base:

```
class VectorBase {                                // one copy, works on void*
protected:
    void* data_ = nullptr;
    std::size_t size_ = 0, cap_ = 0;
    void growRaw(std::size_t elemSize); // the complex logic, instantiated ONCE
};

template <class T>
class Vector : private VectorBase {               // thin, inlinable, type-safe wrapper
public:
    void push_back(const T& x) { if (size_ == cap_) growRaw(sizeof(T)); ... }
};
```

This is exactly what real STL implementations do for `std::vector<T*>` (all pointer instantiations share one implementation) and what `std::function` does with type erasure.

2. `extern template` — declare that an instantiation exists elsewhere, so this TU doesn't generate it:

```
// widget.h
template <class T> class Processor { /* ... */ };
extern template class Processor<int>;           // "don't instantiate here"

// widget.cpp
template class Processor<int>;                  // explicit instantiation
DEFINITION — once
```

Real effect on large codebases: significant. libstdc++ does this for `std::basic_string<char>` and the common stream types.

3. Don't template what doesn't need it.

```
template <class Container> void process(const Container& c); // N instantiations
void process(std::span<const int> c);                       // ONE function
 (§3.8)
void log(std::string_view msg);                             // ONE function
```

`span` and `string_view` exist partly to *de-templatize* interfaces.

4. Measure.

```
clang -ftime-trace file.cpp           # produces a Chrome-trace JSON — open in
chrome://tracing                     # or feed to ClangBuildAnalyzer

g++ -ftime-report                     # Clang-based template instantiation profiler
templight                             # find the fattest symbols
nm -C --size-sort binary | tail -50  # size attribution by symbol/section
bloaty binary
```

5. Structural fixes: precompiled headers, unity builds, `ccache`, `include-what-you-use`, and — the real answer — **modules** (§17).

⚠ Recursive templates also hit **instantiation depth limits** (`-ftemplate-depth=`, default around 900). If you hit it, you almost certainly want an iterative fold expression or `if constexpr` instead of recursion.

10.11 `declval`, dependent names, and ADL

`std::declval`

Produces a `T&&` in an **unevaluated context** without requiring `T` to be constructible:

```
template <class T, class U>
using AddResult = decltype(std::declval<T>() + std::declval<U>());
```

```
static_assert(std::is_same_v<AddResult<int, double>, double>);
```

⚠ It has **no definition** — using it in evaluated code is a link error, deliberately. It exists only for `decltype`, `sizeof`, `noexcept`, and `requires`.

Note `declval<T>()` yields `T&&` (an xvalue), so for member access on a type whose methods are lvalue-ref-qualified you may want `declval<T&>()`.

ADL and templates

Because dependent names are resolved at instantiation (§10.2), **ADL is how generic code finds user customizations**:

```
namespace lib { struct Widget {}; void serialize(const Widget&); }

template <class T>
void save(const T& x) { serialize(x); }           // unqualified – ADL finds
lib::serialize
```

This is the classic C++ customization-point mechanism, together with the `using std::swap`; two-step (§5.9). Its weakness: any name in scope can hijack the call, and the customization point isn't discoverable.

Niebloids / CPOs — why `std::ranges::sort` is an object

The C++20 ranges algorithms are **function objects**, not function templates:

```
std::ranges::sort(v);           // `sort` is a const object with operator()
```

Because they're objects, ordinary unqualified lookup finds them and **ADL is suppressed** — so `std::ranges::sort(v)` can never be hijacked by an ADL-visible `sort` in the argument's namespace. Customization-point objects (`std::ranges::begin`, `swap`, `size`) go further: they encapsulate the "member if available, else ADL, else default" cascade in one place, so callers never have to write the two-step by hand.

That's why you'll see `std::ranges::swap(a, b)` recommended over `using std::swap; swap(a, b);` in new code.

Gotcha checklist for this section

- ☐ Only the **members you use** of a class template get instantiated
- ☐ **Non-dependent names are bound at definition time** — a function declared later won't be found
- ☐ MSVC needs `/permissive-` for standard two-phase lookup
- ☐ `typename` before dependent types; `template` before dependent member templates
- ☐ **Dependent base members are invisible** — use `this->x` or `using Base<T>::x`
- ☐ **Function templates can't be partially specialized** — overload instead

- ☐ Full specializations of function templates are chosen **after** overload resolution — adding an overload silently disables them
- ☐ SFINAE only applies to the **immediate context**, not the function body
- ☐ Raw type traits in **requires** clauses **don't subsume** — always name your concepts
- ☐ Concepts constrain the caller; they do **not** check the template body
- ☐ `Numeric auto a`, `Numeric auto b` are two **independent** types
- ☐ `std::vector v{v1}` deduces a **copy**, not a nested vector
- ☐ CTAD **decays** and applies no conversions — `std::pair p{"a","b"}` is `const char*`
- ☐ CTAD is all-or-nothing — no partial argument lists
- ☐ `if constexpr` discarded branches are still **parsed**; non-dependent errors still fire
- ☐ `if constexpr` has no special power outside a template
- ☐ `static_assert(false)` in an `else` needs `dependentFalse<T>` until C++26
- ☐ `std::declval` has no definition — unevaluated contexts only
- ☐ Alias templates can't be specialized and can't be deduced through
- ☐ Every distinct instantiation is a **separate copy** in the binary

Flags and tools

```

/permissive-                # MSVC: standard two-phase lookup – essential
-ftemplate-backtrace-limit=0 # see the whole instantiation chain
-ftemplate-depth=1024
-fconcepts-diagnostics-depth=3 # GCC: better concept failure messages
-ftime-trace                 # Clang: per-template compile time attribution
-ftime-report                # GCC
-fno-implicit-templates     # force explicit instantiation everywhere

Tools:
  ClangBuildAnalyzer         # aggregates -ftime-trace across a build
  temlight / temlight-tools  # instantiation profiler
  include-what-you-use
  bloaty, nm -C --size-sort  # where the binary size went
  cppinsights.io             # ★ shows what the compiler ACTUALLY generated
  Compiler Explorer (godbolt.org)

clang-tidy:
  modernize-use-constraints  # enable_if -> requires
  modernize-type-traits      # ::value/::type -> _v/_t
  bugprone-forwarding-reference-overload

```

cppinsights.io deserves special mention: paste a template, and it shows you the instantiated code, the synthesized deduction guides, the lambda's closure class, and the implicit special members. It is the single best tool for building intuition here.

Exercises worth doing

1. Paste a class with a lambda, a range-for, and `auto` into **cppinsights.io**. Read the generated code line by line.

2. Write the `hasSize` detection idiom with `void_t`. Then rewrite it as a two-line C++20 concept. Compare the error messages when you pass a type that fails.
3. Reproduce the Dimov/Abrahams trap: primary, specialization, then add an overload. Confirm your specialization is skipped.
4. Write `template <class T> requires std::is_integral_v<T>` and a second more-constrained overload with raw traits. Get the ambiguity error. Fix it by naming the concepts.
5. Write a CRTP base that accesses a member of the derived class through a dependent base and get the error. Fix it three ways (`this->`, qualification, `using`).
6. Build a 256-entry lookup table with `constexpr` and confirm with `objdump -s -j .rodata` that it's in the binary.
7. Take a header with a heavily used class template, add `extern template` + an explicit instantiation, and measure compile time and binary size before/after with `-ftime-trace` and `bloaty`.
8. Write a deduction guide for a container that takes an iterator pair.
9. Implement `std::remove_reference`, `std::conditional`, and `std::is_same` from scratch using partial specialization.

Chapter 11 — The STL — Containers

What this chapter covers

- **vector** — the default. Capacity vs size, `reserve`, growth, iterator invalidation
- `array`, `deque`, `list`, `forward_list`
- `map` / `set` (ordered, node-based) vs `unordered_map` / `unordered_set` (hash)
- `multimap`, `multiset`, `flat_map`/`flat_set` (C++23)
- **Iterator invalidation rules** — memorize them per container
- `emplace` vs `insert` vs `push_back`; `try_emplace`, `insert_or_assign`
- Custom hash and equality; `std::hash` specialization
- Container adaptors: `stack`, `queue`, `priority_queue`
- `string` — SSO, `string_view` ⚠ (dangling!)
- `span`, `mdspan` (C++23)
- `std::vector<bool>` — the standard's most famous mistake
- Allocators, PMR (`<memory_resource>`)

The most reference-heavy section in Part 1. Two things here are worth actually memorizing: the **iterator invalidation table** and the **complexity/locality trade-offs**. Everything else you can look up — but picking the wrong container is a design decision that gets baked in, and invalidation bugs are silent until they aren't.

11.1 `std::vector` — the default

Layout and the size/capacity distinction

```
template <class T, class A = std::allocator<T>>
class vector {
    T* begin_;    // start of data
```

```

    T* end_;           // one past the last ELEMENT      -> size()      = end_ -
    begin_
    T* capEnd_;        // one past the last ALLOCATED slot -> capacity() = capEnd_ -
    begin_
};
static_assert(sizeof(std::vector<int>) == 24);           // three pointers, on LP64

```

```

std::vector<int> v;
v.reserve(100);      // capacity 100, size 0   - allocates, constructs NOTHING
v.resize(100);        // capacity 100, size 100 - constructs 100 zero-initialized
ints
v.reserve(10);        // no-op - reserve never shrinks
v.shrink_to_fit();    // non-binding REQUEST; may do nothing

```

⚠ **reserve** vs **resize** is the most common beginner error: `v.reserve(n); v[0] = x;` is UB because there is no element 0.

Growth

Reallocation is **geometric**, which makes **push_back** amortized $O(1)$:

Implementation	Growth factor
libstdc++ (GCC), libc++ (Clang)	2×
MSVC	1.5×

Each reallocation: allocate new buffer → move (or copy — \$5.5!) every element → destroy the old → free. So:

```

std::vector<Widget> v;
v.reserve(n);                // ✓ ONE allocation, zero reallocations
for (int i = 0; i < n; ++i) v.push_back(...);

```

reserve when you know the size is the single highest-value **vector** habit.

Why it's the default

Asymptotics lie about constants. **vector** stores elements **contiguously**, so iteration is prefetcher-friendly and every element touch is a cache hit. A linked list is $O(1)$ to insert and $O(n)$ to *find where to insert*, with a cache miss per node.

Empirical rule: for insert-in-the-middle workloads, `std::vector` beats `std::list` up to element counts in the **thousands to tens of thousands**, and beats it *always* for iteration. Bjarne's "vector vs list" benchmark is the canonical demonstration.

Choose **list** for stable references and $O(1)$ splice, not for insertion speed.

Other essentials

```
v.data(); // T* — contiguity is GUARANTEED by the
standard
v.at(i); // bounds-checked, throws std::out_of_range
v[i]; // unchecked (§9.4)
v.push_back(x); v.emplace_back(args...); // §11.6
v.pop_back();
std::erase(v, value); // C++20 — replaces the erase-remove idiom (§12)
std::erase_if(v, pred);

// Actually release memory (pre-C++11 idiom, still occasionally needed)
std::vector<T>(v).swap(v);
```

11.2 array, deque, list, forward_list

`std::array<T, N>`

A fixed-size array with a proper interface: no decay (§3.8), knows its size, copyable, comparable, an aggregate (§7.8), and **zero overhead** — `sizeof(std::array<int,4>) == 16`.

```
std::array<int, 3> a{1, 2, 3};
a.size(); a.at(1); a.data(); a.fill(0);
auto b = std::to_array({1, 2, 3}); // C++20 — deduces array<int,3>
auto c = std::to_array<long>({1, 2, 3});
```

Use it anywhere you'd have used `int arr[N]`. Always.

`std::deque<T>`

Segmented: a map of fixed-size chunks. $O(1)$ push/pop at **both** ends, $O(1)$ random access (two indirections).

```
std::deque<int> d;
d.push_front(1); // ✓ vector can't do this
d.push_back(2);
```

⚠ Two things people don't expect:

- **Not contiguous** — no `.data()`, can't pass to a C API
- **Chunk size varies wildly by implementation.** libstdc++ uses 512 bytes; **MSVC uses 16 bytes**, which makes `std::deque` on MSVC memory-hungry and slow enough that many cross-platform codebases avoid it entirely

The unusual invalidation property (see the table): inserting at either **end** invalidates all iterators but **keeps references and pointers valid**.

`std::list<T>` (doubly linked)

```
std::list<int> l{1, 2, 3};
auto it = std::next(l.begin());
l.insert(it, 99);           // O(1) given the iterator
l.splice(l.begin(), other); // O(1) – moves nodes between lists, no allocation
l.sort(); l.merge(other); l.reverse(); // member versions (std::sort needs
random access)
```

Real reasons to choose it: **references and iterators never invalidate**, and O(1) `splice`. Bad reasons: "insertion is O(1)" — finding the position isn't, and the cache misses dominate.

Overhead: two pointers + allocation per element.

`std::forward_list<T>` (singly linked)

Minimal overhead (one pointer per node), and deliberately **no `size()`** (it would cost a member or O(n)). Uses `insert_after` / `erase_after` / `before_begin()`.

Use it only when the per-node pointer actually matters — embedded, or millions of tiny nodes.

Comparison

	Contiguous	Random access	push_back	push_front	insert middle	Per-element overhead
array	✓	O(1)	—	—	—	0
vector	✓	O(1)	amortized O(1)	O(n)	O(n)	0
deque	X	O(1)	O(1)	O(1)	O(n)	small + chunk map
list	X	O(n)	O(1)	O(1)	O(1)*	2 pointers + alloc
forward_list	X	O(n)	—	O(1)	O(1)*	1 pointer + alloc

* given an iterator to the position

11.3 Ordered vs hashed associative containers

`std::map` / `std::set` — ordered, node-based

Red-black tree. $O(\log n)$ for everything, **sorted iteration**, and node-based so references are permanently stable.

```
std::map<std::string, int> m;
m["a"] = 1; // ⚠ default-constructs and INSERTS if
absent
m.at("a"); // throws if absent; works on const maps
m.find("a"); // iterator or end()
m.contains("a"); // C++20 – say what you mean
m.count("a"); // 0 or 1 (the pre-C++20 way to say
contains)
m.lower_bound("a"); m.upper_bound("a"); m.equal_range("a");
```

⚠ **operator[] inserts**. `if (m["key"] == 0)` silently adds an entry, and `operator[]` doesn't exist on a `const map`. Use `find/contains/at` for lookup, `operator[]` only when insert-if-absent is what you want.

Requires a **strict weak ordering** comparator (§12) — violating it is UB, not just wrong results.

`std::unordered_map` / `std::unordered_set` — hashed

Buckets with chaining. $O(1)$ average, $O(n)$ worst case, iteration order unspecified and **changes on rehash**.

```
std::unordered_map<std::string, int> um;
um.reserve(10'000); // ✓ pre-size to avoid rehashing
um.load_factor(); um.max_load_factor(0.7);
um.bucket_count();
```

⚠ `std::unordered_map` is slow *by mandate*

The standard requires a bucket interface (`bucket()`, `begin(n)`, `bucket_count()`) and **reference stability across rehash**. Together those force a node-based, chained implementation: one allocation per element and a pointer chase per lookup. No conforming implementation can be fast.

Open-addressing alternatives, typically **2–5× faster**:

Library	Type
Abseil	<code>absl::flat_hash_map</code> , <code>absl::node_hash_map</code>
Boost 1.81+	<code>boost::unordered_flat_map</code>
ankerl	<code>ankerl::unordered_dense::map</code>
tsl	<code>tsl::robin_map</code> , <code>tsl::hopscotch_map</code>

The trade-off: flat maps **invalidate references on rehash** (elements move). If you were relying on `std::unordered_map`'s reference stability, you need the `node_hash_map` variant.

Choosing

Need	Use
Sorted iteration, range queries (<code>lower_bound</code>)	<code>map</code> / <code>set</code>
Fastest lookup, order irrelevant	a flat hash map (not <code>std::unordered_map</code>)
Stable references across insertion	<code>map</code> , or <code>node_hash_map</code>
Small (< ~32 entries), read-mostly	<code>flat_map</code> or a sorted <code>vector</code>
No allocation allowed	<code>flat_map</code> over a <code>pmr</code> or fixed buffer

Heterogeneous lookup ⚠

By default, looking up a `std::string` key with a `const char*` or `string_view` **constructs a temporary `std::string`** — an allocation per lookup.

```
std::map<std::string, int> m;
m.find("key"); // ⚠ allocates a temporary
               string

std::map<std::string, int, std::less<>> m2; // NOTE: less<> not
less<std::string>
m2.find("key"); // ✓ no temporary –
transparent comparator
m2.find(std::string_view{"key"}); // ✓
```

`std::less<>` has an `is_transparent` typedef that unlocks the templated `find`/`count`/`contains` overloads. For unordered containers this arrived in **C++20** and needs both the hash and the equality to be transparent:

```
struct StringHash {
    using is_transparent = void;
    std::size_t operator()(std::string_view sv) const { return
std::hash<std::string_view>{}(sv); }
};
std::unordered_map<std::string, int, StringHash, std::equal_to<>> um;
um.find("key"); // ✓ no allocation
```

Node handles (C++17)

Move nodes between containers, or change a key, with no reallocation:

```
auto node = m.extract("oldKey");
node.key() = "newKey"; // mutate the key – impossible any other
way
m.insert(std::move(node));
```

```
m1.merge(m2);           // splice all non-conflicting nodes out
of m2
```

11.4 multimap/multiset and flat_map/flat_set

```
std::multimap<std::string, int> mm;
mm.emplace("a", 1);
mm.emplace("a", 2);
auto [first, last] = mm.equal_range("a");           // the only sane way to read them
for (auto it = first; it != last; ++it) { ... }
```

No `operator[]`, and `count()` is $O(\log n + k)$. Equivalent keys are stored in insertion order (guaranteed since C++11).

`std::flat_map / flat_set` (C++23)

Container **adaptors**, not new containers: a `flat_map` is two parallel sorted vectors (keys and values).

```
std::flat_map<int, std::string> fm;
fm.emplace(1, "one");
```

	map	flat_map
Lookup	$O(\log n)$, pointer chasing	$O(\log n)$, cache-friendly binary search
Iteration	pointer chasing	contiguous — much faster
Insert / erase	$O(\log n)$	$O(n)$ — shifts the vector
Reference stability	permanent	none
Memory	node + 2-3 pointers per element	packed

Use it for **read-mostly** maps: config tables, lookup tables built once, small maps. Available today without C++23 as `boost::container::flat_map`; `absl::btree_map` is a good middle ground (B-tree: cache-friendly *and* $O(\log n)$ insert).

11.5 ⚠ Iterator invalidation — the table to memorize

Container	Operation	Iterators	References/pointers
vector	<code>push_back/insert</code> , realloc	all	all
	<code>push_back/insert</code> , no realloc	at/after insertion point	at/after insertion point

Container	Operation	Iterators	References/pointers
	<code>erase</code>	at/after erase point	at/after erase point
	<code>reserve</code> , <code>resize</code> (grow), <code>shrink_to_fit</code>	all	all
	<code>clear</code>	all	all
deque	<code>push_front</code> / <code>push_back</code>	all	none ⚠
	<code>insert</code> (middle)	all	all
	<code>erase</code> (middle)	all	all
	<code>erase</code> (at either end)	only the erased	only the erased
list / forward_list	<code>insert</code> , <code>splice</code>	none	none
	<code>erase</code>	only the erased	only the erased
map / set / multi*	<code>insert</code> , <code>emplace</code>	none	none
	<code>erase</code>	only the erased	only the erased
unordered_*	<code>insert</code> causing rehash	all	none ⚠
	<code>insert</code> without rehash	none	none
	<code>erase</code>	only the erased	only the erased
flat_map / flat_set	any insert/erase	all	all
string	any capacity change	all	all

Three rows people get wrong:

1. **deque end-insertion**: iterators die, references live.
2. **unordered_* rehash**: iterators die, references live (it's node-based).
3. **vector no-realloc insert**: everything *before* the insertion point survives.

⚠ And the universal one: **`end()` is invalidated by essentially every modification** on every container.

```
for (auto it = v.begin(); it != v.end(); ) {
    if (pred(*it)) it = v.erase(it);    // ✓ erase RETURNS the next valid
    iterator
    else ++it;
}
std::erase_if(v, pred);                // ✓ C++20 – just do this
```

11.6 `emplace` vs `insert` vs `push_back`

Sequence containers

```
v.push_back(widget);           // copy
v.push_back(std::move(widget)); // move
v.push_back(Widget{a, b});     // construct temporary, then move
v.emplace_back(a, b);          // construct IN PLACE from the arguments – no
                                temporary
auto& ref = v.emplace_back(a, b); // C++17: returns a reference to the new element
```

`emplace_back` perfectly forwards its arguments to `T`'s constructor in the container's storage (§5.4).

⚠ `emplace_back` is not automatically better:

- With an **existing object**, `push_back(obj)` and `emplace_back(obj)` do the same thing
- `emplace_back` uses **direct-initialization**, so it will call **explicit** constructors that `push_back` would reject — bypassing a safety check you deliberately added
- It generates a new template instantiation per argument-list shape (§10.10)
- ⚠ **Leak risk with raw `new`:**

```
v.emplace_back(new Widget);      // if the vector reallocates and THROWS, the
Widget leaks
v.push_back(std::make_unique<Widget>()); // ✓ ownership is established before
the call
```

Use `push_back` for objects you already have; `emplace_back` when you're constructing from arguments.

Associative containers — the important one

```
std::map<std::string, std::unique_ptr<Widget>> m;

m.insert({key, std::move(p)});           // constructs a pair, then may discard
it
m.emplace(key, std::move(p));             // ⚠ may CONSTRUCT the pair (moving
p!) and
// then discard it if the key
// already exists
// -> your unique_ptr is gone,
// nothing inserted
m.try_emplace(key, std::move(p));         // ✓ C++17 – checks the key FIRST.
// p is untouched if the key
exists.
m.insert_or_assign(key, std::move(p));    // ✓ assigns if present, inserts if
not
```

try_emplace is the correct default for maps with move-only or expensive values. **emplace**'s behaviour here is a genuine footgun: it's allowed to construct the **value_type** before checking, which means it can move from your argument and then throw the result away.

Also note **try_emplace** doesn't construct the mapped value at all when the key exists — so it beats **operator[]** for "insert if absent" on expensive types.

11.7 Custom hash and equality

```
struct Point { int x, y; };

struct PointHash {
    std::size_t operator()(const Point& p) const noexcept {
        std::size_t seed = 0;
        hashCombine(seed, p.x);
        hashCombine(seed, p.y);
        return seed;
    }
};

std::unordered_set<Point, PointHash, /* equality: */ std::equal_to<>> s;
```

Or specialize **std::hash** so it works by default (§10.3):

```
template <>
struct std::hash<Point> {
    std::size_t operator()(const Point& p) const noexcept { /* ... */ }
};
```

Combining hashes properly

```
// ✗ DON'T: XOR is symmetric, so hash({1,2}) == hash({2,1}), and x^x == 0
return std::hash<int>{}(p.x) ^ std::hash<int>{}(p.y);

// ✓ boost::hash_combine — the standard folk formula
template <class T>
void hashCombine(std::size_t& seed, const T& v) {
    seed ^= std::hash<T>{}(v) + 0x9e3779b9 + (seed << 6) + (seed >> 2);
}
```

0x9e3779b9 is the golden-ratio constant; the shifts diffuse the bits. There's no **std::hash_combine** in the standard (a long-standing gap) — use Boost's, Abseil's **absl::Hash**, or **ankerl**'s.

The contract

If $a == b$, then $\text{hash}(a) == \text{hash}(b)$. The reverse is not required.

So: hash exactly the members you compare in `operator==` (§9.5's defaulted-comparison warning applies here too). And mark `operator()` `noexcept` — some containers give stronger exception guarantees when the hash can't throw.

11.8 Container adaptors

```
std::stack<int> s; // adapts deque by default
std::stack<int, std::vector<int>> s2; // ✓ vector is usually faster
for a stack
std::queue<int> q; // adapts deque
std::priority_queue<int> pq; // adapts vector + heap
algorithms
```

Adaptors expose a **restricted** interface: no iteration, no `begin()/end()`, no `find`. That's the point — but it also means you can't debug-print one without draining it.

⚠ `priority_queue` gotchas

```
std::priority_queue<int> pq; // MAX-heap by default
std::priority_queue<int, std::vector<int>, std::greater<>> minPq; // min-heap

pq.push(3); pq.push(1);
pq.top(); // 3 – largest
pq.pop(); // ⚠ returns void; you must top() then pop()
```

The comparator means "**lower priority**", which reads backwards from `std::sort`'s "comes first". A custom comparator for a min-heap of pairs by second element:

```
auto cmp = [](const auto& a, const auto& b) { return a.second > b.second; };
std::priority_queue<P, std::vector<P>, decltype(cmp)> pq(cmp);
```

⚠ **There is no way to update an element's priority.** Options: push a duplicate with the new priority and skip stale entries on pop (the usual Dijkstra trick), or use a `std::set` / indexed binary heap.

11.9 `std::string` and `std::string_view`

SSO — the small string optimization

Short strings live **inside** the string object; no allocation.

Implementation	<code>sizeof(std::string)</code>	SSO capacity
libstdc++ (GCC)	32	15 chars

Implementation	<code>sizeof(std::string)</code>	SSO capacity
libc++ (Clang)	24	22 chars
MSVC	32	15 chars

This is why `std::string` is fine for short keys and identifiers, and why `sizeof(std::string)` differing across implementations is an ABI landmine (§25).

```
s.c_str();           // guaranteed null-terminated
s.data();            // same pointer, also null-terminated since C++11
s.reserve(n); s.capacity(); s.shrink_to_fit();
s.starts_with("x"); s.ends_with("y");    // C++20
s.contains("z");           // C++23
```

`std::string` is a full container — iterators, `<algorithm>`, range-for all work on it.

`std::string_view` ⚠

A non-owning (pointer, length) pair (§3.8). Cheap to copy, no allocation, and the right type for **read-only string parameters**.

```
void log(std::string_view msg);           // ✓ accepts string, const char*,
literal – no copies
log(myString);
log("literal");
log(std::string_view{buf, len});
```

Two traps, both serious:

1. No null terminator guarantee.

```
void cApi(const char*);
void f(std::string_view sv) {
    cApi(sv.data());           // ⚠ UB – sv may be a substring; data() isn't
    null-terminated
    cApi(std::string(sv).c_str()); // ✓ (allocates, but correct)
}
```

2. It dangles like any reference.

```
std::string_view sv = getString();           // ⚠ temporary destroyed at the
semicolon
std::string_view bad() { std::string s; return s; } // ⚠ dangling

struct Config { std::string_view name; }; // ⚠ member: only safe if you
```



```
control the owner
Config c{std::string("temp")};           // dangling immediately (§3.4)

std::map<std::string_view, int> m;
m[std::string("temp")] = 1;              // ⚠ key dangles
```

string_view is for parameters and locals. Returning one, or storing one as a member, requires you to own the lifetime argument.

-Wdangling-gsl (Clang) and ASan with `use-after-scope` catch most of these.

11.10 span and mdspan

`std::span` is covered in §3.8 — non-owning view of a contiguous range, the fix for `(T*, size_t)` parameter pairs, and it dangles exactly like `string_view`.

`std::mdspan` (C++23)

A multidimensional view over contiguous storage — the fix for `matrix[i * cols + j]` arithmetic scattered through numeric code.

```
#include <mdspan>

std::vector<double> storage(rows * cols);
std::mdspan m{storage.data(), rows, cols};           // dynamic extents
m[i, j] = 1.0;                                       // C++23 multidimensional
operator[] (§9.4)

std::mdspan<double, std::extents<std::size_t, 3, 4>> fixed{ptr}; // static
extents
```

Three policy parameters make it more than a convenience:

```
std::layout_right    // row-major (C/C++ default)
std::layout_left     // column-major (Fortran/BLAS/LAPACK)
std::layout_stride    // arbitrary strides – submatrices, slices, padded rows
```

Plus a custom **accessor policy**, which is how you build atomic, checked, or restrict-qualified views over the same buffer. It's non-owning, so the same lifetime rules apply. (Available today as the reference implementation `kokkos/mdspan`.)

11.11 ⚠ std::vector<bool> — the famous mistake

`std::vector<bool>` is a **specialization** that packs 8 bools per byte. It is not a container, and it breaks generic code.

```
std::vector<bool> v(10);

bool* p = &v[0];           // X ERROR – v[0] is a PROXY, not a bool&
v.data();                  // X doesn't exist
auto x = v[0];             // ⚠ x is vector<bool>::reference, not bool (§9.4)
                           //    x stays tied to the bit; assigning to it
                           //    writes to v

template <class C> void f(C& c) { auto& r = c[0]; } // X fails only for
vector<bool>
```

It fails to meet the standard's own `Container` and `SequenceContainer` requirements. It exists because in 1998 it looked like a good demonstration of what specialization could do, and it couldn't be removed without breaking ABI.

⚠ **Also a threading hazard:** for every other container, writing to *distinct* elements from different threads is safe. For `vector<bool>`, two adjacent elements share a byte — concurrent writes are a **data race** (§15).

Alternatives:

Want	Use
A real container of bools	<code>std::vector<char></code> or <code>std::vector<std::uint8_t></code>
Fixed-size bitset	<code>std::bitset<N></code>
Dynamic bitset with real bit ops	<code>boost::dynamic_bitset</code>
A flag per element in a struct	just use <code>bool</code>

11.12 Allocators and PMR

Classic allocators

The allocator is a **template parameter**, so it's part of the type:

```
std::vector<int, MyAlloc<int>> a;
std::vector<int> b;
a = b;           // X different types – won't convert
void f(const std::vector<int>&);
f(a);           // X can't pass
```

This "allocator infects the type" problem is why classic allocators saw so little use outside of specialist code. `std::allocator_traits` provides the uniform interface, and the propagation traits (`propagate_on_container_copy_assignment` etc.) control what happens on copy/move/swap.

PMR — polymorphic memory resources (C++17)

The fix: one allocator type holding a **pointer to a runtime-polymorphic resource**. All `std::pmr::vector<int>`s are the same type regardless of where they allocate.

```
#include <memory_resource>

void process() {
    std::byte buffer[8192];
    std::pmr::monotonic_buffer_resource pool{buffer, sizeof(buffer)};

    std::pmr::vector<std::pmr::string> v{&pool};
    v.emplace_back("hello");           // allocates FROM THE STACK BUFFER
    v.emplace_back("world");
    // zero heap allocations; "deallocation" is the pool going out of scope
}
```

The available resources:

Resource	Behaviour
<code>monotonic_buffer_resource</code>	bump-pointer arena; never frees individual allocations; blazing fast; release all at once
<code>unsynchronized_pool_resource</code>	size-classed pools, single-threaded
<code>synchronized_pool_resource</code>	same, thread-safe
<code>new_delete_resource()</code>	the default — plain <code>new/delete</code>
<code>null_memory_resource()</code>	throws <code>bad_alloc</code> on any allocation — use it to prove a code path doesn't allocate

```
std::pmr::set_default_resource(&myPool);    // change the global default
```

⚠ **Gotchas:**

- The allocator is **not part of the value**. Copying a `pmr::vector` gives the copy the *default* resource, not the source's. Copies between resources are easy to do accidentally.
- Containers must **propagate** the resource to their elements — `std::pmr::vector<std::pmr::string>` works; `std::pmr::vector<std::string>` puts the vector's buffer in the pool but every string still hits the global heap.
- One virtual call per allocation. Almost always dwarfed by what you save from not calling `malloc`.

When PMR pays off: request-scoped arenas in servers (allocate freely, drop the whole arena at the end of the request), parsers, game frame allocators, and anywhere `perf` shows `malloc` in the top five (§24).

Gotcha checklist for this section

- ☐ `reserve(n)` sets **capacity**, not size — `v[0]` afterwards is UB

- `vector` reallocation invalidates **everything**, and copies instead of moves if the move isn't `noexcept` (§5.5)
- `map::operator[]` **inserts** a default-constructed value and doesn't work on `const`
- `deque` end-insertion invalidates iterators but **not** references
- `unordered_*` rehash invalidates iterators but **not** references
- `end()` is invalidated by almost every modification
- `std::unordered_map` is slow by standard mandate — use a flat hash map for hot lookups
- Looking up a `std::string` key with a literal **allocates** unless the comparator is transparent (`std::less<>`)
- `map::emplace` may construct and **discard** the value — losing a moved-from `unique_ptr`. Use `try_emplace`
- `emplace_back` bypasses `explicit` constructors and can leak with a raw `new`
- XOR is a bad hash combiner — use the `hash_combine` formula
- `priority_queue` is a **max**-heap; `pop()` returns `void`; priorities can't be updated
- `string_view::data()` is **not null-terminated** — never hand it to a C API
- `string_view` members and returns dangle
- `vector<bool>` has no `data()`, returns proxies that break `auto`, and races on adjacent elements
- Copying a `pmr` container does **not** carry the memory resource across
- `std::pmr::vector<std::string>` only pools the outer buffer — the inner strings use the global heap

Flags and tools

```
-D_GLIBCXX_DEBUG                # libstdc++ debug mode: checked iterators,
invalidation detection
-D_GLIBCXX_ASSERTIONS          # lighter: bounds checks on operator[], at,
front, back
-D_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_EXTENSIVE # libc++ (newer)
/D_ITERATOR_DEBUG_LEVEL=2      # MSVC
-fsanitize=address              # use-after-free on invalidated iterators,
dangling string_view
-Wdangling-gsl                  # Clang: string_view/span lifetime

⚠ _GLIBCXX_DEBUG changes the ABI – every TU and every library must be built with
it.

clang-tidy:
  performance-inefficient-vector-operation    # missing reserve() before a
push_back loop
  performance-unnecessary-copy-initialization
modernize-loop-convert
  readability-container-contains              # count() > 0 -> contains()
  readability-container-size-empty            # size() == 0 -> empty()
  bugprone-use-after-move
```

Exercises worth doing

1. Insert 10,000 elements in sorted position into a `vector` and a `list`, timing both. Repeat for 100 and 1,000,000. Find the crossover point on your machine.
2. Fill a `vector` with and without `reserve` and count reallocations by instrumenting the element's copy/move constructors (reuse §5's `Tracker`).
3. Write a loop that erases from a `vector` while iterating, the wrong way. Run it under `-D_GLIBCXX_DEBUG` and then under ASan. Compare the diagnostics.
4. Benchmark `std::unordered_map<std::string,int>` against `absl::flat_hash_map` or `ankerl::unordered_dense::map` with a million lookups.
5. Add `std::less<>` to a `std::map<std::string,int>` and measure lookups with `const char*` keys before and after. Count allocations with a custom global `operator new`.
6. Build a `std::map<std::string, std::unique_ptr<T>>`, call `emplace` twice with the same key, and confirm the second call consumed your `unique_ptr`. Fix it with `try_emplace`.
7. Write a function that allocates a `std::pmr::vector` from a stack buffer, then set the default resource to `null_memory_resource()` and confirm the function still doesn't throw.
8. Try to write a generic `template <class C> void f(C& c) { auto& first = c[0]; }` and instantiate it with `vector<int>` and `vector<bool>`.

Chapter 12 — The STL — Iterators, Algorithms, Ranges

What this chapter covers

- Iterator categories and **C++20 iterator concepts**
- Iterator adaptors: `back_inserter`, `reverse_iterator`, `move_iterator`, stream iterators
- `<algorithm>` — the full surface: `sort`, `stable_sort`, `find`, `transform`, `accumulate/reduce`, `partition`, `rotate`, `unique`, `lower_bound`, set operations, heap operations
- **The erase-remove idiom** (and `std::erase/erase_if` in C++20)
- `<numeric>`: `iota`, `accumulate`, `inner_product`, `gcd`, `midpoint`
- Predicates, projections, comparators, **strict weak ordering** ⚠
- **Ranges (C++20)**: `views::filter`, `transform`, `take`, `drop`, `join`, `split`, `zip` (C++23), `enumerate` (C++23)
- Lazy evaluation, range adaptors, pipe syntax, **range algorithms with projections**
- ⚠ Dangling ranges, `owning_view`, `borrowed_range`
- Parallel algorithms (`std::execution::par`)

Stepanov's insight: separate **what** you're computing from **what you're computing it on**. Iterators are the glue, algorithms are the reusable half, and C++20's ranges finally made the whole thing composable. This section is mostly a reference surface, with two things you genuinely must internalize: **strict weak ordering** and **which algorithms don't do what their names say**.

12.1 Iterator categories and C++20 concepts

The classic hierarchy

Each category is a refinement of the one above it.

Category	Adds	Examples
Input	<code>*it</code> (read once), <code>++it</code> , <code>==</code>	<code>istream_iterator</code>
Output	<code>*it = v</code> , <code>++it</code>	<code>back_insert_iterator</code> , <code>ostream_iterator</code>
Forward	multi-pass, default-constructible	<code>forward_list</code> , <code>unordered_map</code>
Bidirectional	<code>--it</code>	<code>list</code> , <code>map</code> , <code>set</code>
Random access	<code>it + n</code> , <code>it - it</code> , <code>it[n]</code> , <code><</code>	<code>deque</code>
Contiguous (C++17 tag)	elements are physically adjacent; <code>std::to_address</code>	<code>vector</code> , <code>array</code> , <code>string</code> , <code>span</code>

Why it matters: `std::sort` requires random access (so it doesn't compile on `std::list` — use the member `list::sort`), and `std::vector`-only optimizations like `memmove`-based `copy` kick in for contiguous iterators.

iterator_traits

```
using It = std::vector<int>::iterator;
std::iterator_traits<It>::value_type;           // int
std::iterator_traits<It>::difference_type;      // ptrdiff_t
std::iterator_traits<It>::reference;            // int&
std::iterator_traits<It>::iterator_category;    // random_access_iterator_tag

// C++20 shorthands
std::iter_value_t<It>; std::iter_reference_t<It>; std::iter_difference_t<It>;
```

C++20 iterator concepts — and the two real changes

```
std::input_or_output_iterator, std::input_iterator, std::output_iterator,
std::forward_iterator, std::bidirectional_iterator,
std::random_access_iterator, std::contiguous_iterator
std::sentinel_for<S, I>, std::sized_sentinel_for<S, I>
```

Change 1: iterator and sentinel are decoupled. `end()` no longer has to be the same type as `begin()`:

```
struct NullSentinel {
    friend bool operator==(const char* p, NullSentinel) { return *p == '\0'; }
};
// A null-terminated string is now a range, with no strlen() pass
std::ranges::for_each(cstr, NullSentinel{}, print);
```

This is what makes infinite ranges (`views::iota(0)`), predicate-terminated ranges, and cheap C-string ranges expressible.

Change 2: the C++17 categories were subtly broken for proxy iterators. The old `ForwardIterator` required `*it` to return a real `T&`, which `vector<bool>` (§11.11) and `zip` violate. C++20's concepts are defined in terms of `iter_reference_t` and don't require a true reference, so proxy iterators finally have a legal category. (This is why some types report a C++17 category of `input` and a C++20 concept of `random_access` simultaneously.)

Writing your own

`std::iterator` as a base class is **deprecated**. Provide the typedefs directly:

```
class MyIter {
public:
    using value_type      = int;
    using difference_type = std::ptrdiff_t;
    using iterator_category = std::forward_iterator_tag; // C++17 compatibility
    using iterator_concept = std::forward_iterator_tag; // C++20

    int& operator*() const;
    MyIter& operator++();
    MyIter operator++(int);
    bool operator==(const MyIter&) const = default;
};
static_assert(std::forward_iterator<MyIter>); // ✓ check it, don't
hope
```

That `static_assert` will save you hours — concept failures at the definition are far cheaper than at the use site.

12.2 Iterator adaptors

```
#include <iterator>

// Insert iterators – turn assignment into insertion
std::copy(src.begin(), src.end(), std::back_inserter(dst)); // push_back
std::copy(src.begin(), src.end(), std::front_inserter(lst)); // push_front
std::copy(src.begin(), src.end(), std::inserter(s, s.begin())); // insert (works on
set/map)

// Reverse
std::reverse_iterator{v.end()}; // or v.rbegin()

// Move – dereference yields an rvalue
std::copy(std::move_iterator{src.begin()}, std::move_iterator{src.end()},
          std::back_inserter(dst)); // MOVES elements out of src
// or, C++20: std::ranges::copy(src | std::views::as_rvalue, ...)

// Stream iterators
std::vector<int> v{std::istream_iterator<int>{std::cin},
```

```
std::istream_iterator<int>{});
std::copy(v.begin(), v.end(), std::ostream_iterator<int>{std::cout, ", "});

// C++20
std::counted_iterator{it, 5};           // an iterator that knows it has 5 left
std::common_iterator{it, sentinel};    // adapt an iterator/sentinel pair to a same-
type pair
```

⚠ **back_inserter defeats sizing.** Each `push_back` may reallocate; the algorithm can't know the final size:

```
dst.reserve(src.size());                // ✓ do this
first
std::copy(src.begin(), src.end(), std::back_inserter(dst));
```

⚠ **reverse_iterator::base() is offset by one.** `rit.base()` points one *past* what `*rit` refers to. This is the correct behaviour (it keeps `rbegin()` ↔ `end()` aligned) and it catches everyone once:

```
auto rit = std::find(v.rbegin(), v.rend(), x);
v.erase(std::next(rit).base());        // erase the element rit refers to
```

12.3 The `<algorithm>` surface

Non-modifying

```
all_of  any_of  none_of
find  find_if  find_if_not  find_end  find_first_of  adjacent_find
count  count_if
mismatch  equal  lexicographical_compare  lexicographical_compare_three_way
(C++20)
search  search_n
for_each  for_each_n
```

Modifying

```
copy  copy_if  copy_n  copy_backward
move  move_backward
fill  fill_n  generate  generate_n
transform                                // 1-range and 2-range forms
replace  replace_if  replace_copy
swap_ranges  iter_swap
reverse  rotate  shift_left  shift_right (C++20)
shuffle  sample (C++17)
```



```
unique    unique_copy
remove    remove_if           // ⚠ see §12.4
```

Partitioning

```
partition    stable_partition    is_partitioned    partition_point    partition_copy
```

Sorting and searching (sorted ranges only)

```
sort    stable_sort    partial_sort    partial_sort_copy    nth_element
is_sorted    is_sorted_until
lower_bound    upper_bound    equal_range    binary_search
merge    inplace_merge
includes    set_union    set_intersection    set_difference    set_symmetric_difference
```

Heaps, min/max, permutations

```
make_heap    push_heap    pop_heap    sort_heap    is_heap    is_heap_until
min    max    minmax    min_element    max_element    minmax_element    clamp
next_permutation    prev_permutation    is_permutation
```

The ones with real detail behind them

sort vs **stable_sort**

	Algorithm	Complexity	Extra memory	Equal elements
sort	introsort (quicksort → heapsort → insertion)	$O(n \log n)$ guaranteed	$O(\log n)$	reordered
stable_sort	merge sort	$O(n \log n)$ with a buffer, $O(n \log^2 n)$ without	$O(n)$ if available	order preserved

sort requires **random access** — that's why `std::list` has a member `sort()`.

nth_element — **the underused one**. $O(n)$ selection, not $O(n \log n)$:

```
std::nth_element(v.begin(), v.begin() + k, v.end());
// v[k] is now the element that would be there if sorted;
// everything before it is <=, everything after is >=. Nothing else is ordered.
auto median = v[v.size()/2];
```

partial_sort — **top-k** in $O(n \log k)$:

```
std::partial_sort(v.begin(), v.begin() + 10, v.end()); // first 10 fully sorted
```

rotate returns the new position of the old first element (since C++11), which makes it composable — it's the primitive behind "move this element to position k".

unique only removes *consecutive* duplicates. Sort first, or you'll silently get partial deduplication.

Binary search family — all four require a **sorted** range (or at least one partitioned by the predicate):

```
std::lower_bound(v.begin(), v.end(), x); // first element NOT LESS than x
std::upper_bound(v.begin(), v.end(), x); // first element GREATER than x
std::equal_range(v.begin(), v.end(), x); // both, as a pair
std::binary_search(v.begin(), v.end(), x); // just a bool
```

⚠ **Never use the free `std::lower_bound` on a `std::set` or `std::map`.** The free function does $O(\log n)$ comparisons but $O(n)$ iterator advances on a bidirectional iterator. Use the member `s.lower_bound(x)`, which walks the tree.

Search with a searcher (C++17) — for repeated substring searches:

```
std::boyer_moore_searcher searcher{needle.begin(), needle.end()};
auto it = std::search(hay.begin(), hay.end(), searcher); // preprocesses once,
reuse it
```

std::shuffle needs a proper URBG. **std::random_shuffle** was removed in C++17, and **rand()** should not be used:

```
std::mt19937 gen{std::random_device{}}();
std::shuffle(v.begin(), v.end(), gen);
```

12.4 The erase-remove idiom ⚠

Why it exists

std::remove **doesn't remove anything**. It can't — algorithms only have iterators, not the container. It compacts the surviving elements to the front and returns the new logical end; the tail is left in a valid-but-undefined (moved-from) state, and **size()** is unchanged.

```
std::vector<int> v{1, 2, 3, 2, 4};
auto newEnd = std::remove(v.begin(), v.end(), 2);
```

```
// v is now {1, 3, 4, ?, ?}, v.size() == 5 ⚠️

v.erase(newEnd, v.end()); // the idiom
v.erase(std::remove(v.begin(), v.end(), 2), v.end()); // usually written as
one line
v.erase(std::remove_if(v.begin(), v.end(), pred), v.end());
v.erase(std::unique(v.begin(), v.end(), v.end())); // same shape for
unique
```

Forgetting the `erase` half is a classic bug that produces a container with stale trailing elements and no error.

C++20: just use `std::erase` / `std::erase_if`

```
std::erase(v, 2); // free functions, in <vector>, <string>,
<deque>, <list>...
std::erase_if(v, pred);
std::erase_if(myMap, [](const auto& kv) { return kv.second == 0; }); // works on
map/set too
```

⚠️ **Never use `std::remove` on an associative container.** The elements are `const`-keyed and node-based; use the member `erase` or `std::erase_if`.

⚠️ And the erase-while-iterating loop for associative containers (pre-C++20):

```
for (auto it = m.begin(); it != m.end(); ) {
    if (pred(*it)) it = m.erase(it); // erase returns the next iterator
    else ++it;
}
```

12.5 <numeric>

```
std::iota(v.begin(), v.end(), 0); // fill with 0,1,2,3...
std::accumulate(v.begin(), v.end(), 0); // left fold, sequential
std::accumulate(v.begin(), v.end(), 0, std::multiplies{});
std::inner_product(a.begin(), a.end(), b.begin(), 0); // dot product
std::partial_sum(v.begin(), v.end(), out.begin()); // running total
std::adjacent_difference(v.begin(), v.end(), out.begin());

std::gcd(a, b); std::lcm(a, b); // C++17
std::midpoint(a, b); // C++20 ⚠️ overflow-safe
std::lerp(a, b, t); // C++20

// C++17 parallel-friendly versions
std::reduce(v.begin(), v.end());
std::transform_reduce(v.begin(), v.end(), 0, std::plus{}, [](int x){ return x*x;
```

```
});
std::inclusive_scan(...); std::exclusive_scan(...);
```

⚠ The `accumulate` init-type trap

The accumulator's type is the type of the **init** argument, not the element type:

```
std::vector<double> v{1.5, 2.5, 3.5};
std::accumulate(v.begin(), v.end(), 0);      // ⚠ 0 is an INT -> accumulates in
int -> 6
std::accumulate(v.begin(), v.end(), 0.0);    // ✓ 7.5
```

Same trap with `int` overflow on large sums — use `0LL` or `std::int64_t{0}`.

⚠ `accumulate` vs `reduce`

`accumulate` is a strict **left fold, in order**. `reduce` may reorder and regroup — so it requires the operation to be **associative and commutative**, and it can give different results for floating point:

```
std::accumulate(v.begin(), v.end(), 0.0);      // deterministic
std::reduce(std::execution::par, v.begin(), v.end(), 0.0); // parallel, may
differ in the low bits
```

For reproducible float results, use `accumulate`. For speed on large ranges, `reduce`.

⚠ `midpoint` exists for a reason

```
int mid = (low + high) / 2;      // ⚠ overflows – the famous binary-search
bug
int mid = low + (high - low) / 2; // ✓ the manual fix
int mid = std::midpoint(low, high); // ✓ C++20, and it rounds toward `low`
```

12.6 Predicates, comparators, and strict weak ordering ⚠

Strict weak ordering — the contract

`std::sort`, `std::map`, `std::set`, `lower_bound`, and the heap algorithms all require the comparator to be a **strict weak ordering**:

1. **Irreflexive**: `comp(a, a)` is false
2. **Asymmetric**: if `comp(a, b)` then `!comp(b, a)`
3. **Transitive**: `comp(a, b) && comp(b, c) ==> comp(a, c)`
4. **Transitivity of incomparability**: if `a` and `b` are equivalent and `b` and `c` are equivalent, then `a` and `c` are equivalent

⚠ Violating it is UB, not "wrong order"

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a <= b; }); // ⚠ NOT
irreflexive
```

This doesn't produce a badly-sorted vector. In libstdc++, `sort`'s insertion-sort phase uses "the comparator will eventually return false" as its loop guard — with `<=`, it runs **past the end of the buffer** and corrupts memory. It's a genuine, reproducible segfault.

Other real violations:

```
// NaN — comp(a,b) and comp(b,a) are both false, but a != b. Breaks rule 4.
std::sort(floats.begin(), floats.end()); // ⚠ UB if any NaN is present

// A "comparator" that isn't transitive
[](const P& a, const P& b) { return a.x < b.x || a.y < b.y; }; // ⚠ wrong AND
non-transitive
[](const P& a, const P& b) { return std::tie(a.x, a.y) < std::tie(b.x, b.y); };
// ✓

// Random or stateful comparators
[](auto, auto) { return rand() % 2; }; // ⚠
```

`std::tie` (or, better, a defaulted `operator<=>`, §9.5) is how you write multi-field comparisons without getting them wrong.

Debug modes will catch some of this: `-D_GLIBCXX_DEBUG` validates the ordering on every `sort`.

Predicates must be pure

Algorithms may **copy** predicates an unspecified number of times, and may call them in an unspecified order. A stateful predicate is therefore unreliable:

```
int count = 0;
std::remove_if(v.begin(), v.end(), [count](int) mutable { return ++count > 3; });
// ⚠ each copy of the lambda has its own `count`. Results are unspecified.
```

Capture by reference if you truly need shared state — and know that the call order is still unspecified for most algorithms (`for_each` is the notable one that guarantees in-order).

Projections (C++20)

The ranges algorithms take an optional **projection** — a function applied to each element before the comparator or predicate sees it. This eliminates most one-line lambdas:

```
std::ranges::sort(people, {}, &Person::age);           // sort by age
std::ranges::sort(people, std::greater{}, &Person::name); // descending by
name
std::ranges::find(people, "Alice", &Person::name);
std::ranges::max_element(people, {}, &Person::score);
auto n = std::ranges::count_if(people, isAdult, &Person::age);
```

`{}` means "the default comparator" (`std::ranges::less`). The projection can be any callable, including a member pointer (thanks to `std::invoke`, §6.11).

12.7 Ranges (C++20)

What changed

```
std::sort(v.begin(), v.end());           // C++98: pass two halves of one thing
std::ranges::sort(v);                    // C++20: pass the thing
```

Three real improvements: **you can pass a container**, **algorithms are constrained** (so errors say "not a random_access_range" instead of 200 lines), and **views compose**.

The `std::ranges::` algorithms are **niebloids** — function objects, not templates — so ADL can't hijack them (§10.11).

Views

Lazy, non-owning, composable adaptors. Nothing is computed until you iterate.

```
namespace rv = std::views;

auto result = data
    | rv::filter([](int n) { return n % 2 == 0; })
    | rv::transform([](int n) { return n * n; })
    | rv::take(5);

for (int x : result) { ... }           // the work happens HERE, one element at a
time
```

No intermediate containers are created. Each element flows through the whole pipeline before the next one starts.

The catalogue

C++20

`filter`, `transform`

the two workhorses

C++20

<code>take, take_while, drop, drop_while</code>	slicing
<code>reverse</code>	requires bidirectional
<code>join</code>	flatten a range of ranges
<code>split, lazy_split</code>	split on a delimiter
<code>keys, values, elements<N></code>	project tuples/pairs
<code>iota</code>	<code>views::iota(0)</code> is infinite; <code>views::iota(0, 10)</code> is 0..9
<code>single, empty, counted, common, all</code>	utilities

C++23

<code>zip, zip_transform</code>	iterate several ranges in lockstep
<code>enumerate</code>	index + element pairs
<code>adjacent<N>, pairwise</code>	sliding tuples
<code>chunk, slide, stride</code>	windowing
<code>chunk_by</code>	group by a binary predicate
<code>cartesian_product</code>	
<code>join_with</code>	join with a separator
<code>as_rvalue, as_const</code>	
<code>repeat</code>	

`ranges::to<C>()` materialize a view into a container

```
// C++23 – the ones you'll reach for constantly
for (auto [i, name] : names | rv::enumerate) { ... }
for (auto [a, b]    : rv::zip(xs, ys))          { ... }

auto v = data | rv::filter(pred) | rv::transform(f) | std::ranges::to<std::vector>
();
```

Before C++23, materializing is manual:

```
std::vector<int> out;
std::ranges::copy(data | rv::filter(pred), std::back_inserter(out));
```

⚠ Views gotchas

1. `filter_view` caches `begin()`. Finding the first element that satisfies the predicate is $O(n)$, so the view caches it — which means `begin()` is **not `const`** and a `filter_view` is not a `const`-iterable range:

```
void f(const auto& view) { for (auto x : view) {} } // ⚠ fails for a filtered view
```

This propagates: anything built on top of `filter` inherits the problem. It's the single most common ranges complaint.

2. Views must be cheap to copy. They're semiregular value types; copying one copies the *adaptor*, not the data. That means a view holding a large lambda by value costs on every copy.

3. Multiple passes re-evaluate. A `transform_view` recomputes `f` every time you dereference. Iterating twice runs the function twice:

```
auto v = data | rv::transform(expensiveFn);
auto total = std::ranges::fold_left(v, 0, std::plus{}); // C++23
auto max    = std::ranges::max(v);                      // ⚠ expensiveFn runs AGAIN
```

4. Compile times and debug performance. Range pipelines instantiate deeply (\$10.10) and can be dramatically slower at `-O0`. Measure at `-O2` before drawing conclusions.

12.8 ⚠ Dangling ranges, `borrowed_range`, `owning_view`

The library actively defends against returning iterators into a temporary:

```
std::vector<int> makeVec();

auto it = std::ranges::find(makeVec(), 5);
// `it` has type std::ranges::dangling — a tag type with NO operations.
*it;      // ✗ COMPILE ERROR, not a runtime use-after-free.
```

This is a real improvement over `std::find(makeVec().begin(), makeVec().end(), 5)`, which was silent UB.

`borrowed_range`

A range whose iterators stay valid even after the range object dies — because it doesn't own the data:

```
std::ranges::borrowed_range<std::span<int>>; // ✓ true
std::ranges::borrowed_range<std::string_view>; // ✓ true
std::ranges::borrowed_range<std::vector<int>>; // ✓ true for LVALUES
(ref_view)
```



```
std::ranges::borrowed_range<std::vector<int>&&>;    // X false – rvalue owns
its data
```

So `std::ranges::find(someSpan, x)` on a temporary span is fine; on a temporary vector it isn't. The library encodes this in the return type.

`views::all, ref_view, owning_view`

```
std::vector<int> v;
auto a = v | rv::all;           // ref_view<vector<int>> – refers to v
auto b = makeVec() | rv::all;   // owning_view<vector<int>> – MOVES the vector
                                in, owns it
```

`owning_view` (added by a C++20 defect report) is what makes `makeVec() | views::filter(...)` safe — the pipeline takes ownership of the temporary.

⚠ The related range-for trap was fixed in C++23 (P2718R0):

```
for (auto x : getObject().getVectorRef()) { ... }
// C++20: the getObject() temporary is NOT lifetime-extended -> UB (§3.4)
// C++23: all range-init temporaries are extended -> fine
```

12.9 Parallel algorithms

```
#include <execution>

std::sort(std::execution::par, v.begin(), v.end());
std::transform(std::execution::par_unseq, in.begin(), in.end(), out.begin(), f);
std::reduce(std::execution::par, v.begin(), v.end());
```

The four policies

Policy	Meaning
seq	sequential, in order — the default behaviour, stated explicitly
par	may run on multiple threads; each element's work is sequenced within a thread
par_unseq	multiple threads and vectorized — element work may be interleaved within a thread
unseq (C++20)	single thread, vectorized

⚠ `par_unseq` and `unseq` forbid any **synchronization** in the callable — no mutexes, no allocation, no I/O. Interleaved execution within one thread means a lock could deadlock against itself. `par` allows locks but you still must avoid data races.

Practical notes

```
# GCC requires Intel TBB
g++ -std=c++20 main.cpp -ltbb
```

libc++ support has historically lagged; check before relying on it.

⚠ **If an element access function throws, `std::terminate` is called** — parallel algorithms don't propagate exceptions the way you'd hope.

⚠ **Parallelism has a floor.** Thread pool startup and the scatter/gather overhead mean `par` is a pessimization for small ranges or trivial per-element work. `std::sort(par)` typically needs ~100k+ elements to pay for itself. **Always measure.**

⚠ **The `std::ranges::` algorithms have no parallel overloads.** You must use the iterator-pair forms to get an execution policy — a known and awkward gap.

Alternatives when you need more control: **TBB** directly, **OpenMP**, or C++26's `std::execution` senders/receivers (§15, §18).

Gotcha checklist for this section

- ☐ `std::remove` **does not remove** — pair it with `erase`, or use C++20 `std::erase/erase_if`
- ☐ `std::unique` only collapses **consecutive** duplicates — sort first
- ☐ `std::accumulate(v.begin(), v.end(), 0)` accumulates in `int` regardless of element type
- ☐ `accumulate` is ordered; `reduce` may reorder — different float results
- ☐ `(low + high) / 2` overflows — use `std::midpoint`
- ☐ A comparator using `<=` violates strict weak ordering → **memory corruption**, not just bad order
- ☐ NaN in a range makes `std::sort` UB
- ☐ Multi-field comparators need `std::tie` or defaulted `<=>`, not `a.x < b.x || a.y < b.y`
- ☐ Predicates are **copied** an unspecified number of times — no mutable state
- ☐ Free `std::lower_bound` on a `set/map` is O(n) — use the member
- ☐ `back_inserter` prevents sizing — `reserve` first
- ☐ `reverse_iterator::base()` is offset by one
- ☐ `std::random_shuffle` was removed; `std::shuffle` needs a URBG
- ☐ `filter_view::begin()` is O(n), cached, and **not const** — filtered views aren't const-iterable
- ☐ A `transform_view` re-runs its function on **every** pass
- ☐ `ranges::find(temporary, x)` returns `std::ranges::dangling` — a compile error, by design
- ☐ `par_unseq` forbids locks, allocation, and I/O in the callable
- ☐ An exception from a parallel algorithm's callable calls `std::terminate`
- ☐ GCC's parallel algorithms need `-ltbb`
- ☐ `std::ranges::` algorithms have no execution-policy overloads

Flags and tools

```

-D_GLIBCXX_DEBUG           # validates strict weak ordering, checks iterator
ranges
-D_GLIBCXX_ASSERTIONS      # lighter-weight checks
/D_ITERATOR_DEBUG_LEVEL=2  # MSVC
-fsanitize=address          # the sort-comparator overrun shows up here
-ltbb                      # GCC parallel algorithms
-fconcepts-diagnostics-depth=3 # readable ranges/concept errors on GCC

clang-tidy:
  modernize-loop-convert
  modernize-use-ranges      # newer: std:: -> std::ranges::
  readability-use-anyofallof
  performance-inefficient-algorithm # e.g. std::find on a set instead of set::find
  bugprone-inaccurate-erase      # the missing half of erase-remove

```

Exercises worth doing

1. Sort a vector with a `<=` comparator under ASan and `-D_GLIBCXX_DEBUG`. Read both diagnostics.
2. Write the erase-remove idiom, then deliberately omit the `erase`. Print `size()` and the contents. Then replace the whole thing with `std::erase_if`.
3. `std::accumulate` a `vector<double>` with init `0` and with `0.0`. Explain the difference to yourself out loud.
4. Implement top-10-by-score three ways: full `sort`, `partial_sort`, `nth_element`. Benchmark all three on a million elements.
5. Write a pipeline with `views::filter | views::transform`, add a print statement inside each lambda, and iterate. Confirm the interleaved (per-element) evaluation order.
6. Pass a `filter_view` to a function taking `const auto&`. Read the error. Then fix it.
7. Call `std::ranges::find` on a function returning a vector by value. Try to dereference the result.
8. Rewrite five hand-written loops in your codebase as ranges pipelines. Compare compile time and `-O2` assembly for one of them.
9. Benchmark `std::sort` vs `std::sort(std::execution::par, ...)` at 1k, 100k, and 10M elements. Find your crossover.

Chapter 13 — Error Handling

What this chapter covers

- **Exceptions:** `throw`, `try/catch`, catch by `const&`, rethrow
- **Exception safety guarantees:** no-throw, strong, basic
- **RAII makes exception safety tractable** — this is the whole argument
- `noexcept`, `std::terminate`, and destructors that throw ⚠
- `std::exception` hierarchy, custom exceptions, `nested_exception`
- `std::optional`, `std::expected (C++23)`, `std::variant` for errors
- Error codes, `std::error_code` / `error_condition` / `system_error`
- ⚠ **The exception debate:** exceptions banned in games/embedded/HFT; `-fno-exceptions` and what you lose

- `assert`, `NDEBUG`, **contracts (C++26)**
- ⚠ Never let an exception escape a destructor, a `noexcept` function, or a C callback

C++ has more error-handling mechanisms than any comparable language — exceptions, error codes, `optional`, `expected`, `variant`, `error_code`, assertions, and soon contracts. That's not redundancy: they answer different questions. The skill is knowing which failure mode you're modelling, and knowing that **exception safety is a property of the types you use, not the code you write**.

13.1 Exceptions: mechanics

Throw and catch

```
void f() {
    if (bad) throw std::runtime_error("something failed"); // throw BY VALUE
}

try {
    f();
} catch (const std::invalid_argument& e) { // most derived first
    ...
} catch (const std::runtime_error& e) {
    log(e.what());
} catch (const std::exception& e) { // base last
    ...
} catch (...) { // everything, including non-std types
    ...
}
```

Rules that matter:

- **Throw by value, catch by `const&`**. Catching by value slices (§8.6) and copies; catching by non-const reference is pointless.
- **Catch clauses are tried in order, first match wins** — no best-match resolution like overloading. Put derived types first or the base handler swallows everything.
- Only these conversions apply when matching: derived→base, non-const→const, array/function decay. **No user-defined conversions**, no integral promotions.

Rethrowing ⚠

```
catch (const std::exception& e) {
    log(e.what());
    throw; // ✓ RETHROW – preserves the original exception object
           // and its type
}

catch (const std::exception& e) {
    throw e; // ✗ throws a COPY, SLICED to std::exception. Type
```

```
information lost.
}
```

This is a genuinely common bug: a `catch (const std::exception&)` that logs and does `throw e;` silently destroys the derived type for every handler upstream.

`exception_ptr` — moving exceptions across threads

```
std::exception_ptr eptr;

try { risky(); }
catch (...) { eptr = std::current_exception(); }    // capture, type-erased

// ...later, possibly on another thread:
if (eptr) std::rethrow_exception(eptr);             // rethrow in this context
```

This is the machinery behind `std::future` and `std::promise` (§15) — an exception thrown in a worker thread is captured, transported, and rethrown when you call `.get()`.

Cost model ⚠️

Modern implementations use **table-based ("zero-cost") exceptions**:

Cost	
Non-throwing path	~zero runtime cost. No checks, no setup.
Binary size	+10–20% for unwind tables (<code>.eh_frame</code> , <code>.gcc_except_table</code>)
Optimization	some barriers — the optimizer must preserve object state at throw points
Throwing path	expensive : ~1–10 μs. Allocates the exception, walks the stack, consults unwind tables, runs destructors.
⚠️ Throughput under contention	the unwinder consults the dynamic loader's data structures, which take a process-global lock on glibc. Many threads throwing simultaneously serialize catastrophically.

That last row is the empirical finding behind much of the anti-exception sentiment (Herb Sutter's measurements in P0709). Exceptions are cheap when rare and terrible when frequent.

Design rule: exceptions are for the exceptional. A failure that happens on 1% of calls should not be an exception. A failure that happens on 0.001% of calls should be.

13.2 Exception safety guarantees

Four levels, from strongest:

Guarantee	Meaning
No-throw (<code>noexcept</code>)	The operation cannot fail. Required for destructors, swap, move operations.
Strong	Commit-or-rollback. If it throws, the program state is exactly as before.
Basic	No leaks, all invariants intact, object still usable — but in an unspecified valid state.
None	Anything may happen. Not acceptable in production code.

```
// NO-THROW
void swap(Buffer& a, Buffer& b) noexcept;

// STRONG — via copy-and-swap (§5.9): the throwing part happens before any
mutation
Buffer& operator=(Buffer other) { swap(*this, other); return *this; }

// BASIC — v is left valid but its contents are unspecified
void appendAll(std::vector<T>& v, const std::vector<T>& src) {
    for (const auto& x : src) v.push_back(x);    // if one push_back throws, some
are in
}

// STRONG version of the same
void appendAll(std::vector<T>& v, const std::vector<T>& src) {
    std::vector<T> tmp = v;                      // all the work...
    for (const auto& x : src) tmp.push_back(x);  // ...on a copy
    v = std::move(tmp);                         // commit with a noexcept
operation
}
```

The recipe for the strong guarantee

Do all the work that can throw first, on the side. Commit with operations that cannot throw.

`vector::push_back` implements this: it allocates the new buffer (may throw), relocates (must not throw — hence `move_if_noexcept`, §5.5), and only then swaps the pointers and frees the old buffer.

⚠ **Strong isn't always achievable or worth it.** It costs a full copy for container operations.

`vector::push_back` gives you strong; `vector::insert` in the middle gives you basic when the element type's move can throw. **Aim for basic everywhere, strong where it's cheap or where the caller genuinely needs rollback.**

13.3 RAII makes exception safety tractable — the whole argument

Without RAII, every function with N resources and M throw points needs cleanup at every combination:

```

void bad() {
    FILE* f = fopen("a", "r");
    if (!f) throw std::runtime_error("open");
    char* buf = (char*)malloc(1024);
    if (!buf) { fclose(f); throw std::bad_alloc{}; }
    try {
        parse(f, buf);                // may throw
    } catch (...) {
        free(buf); fclose(f); throw;    // manual cleanup at EVERY exit
    }
    free(buf); fclose(f);
}

```

Add a third resource and the combinations multiply. Add a throw in the middle and someone forgets a path.

```

void good() {
    File f{"a", "r"};                // RAII
    std::vector<char> buf(1024);      // RAII
    parse(f, buf);                    // may throw – everything is cleaned up,
    in order
}

```

Exception safety is a property of the types you use, not the code you write.

This is why §4 (RAII) and §13 are the same argument told twice. Languages with exceptions and *without* deterministic destruction had to invent *finally* and then discover that everyone forgets to write it. C++ made the cleanup a property of the type, so you cannot forget.

Corollary: **a function whose locals are all RAII types is automatically at least basic-safe**, with no effort. That's the payoff.

13.4 `noexcept`, `std::terminate`, and destructors ⚠

`noexcept` mechanics are in §6.6. The error-handling-specific parts:

Destructors are implicitly `noexcept`

If an exception escapes a destructor, `std::terminate` is called — immediately, with no further unwinding. And a destructor running *during* unwinding that throws is the classic double-exception case the language cannot resolve.

```

class Transaction {
    bool committed_ = false;
public:
    void commit() { doCommit(); committed_ = true; } // ✓ CAN throw – caller
    sees it
}

```

```

    ~Transaction() {
        if (!committed_) {
            try { rollback(); }
            catch (...) { logError(); }           // ✓ swallow – no other
option
        }
    }
};

```

The pattern: give the class an explicit `close()/commit()/flush()` that may throw, and make the destructor the silent best-effort fallback. `std::ofstream` does exactly this — `close()` can set the failbit, the destructor cannot report anything.

`std::uncaught_exceptions()` (C++17, note the plural)

Returns how many exceptions are currently in flight. This is how `scope_fail` (§4.10) distinguishes normal exit from exceptional exit:

```

class ScopeFail {
    int count_ = std::uncaught_exceptions();
    F f_;
public:
    ~ScopeFail() noexcept {
        if (std::uncaught_exceptions() > count_) f_(); // we're unwinding
    }
};

```

The C++98 singular `uncaught_exception()` was broken for nested cases and is removed in C++20.

`std::terminate`

Called on: an exception escaping `noexcept`, escaping a destructor during unwinding, escaping a thread's entry function, escaping `main`, no matching handler, a `std::thread` destroyed while joinable, exceptions from a parallel algorithm's callable (§12.9).

```

std::set_terminate([]{
    if (auto e = std::current_exception()) {
        try { std::rethrow_exception(e); }
        catch (const std::exception& ex) { logFatal(ex.what()); }
        catch (...) { logFatal("unknown exception"); }
    }
    std::abort();
});

```

Worth installing in production — the default handler tells you almost nothing.

13.5 The exception hierarchy and custom exceptions

```

std::exception
├── std::logic_error           // a bug: preconditions violated
│   ├── std::invalid_argument
│   ├── std::domain_error
│   ├── std::length_error
│   ├── std::out_of_range     // vector::at, map::at
│   └── std::future_error
├── std::runtime_error        // a runtime condition
│   ├── std::range_error
│   ├── std::overflow_error
│   ├── std::underflow_error
│   ├── std::system_error     // holds an error_code (§13.7)
│   └── std::format_error
├── std::bad_alloc
├── std::bad_cast             // dynamic_cast<T&> failure
├── std::bad_typeid
├── std::bad_optional_access
├── std::bad_variant_access
├── std::bad_expected_access<E> // C++23
├── std::bad_function_call
└── std::bad_weak_ptr

```

The `logic_error` / `runtime_error` split is a design statement: `logic_error` means *the programmer made a mistake* (arguably it should have been an assertion, §13.9), `runtime_error` means *the world didn't cooperate*.

Custom exceptions

```

class ParseError : public std::runtime_error {
    std::size_t line_;
public:
    ParseError(const std::string& msg, std::size_t line)
        : std::runtime_error(msg), line_(line) {}
    std::size_t line() const noexcept { return line_; }
};

```

Requirements worth respecting:

- **Derive from `std::exception`** (usually via `runtime_error`, which handles the string storage)
- **Copy construction must not throw** — an exception whose copy throws leads to `terminate`. `runtime_error` stores its message in a refcounted buffer for exactly this reason. ⚠ A custom exception with a `std::string` member is copy-throwing.
- **`what()` must be `noexcept`** and should not allocate
- Keep them small and cheap to copy

`nested_exception` — error context chains

```

void low() { throw std::runtime_error("disk read failed"); }

void mid() {
    try { low(); }
    catch (...) { std::throw_with_nested(std::runtime_error("loading config")); }
}

void printChain(const std::exception& e, int depth = 0) {
    std::println("{:{}{}}", "", depth * 2, e.what());
    try { std::rethrow_if_nested(e); }
    catch (const std::exception& inner) { printChain(inner, depth + 1); }
    catch (...) {}
}
// loading config
// disk read failed

```

This is C++'s answer to a stack of "caused by" lines. Underused, and genuinely useful for anything with layers.

13.6 optional, expected, and variant for errors

The three shapes

```

std::optional<Config> parse(std::string_view);           // failed – no reason given
std::expected<Config, ParseError> parse(...);           // failed – here's why (C++23)
std::variant<Config, ParseError> parse(...);           // the pre-C++23 workaround

```

`std::expected<T, E>` (C++23)

The best of the three: a value or an error, in one return, with no allocation and no unwinding.

```

#include <expected>

std::expected<int, std::string> parseInt(std::string_view s) {
    if (s.empty()) return std::unexpected("empty input");
    int value;
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), value);
    if (ec != std::errc{}) return std::unexpected("not a number");
    return value;
}

auto r = parseInt("42");
if (r) use(*r);           // operator* / operator-> ; UB if empty
r.value();               // throws std::bad_expected_access if empty
r.error();               // the E – UB if it HAS a value
r.value_or(0);
r.has_value();

```

The monadic interface is what makes it pleasant — errors short-circuit without any `if` ladder:

```
auto result = readFile(path)
    .and_then(parseJson)           // only runs on success
    .and_then(validate)
    .transform(toConfig)           // maps the value
    .transform_error(addContext); // maps the error
if (!result) log(result.error());
```

Available pre-C++23 as `tl::expected` (header-only, widely used) or Boost.Outcome.

Choosing

Situation	Use
Absence is normal, no reason needed	<code>std::optional<T></code>
Failure is expected and routine (parsing, lookup, I/O, validation)	<code>std::expected<T,E></code>
Failure is genuinely exceptional, or you're deep in a call stack	exception
A constructor can fail	exception (or a factory returning <code>expected</code>)
Crossing a C ABI boundary	error code (§25)
A programmer bug / broken precondition	assert / contract (§13.9)

⚠ The one thing `expected` can't do: **constructors have no return value**. A constructor that can fail must either throw or be replaced by a static factory returning `expected` (with a private constructor).

⚠ And `expected` is only checked if you check it. Mark your functions `[[nodiscard]]` (§6.7) — `std::expected` is itself `[[nodiscard]]`-friendly and ignoring one should be a warning.

13.7 `std::error_code` and `system_error`

The bridge between C-style integer error codes and typed C++ error handling.

```
#include <system_error>

std::error_code ec;
auto size = std::filesystem::file_size(path, ec); // non-throwing overload
if (ec) {
    std::println("{} ({})", ec.message(), ec.value());
    if (ec == std::errc::no_such_file_or_directory) { ... } // portable
    comparison
}
```

The three types

Type	Role
<code>std::error_code</code>	a specific error: an integer value + a pointer to its category. Cheap, no allocation.
<code>std::error_condition</code>	a portable abstraction you compare against (<code>std::errc::permission_denied</code>)
<code>std::error_category</code>	maps values to messages, and specific codes to portable conditions

The comparison `ec == std::errc::no_such_file_or_directory` works across platforms because the category translates a Windows `ERROR_FILE_NOT_FOUND` and a POSIX `ENOENT` to the same condition.

`std::system_error`

The exception form — carries an `error_code`:

```
try { std::filesystem::file_size(path); }           // throwing overload
catch (const std::filesystem::filesystem_error& e) {
    e.code(); e.path1(); e.what();
}
```

The dual-API pattern

`<filesystem>` and the networking proposals all offer **both**: a throwing overload and an `error_code&` overload. It's a good pattern for library code — callers who want exceptions get them, callers in `-fno-exceptions` builds or hot loops get codes.

```
Result doThing(const Input& in);                    // throws
Result doThing(const Input& in, std::error_code& ec); // sets ec, never throws
```

Custom categories

The boilerplate is real (a singleton category class, `make_error_code`, and an `is_error_code_enum` specialization), which is why most projects reach for `expected<T, MyEnum>` instead unless they need to interoperate with `std::system_error`.

13.8 ⚠ The exception debate and `-fno-exceptions`

Who bans them, and why

Domain	Reason
Games (Unreal, most engines)	binary size, unpredictable frame-time spikes, console toolchain history

Domain	Reason
Embedded / automotive	MISRA C++ / AUTOSAR forbid them; no heap for the exception object; freestanding targets may lack an unwinder; certification requires bounded WCET
HFT	throw latency is unbounded and the unwinder takes a global lock
LLVM, Chromium, parts of Google	build size, historical portability, and a preference for auditable error paths

The technical objections, fairly stated:

1. **Unbounded, unpredictable latency** on the throwing path
2. **Poor multithreaded scaling** when throws are frequent (the loader lock)
3. **Binary size**: +10–20% for unwind tables
4. **Invisible control flow** — every call is a potential early return, which is hard to audit
5. **Not available** on some freestanding/certified toolchains

What you actually lose with `-fno-exceptions`

```
g++ -fno-exceptions -fno-rtti ...
```

- `throw` becomes a call to `std::terminate`; `try/catch` won't compile
- `operator new` **aborts instead of throwing `bad_alloc`** — use `new (std::nothrow)`
- `vector::at`, `map::at`, `std::stoi`, `std::stod`, `optional::value`, `variant` alternatives, `std::function` invocation of an empty target — all abort
- `dynamic_cast<T&>` (the reference form) can't report failure — use the pointer form
- **Constructors can no longer fail**, so you need two-phase init or static factories
- Much of the standard library becomes usable-but-limited; some third-party libraries won't compile at all

The counter-argument

Bjarne's, and it's not weak: **banning exceptions in practice produces unchecked error codes**, which is how you get silent data corruption instead of a loud crash. Error codes are only better if they're actually checked, and empirically they often aren't. `[[nodiscard]]` and `std::expected` narrow that gap considerably.

The pragmatic middle, which is where most modern codebases land

`std::expected` for expected failures in the hot path. Exceptions for genuinely exceptional conditions and for constructor failure. A `catch (...)` boundary at the top of each thread and at every C ABI boundary.

What's coming

Herb Sutter's **P0709 "static exceptions"** proposes value-based, deterministic, allocation-free exceptions that use the ordinary return path — aiming to satisfy both camps. Not adopted, still under discussion, worth watching.

13.9 `assert`, `NDEBUG`, and contracts (C++26)

The distinction that matters

Assertions are for programmer bugs. Exceptions and `expected` are for runtime conditions.

A failed precondition means *the code is wrong* — there is nothing to recover from and no point returning an error. A failed file open means *the world is uncooperative* — that's a normal outcome you must handle.

`assert`

```
#include <cassert>
assert(index < size);           // compiled out entirely when NDEBUG is defined
```

⚠ Two hazards:

```
assert(v.pop() == 5);           // ⚠ the SIDE EFFECT disappears in release builds
assert(x = 5);                  // ⚠ assignment, always true – write `==`
```

Never put a side effect inside `assert`.

⚠ Also: `NDEBUG` differing between translation units is an **ODR violation** if it changes any class layout or inline function body (§6.4).

Many production codebases define their own always-on assertion macro (`CHECK` in Abseil, `assert`-like macros in LLVM), because compiling out all your invariant checks in the build you actually ship is a questionable trade.

`static_assert`

```
static_assert(sizeof(void*) == 8, "64-bit only");
static_assert(std::is_nothrow_move_constructible_v<T>); // §5.5
static_assert(condition);                                // C++17: message
optional
```

`[[assume]]` (C++23)

```
[[assume(x > 0)]];           // tells the optimizer; does NOT check. UB if
false.
```

Sharp. This is `__builtin_unreachable`-grade — a wrong assumption is silent UB, and it will not tell you.

Contracts (C++26)

Voted into C++26. Preconditions, postconditions, and assertions as **language** features rather than macros:

```
int divide(int a, int b)
    pre(b != 0)                // precondition – checked on entry
    post(r: r * b <= a)        // postcondition, naming the result
{
    contract_assert(a >= 0);    // in-body assertion
    return a / b;
}
```

Why it's better than `assert`:

- Contracts are part of the **declaration**, so they're visible in the header and to callers, tools, and documentation
- Multiple **evaluation semantics** chosen at build time: *ignore*, *observe* (log and continue), *enforce* (terminate), *quick-enforce* (trap)
- The compiler may use them for optimization when enforced
- A violation handler can be installed centrally

Compiler support is arriving now; treat this as forward-looking rather than something to build on today.

13.10 ⚠ Never let an exception escape

Five boundaries where an escaping exception means `std::terminate` or UB:

```
// 1. Destructors (§13.4)
~T() { try { cleanup(); } catch (...) { log(); } }

// 2. noexcept functions
void f() noexcept { try { risky(); } catch (...) { handle(); } }

// 3. C callbacks and any C ABI boundary (§25) – UB, not just terminate
extern "C" int callback(void* ctx) noexcept {
    try { return static_cast<Impl*>(ctx)->run(); }
    catch (const std::exception& e) { logError(e.what()); return -1; }
    catch (...) { return -1; }
}

// 4. Thread entry functions
std::thread t([]{
    try { work(); }
    catch (...) { /* std::terminate if you don't */ }
});
// ✓ or use std::async / packaged_task, which CAPTURE the exception into the
// future:
auto fut = std::async(std::launch::async, work);
try { fut.get(); } catch (const std::exception& e) { ... } // rethrown here

// 5. main
```

```
int main() try { return realMain(); }
catch (const std::exception& e) { std::println(stderr, "fatal: {}", e.what());
return 1; }
```

Item 3 is the one that produces genuinely undefined behaviour rather than a clean `terminate`: C frames have no unwind information, so the unwinder walks into memory it doesn't understand. Every `extern "C"` function you write should be `noexcept` with a `catch (...)` inside.

Gotcha checklist for this section

- ☐ `throw e;` inside a `catch` **slices**; bare `throw;` rethrows the original
- ☐ Catch by `const&`; catch clauses match **in order**, not best-match
- ☐ Throwing is $\sim\mu\text{s}$ and **serializes across threads** on glibc — never for control flow
- ☐ Destructors are implicitly `noexcept` — a throw during unwinding calls `terminate`
- ☐ Give resource classes an explicit `commit()/close()` that can throw
- ☐ A custom exception with a `std::string` member has a **throwing copy constructor**
- ☐ `what()` must be `noexcept` and shouldn't allocate
- ☐ Strong guarantee = do the throwing work on a copy, commit with `noexcept` operations
- ☐ A move that isn't `noexcept` downgrades `vector`'s guarantees and performance (§5.5)
- ☐ `assert` side effects vanish under `NDEBUG`
- ☐ `NDEBUG` mismatched across TUs is an ODR violation
- ☐ `[[assume]]` doesn't check — a false assumption is silent UB
- ☐ Constructors can't return `expected` — throw, or use a static factory
- ☐ `-fno-exceptions` makes `new`, `at()`, `stoi`, `optional::value` **abort**
- ☐ An exception escaping a `std::thread`'s function calls `terminate` — use `std::async`
- ☐ An exception crossing a C ABI boundary is **UB**, not `terminate`

Flags and tools

```
-fno-exceptions -fno-rtti      # if your domain requires it
-fexceptions              # force on (needed when compiling C that must
be unwindable)
-Wnoexcept                # GCC: functions that could be noexcept
-Wterminate               # a throw in a noexcept context
-Wexceptions              # Clang
-D_GLIBCXX_ASSERTIONS     # keep library preconditions checked in release
-fstack-protector-strong  # unrelated, but pairs with hardened error
handling (§23)

clang-tidy:
  bugprone-exception-escape # ★ finds throws escaping noexcept/dtors
  cert-err58-cpp            # static objects whose ctor can throw
  cert-err60-cpp            # exception types must be nothrow-copyable
  misc-throw-by-value-catch-by-reference
  bugprone-unused-return-value
  modernize-use-nodiscard
```



```
hicpp-exception-baseclass      # throw only types derived from
std::exception
```

Exercises worth doing

1. Write a `catch` block that does `throw e;` and one that does `throw;`, both catching by `const std::exception&`, and observe the type at the outer handler.
2. Implement `appendAll` with the basic guarantee, then with the strong guarantee. Inject a throw at element 5 and verify the container state in both.
3. Write a class whose destructor throws, use it inside another throw's unwinding, and watch `terminate` fire. Then install a `set_terminate` handler that prints the exception.
4. Build the `throw_with_nested` / `rethrow_if_nested` chain across three layers and write the recursive printer.
5. Rewrite a function that returns `bool` + out-parameter as one returning `std::expected` (or `tl::expected`). Then chain three of them with `and_then`.
6. Compile a small program with and without `-fno-exceptions`; diff the binary size and try calling `vector::at` out of range in each.
7. Benchmark a function that throws once per 1,000 calls vs one returning `expected`, single-threaded and then on 8 threads. The multithreaded result is the interesting one.
8. Write an `extern "C"` callback that lets an exception escape. Run it under a debugger and look at where it goes.

Chapter 14 — Undefined Behavior ⚠

What this chapter covers

- UB / unspecified / implementation-defined — the same three categories as C
- The C++-specific catalogue: signed overflow, OOB, null deref, use-after-free, **use-after-move misuse**, dangling references/`string_view`/iterators, **iterator invalidation**, data races, reading uninitialized, strict aliasing, infinite loops without side effects, misuse of `const_cast`, slicing-related, static init order fiasco
- How optimizers exploit it (identical mechanics to C)
- **Sanitizers are mandatory**: ASan, UBSan, TSan, MSan
- `std::launder`, object lifetime rules, `start_lifetime_as` (C++23)

The mechanics are identical to C — same three categories, same optimizer exploitation, same "it worked at `-O0`" failure mode. What's different in C++ is the **catalogue**: object lifetimes, iterator invalidation, dangling views, moved-from misuse, and the fact that a `std::vector` gives you three new ways to have a dangling pointer that a C array never did.

The single most important sentence in this section: **sanitizers are not optional tooling, they are part of the language**. C++ is not memory-safe, and the sanitizers are how you get most of the way there anyway.

14.1 The categories

Category	Meaning	Example
Undefined behavior	No requirements whatsoever. The compiler may assume it never happens.	signed overflow, OOB access, use-after-free, data race
Unspecified behavior	Several valid options; the implementation picks one and need not document or be consistent	order of evaluation of function arguments; whether string literals are shared
Implementation-defined	Same, but the implementation must document its choice	<code>sizeof(int)</code> , <code>char</code> signedness, vtable layout, <code>std::string</code> layout
Erroneous behavior (C++26)	Well-defined-but-wrong: produces a specified (garbage) value, is diagnosable, and does not poison the program	reading an uninitialized variable
IFNDR	Ill-formed, no diagnostic required — the whole program is invalid, silently	ODR violations (§6.4)

Two of these deserve emphasis:

Erroneous behavior (C++26, P2795) is a genuinely new fourth category and a good sign for the language's direction. Reading an uninitialized variable moves from "the compiler may delete your entire function" to "you get an erroneous value, and the implementation is encouraged to diagnose it." It converts a nuclear-grade UB into an ordinary bug.

IFNDR is worse than UB in one respect: UB is a property of an execution path you might not take. IFNDR means the *program* is broken. A mismatched `NDEBUG` across two translation units doesn't corrupt on a particular path — it silently gives you two different definitions of the same class and lets the linker pick.

Why UB exists

Three reasons, all still valid:

1. **Optimization:** assuming no signed overflow lets the compiler promote `int` loop counters to 64-bit registers, hoist bounds computations, and vectorize.
2. **Hardware diversity:** the standard can't mandate what happens on a null dereference across every architecture C++ targets.
3. **Zero-overhead:** the alternative is a runtime check on every operation, which is what C++ explicitly refuses to pay for.

The contract is: **the programmer promises it never happens; the compiler optimizes on that promise.**

14.2 The C++ catalogue

A. Memory and lifetime

```
// Out of bounds
std::vector<int> v(10);
```

```

v[10]; // ⚠ UB – operator[] is unchecked
v.at(10); // ✓ throws
int a[10]; a[10] = 1; // ⚠ UB
p + n; // ⚠ UB if it leaves the array (§3.8)

// Null / invalid dereference
T* p = nullptr; *p;
(*static_cast<T*>(nullptr)).f();

// Use after free / double free
delete p; p->f();
delete p; delete p;

// Dangling – the C++ specialities
const std::string& r = getObj().name(); // §3.4
std::string_view sv = getString(); // §11.9
std::span<int> s = makeVector(); // §3.8
auto f = [&local]{ ... }; return f; // §6.9
int& r2 = v[0]; v.push_back(x); r2 = 1; // §11.5 iterator/reference
invalidation

// Object lifetime
new (buf) T{}; ~T(); obj.f(); // after the destructor started – §14.4
T* p = (T*)malloc(sizeof(T)); p->method(); // no object was ever created there

// Misaligned access
auto* p = reinterpret_cast<double*>(charBuffer + 1); // ⚠ UB even before
dereferencing

```

B. Values and arithmetic

```

int x = INT_MAX; ++x; // ⚠ signed overflow is UB
unsigned u = UINT_MAX; ++u; // ✓ defined – wraps to 0
INT_MIN / -1; INT_MIN % -1; // ⚠ UB (overflow)
x / 0; x % 0; // ⚠ UB
x << 32; x << -1; // ⚠ shift >= width, or negative
(int)hugeDouble; // ⚠ UB if out of range for int

int u; std::cout << u; // ⚠ uninitialized read (→ erroneous in C++26)

// The favourite UBSan demo
char storage = 7;
bool b = *reinterpret_cast<bool*>(&storage);
if (b) ... else ...; // ⚠ a bool holding neither 0 nor 1 – BOTH
branches // can be skipped. Genuinely observed.

enum class E { A = 0, B = 1 };
E e = static_cast<E>(99); // ⚠ out-of-range enum value

```

C. C++-specific

```

// Use-after-move MISUSE – the move itself is fine (§5.6)
std::string s2 = std::move(s1);
s1.size();           // ✓ legal – no precondition
s1.front();          // ⚠ UB if s1 is empty – front() has a
precondition

// Polymorphic deletion without a virtual destructor (§8.3)
Base* p = new Derived; delete p;

// Mismatched new/delete forms
T* p = new T[10]; delete p;           // ⚠ must be delete[]
T* q = new T; delete[] q;             // ⚠

// Pure virtual call from a constructor (§8.2)

// const_cast then MODIFY an originally-const object
const int ci = 42;
*const_cast<int*>(&ci) = 7;           // ⚠ UB. Casting away const to READ is fine.

// Strict aliasing / union punning (§3.9)
float f; int i = *reinterpret_cast<int*>(&f);
union U { float f; int i; }; u.f = 1.0f; use(u.i); // ⚠ UB in C++ (legal in C)

// Infinite loop with no side effects – the forward-progress guarantee
while (true) {}                       // ⚠ UB in C++ (C is more lenient here)

// Flowing off the end of a non-void function
int f(int x) { if (x) return 1; }      // ⚠ UB if x == 0. Compilers often emit
NOTHING,                               // so control falls into the next function.

// Modifying a string literal
char* p = (char*)"hello"; p[0] = 'H';

// Static initialization order fiasco (§7.6)
// – the ORDER is unspecified; using an uninitialized object is UB

// Data races (§15)
// Exception escaping a C ABI boundary (§13.10)

```

⚠ Two rows people underestimate:

- **Flowing off the end of a non-void function.** `-Wreturn-type` catches it, but if you ignore the warning, GCC and Clang genuinely emit no `ret` instruction and execution falls into whatever function follows in the object file. This produces some of the most bewildering bug reports in existence.
- **Use-after-move is not automatically UB.** Moving from an object and then calling `size()`, `clear()`, or `operator=` is perfectly legal. It becomes UB only when you call an operation with a **precondition** the moved-from state doesn't satisfy.

14.3 How optimizers exploit UB

The reasoning chain is always the same:

"This construct would be UB if condition X held. The programmer promised X never holds. Therefore I may assume **!X** and delete everything that only executes when X."

The classics

```
// 1. The null check disappears
void f(int* p) {
    int x = *p;           // dereference => p is not null (else UB)
    if (p == nullptr) return; // therefore this is dead code. DELETED.
    use(x);
}
```

This is the **Linux kernel tun driver CVE-2009-1897** almost exactly: a dereference before a null check, the check optimized away, and a mapped page at address 0 turning it into a local root exploit.

```
// 2. Signed overflow checks fold to false
bool willOverflow(int x) { return x + 1 < x; } // -> `return false;`
                                              // (works fine with unsigned)

// 3. Loop induction variables get widened
for (int i = 0; i < n; ++i) arr[i] = 0;
// i can't overflow => i is monotonically increasing => promote to 64-bit,
// no sign-extension per iteration, vectorize freely.
// With `unsigned i`, wraparound is defined, so the compiler must preserve it.

// 4. The loop that never terminates gets deleted
while (x != 1) { x = (x % 2) ? 3*x+1 : x/2; } // no side effects => forward
progress                                     // => compiler may assume it
                                              // and may delete it entirely
terminates,
```

Time travel ⚠

UB is **not** "the bad thing happens when you reach that line." The compiler's assumption applies to the whole function, so UB can affect code that runs **before** the UB would have executed:

```
void f(int x) {
    printf("about to divide\n");
    int y = 1 / x;           // UB if x == 0
    use(y);
}
```

```
// The compiler may hoist, reorder, or delete the printf. Calling f(0) can print nothing.
```

UB is not "implementation-defined at runtime." It is a compile-time licence to assume.

Escape hatches (and their cost)

```
-fwrapv          # signed overflow wraps (defined). Costs some loop
optimizations.
-ftrapv          # signed overflow traps at runtime. Slower than -fwrapv.
-fno-strict-aliasing # type punning through pointers is allowed. Linux builds
this way.
-fno-delete-null-pointer-checks # keep null checks even after a dereference
```

These are legitimate for legacy code you can't fix, and they are **not** a substitute for fixing it. They also don't compose across libraries — a dependency compiled without them still has the original assumptions baked in.

14.4 Object lifetime, `launder`, `start_lifetime_as`

Aliasing and `std::launder` are covered in §3.9. The lifetime model itself:

When does an object exist?

Lifetime begins when storage is obtained *and* initialization completes. **Lifetime ends** when the destructor call starts, or the storage is released/reused.

Before or after that window, you may:

- ✓ hold the pointer, convert it to `void*`, compare it
- ✗ dereference it, call non-static member functions, access non-static data members, bind a reference to it, `static_cast` it to a derived type

```
alignas(T) std::byte buf[sizeof(T)];
T* p = reinterpret_cast<T*>(buf);
p->method();                // ⚠ UB – no object has been created here

T* q = new (buf) T{};        // ✓ placement new begins the lifetime
q->method();                 // ✓
q->~T();                     // lifetime ends
q->method();                 // ⚠ UB
```

Reusing storage and `std::launder`

```
struct S { const int x; };    // const member – compiler may cache it
```

```

S* p = new S{1};
p->~S();
S* q = new (p) S{2};           // new object in the same storage

p->x;                           // ⚠ UB – p names the OLD object
q->x;                           // ✓ 2
std::launder(p)->x;           // ✓ 2 – re-derives a valid pointer

```

You need `launder` only when the type has `const`/reference members or a vtable and you're placement-newing over existing storage. Allocator, `variant`, and small-buffer-optimized implementations need it; application code essentially never does.

Implicit-lifetime types (C++20, P0593)

C++20 retroactively legalized a huge amount of existing practice: operations like `malloc`, `memcpy`, and `bit_cast` **implicitly create objects** of *implicit-lifetime types* (scalars, arrays, and aggregates with trivial destructors) where doing so would make the program well-defined.

```

struct Header { std::uint32_t magic, len; };           // implicit-lifetime type
void* mem = std::malloc(sizeof(Header));
auto* h = static_cast<Header*>(mem);
h->magic = 0xC0DE;                                   // ✓ legal in C++20 –
object implicitly created                             // (was technically UB
before)

```

`std::start_lifetime_as` (C++23)

For the case P0593 doesn't cover — bytes you didn't allocate, like a network buffer or an `mmap`d file:

```

std::byte buffer[1024];
recv(sock, buffer, sizeof buffer, 0);

auto* h = std::start_lifetime_as<Header>(buffer);     // ✓ C++23, no copy, no
UB
auto* arr = std::start_lifetime_as_array<std::uint32_t>(buffer, 256);

```

Before C++23 the only strictly-correct way to read a byte buffer as a struct was `std::memcpy` into a real object (which the optimizer usually eliminates anyway — so `memcpy` remains the portable answer today).

For value-level punning, `std::bit_cast` (C++20) is the tool (§3.9).

14.5 Sanitizers — mandatory ⚠

If you take one action item from this entire document: **turn on ASan and UBSan in your test builds today**. They find more real bugs per hour than any other C++ practice.

The four

Sanitizer	Catches	Slowdown	Memory	Notes
ASan	heap/stack/global OOB, use-after-free, use-after-scope, use-after-return, double-free, leaks	~2×	~3×	the highest-value one
UBSan	signed overflow, bad shifts, null deref, misalignment, invalid bool/enum, missing return, float→int overflow	~1.2×	~1×	nearly free — run it always
TSan	data races, lock-order inversions	5–15×	5–10×	⚠ incompatible with ASan
MSan	reads of uninitialized memory	~3×	~2×	⚠ requires all code instrumented, including libc++

Practical invocations

```
# The everyday combination – run this on your whole test suite
g++ -std=c++23 -g -O1 -fno-omit-frame-pointer \
    -fsanitize=address,undefined \
    -fno-sanitize-recover=all \
    main.cpp

# Extra ASan checks worth enabling
-fsanitize-address-use-after-scope           # dangling lambda captures, §6.9
ASAN_OPTIONS=detect_stack_use_after_return=1:strict_string_checks=1:check_initialization_order=1

# UBSan sub-checks not in the default set
-fsanitize=undefined,integer,nullability,implicit-conversion

# Threads – separate build, cannot combine with ASan
g++ -g -O1 -fsanitize=thread main.cpp

# Uninitialized reads – needs an instrumented standard library
clang++ -fsanitize=memory -fsanitize-memory-track-origins \
    -stdlib=libc++ -nostdinc++ -I<msan-libcxx>/include ...
```

-fno-sanitize-recover=all makes UBSan **abort** on the first finding rather than printing and continuing — essential in CI, where a printed warning gets ignored.

Complementary tools

Tool	Role
------	------

Tool	Role
Valgrind (memcheck)	no recompile needed; catches uninitialized reads without MSan's build burden; ~20× slower; misses stack OOB that ASan catches
<code>-D_GLIBCXX_DEBUG / _LIBCPP_HARDENING_MODE</code>	library-level: iterator invalidation, strict weak ordering, bounds (§11, §12)
<code>-D_FORTIFY_SOURCE=3</code>	cheap runtime bounds checks on libc calls; usable in production (§23)
<code>-fstack-protector-strong,</code> CFI	production hardening (§23)
Static analysis — clang-tidy, cppcheck, PVS-Studio, Coverity	finds paths tests never execute (§21)

⚠ The fundamental limitation

Sanitizers only find bugs on paths you actually execute. They are dynamic tools. That is why they must be paired with:

- **High test coverage** (§22) — a sanitizer on 40%-covered code checks 40% of your program
- **Fuzzing** (§22) — libFuzzer/AFL++ under ASan+UBSan is the single most productive bug-finding setup that exists, and it's how OSS-Fuzz finds thousands of bugs in widely-used libraries

The CI matrix that's worth having

```
- { compiler: gcc, sanitizers: "address,undefined" }
- { compiler: clang, sanitizers: "address,undefined" }
- { compiler: clang, sanitizers: "thread" }
- { compiler: gcc, flags: "-D_GLIBCXX_DEBUG" }
- { compiler: msvc, flags: "/fsanitize=address /permissive- /W4" }
- fuzz targets, 15 minutes per PR, under ASan+UBSan
```

14.6 Avoiding UB by construction

The goal is to make whole categories unreachable rather than remembering to avoid them.

Instead of	Use	Kills
<code>v[i]</code> in non-hot code	<code>v.at(i)</code> , <code>gsl::at(v, i)</code> , hardened STL	OOB
<code>T* ptr, size_t len</code>	<code>std::span</code> , <code>std::string_view</code>	OOB, length mismatch
raw <code>new/delete</code>	<code>make_unique</code> , containers, values	leaks, UAF, double-free, mismatched delete
uninitialized members	NSDMI (<code>int x = 0;</code> , §7.3)	uninitialized reads

Instead of	Use	Kills
<code>reinterpret_cast</code> for punning	<code>std::bit_cast</code> , <code>memcpy</code>	strict aliasing
C-style casts	named casts	silent const/type violations
<code>int</code> for sizes with wraparound intent	be deliberate; UBSan catches signed overflow, nothing catches unsigned wrap	overflow bugs
raw loops	<code><algorithm></code> , ranges (§12)	off-by-one, invalidation
non-owning members	own by value, or document the lifetime contract	dangling
<code>assert</code> only in debug	keep assertions in release, or use contracts (§13.9)	undetected precondition violations

⚠ The signed/unsigned row is a real debate. Signed overflow is UB — but that means **UBSan can catch it**. Unsigned wraparound is *defined*, so it's silent, and `for (unsigned i = v.size() - 1; i >= 0; --i)` loops forever on an empty vector. The Google style guide and the Core Guidelines both recommend **signed** types for arithmetic and indices for exactly this reason, despite `size()` returning unsigned. `std::ssize()` (C++20) exists to help.

Where the language is going

- **Erroneous behavior** (C++26) removes uninitialized reads from the UB list
- **Contracts** (C++26, §13.9) make preconditions checkable and enforceable
- **Safety profiles** and the **Safe C++** proposals (§23) aim to make subsets of the language statically memory-safe
- **Hardened standard library modes** are becoming production-viable, not just debug tools

Gotcha checklist for this section

- ☐ UB is a **compile-time licence to assume**, not a runtime outcome — it can affect code *before* the UB line
- ☐ Signed overflow is UB; unsigned wrap is **defined** (and therefore silent)
- ☐ A `bool` holding a value other than 0/1 can make **both** branches unreachable
- ☐ Flowing off the end of a non-void function emits no `ret` — execution falls into the next function
- ☐ `while (true) {}` with no side effects is UB in C++ (forward-progress guarantee)
- ☐ Casting away `const` to **read** is fine; to **modify an originally-const object** is UB
- ☐ Union type punning is UB in C++, legal in C — use `bit_cast`
- ☐ Use-after-move is only UB for operations **with preconditions**
- ☐ `delete` vs `delete[]` mismatch, and polymorphic delete without a virtual destructor
- ☐ Misaligned pointers are UB **on formation**, not just on dereference
- ☐ ODR violations are IFNDR — worse than UB, because the whole program is invalid
- ☐ ASan and TSan **cannot be combined** — separate CI jobs
- ☐ MSan needs an instrumented standard library or it drowns in false positives
- ☐ Sanitizers only check **executed** paths — pair with coverage and fuzzing
- ☐ `-D_GLIBCXX_DEBUG` changes the ABI — all TUs and libraries must match

Flags reference

```
# Warnings that catch UB at compile time
-Wall -Wextra -Wpedantic
-Wreturn-type          # ★ missing return – one of the worst UBs
-Wuninitialized -Wmaybe-uninitialized
-Wnull-dereference
-Wstrict-aliasing=3
-Warray-bounds=2
-Wshift-overflow=2
-Wconversion -Wsign-conversion
-Wcast-align
-Wdangling-reference    # GCC 13+
-Wdangling-gsl          # Clang
-Werror                 # once the build is clean

# Runtime
-fsanitize=address,undefined -fno-sanitize-recover=all
-fsanitize-address-use-after-scope
-fsanitize=thread        # separate build
-D_GLIBCXX_ASSERTIONS    # cheap; safe for release
-D_GLIBCXX_DEBUG         # heavy; test builds only
-D_FORTIFY_SOURCE=3 -O2   # production-safe

# Escape hatches for legacy code
-fwrapv -fno-strict-aliasing -fno-delete-null-pointer-checks
```

Exercises worth doing

1. Write `bool willOverflow(int x) { return x + 1 < x; }` and compile at `-O2` on Godbolt. Then add `-fwrapv` and diff.
2. Write a function with a dereference before a null check. Confirm at `-O2` that the check is gone from the assembly.
3. Write `int f(int x) { if (x > 0) return 1; }` and call `f(-1)`. Read the generated assembly and figure out where control actually goes.
4. Do the `bool` punning trick (`char storage = 7`) and print both branches' output at `-O0` and `-O2`. Then run it under UBSan.
5. Take a project you have and turn on `-fsanitize=address,undefined` for its test suite. Count the findings. (There will be findings.)
6. Set up a libFuzzer target under ASan+UBSan for any function that parses input. Run it for 10 minutes.
7. Write a placement-new over an object with a `const` member, access it through the old pointer, and reason about why `std::launder` is required.
8. Compare `for (unsigned i = v.size() - 1; i >= 0; --i)` and the `std::ssize` version on an empty vector.

Chapter 15 — Concurrency

What this chapter covers

- The **C++11 memory model**, data races as UB, happens-before
- `std::thread`, `std::jthread` (C++20) with cooperative cancellation (`stop_token`)
- `mutex`, `recursive_mutex`, `shared_mutex`, `timed_mutex`
- `lock_guard`, `unique_lock`, `scoped_lock` — RAII locking
- `condition_variable` — always in a predicate loop
- `std::atomic`, memory orders, `atomic_ref`, `atomic<shared_ptr>`
- `future`, `promise`, `packaged_task`, `async` (`△` and its footguns)
- `latch`, `barrier`, `semaphore` (C++20)
- `std::execution` / **senders & receivers** (C++26) — structured async
- Thread pools, work stealing, false sharing (`hardware_destructive_interference_size`)
- Lock-free programming, ABA, and why you should use a mutex
- Coroutines (see below)

C++11 did something no mainstream language had done before: it gave a **systems language a formal memory model**. Before that, threads were a library bolted onto a language that officially didn't know they existed, and every guarantee was a vendor promise. Everything in this section rests on that model — and the model's central rule is that a data race is undefined behavior, full stop.

Second theme: **TSan gets its own CI job**. Concurrency bugs are the least reproducible, most timing-dependent, most expensive-to-debug category in the language, and dynamic race detection is the only tool that reliably finds them.

15.1 The C++11 memory model

Memory locations

The unit of the race rule is a **memory location**: an object of scalar type, or a maximal sequence of adjacent bit-fields of non-zero width.

```
struct S {
    int    a;           // distinct location
    int    b;           // distinct location
    unsigned c : 3;      // }
    unsigned d : 5;      // } ONE memory location – c and d share it
    char   e;           // distinct location
};
```

Consequence: two threads writing `s.a` and `s.b` concurrently is **safe**. Two threads writing `s.c` and `s.d` concurrently is a **data race**.

Same for containers: concurrent writes to *distinct elements* of a `vector`, `array`, or `map` are fine. `△` Except `std::vector<bool>` (§11.11), where adjacent elements share a byte.

The data race rule

Two accesses to the **same memory location**, where at least one is a **write**, not ordered by **happens-before**, and not both **atomic** — is a **data race**, which is **undefined behavior**.

Not "you get a torn value." Not "you read a stale value." UB, with all the compile-time consequences from §14 — including the compiler assuming the race can't happen and optimizing on that basis.

Happens-before

The relation that saves you. It's built from:

1. **Sequenced-before** — ordinary program order within a single thread
2. **Synchronizes-with** — established by:
 - a **mutex** unlock and a subsequent lock of the same mutex
 - an atomic **release** store and an **acquire** load that reads it
 - **std::thread** construction (creator → new thread) and **join()** (thread → joiner)
 - **promise::set_value** → **future::get**
 - **latch::count_down** → **latch::wait**
3. **Transitivity** — inter-thread happens-before chains through both of the above

If A happens-before B, then B sees everything A did. If neither happens-before the other and they touch the same location, you have a race.

The two contracts to remember

- **const member functions must be safe to call concurrently** (§7.5). Every standard type honours this, and every standard algorithm assumes it. A **mutable** cache in a **const** method is a synchronization obligation.
- **Distinct objects are independent**. Two threads using two separate **std::vectors** never need synchronization.

15.2 **std::thread** and **std::jthread**

std::thread — the C++11 version, with sharp edges

```
std::thread t{work, arg1, arg2};
t.join();                      // or t.detach()
```

⚠ Three traps:

```
// 1. Destroying a joinable thread calls std::terminate
{
    std::thread t{work};
}                                // ⚠ terminate – you must join or detach
first                            // (and an exception before the join())
does it too)
```

```
// 2. Arguments are COPIED (or moved) into the thread
std::atomic<int> counter;
std::thread t{worker, counter};      // X won't compile (atomic isn't copyable) –
good
std::vector<int> data;
std::thread t2{worker, data};        // ⚠️ COMPILES, and silently copies the
vector
std::thread t3{worker, std::ref(data)}; // ✓ (§3.7)

// 3. An exception escaping the thread function calls terminate (§13.10)
std::thread t4{[] { try { work(); } catch (...) { log(); } };
```

`std::thread::hardware_concurrency()` returns a **hint** — it may return 0, it doesn't know about cgroup/container limits, and it doesn't account for hyperthreading policy.

`std::jthread` (C++20) — use this instead

Two improvements: it **joins in its destructor**, and it carries a **stop token** for cooperative cancellation.

```
void worker(std::stop_token st, Queue& q) {
    while (!st.stop_requested()) {
        auto item = q.tryPop();
        if (item) process(*item);
    }
}

{
    std::jthread t{worker, std::ref(queue)}; // stop_token is injected
    automatically
    doOtherWork();
} // destructor: request_stop() THEN join(). No terminate, no leak.
```

```
std::stop_source src;
std::stop_token tok = src.get_token();
std::stop_callback cb{tok, []{ wakeUpTheIOLayer(); }}; // runs on request_stop()
src.request_stop();
```

⚠️ Cancellation is **cooperative** — nothing preempts a thread. Your loop must check `stop_requested()`, and any blocking wait must be interruptible (`condition_variable_any::wait` takes a `stop_token` in C++20 for exactly this).

`thread_local`

```
thread_local int counter = 0; // one instance per thread
```

Constructed on first use per thread, destroyed at thread exit. ⚠ Access is not free (a TLS lookup, sometimes through a function call in shared libraries), and destruction order at thread exit has the same hazards as static destruction (§7.6).

15.3 Mutexes

Type	Use
<code>std::mutex</code>	the default
<code>std::recursive_mutex</code>	same thread may lock repeatedly — ⚠ usually a design smell
<code>std::timed_mutex</code>	<code>try_lock_for</code> / <code>try_lock_until</code>
<code>std::shared_mutex</code> (C++17)	reader/writer: many shared, one exclusive
<code>std::shared_timed_mutex</code> (C++14)	both

⚠ **`recursive_mutex` almost always means the design is wrong.** It usually appears when a public locking method calls another public locking method. The fix is a private unlocked `xImpl()` that both public wrappers call while holding the lock once.

⚠ **`shared_mutex` is frequently slower than `std::mutex`.** The shared bookkeeping (reader counting, writer starvation avoidance) costs more than the concurrency saves unless critical sections are long and reads massively outnumber writes. **Measure before switching** — the intuition "reads are concurrent so it must be faster" is wrong for short critical sections.

15.4 RAIL locking

```
std::lock_guard  lg{m};           // minimal: lock in ctor, unlock in dtor. Not
movable.
std::unique_lock ul{m};           // movable, deferrable, unlockable,
relockable
std::shared_lock sl{sharedMtx};   // reader lock for shared_mutex
std::scoped_lock sc{m1, m2, m3};  // C++17: multiple mutexes, DEADLOCK-FREE
ordering
```

```
std::unique_lock ul{m, std::defer_lock};    // don't lock yet
std::unique_lock ul2{m, std::try_to_lock};   // try, check ul2.owns_lock()
std::unique_lock ul3{m, std::adopt_lock};    // already locked; just take
ownership
ul.unlock(); ul.lock();                     // manual control when you need it
```

Use `lock_guard` by default, `unique_lock` when a `condition_variable` or manual unlock requires it, `scoped_lock` for two or more mutexes.

⚠ The unnamed-temporary bug

```
std::lock_guard{m};           // ⚠ temporary – locks and IMMEDIATELY
unlocks                      //
std::lock_guard lg{m};        // ✓ named – held for the scope
```

The first form is a one-character mistake with no diagnostic. `std::lock_guard` is marked `[[nodiscard]]` in newer library versions specifically to catch it (§6.7); `-Wunused-value` sometimes helps.

Deadlock avoidance

```
// ✗ classic AB-BA deadlock
void t1() { std::lock_guard a{m1}; std::lock_guard b{m2}; }
void t2() { std::lock_guard b{m2}; std::lock_guard a{m1}; }

// ✓ scoped_lock uses std::lock's deadlock-avoidance algorithm
void t1() { std::scoped_lock lk{m1, m2}; }
void t2() { std::scoped_lock lk{m2, m1}; } // same result, no deadlock
```

The four rules:

1. **Consistent lock ordering** globally (a documented hierarchy, or address order)
2. **scoped_lock for multiple mutexes** — never nest lock guards by hand
3. **Never call user code (callbacks, virtual functions) while holding a lock** — you don't know what it locks
4. **Hold locks for the shortest possible time** — copy the data out, release, then process

A **hierarchical mutex** (each mutex has a level; you may only acquire strictly lower levels) is a cheap way to make rule 1 enforceable at runtime; it's ~30 lines and worth having in any codebase with more than three mutexes.

15.5 condition_variable

The pattern — memorize this shape

```
std::mutex          m;
std::condition_variable cv;
std::queue<Task>    q;
bool               done = false;

// Producer
{
    std::lock_guard lk{m};
    q.push(task);
}
cv.notify_one();           // notify after releasing (or while holding –
```



```
both work)

// Consumer
{
    std::unique_lock lk{m}; // MUST be unique_lock
    cv.wait(lk, []{ return !q.empty() || done; }); // predicate form – always
    if (!q.empty()) { auto t = q.front(); q.pop(); }
}
```

⚠ Always use the predicate overload

`cv.wait(lk)` without a predicate is almost always a bug, for two independent reasons:

1. **Spurious wakeups** — `wait` may return without any notification. This is permitted and does happen.
2. **Lost wakeups** — if the producer notifies *before* the consumer starts waiting, the notification is gone forever. The predicate form checks the condition **before** waiting, so an already-satisfied condition doesn't block.

The predicate form is exactly:

```
while (!pred()) cv.wait(lk);
```

which is why the loop is non-negotiable.

Other notes

- **notify_one vs notify_all**: use `notify_one` when any single waiter can handle the event; `notify_all` when the condition may satisfy several waiters, or when waiters are waiting on *different* predicates on the same cv.
- **condition_variable_any** works with any lockable type (e.g. `shared_lock`, or a custom mutex) at a small cost. C++20 gives it a `stop_token` overload — the correct way to make a blocking wait cancellable:

```
cv.wait(lk, stopToken, []{ return !q.empty(); }); // returns false if stop was
requested
```

15.6 `std::atomic` and memory orders

Basics

```
#include <atomic>

std::atomic<int> counter{0};
counter.fetch_add(1); // atomic RMW
++counter;           // same
```

```
int v = counter.load();
counter.store(5);
int old = counter.exchange(10);

int expected = 10;
counter.compare_exchange_strong(expected, 20); // CAS: if == expected, set to 20
// else load current into
`expected`

std::atomic<T>::is_always_lock_free; // ★ compile-time constant – check this
counter.is_lock_free(); // runtime
```

`std::atomic<T>` requires `T` to be trivially copyable (§7.1). If `T` is too large, the implementation falls back to a hidden lock — hence the `is_always_lock_free` check.

C++20 additions: `fetch_add` for floating point, `std::atomic<std::shared_ptr<T>>`, and `wait/notify_one/notify_all` (futex-based blocking without a condition variable).

`std::atomic_ref` (C++20)

Atomic operations on an object that isn't declared atomic — for cases where the storage is shared with non-atomic code:

```
std::vector<int> data(1000);
// in parallel:
std::atomic_ref{data[i]}.fetch_add(1); // ⚠ ALL concurrent accesses must go
through atomic_ref
```

Memory orders

Order	Guarantee
<code>memory_order_relaxed</code>	atomicity only — no ordering with respect to other operations
<code>memory_order_acquire</code>	on a load: no later reads/writes move before it; synchronizes-with a matching release
<code>memory_order_release</code>	on a store: no earlier reads/writes move after it
<code>memory_order_acq_rel</code>	both, for read-modify-write operations
<code>memory_order_seq_cst</code>	the default — <code>acq_rel</code> plus a single global total order across all <code>seq_cst</code> operations
<code>memory_order_consume</code>	⚠ don't use — its specification is broken, every implementation promotes it to <code>acquire</code> , and its deprecation is under discussion

The release/acquire pattern — the one you'll actually write

```

Data          data;
std::atomic<bool> ready{false};

// Producer
data = compute();           // ordinary, non-atomic write
ready.store(true, std::memory_order_release); // everything above is
visible...

// Consumer
if (ready.load(std::memory_order_acquire)) { // ...to anyone who sees
    `true` here
    use(data);                             // ✓ NOT a race – happens-
before established
}

```

This is the whole point of release/acquire: it lets you publish **non-atomic** data safely by making one atomic flag the synchronization point.

```

// Relaxed: fine when you only need the count, not ordering
std::atomic<int> hits{0};
hits.fetch_add(1, std::memory_order_relaxed); // statistics counters,
refcount increments

```

⚠ **seq_cst is the default because it's the only one most people can reason about correctly.** Relaxing it requires a proof and a benchmark showing it matters. On x86, acquire/release are free anyway (the hardware model is already strong) and only **seq_cst** stores cost extra — so the optimization usually only pays on ARM/POWER.

compare_exchange_weak vs strong

```

// weak may fail SPURIOUSLY (LL/SC architectures: ARM, POWER) – use it in a loop,
// where you're looping anyway, because it's cheaper
int expected = current.load();
while (!current.compare_exchange_weak(expected, expected + 1)) { /* expected
updated */ }

// strong never fails spuriously – use it when you're NOT looping
if (state.compare_exchange_strong(expected, newState)) { ... }

```

Fences

```

std::atomic_thread_fence(std::memory_order_release); // ordering without tying
it to one variable

```

Rarely needed; almost always the ordering belongs on a specific atomic operation instead.

15.7 Futures: `future`, `promise`, `packaged_task`, `async`

```
// promise/future – manual
std::promise<int> p;
auto fut = p.get_future();
std::jthread t{[&p]{ p.set_value(compute()); }}; // or p.set_exception(...)
int result = fut.get(); // blocks; RETHROWS if
set_exception

// packaged_task – a callable that fulfils a future
std::packaged_task<int()> task{compute};
auto fut2 = task.get_future();
std::jthread t2{std::move(task)};

// async
auto fut3 = std::async(std::launch::async, compute);
```

⚠ `std::async` footguns

1. The returned future's destructor *blocks*.

```
std::async(std::launch::async, work); // ⚠ the temporary future is
destroyed at the                      // semicolon, and its destructor
JOINS.                               // This line is fully SYNCHRONOUS.
auto f = std::async(std::launch::async, work); // ✓ runs concurrently
```

This is unique to `async`-created shared states, and it surprises everyone once. (It's also the only sane behaviour available, given `async` has no way to own a thread otherwise — but it's still a trap.)

2. The default launch policy may defer.

```
auto f = std::async(work); // policy = async | deferred
                           // ⚠ the implementation MAY choose
deferred,                  // in which case `work` doesn't run
until .get()
auto f = std::async(std::launch::async, work); // ✓ ALWAYS specify the policy
```

If it defers and you never call `.get()`, the work never runs at all.

3. There is no thread pool. `std::async` typically spawns a fresh thread per call. Calling it in a loop over 10,000 items is a resource catastrophe.

Other notes: `future::get()` may be called only **once** (it moves the value out) — use `std::shared_future` for multiple consumers. Exceptions propagate cleanly through `get()`, which makes `promise/future` the correct way to get an exception out of a worker thread (§13.10).

15.8 `latch`, `barrier`, `semaphore` (C++20)

```
#include <latch>
#include <barrier>
#include <semaphore>

// latch — single-use countdown
std::latch startGate{1};
std::latch allDone{workerCount};
// workers:
startGate.wait();           // all block until the main thread opens the
                             gate
doWork();
allDone.count_down();
// main:
startGate.count_down();     // release everyone at once (good for
                             benchmarks)
allDone.wait();

// barrier — REUSABLE, with an optional completion function
std::barrier sync{4, []() noexcept { std::println("phase complete"); }};
for (int phase = 0; phase < 10; ++phase) {
    computePhase(phase);
    sync.arrive_and_wait();  // all 4 threads sync here, every phase
}

// semaphore — counted resource access
std::counting_semaphore<10> connections{10};
connections.acquire();
useConnection();
connections.release();

std::binary_semaphore signal{0};
signal.acquire();           // a lighter-weight signalling primitive than
                             cv+mutex
```

`latch` is a one-shot gate; `barrier` is a recurring rendezvous; `semaphore` counts permits. All three are implemented on `futexes` where available, so they're cheaper than a `condition_variable` + `mutex` for these specific shapes.

15.9 `std::execution` — senders and receivers (C++26)

The standard's answer to structured asynchrony: a composable, allocation-free framework covering thread pools, GPUs, and coroutines under one model. Adopted for C++26; usable today via **NVIDIA's `stdexec`**

reference implementation.

The four concepts:

Concept	Role
scheduler	a handle to an execution context (a thread pool, a GPU stream, the current thread)
sender	a lazy description of async work — a <i>recipe</i> , not a running task
receiver	the continuation: <code>set_value</code> / <code>set_error</code> / <code>set_stopped</code>
operation state	the connected, ready-to-start object — created in place, no allocation

```
#include <stdexec/execution.hpp>
#include <exec/static_thread_pool.hpp>

exec::static_thread_pool pool{8};
auto sched = pool.get_scheduler();

auto work= stdexec::schedule(sched)
    | stdexec::then([]{ return loadData(); })
    | stdexec::then([](Data d){ return transform(d); })
    | stdexec::let_value([](Result r){ return saveAsync(r); });

auto [result] = stdexec::sync_wait(std::move(work)).value();
```

Key algorithms: `just`, `then`, `let_value`, `when_all`, `bulk`, `on`, `into_variant`, `sync_wait`, `start_detached`.

Why it matters: it's **lazy** (nothing runs until started), **structured** (cancellation and error handling propagate through the graph automatically via `set_stopped`/`set_error`), and **allocation-free** (the operation state is composed in place). It also interoperates with coroutines (§16) — a sender is awaitable, and a coroutine task is a sender.

15.10 Thread pools, work stealing, false sharing

There is no standard thread pool (yet)

Until `std::execution` lands in implementations, use: **TBB** (`tbb::task_group`, mature), **Asio** (`io_context` + `thread_pool`), **folly**, **taskflow**, or a hand-rolled one. Do **not** use `std::async` as a pool (§15.7).

Work stealing: each worker owns a deque, pushes and pops from its own **back**, and steals from other workers' **fronts**. This minimizes contention (owner and thieves touch opposite ends) and gives good locality (a worker's most recent task is usually cache-warm).

⚠ False sharing

Two threads writing to different variables that land on the **same cache line** cause that line to ping-pong between cores. The code is correct; the performance is catastrophic.

```

struct Counters {
    std::atomic<int> a;      // }
    std::atomic<int> b;      // } same 64-byte cache line – 10-100x slowdown
};

struct alignas(std::hardware_destructive_interference_size) PaddedCounter {
    std::atomic<int> value;
};

std::vector<PaddedCounter> counters(threadCount); // ✓ each on its own line

```

```

std::hardware_destructive_interference_size // min offset to AVOID sharing
(usually 64)
std::hardware_constructive_interference_size // max size to PROMOTE sharing

```

⚠ GCC emits `-Winterference-size` warnings because these constants are ABI-sensitive (their value is baked into your class layouts, and it differs by `-mtune`). Many codebases just define `constexpr size_t kCacheLine = 64;` and move on — which is right for x86-64 and ARM64, though Apple Silicon's L2 line is 128.

False sharing is invisible in code review and obvious in `perf c2c` output. Suspect it whenever adding threads makes things slower.

15.11 Lock-free programming, ABA, and why you should use a mutex

The hierarchy

Property	Guarantee
Wait-free	every thread completes in a bounded number of steps
Lock-free	<i>some</i> thread makes progress; the system as a whole never stalls
Obstruction-free	a thread running alone completes
Blocking	a suspended thread can stall everyone (ordinary mutexes)

Lock-free does **not** mean "faster." It means "no thread holding a lock can block the others" — a *progress* guarantee, which matters for real-time deadlines and for signal handlers, not usually for throughput.

The ABA problem

```

// Thread 1: reads head = A, prepares to CAS head from A to A->next (= B)
// Thread 2: pops A, pops B, pushes A back. head is A again – but A->next is now C.
// Thread 1: CAS succeeds (head is still A!) and sets head = B – a freed node.
Corruption.

```

The CAS compared the *pointer value*, which is unchanged, while the *state* changed underneath. Solutions:

- **Tagged pointers / generation counters** — pack a counter into unused pointer bits, or use a double-width CAS
- **Hazard pointers** (`std::hazard_pointer`, C++26)
- **RCU** (`std::rcu_domain`, C++26)
- **Epoch-based reclamation**

⚠ **Memory reclamation is the genuinely hard part of lock-free programming.** Knowing when it's safe to `delete` a node that another thread might still be reading is harder than the algorithm itself, and it's where nearly all lock-free bugs live.

Why you should use a mutex

1. **An uncontended `std::mutex` is a single atomic RMW** — roughly the cost of one `fetch_add`. It only enters the kernel when contended.
2. **Under contention, mutexes often beat lock-free structures**, because a blocked thread sleeps and stops competing for the cache line, while a lock-free CAS loop keeps spinning and keeps the line hot.
3. Lock-free code is **extremely** hard to get right, nearly impossible to test exhaustively, and its bugs are timing-dependent and non-reproducible.
4. TSan can miss races in hand-written atomic code that it would catch through a mutex.

Default to a mutex. Reach for atomics for counters, flags, and single-word state. Reach for a lock-free data structure only with a measured problem, a real-time constraint, or a published algorithm — and then use an existing implementation.

Existing implementations worth using: `moodycamel::ConcurrentQueue`, `folly::MPMCQueue`, `boost::lockfree`, `rigtorp::SPSCQueue`.

Gotcha checklist for this section

- ☐ A data race is **UB**, not a stale read — the compiler optimizes assuming it can't happen
- ☐ Adjacent bit-fields share a memory location; distinct scalars don't
- ☐ `std::vector<bool>` races on adjacent elements
- ☐ Destroying a joinable `std::thread` calls `terminate` — use `jthread`
- ☐ `std::thread` **copies** its arguments — use `std::ref`
- ☐ An exception escaping a thread function calls `terminate`
- ☐ `std::lock_guard{m};` (unnamed) locks and immediately unlocks
- ☐ `recursive_mutex` almost always means the design is wrong
- ☐ `shared_mutex` is often **slower** than `mutex` for short critical sections
- ☐ Never hold a lock while calling a callback or virtual function
- ☐ **Always use the predicate form of `cv.wait`** — spurious *and* lost wakeups
- ☐ `condition_variable` requires `unique_lock`, not `lock_guard`
- ☐ `std::async(...)` as a bare statement is **synchronous** — the temporary future's destructor blocks
- ☐ `std::async` without an explicit policy may **defer** and never run
- ☐ `std::async` is not a thread pool
- ☐ `future::get()` may be called once — use `shared_future` otherwise
- ☐ `memory_order_consume` is broken — never use it

- ☐ `compare_exchange_weak` fails spuriously — only in loops
- ☐ Two atomics on one cache line = **false sharing**, 10-100× slowdown
- ☐ ABA: the CAS compares the pointer, not the state
- ☐ `const` member functions must be concurrency-safe (§7.5)

Flags and tools

```
-fsanitize=thread                # ★ TSan – separate build, incompatible with
ASan
-fsanitize=thread -O1 -g -fno-omit-frame-pointer
TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1

-pthread                        # required on GCC/Clang for <thread>
-Winterference-size             # GCC: hardware_*_interference_size ABI warning

perf c2c                        # ★ false sharing / cache-line contention
analysis
perf lock                       # lock contention
valgrind --tool=helgrind        # alternative race detector (different
tradeoffs)
valgrind --tool=drd
Intel Inspector / VTune        # threading analysis

clang-tidy:
  concurrency-mt-unsafe         # calls to non-thread-safe functions (strtok,
asctime, ...)
  bugprone-spuriously-wake-up-functions # cv.wait without a predicate
  cert-con54-cpp
  misc-const-correctness
```

TSan is the single highest-value tool here. Its limitation is the same as every dynamic tool (§14.5): it only sees the interleavings that actually execute. Run it over your whole test suite, repeatedly, and consider stress tests with randomized delays.

Exercises worth doing

1. Write a counter incremented by 8 threads without synchronization. Run it under TSan. Then fix it with `atomic` and with a `mutex`, and benchmark all three.
2. Write `std::async(std::launch::async, longTask);` as a bare statement and time the enclosing function. Then assign it to a named variable and time it again.
3. Build the false-sharing demo: 8 threads incrementing 8 adjacent `atomic<int>`s, then the same with `alignas(64)` padding. Measure. Then confirm with `perf c2c`.
4. Write a `condition_variable` consumer without a predicate and construct a lost-wakeup by notifying before the consumer waits.
5. Implement the release/acquire publication pattern, then change both to `relaxed` and run under TSan on an ARM machine if you have one.
6. Write a lock-free stack with `compare_exchange_weak`, then work out where the ABA problem is and when it would be safe to free a popped node.

7. Deliberately create an AB-BA deadlock, catch it with TSan's deadlock detection, then fix it with `std::scoped_lock`.
8. Convert a `std::thread` + manual join to `std::jthread` with a `stop_token`, and make a blocking `cv` wait cancellable.

Chapter 16 — Coroutines (C++20)

What this chapter covers

- `co_await`, `co_yield`, `co_return`
- **The promise type / awaitable / awaiter machinery** — why the standard shipped a framework, not a library
- Generators (`std::generator`, C++23), lazy sequences
- Async tasks, the executor problem
- Libraries you'll actually use: **cppcoro**, **libunifex**, **stdexec**, **Boost.Asio** awaitables
- ⚠ Lifetime traps: dangling references across suspension points

The most powerful and least finished feature in modern C++. C++20 shipped the **compiler transformation** and left the **library** for later — so you get `co_await`, and then you have to build or import the `task<T>` you actually want. C++23 added `std::generator`; C++26's `std::execution` (§15.9) is meant to close the rest of the gap.

Read this section knowing that the machinery below is what *library authors* write. Application code should be using someone else's `task` type.

16.1 What a coroutine actually is

A function that can **suspend** and later **resume** where it left off.

```
Generator<int> counter() {
    for (int i = 0; ; ++i)
        co_yield i;           // suspends HERE; locals survive; resumes here next
time
}
```

It's a property of the body, not the signature ⚠

Any function whose body contains `co_await`, `co_yield`, or `co_return` **is** a coroutine. There is no marker on the declaration.

```
Task<int> compute();           // is this a coroutine? YOU CANNOT TELL.
```

This is a genuine design wart: whether a function is a coroutine changes its parameter lifetime semantics dramatically (§16.7), and callers can't see it. Conventions (naming, return-type naming) are your only defence.

Stackless

C++ coroutines are **stackless**: only the coroutine's own frame is saved, not an entire call stack. You can only suspend from the coroutine body itself, never from a function it calls.

	Stackless (C++20)	Stackful (fibers, Boost.Context, Go)
Suspend from a nested call	✗	✓
Memory per instance	just the frame (tens of bytes)	a whole stack (KBs–MBs)
Switch cost	a few instructions	register save/restore
Compiler support needed	yes	no (it's a library)

The transformation

The compiler rewrites the body into a state machine and allocates a **coroutine frame**:

Coroutine frame (heap, usually):

promise_type object

COPIES of all parameters

object

locals that live across a suspend

resume function pointer

destroy function pointer

current suspension point index

← the customization point

⚠ references copy the reference, not the object

HALO (Heap Allocation eLision Optimization): if the compiler can prove the coroutine's lifetime is strictly nested within the caller's, it may put the frame on the stack instead. It is **not guaranteed** — treat the allocation as present unless you've checked the assembly.

16.2 The three keywords

```
co_await expr;    // suspend until `expr` is ready; the awaiter decides what
                  // that means
co_yield value;   // equivalent to: co_await promise.yield_value(value)
co_return value;  // finish: calls promise.return_value(value) or return_void()
```

⚠ **Where you cannot use them:** constructors, destructors, `constexpr` functions, `constexpr` functions, `main`, variadic functions, and functions with a **deduced return type** (`auto` without a trailing return type).

16.3 The machinery: promise type, awaitable, awaiter

The promise type

The compiler finds it via `std::coroutine_traits<ReturnType, Args...>::promise_type` — in practice, a nested `ReturnType::promise_type`.

```
struct promise_type {
    Task get_return_object();           // what the CALLER receives
    Awaitable initial_suspend();       // suspend before the body runs?
    (lazy vs eager)
    Awaitable final_suspend() noexcept; // ⚠ MUST be noexcept
    void return_value(T v);            // for co_return v; } pick exactly
    void return_void();                // for co_return; } one
    void unhandled_exception();        // an exception escaped the body

    // optional
    Awaitable yield_value(T v);        // enables co_yield
    auto await_transform(auto&&);      // intercept EVERY co_await in this
    coroutine
    static void* operator new(std::size_t); // custom frame allocation
};
```

`std::suspend_always` and `std::suspend_never` are the two trivial awaitables you'll return from `initial_suspend/final_suspend`.

- `initial_suspend()` -> `suspend_always` = **lazy** (nothing runs until you resume) — correct for `task`
- `initial_suspend()` -> `suspend_never` = **eager** (body runs immediately up to the first suspend)

coroutine_handle

```
std::coroutine_handle<promise_type> h;
h.resume(); h();           // resume execution
h.destroy();               // destroy the frame - ⚠ YOU own this
h.done();                  // has it reached final_suspend?
h.promise();               // access the promise object
std::coroutine_handle<promise_type>::from_promise(p);
h.address(); std::coroutine_handle<>::from_address(p); // type-erased, for C
APIs
std::noop_coroutine();     // a handle that does nothing when resumed
```

⚠ `coroutine_handle` is a **non-owning, raw handle** — it's the `T*` of coroutines. Something must call `destroy()` exactly once. That "something" is the destructor of your `task/generator` wrapper, which is why you should not be passing raw handles around.

The awaiter protocol

`co_await expr` needs an **awaiter** — obtained from `expr` via `await_transform`, then `operator co_await`, or `expr` itself. The awaiter provides:

```
struct Awaiter {
    bool await_ready();           // true -> skip suspension entirely
    ??? await_suspend(std::coroutine_handle<> h);
    T      await_resume();       // the RESULT of the co_await
    expression
};
```

`await_suspend` has three legal return types:

Returns	Meaning
<code>void</code>	stay suspended; control returns to whoever resumed us
<code>bool</code>	<code>true</code> = stay suspended, <code>false</code> = resume immediately (the "it was already ready" fast path)
<code>std::coroutine_handle<></code>	symmetric transfer : suspend this one and immediately resume <i>that</i> one

Symmetric transfer is important, not a curiosity. Without it, coroutine A resuming B resuming C grows the real stack with each hop, and a long chain of `co_await`s overflows it. Returning a handle makes the transfer a tail call, so the stack stays flat.

⚠ Why the standard shipped a framework, not a library

C++20 gave you every hook a library author needs and **no library**. There is no `std::task`, no `std::lazy`, no `std::when_all`, no scheduler. The reason is honest: nobody agreed on the executor model, and shipping a `task` without one would have baked in the wrong answer.

The result is that every project either writes ~200 lines of promise-type boilerplate or takes a dependency. That is a real usability failure, and it's why coroutine adoption has been slower than the feature's power warrants. C++23's `std::generator` and C++26's `std::execution` are the fix arriving in stages.

16.4 Generators

A complete minimal generator

```
#include <coroutine>

template <class T>
class Generator {
public:
    struct promise_type {
        T current_;
        std::exception_ptr error_;

        Generator get_return_object() {
            return
Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
    };
};
```

```

    std::suspend_always initial_suspend() noexcept { return {}; } // lazy
    std::suspend_always final_suspend() noexcept { return {}; }
    std::suspend_always yield_value(T v) { current_ = std::move(v); return {}; }
}

void return_void() {}
void unhandled_exception() { error_ = std::current_exception(); }
};

using handle = std::coroutine_handle<promise_type>;

explicit Generator(handle h) : h_(h) {}
~Generator() { if (h_) h_.destroy(); } // RAII - §4
Generator(Generator&& o) noexcept : h_(std::exchange(o.h_, {})) {}
Generator& operator=(Generator&& o) noexcept {
    if (this != &o) { if (h_) h_.destroy(); h_ = std::exchange(o.h_, {}); }
    return *this;
}
Generator(const Generator&) = delete; // move-only
Generator& operator=(const Generator&) = delete;

struct iterator {
    handle h_;
    bool operator==(std::default_sentinel_t) const { return !h_ || h_.done(); }
}

iterator& operator++() {
    h_.resume();
    if (h_.promise().error_) std::rethrow_exception(h_.promise().error_);
    return *this;
}
const T& operator*() const { return h_.promise().current_; }
};

iterator begin() { if (h_) { h_.resume(); } return iterator{h_}; }
std::default_sentinel_t end() { return {}; }

private:
    handle h_;
};

```

```

Generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) { co_yield a; auto t = a + b; a = b; b = t; }
}

for (int x : fibonacci()) { if (x > 100) break; std::print("{} ", x); }

```

Note how much of §4 and §5 is in there: move-only, RAII destructor, `std::exchange` in the move constructor, deleted copies.

`std::generator` (C++23)

The standard version, which you should use instead:

```
#include <generator>

std::generator<int> range(int lo, int hi) {
    for (int i = lo; i < hi; ++i) co_yield i;
}

// Recursive yielding – the killer feature
std::generator<const Node&> traverse(const Node& n) {
    co_yield n;
    for (const auto& child : n.children)
        co_yield std::ranges::elements_of(traverse(child)); // yields the whole subtree
}

for (const auto& node : traverse(root)) { ... }
```

`std::generator` models `input_range`, so it composes with ranges (§12):

```
auto evens = range(0, 100) | std::views::filter([](int n){ return n % 2 == 0; });
```

⚠ Naive recursive generators without `elements_of` are **O(depth) per element** — each value bubbles up through every level. `elements_of` uses symmetric transfer to make it O(1).

Generators vs range views

	std::generator	views::filter/transform
Expressing recursive traversal	trivial	very hard
Arbitrary control flow, state machines	trivial	hard
Per-element cost	frame allocation + indirect resume (~ns)	typically inlined to zero
Composability	as an input_range	full pipeline

Use generators when the *producer* has complex control flow. Use views when the transformation is a pipeline over an existing range.

16.5 Async tasks and the executor problem

A `task<T>` sketch

```

template <class T>
class Task {
public:
    struct promise_type {
        T value_;
        std::exception_ptr error_;
        std::coroutine_handle<> continuation_ = std::noop_coroutine();

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; } // LAZY

        struct FinalAwaiter {
            bool await_ready() noexcept { return false; }
            std::coroutine_handle<> await_suspend(
                std::coroutine_handle<promise_type> h) noexcept {
                return h.promise().continuation_; // ★ symmetric transfer
to the awaiter
            }
            void await_resume() noexcept {}
        };
        FinalAwaiter final_suspend() noexcept { return {}; }

        void return_value(T v) { value_ = std::move(v); }
        void unhandled_exception() { error_ = std::current_exception(); }
    };

    auto operator co_await() && {
        struct Awaiter {
            std::coroutine_handle<promise_type> h_;
            bool await_ready() { return false; }
            std::coroutine_handle<> await_suspend(std::coroutine_handle<> caller)
{
                h_.promise().continuation_ = caller; // remember who to resume
                return h_; // and start the task –
symmetric transfer
            }
            T await_resume() {
                if (h_.promise().error_)
std::rethrow_exception(h_.promise().error_);
                return std::move(h_.promise().value_);
            }
        };
        return Awaiter{h_};
    }
    // ... RAII destructor, move-only, as in Generator
private:
    std::coroutine_handle<promise_type> h_;
};

```



```
Task<Config> loadConfig() {
    auto raw = co_await readFile("config.json");    // suspends; resumes when I/O
    completes
    co_return parse(raw);
}
```

⚠ The executor problem

A coroutine has no idea what thread it should resume on. `co_await someTask` suspends the caller and starts the callee — but when an *async I/O* operation completes, something must call `resume()`, and that something runs on whatever thread the I/O layer chose.

C++20 provides no scheduler, so every framework invented its own:

```
co_await asio::post(executor, asio::use_awaitable);    // Asio: hop to a specific
executor
co_await cppcoro::resume_on(threadPool);                // cppcoro
co_await stdexec::on(scheduler, work);                 // stdexec / C++26
```

This is exactly the gap `std::execution` (§15.9) fills: a sender is awaitable, a coroutine task is a sender, and `schedule(sched)` is how you say where to resume. Until then, **pick one framework and stay inside it** — mixing coroutine libraries means mixing incompatible scheduling models.

16.6 Libraries you'll actually use

Library	What it gives you	Status
Boost.Asio	<code>asio::awaitable<T></code> , <code>co_spawn</code> , <code>use_awaitable</code> , full async I/O and timers	the most production-ready today , especially for networking
stdexec (NVIDIA)	the <code>std::execution</code> reference implementation (§15.9); schedulers, senders, coroutine interop	the future; use it if you're targeting C++26
cppcoro (Lewis Baker)	<code>task</code> , <code>generator</code> , <code>async_generator</code> , <code>when_all</code> , <code>sync_wait</code> , <code>async_mutex</code> , <code>static_thread_pool</code>	foundational, widely copied, lightly maintained
libunifex (Meta)	sender/receiver predecessor to stdexec	largely superseded by stdexec
folly::coro	<code>Task</code> , <code>AsyncGenerator</code> , <code>collectAll</code> , integrated with folly's executors	excellent, but a heavy dependency
concurrencycpp	<code>task</code> , executors, timers — small and self-contained	good for smaller projects
QCoro	coroutine support for Qt types	if you're in Qt

Do not write your own `task<T>` for production. The boilerplate above is educational; getting symmetric transfer, cancellation, exception propagation, allocator support, and executor affinity all

correct is a serious project.

16.7 ⚠ Lifetime traps

This is where coroutines bite hardest, because the frame outlives the calling expression.

1. Reference parameters — the big one

Parameters are copied into the frame. But copying a *reference* copies the reference, not the referent.

```
Task<int> process(const std::string& data) {           // ⚠ REFERENCE parameter
    co_await something();                             // suspends here
    co_return data.size();                             // data may be long dead
}

auto t = process("literal");                          // temporary std::string dies at the semicolon
co_await std::move(t);                                // ⚠ USE-AFTER-FREE – the coroutine resumes
later
```

The temporary's lifetime ends at the end of the full expression (§3.4); the coroutine's does not.

Rule: coroutine parameters should be taken by value. No `const&`, no `string_view`, no `span`, no raw pointers — unless you can prove the referent outlives the whole coroutine.

The by-value version costs a move and is correct:

```
Task<int> process(std::string data) { ... }           // ✓
```

2. Lambda coroutines

```
auto makeTask = [&captured]() -> Task<int> {         // ⚠ the CLOSURE is a
    temporary                                         // dangling
    co_await something();
    co_return captured;                             // captured by reference –
};                                                    // dangling
co_spawn(ctx, makeTask(), ...);
// the lambda object itself may also be destroyed while the coroutine runs
```

Two separate dangers: the captures dangle, **and the closure object** (which `this` points to inside the body) may die. Fixes: make the lambda's captures parameters of the coroutine instead, or keep the closure alive for the coroutine's whole lifetime.

3. `this` in member coroutines

```
class Session {
    Task<void> run() { // captures `this`
    implicitly
        co_await read();
        use(member_); // ⚠ Session must
    outlive this coroutine
    }
};
```

Same problem, same shape as the async callback case (§4.4). The standard fix is the same: `co_await` while holding a `shared_ptr` to self via `enable_shared_from_this`.

4. What is safe

Temporaries created **inside** a `co_await` expression are fine — the coroutine frame keeps them alive across the suspension, because the full-expression hasn't ended:

```
co_await doThing(std::string{"temp"}); // ✔ the temporary lives in the frame
```

5. Ownership of the handle

```
Generator<int> g = makeGen();
auto it = g.begin();
// ... g is destroyed ...
*it; // ⚠ the frame is gone
```

An iterator or handle referring to a destroyed coroutine dangles like any other pointer.

Detection

ASan catches most of these at runtime (`-fsanitize=address`), and modern GCC/Clang have some coroutine-aware lifetime warnings, but **there is no reliable static check for the reference-parameter trap**. Discipline — parameters by value — is the actual defence.

16.8 The cost model

Cost	Detail
Frame allocation	one heap allocation per coroutine invocation unless HALO elides it — typically 30–100 ns
Frame size	promise + parameter copies + all locals live across a suspension
Resume	an indirect call through the frame's resume pointer; not inlinable

Cost	Detail
co_yield per element	a suspend + resume round trip, ~few ns

Mitigations:

```
struct promise_type {
    static void* operator new(std::size_t n) { return myPool.allocate(n); } //
pool the frames
    static void operator delete(void* p, std::size_t n) { myPool.deallocate(p,
n); }
};
```

Or, for a coroutine used in a hot loop, use `std::pmr` (§11.12) or check whether HALO fired (look for the absence of a call to `operator new` in the assembly).

Practical guidance: coroutines are a *clarity* feature that costs a little performance. For a generator over millions of elements in a tight loop, a range view or a plain loop is likely 5–10× faster. For async I/O, where you're waiting microseconds to milliseconds anyway, the frame cost is noise and the readability gain is enormous.

Gotcha checklist for this section

- ☐ You **cannot tell from the signature** whether a function is a coroutine
- ☐ **Take coroutine parameters by value** — `const&`, `string_view`, and `span` parameters dangle across suspensions
- ☐ Lambda coroutines: both the captures **and the closure object** can die while the coroutine lives
- ☐ A member coroutine requires the object to outlive it — use `shared_from_this`
- ☐ `coroutine_handle` is non-owning — exactly one thing must call `destroy()`
- ☐ `final_suspend()` **must** be `noexcept`
- ☐ Pick exactly one of `return_value` / `return_void`
- ☐ `initial_suspend` returning `suspend_never` makes the coroutine **eager** — usually wrong for `task`
- ☐ Without **symmetric transfer**, chained coroutines grow the real stack and can overflow it
- ☐ Naive recursive generators are O(depth) per element — use `std::ranges::elements_of`
- ☐ HALO is an optimization, **not a guarantee** — assume the frame is heap-allocated
- ☐ No coroutines in constructors, destructors, `constexpr` functions, `main`, or deduced-return-type functions
- ☐ C++20 gives you no scheduler — don't mix coroutine libraries

Flags and tools

```
-std=c++20 -fcoroutines           # GCC 10 needs the flag explicitly; GCC 11+
implies it
/await:strict                     # MSVC: standard C++20 coroutines (not the old
TS)
```

```

-stdlib=libc++                # Clang: <coroutine> availability
-fsanitize=address -fsanitize-address-use-after-scope # ★ the main defence

-fcoro-elide                    # (Clang) frame elision hints
# Verify HALO: look for a call to operator new in the -O2 assembly

clang-tidy:
  cppcoreguidelines-avoid-reference-coroutine-parameters # ★ exactly the §16.7
  trap
  cppcoreguidelines-avoid-capturing-lambda-coroutines    # ★ the lambda trap

```

Those two clang-tidy checks exist specifically for the two worst coroutine bugs. Turn them on before you write your first coroutine.

Exercises worth doing

1. Type the `Generator<T>` above from scratch without copying it. Then use it for a Fibonacci sequence and a binary-tree traversal.
2. Add `printf` to every promise-type member (`get_return_object`, `initial_suspend`, `yield_value`, `final_suspend`, the destructor) and trace the exact call order for one `for` loop over a generator.
3. Write the reference-parameter bug: a `Task<int>` taking `const std::string&`, called with a literal. Run it under ASan and read the report.
4. Convert `await_suspend` from returning `void` to returning a `coroutine_handle`. Chain 100,000 tasks and observe the stack overflow disappear.
5. Write a recursive tree generator both naively and with `std::ranges::elements_of`, and benchmark on a deep tree.
6. Benchmark a `std::generator` producing 10 million ints against a `views::iota` pipeline and a plain loop.
7. Add a pooling `operator new` to a promise type and measure the difference.
8. Write a small TCP echo server with Boost.Asio's `awaitable<T>` and `co_spawn`. Compare it to the same server written with callbacks.

Chapter 17 — Modules (C++20)

What this chapter covers

- `export module`, `import`, module partitions, the global module fragment
- **Why**: compile-time reduction, real encapsulation, no macro leakage
- `import std;` (C++23)
- Build-system support status and the practical migration story
- Headers, `#include` order, and why modules were needed

The biggest change to the C++ compilation model since 1972 — and the one with the widest gap between "the standard says" and "you can use it." The compiler front ends work. The ecosystem largely doesn't. This section covers the feature properly and is honest about the current state, which as of mid-2026 is **"solved in the compiler, unsolved everywhere else."**

17.1 Why modules were needed

What `#include` actually does

`#include` is **textual substitution**. The preprocessor pastes the file's contents into your translation unit, and then the compiler parses all of it, from scratch, for every TU.

```
#include <vector>           // ~20,000 lines after preprocessing
#include <string>            // ~15,000
#include <algorithm>         // ~30,000
// A "small" file is now 65,000+ lines. Multiply by 500 source files.
```

The consequences, all of which modules address:

- 1. Quadratic compile times.** $N \text{ source files} \times M \text{ headers} = N \times M$ parses of the same text. Templates make it worse: every TU instantiates its own copies (§10.10), and the linker discards the duplicates.
- 2. No encapsulation.** Everything in a header is visible to everything that includes it — implementation details, helper types, internal `using` declarations. `namespace detail` is a naming convention, not a barrier.
- 3. Macro leakage.** A macro defined in one header affects every file included after it, forever.

```
#include <windows.h>        // defines min, max, near, far, small, ...
#include <algorithm>
std::min(a, b);             // ⚠ breaks — `min` is a macro now
```

Hence `#define NOMINMAX`, hence `#undef` blocks, hence `(std::min)(a, b)` with defensive parentheses.

- 4. `#include` order matters.** A header that forgets its own dependencies compiles only if you happened to include something else first. This is why "include what you use" is a tool and not just a slogan.
- 5. Include guards are a workaround** for a problem the language created, and `#pragma once` is a non-standard workaround for the workaround.
- 6. ODR violations are invisible** (§6.4) — the same header can expand differently in two TUs based on macros defined before it.

Modules fix all six by making the unit of compilation a **compiled, semantic artifact** rather than a pile of text.

17.2 The syntax

A module interface unit

```
// math.cppm (.cppm on GCC/Clang, .ixx on MSVC by convention)
export module math;                               // module declaration — must be first

import <cstdint>;                                   // (header units — see the status caveat
```

```

below)

export int add(int a, int b) { return a + b; } // visible to importers

int helper(int x) { return x * 2; }           // NOT exported – truly private

export class Matrix {                         // exported type
public:
    double at(int r, int c) const;
private:
    std::vector<double> data_;                // implementation detail,
    genuinely hidden
};

export namespace geometry {                  // export a whole namespace
    double area(const Shape&);
}

export {                                      // export a block
    int f();
    int g();
}

```

```

// main.cpp
import math;

int main() {
    add(1, 2);           // ✓
    helper(3);           // ✗ not exported – a real compile error, not a convention
}

```

`helper` isn't merely discouraged from use; it is **not in the importer's translation unit at all**. That's the encapsulation win.

Module implementation units

```

// math_impl.cpp
module math;                               // NO `export` keyword – this is an
implementation unit

double Matrix::at(int r, int c) const { return data_[r * cols_ + c]; }

```

Implementation units implicitly import their interface and can see everything in it, exported or not.

Module partitions

For splitting a large module without exposing the split to consumers:

```
// math-matrix.cppm — an interface partition
export module math:matrix;
export class Matrix { ... };

// math-vector.cppm
export module math:vector;
export class Vector { ... };

// math.cppm — the primary interface unit
export module math;
export import :matrix;           // re-export the partition
export import :vector;

// math-detail.cppm — an INTERNAL partition (no `export` on the module
// declaration)
module math:detail;
int internalHelper();
```

Consumers write `import math;` and never see the partitions. Note the primary interface unit **must export import** every interface partition, or their contents aren't visible.

The global module fragment

For `#includes` that must be textually processed — usually because the header uses macros, or isn't module-aware:

```
module;                                // global module fragment starts here
#include <windows.h>                    // preprocessor-only region
#include <legacy_c_api.h>
export module mymodule;                // the module declaration ends it

export void doThing();
```

⚠ The global module fragment may contain **only preprocessor directives**. Names declared there attach to the *global module*, so they're not exported — which is exactly what you want when wrapping a C library.

Private module fragment

```
export module widget;
export class Widget { void f(); };

module :private;                       // everything below is invisible to
importers,                             // and changing it does NOT trigger their
void Widget::f() { ... }               // rebuild
```

Useful for single-file modules where you still want implementation changes to avoid a downstream rebuild.

17.3 What you actually gain

Compile time

The real numbers are good where the feature works. *One reported case replaced roughly nine seconds of including VulkanHpp with a negligible amount of milliseconds, and Boost's per-library module prototype has shown build-time reductions around 45%.*

The mechanism: a module interface is compiled **once** into a BMI (Binary Module Interface — `.pcm` on Clang, `.gcm` on GCC, `.ifc` on MSVC), and importers load that pre-parsed artifact instead of re-parsing text.

⚠ The cost is a **serialized build**: a module's BMI must exist before anything that imports it can compile. Where headers gave you embarrassing parallelism, modules give you a dependency graph. Build systems must scan sources to discover that graph *before* compiling (the P1689 dependency-scanning format), which is why build-system support was the hard part.

Real encapsulation

Non-exported entities are invisible. Not "in `namespace detail`" — **invisible**. This is the benefit that survives even if compile times don't improve for your project.

No macro leakage

Macros do not cross module boundaries in either direction. A module can `#include <windows.h>` in its global module fragment and its importers are unaffected.

Order independence

`import` order never matters. Module units are semantic, not textual.

One-definition sanity

Each module is compiled once, so a whole class of ODR violations (§6.4) simply cannot occur.

17.4 `import std;` (C++23)

The single most appealing thing about modules for everyday code:

```
import std;                                // the ENTIRE standard library, one import

int main() {
    std::vector<int> v{1, 2, 3};
    std::println("{} ", v.size());
}
```

```
import std.compat;                        // std:: plus the global-namespace C names
 (:::printf etc.)
```

This compiles on GCC 16.1, Clang 18+, and MSVC — the compiler side is done. The build-system side is not (§17.5).

⚠ Note `import std;` is **not** a drop-in for `#include <vector>` in a header-based project: macros from the standard library (`assert`, `errno`, `offsetof`, `NULL`) are *not* exported by a module. If you need those, include the header.

17.5 ⚠ Build system and tooling status (as of mid-2026)

This is the honest assessment, and it's the reason most codebases haven't migrated.

Compilers: ready

Compiler	Named modules	<code>import std;</code>	Extension
MSVC	✓ (VS 2022 17.4+)	✓	<code>.ixx</code>
Clang	✓ (16+)	✓ (18+)	<code>.cppm</code>
GCC	✓ (14+)	✓ (recent)	<code>.cppm</code>

CMake: named modules yes, everything else no

Named modules work with Ninja generators from CMake 3.28 onward. `import std` remains gated behind `CMAKE_EXPERIMENTAL_CXX_IMPORT_STD` with a version-specific GUID that changes between CMake releases. Header units (`import <header.hpp>`) are not supported at all, and Visual Studio generators cannot build BMIs for imported targets.

The working setup:

```
cmake_minimum_required(VERSION 3.28)
project(myapp CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_library(math)
target_sources(math
    PUBLIC
        FILE_SET CXX_MODULES FILES # ★ the key mechanism
            math.cppm
            math-matrix.cppm
    PRIVATE
        math_impl.cpp
)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE math)
```

Use the **Ninja** generator. `FILE_SET CXX_MODULES` is how CMake learns which sources are module interfaces so it can scan and order them.

IDEs: the weak link

Visual Studio 2026 still labels its IntelliSense modules support experimental — a label that has been unchanged for seven years. clangd has experimental support, but editing a module interface does not refresh the BMI, so you must rebuild the project, and the clangd version must match the Clang version exactly. CLion 2026.1 offers partial support. In VS Code, the cpptools extension reports `import std;` as an error with GCC 15+, while clangd with `compile_commands.json` works better.

For a language where "go to definition" and "find references" are load-bearing, this matters more than it sounds.

Libraries: almost nothing ships modules

Six years after C++20, barely any C++ library comes with a module definition. That means importing a third-party library still means `#include` in your global module fragment — which works, but caps how much you gain.

The transitional pattern most libraries are experimenting with is a thin module wrapper that re-exports the header contents via `using` declarations, rather than a native module implementation.

Other build systems

Bazel, Meson, and hand-rolled build systems require noticeably more glue than CMake. If your build is heavily customized, budget real time.

The summary

Modules are a solved problem in the compiler frontend and an unsolved problem everywhere else.

17.6 The practical migration story

Should you migrate?

Migrate now if	Wait if
New project, CMake + Ninja, one recent compiler	You support multiple compilers or old versions
You control your whole dependency tree	You depend on many header-only libraries
Your team tolerates rough IDE support	Your workflow depends on IntelliSense/clangd
Build time is a measured, painful bottleneck	Your build is already fast enough
You have DevOps capacity for build-system work	Nobody owns the build

If you do: incremental, from the leaves

- 1. **Keep a header fallback.** Ship both until your minimum compiler requirement guarantees modules.

2. Start with leaf libraries — self-contained code with few dependencies. Don't start with the module everything else depends on.

3. Wrap, don't rewrite. The cheapest first step is a module that includes your existing headers in the global module fragment and exports the names:

```
module;
#include "widget.h"
export module widget;

export using ::Widget;
export using ::makeWidget;
```

Consumers switch to `import widget;` one at a time, and you haven't touched the implementation.

4. Measure. Modules are supposed to make the build faster. Verify with `-ftime-trace` / ClangBuildAnalyzer (§10.10, §19) before and after — the serialization cost can eat the parsing win on a highly parallel build.

5. Adopt `import std;` last, since it's the least build-system-ready piece.

The intermediate options that still work

If modules aren't viable yet, the older compile-time levers from §19 remain available and are far less disruptive: **precompiled headers** (for some workloads, still faster than modules today), **unity builds**, **ccache**, **include-what-you-use**, **extern template** (§10.10), and reducing header weight by forward-declaring and moving definitions into `.cpp` files.

Gotcha checklist for this section

- ☐ `export module m;` must be the **first** declaration (after the global module fragment)
- ☐ The global module fragment may contain **only preprocessor directives**
- ☐ A primary interface unit must `export import` its interface partitions or their contents are invisible
- ☐ Module implementation units say `module m;` with **no export**
- ☐ **Macros do not cross module boundaries** — `import std;` gives you no `assert`, `errno`, or `NULL`
- ☐ Modules **serialize the build** — the BMI must exist before importers compile
- ☐ BMIs are compiler- **and version**-specific; they are build artifacts, never distributables
- ☐ CMake needs 3.28+, the **Ninja** generator, and `FILE_SET CXX_MODULES`
- ☐ `import std;` in CMake is still behind an experimental, version-keyed variable
- ☐ Header units (`import <vector>;`) have essentially no build-system support
- ☐ IDE support is partial everywhere; clangd needs a full rebuild after editing an interface
- ☐ Almost no third-party libraries ship module interfaces yet

Flags and tools

```
# GCC
g++ -std=c++20 -fmodules-ts -c math.cppm          # BMIs land in gcm.cache/
g++ -std=c++23 -fmodules-ts -fsearch-include-path bits/std.cc  # build the std
```

```

module

# Clang
clang++ -std=c++20 --precompile math.cppm -o math.pcm
clang++ -std=c++20 -fmodule-file=math=math.pcm main.cpp math.pcm

# MSVC
cl /std:c++20 /interface /TP math.ixx          # produces math.ifc
cl /std:c++20 /reference math=math.ifc main.cpp

# CMake
cmake -G Ninja -DCMAKE_CXX_STANDARD=20 ..      # Ninja is required

```

```

P1689          # the standard dependency-scanning format that made build
support possible
clang-scan-deps # Clang's implementation of it

```

Exercises worth doing

1. Write a two-module project (`math` with an exported and a non-exported function) and build it by hand with your compiler's raw flags, no build system. You'll learn what BMIs are.
2. Confirm the non-exported function is genuinely invisible — try to call it, and try to declare it yourself in the importer.
3. Split that module into two partitions plus a primary interface unit. Then remove one `export import` and read the error.
4. Build the same project with CMake 3.28+, Ninja, and `FILE_SET CXX_MODULES`.
5. Wrap an existing header of yours in a module using the global module fragment and `export using`. Switch one consumer to `import`.
6. Measure: build a TU that includes `<vector>`, `<string>`, `<algorithm>`, `<ranges>` a hundred times vs one that uses `import std;`. Compare with `-ftime-trace`.
7. Try `import std;` with your current CMake setup. However far you get, you'll have an accurate sense of the ecosystem's state.

Chapter 18 — C++23 & C++26 Highlights

What this chapter covers

- **C++23:** `std::expected`, `std::print/println`, `std::mdspan`, `std::flat_map`, `std::generator`, `views::zip/enumerate/chunk`, deducing `this`, multidimensional `operator[]`, `if constexpr`, `std::stacktrace`, `import std;`
- **C++26 (as it stands):** `static reflection`, `contracts`, `std::execution` (senders/receivers), `std::hive`, erroneous behavior (replacing some UB), pack indexing, `std::simd`

Mostly a consolidation section — many of these features appear in earlier sections where they belong topically, and this is where they're gathered with adoption status attached. C++23 is the *quality-of-life*

release: it fixes things C++20 shipped incomplete. C++26 is the *transformational* one, and it is now **done**.

18.1 C++23 — the consolidation release

The headline features

Feature	What it does	Section
<code>std::expected<T,E></code>	value-or-error return type with monadic chaining	§13.6
<code>std::print</code> / <code>std::println</code>	fast, safe, direct-to-stdout formatted output	below
<code>std::mdspan</code>	multidimensional non-owning view	§11.10
<code>std::flat_map</code> / <code>flat_set</code>	sorted-vector map adaptors — cache-friendly, read-mostly	§11.4
<code>std::generator</code>	the standard coroutine generator, with recursive yield	§16.4
<code>views::zip</code> / <code>enumerate</code> / <code>chunk</code> / <code>slide</code> / <code>stride</code>	the range adaptors C++20 was missing	§12.7
<code>std::ranges::to<C>()</code>	materialize a view into a container	§12.7
Deducing this	one function template for const/non-const/lvalue/rvalue; retires most CRTP	§7.9, §8.10
Multidimensional operator[]	<code>m[i, j]</code>	§9.4
<code>if constexpr</code>	the un-misusable form of <code>is_constant_evaluated</code>	§6.5
<code>std::stacktrace</code>	portable stack capture	below
<code>import std;</code>	the whole standard library in one import	§17.4

`std::print` / `std::println`

```
#include <print>

std::println("Hello, {}", name);
std::println("{:>10.3f}", value);           // full <format> spec
std::print(stderr, "error: {}\n", msg);
std::println!("{}", std::vector{1, 2, 3});   // [1, 2, 3] – containers
format directly
```

Why it matters beyond convenience: `std::print` writes to the stream **directly**, avoiding the `std::format` → `std::string` → `std::cout` round-trip. It's typically faster than both `printf` and `iostream`, it's type-safe, and format-string errors are **compile-time** errors (the format string is a `constexpr` parameter — §6.5).

```
std::println("{} ", 42, 43);           // X compile error – too many arguments
printf("%s", 42);                     // ⚠ UB, warning at best
```

Make a custom type printable by specializing `std::formatter` (one of the two standard templates you're meant to specialize, alongside `std::hash` — §10.3).

`std::stacktrace`

```
#include <stacktrace>

auto trace = std::stacktrace::current();
std::println("{} ", trace);

for (const auto& frame : trace)
    std::println("{} at {}:{}", frame.description(), frame.source_file(),
frame.source_line());
```

Pairs naturally with a `set_terminate` handler (§13.4) or a custom assertion macro. ⚠ Requires linking a support library on some toolchains (`-lstdc++exp` on GCC) and needs debug info to produce useful output.

The smaller ones you'll use constantly

```
// Monadic optional (matching expected's interface, §13.6)
opt.and_then(f).transform(g).or_else(h);

// std::to_underlying – replaces static_cast<std::underlying_type_t<E>>(e)   §9.3
auto raw = std::to_underlying(Color::Red);

// std::unreachable() – mark a branch as impossible (⚠ UB if reached)
switch (x) { case 1: return a; case 2: return b; default: std::unreachable(); }

// std::move_only_function – the std::function that accepts move-only callables
§6.10
std::move_only_function<void()> f = [p = std::make_unique<T>()] { };

// std::byteswap, saturating arithmetic helpers
// std::out_ptr / std::inout_ptr – smart pointers across C APIs   §25
sqlite3* raw;
sqlite3_open("x.db", std::out_ptr(myUniquePtr));

// auto(x) / auto{x} – explicit decay-copy in one expression
process(auto(v.front()));           // copy, so it survives a mutation of v

// static operator() and static operator[]   §9.4
// [[assume(expr)]]   ⚠ §13.9
// size_t literal: 42uz
// #warning, #elifdef / #elifndef
// std::start_lifetime_as   §14.4
```

⚠ C++23 also relaxed `constexpr` substantially (non-literal variables, `static` in `constexpr` functions, relaxed `goto`) and made `std::unique_ptr` `constexpr`.

18.2 C++26 — status: **done**

The ISO C++ committee completed technical work on C++26 at the London Croydon meeting, finishing on Saturday 28 March 2026, resolving the remaining international ballot comments — 411 national body comments from the summer Committee Draft ballot — with no features added or removed at that meeting. The standard is now in its Draft International Standard phase for ISO approval, with publication expected later in 2026.

Feature freeze was actually a year earlier — the content has been stable since mid-2025, which is why compiler support is unusually far along for a just-shipped standard.

Herb Sutter has described C++26 as the most compelling release since C++11.

Static reflection ★ — the big one

Compile-time introspection of the program by the program. *Sutter's framing on adoption: reflection is more transformational than any ten other major features combined, and will dominate the next decade of C++.*

It uses a new ^^ operator (colloquially the "cat-ears" operator) to produce a reflection, rather than the Reflection TS's `reflExpr` keyword.

```
#include <meta>

constexpr std::meta::info r = ^^int;           // reflect a type
constexpr auto rf = ^^myFunction;             // reflect a function
constexpr auto rn = ^^std::vector;            // reflect a template

typename [:r:] x = 42;                         // SPLICE it back into code:
`int x = 42;`
```

Two operators: ^^ lifts code into a `std::meta::info` value; `[: :]` splices an `info` back into code. Everything in between is **ordinary `constexpr` C++** operating on values — not a separate metaprogramming language.

The example that makes the case, and that you'll see in every talk about this:

```
template <typename E>
constexpr std::string_view enum_to_string(E value) {
    std::string_view result = "<unknown>";
    [: expand(std::meta::enumerators_of(^E)) :] >> [&] <auto e>{
        if (value == [:e:]) result = std::meta::identifier_of(e);
    };
    return result;
}
```


No external code generator, no X-macro tricks, and no *magic_enum* — which was always a workaround for a missing feature and was bounded by template recursion limits. Every serialization library has the same shape.

What this replaces across the ecosystem: enum↔string, JSON/protobuf serialization, ORM field mapping, struct-of-arrays generation, mocking frameworks, *BOOST_HANA_ADAPT_STRUCT*-style registration, and most code generators in a typical build.

```
// Generic struct serialization, sketched
template <class T>
std::string serialize(const T& obj) {
    std::string out = "{";
    [ : expand(std::meta::nonstatic_data_members_of(^T)) : ] >> [&<auto m>{
        out += std::format("{}{}": {},", std::meta::identifier_of(m), obj.[:m:]);
    }];
    return out + "}";
}
```

C++26 also adds **annotations** — user-defined attributes attached to declarations and readable through reflection, which is how you'll express things like "serialize this field under a different name."

⚠ Cost: reflection runs in the compiler's constant evaluator. Heavy use has real compile-time cost, and the tooling story (debuggers, IDEs) for generated code is immature.

Contracts

Preconditions, postconditions, and assertions as language features (§13.9):

```
int divide(int a, int b)
    pre(b != 0)
    post(r: r * b <= a)
{
    contract_assert(a >= 0);
    return a / b;
}
```

Four enforcement modes — *ignore*, *observe*, *enforce*, and *quick-enforce* — selectable at build time. Contracts are part of the *declaration*, so they're visible to callers, tools, and documentation, unlike *assert*.

The feature drew criticism from some committee members over safety, complexity, and implementation details — worth knowing if you encounter strong opinions about it.

std::execution — senders and receivers

The structured async framework (§15.9). Schedulers, senders, receivers, operation states; *then*, *let_value*, *when_all*, *bulk*, *sync_wait*. It's also the answer to the coroutine executor problem (§16.5). Reference implementation: NVIDIA's *stdexec*.

std::simd

Portable data-parallel types — explicit vectorization without intrinsics:

```
#include <simd>

std::simd<float> a = ..., b = ...;
auto c = a * b + a;           // compiles to SIMD instructions for the
                               target
```

Fills the gap between "hope the auto-vectorizer notices" and "write AVX-512 intrinsics by hand" (§24). Derived from Vc/[std::experimental::simd](#).

std::hive

A bucketed container for large numbers of objects with frequent, unordered insertion and erasure — **stable references and pointers**, no reallocation, no element shuffling on erase. Erased slots are recycled.

The classic use case is a game's particle or entity pool: [vector](#) invalidates on erase, [list](#) is a cache disaster, [hive](#) is neither. (Formerly [plf::colony](#).)

Erroneous behavior

A new fourth category between defined and undefined (§14.1). Reading an uninitialized variable becomes **erroneous** rather than UB: you get a specified (wrong) value, implementations are encouraged to diagnose it, and it does **not** license the optimizer to delete your function. A meaningful reduction in the UB surface.

Pack indexing

```
template <class... Ts>
using First = Ts...[0];

template <class... Ts>
auto firstArg(Ts... args) { return args...[0]; }
```

Removes a large amount of [std::index_sequence](#) boilerplate (§6.12, §10.6).

The rest, briefly

```
void f(int) = delete("use the string overload instead"); // deletion with a
reason §7.2
std::optional<T&> // optional references
at last §3.7
std::function_ref // non-owning callable
param §6.10
std::hazard_pointer, std::rcu_domain // lock-free
reclamation §15.11
```

```
static_assert(false) // works in
uninstantiated templates §10.8
constexpr exceptions, constexpr placement new // §6.5
std::is_debugger_present(), std::breakpoint() // debugging support
saturating arithmetic, std::text_encoding
```

⚠ Removed before shipping

Trivial relocatability — the *trivially_relocatable_if_eligible* and *replaceable_if_eligible* specifiers — was removed from C++26 due to implementation bugs and deferred to a later standard. This was to have let *vector* relocate elements with *memcpy* instead of move-construct-then-destroy (§5, §11.1). It'll be back.

Compiler support

As of 30 April 2026, GCC 16.1 supports most C++26 features. Throughout C++26's development, both GCC and Clang consistently had about two-thirds of the features implemented, and GCC has reflection and contracts merged in trunk.

MSVC had no public reflection support as of late April 2026, and no published ETA. For reflection today, use GCC trunk, Bloomberg's Clang implementation, or Compiler Explorer.

```
-std=c++2c # the flag; -std=c++26 as compilers finalize naming
```

The safety context

Worth knowing because it shapes where the language goes next: WG21 rejected the Safe C++ proposal in favour of Profiles, which was contentious (§23). C++26's safety answer is contracts plus a **hardened standard library**, and the committee has adopted a three-year C++29 cycle in which further memory-safety work is a major focus.

Also of note for anyone tracking the process: Herb Sutter stepped down as WG21 convenor after serving since 2002, with Guy Davidson taking over from 1 January 2026.

18.3 Practical adoption

What to actually enable

Standard	Verdict
C++17	the safe floor — universal support, huge wins over C++14
C++20	the right default for new projects. Concepts, ranges, <i><=></i> , <i>jthread</i> , <i>format</i> , coroutines. Modules with caveats (§17)
C++23	ready now on recent toolchains; <i>expected</i> , <i>print</i> , <i>mdspan</i> , deducing <i>this</i> are all worth having

Standard	Verdict
C++26	shipped, but pick features individually — reflection is GCC/Clang-only today

Feature-test macros — the right way to adopt incrementally

```
#include <version>

#if __cpp_lib_expected >= 202202L
    using Result = std::expected<Config, Error>;
#else
    using Result = tl::expected<Config, Error>;
#endif

#if __cpp_lib_print >= 202207L
    std::println("{} ", x);
#else
    std::cout << std::format("{}\n", x);
#endif

#if defined(__cpp_impl_reflection)
    // reflection-based path
#endif
```

`<version>` gives you every `__cpp_lib_*` macro in one header; `__cpp_*` (no `lib`) covers language features. This beats compiler-version checks, which get wrong quickly.

The library substitutes that let you adopt early

Standard feature	Use today
<code>std::expected</code>	<code>tl::expected</code> , Boost.Outcome
<code>std::format</code> / <code>print</code>	fmt (the original; <code>std::format</code> is derived from it)
<code>std::mdspan</code>	<code>kokkos/mdspan</code> (the reference implementation)
<code>std::flat_map</code>	<code>boost::container::flat_map</code>
<code>std::execution</code>	<code>stdexec</code>
<code>std::generator</code> , <code>task</code>	<code>cppcoro</code> , <code>std::generator</code> backports
<code>std::hive</code>	<code>plf::colony</code>
<code>std::simd</code>	Vc, <code>std::experimental::simd</code> , Highway
<code>std::function_ref</code>	<code>tl::function_ref</code> , <code>absl::FunctionRef</code>
reflection (enum↔string)	<code>magic_enum</code> (bounded, but works today)

Adopting the library version now means the migration later is mostly a namespace change.

Where Part 1 ends

Sections 1–18 cover the language. The recurring themes, if you want the compressed version:

1. **Value semantics + RAII** is the foundation everything else rests on (§3, §4)
2. **Ownership is a type-level decision**, not a convention (§3.6, §4.8)
3. **The compiler generates and suppresses more than you think** — special members, moves, conversions (§7.1, §5.8)
4. **UB is a compile-time licence, and sanitizers are mandatory** (§14)
5. **Templates are a code generator with their own lookup and overload rules** (§10)
6. **Every abstraction has a cost model** — vtables, `shared_ptr`, `std::function`, coroutines, exceptions — and knowing it is the job (§8, §4.7, §6.10, §16.8, §13.1)

Part 2 covers what surrounds the language: toolchain, build systems, debugging, testing, security, performance, interop, and the domain-specific dialects that real C++ jobs are actually written in.

Exercises worth doing

1. Replace every `printf/iostream` call in a small project with `std::println`. Then deliberately mismatch a format argument and read the compile error.
2. Rewrite a function returning `bool` + out-param as `std::expected`, then chain three of them with `and_then`.
3. Take a class with duplicated `const`/non-const accessors and collapse them with deducing `this`.
4. On Compiler Explorer with GCC trunk or the Bloomberg Clang, write `enum_to_string` with reflection. Then compare it to what `magic_enum` does.
5. Add `pre/post` contracts to a function and build it under each of the four enforcement modes.
6. Guard a new-standard feature with the right `__cpp_lib_*` macro and a fallback, and verify both paths compile.

PART 2 — What Industry Actually Expects

The language is roughly half the job. This part is the other half: the toolchain that verifies every performance claim Part 1 made, the build systems and packaging that C++ notoriously lacks a standard answer for, the sanitizers and fuzzers that recover most of memory safety, and the domain dialects — games, finance, embedded, graphics, HPC, systems — that determine which two-thirds of the language you're actually allowed to use.

Chapter 19 — Toolchain & Compilation

What this chapter covers

- GCC / Clang / MSVC differences; `-std=`, `/std:`
- **Warnings:** `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wnon-virtual-dtor -Wold-style-cast -Wuseless-cast -Woverloaded-virtual, /W4 /permissive-`
- `-Werror` policy

- Optimization: `-O0..-O3 -Os -Ofast, -g, -march/-mtune`
- **Reading generated assembly, Godbolt** — non-negotiable for C++
- `-E, -S`, template instantiation dumps
- **LTO**, `-ffunction-sections -Wl,--gc-sections`
- **Compile-time management**: PCH, unity builds, `ccache, include-what-you-use, -ftime-trace`
- Name mangling, `nm -C, c++filt`
- Cross-compilation, sysroots

Part 2 begins. Part 1 was the language; Part 2 is everything a working C++ engineer is expected to know that isn't in the standard. This section is the foundation of that: if you can't read the assembly your compiler produced, you cannot verify any of the zero-overhead claims Part 1 made, and in C++ that verification is part of the job.

19.1 The three compilers

	GCC	Clang	MSVC
Standard flag	<code>-std=c++20</code>	<code>-std=c++20</code>	<code>/std:c++20</code>
Bleeding edge	<code>-std=c++2c</code>	<code>-std=c++2c</code>	<code>/std:c++latest</code>
Default stdlib	libstdc++	libstdc++ (Linux) / libc++ (macOS, <code>-stdlib=libc++</code>)	MSVC STL
ABI	Itanium C++ ABI	Itanium	MSVC ABI (incompatible)
Diagnostics	good, improved a lot	best — clearest template and concept errors	improving; historically weak
Tooling ecosystem	—	clang-tidy, clang-format, sanitizers, clangd all live here	<code>/analyze</code> , Visual Studio debugger

Also in the wild: **AppleClang** (Xcode's fork, version numbers don't match upstream LLVM), **ICX** (Intel's LLVM-based compiler), **NVCC** (CUDA, wraps a host compiler), and **EDG** (the front end inside several commercial tools).

MSVC: the flags you must set

```
/std:c++20
/permissive-          ★ standard conformance – enables two-phase lookup (§10.2)
/Zc:__cplusplus       ★ without this, __cplusplus reports 199711L. Forever.
/Zc:preprocessor      standard-conforming preprocessor
/EHsc                standard exception handling (assume extern "C" doesn't
throw)
/utf-8               source and execution charset
/W4                  warning level
```

`/permissive-` and `/Zc:__cplusplus` are not optional for portable code. Without the first, MSVC accepts template code that GCC and Clang reject (§10.2); without the second, every feature-test check that keys off `__cplusplus` silently takes the wrong branch.

Build with more than one compiler

This is the highest-value, lowest-effort correctness practice in C++. Different front ends catch different things:

- MSVC without `/permissive-` accepts non-dependent-name errors GCC/Clang reject
- Clang has the sharpest diagnostics for lifetime and template issues
- GCC catches some optimizer-visible UB others miss
- libstdc++ / libc++ / MSVC STL have different debug modes, different `std::string` layouts (§11.9), different unspecified behaviours you may have accidentally depended on

A CI matrix of three compiler front ends finds bugs that no amount of testing on one will.

19.2 Warnings

The baseline set

```
# GCC / Clang – the everyday set
-Wall -Wextra -Wpedantic
-Wshadow                # a local hiding a member or outer variable
-Wconversion -Wsign-conversion # implicit narrowing / signedness changes (§2,
§14)
-Wnon-virtual-dtor      # polymorphic base without a virtual destructor (§8.3)
-Woverloaded-virtual    # a derived function HIDING a virtual overload
(§6.1, §8.4)
-Wold-style-cast        # C-style casts (§2)
-Wcast-align -Wcast-qual
-Wunused -Wuninitialized
-Wnull-dereference
-Wdouble-promotion
-Wformat=2
-Wimplicit-fallthrough
-Wmisleading-indentation
-Wduplicated-cond -Wduplicated-branches # GCC
-Wlogical-op -Wuseless-cast             # GCC
-Wsuggest-override                     # GCC (Clang: -Winconsistent-
missing-override)
-Wdangling-reference                   # GCC 13+ (§3.4)
-Wdangling-gsl                         # Clang – string_view/span
(§11.9)
-Wreturn-type                         ★ missing return = UB
(§14.2)
-Wdeprecated-copy                    # implicit copy where a dtor is declared (§4.9)
-Wpessimizing-move -Wredundant-move  # (§5.8)
```

```
# MSVC
/W4 /permissive- /Zc:__cplusplus /Zc:preprocessor
/w14242 /w14254 /w14263 /w14265 /w14287 /we4289 /w14296
/w14311 /w14545 /w14546 /w14547 /w14549 /w14555
/w14619 /w14640 /w14826 /w14905 /w14906 /w14928
```

(That MSVC list — promoting useful off-by-default warnings to level 1 — is the widely-circulated "cppbestpractices" set. Worth pasting once and forgetting about.)

-Weverything (Clang)

Not a baseline. It includes mutually contradictory warnings and warnings about future standards. Use it as a *discovery* tool: turn it on once, read what it says, promote the useful ones into your real set, and turn it back off.

⚠ Silence third-party headers with **-isystem**

```
-I include/           # your headers – warnings ON
-isystem third_party/ # ★ their headers – warnings SUPPRESSED
```

Without this, a noisy dependency drowns your own diagnostics and makes **-Werror** impossible. In CMake, `target_include_directories(... SYSTEM ...)`, and `FetchContent/find_package` imported targets usually get this automatically.

Local suppression

```
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wdeprecated-declarations"
legacyCall();
#pragma GCC diagnostic pop
```

Always push/pop, always narrow scope, always leave a comment saying why.

19.3 **-Werror** policy

The nuanced answer, which most projects get wrong in one direction or the other:

Context	-Werror?
Your CI , with a pinned compiler version	✓ yes — this is the point
Local developer builds	usually yes, with an easy opt-out flag

Context	-Werror?
Released source tarballs that users compile	X no — a newer compiler adds warnings and breaks their build for no reason
A new codebase	✓ from day one; it never gets easier later
An existing codebase with 4,000 warnings	enable per-warning, or per-directory, incrementally

```
option(MYPROJ_WERROR "Treat warnings as errors" ${PROJECT_IS_TOP_LEVEL})
```

The pattern: on by default when you're the top-level project, off when you're someone's dependency.

19.4 Optimization levels

Flag	Meaning
-O0	no optimization. Fast compiles, readable debugging, very slow code (C++ abstractions cost real cycles here)
-O1	basic; a reasonable sanitizer build level
-O2	the production default. Nearly all optimizations that don't blow up code size
-O3	-O2 + aggressive inlining, loop unrolling, vectorization. Sometimes slower than -O2 (icache pressure) — measure
-Os	optimize for size
-Oz	Clang: size at any speed cost
-Og	optimized but debuggable — the right level for a debug build
⚠ -Ofast	-O3 + -ffast-math — breaks IEEE 754

⚠ **Never use -Ofast (or -ffast-math) casually.** It permits reassociating floating-point operations, assumes no NaN or infinity, and flushes denormals. Results change, `std::isnan` stops working, and — because it sets FTZ/DAZ in the MXCSR register at startup — it can silently change the behaviour of *other libraries loaded into your process*.

Debug info is independent of optimization

```
-g          # full debug info — orthogonal to -O
-g3         # + macro definitions
-gsplit-dwarf # debug info into separate .dwo files — much faster links
-Wl,--gdb-index # faster gdb startup
```

Ship release builds with debug info, then strip it into a separate file. Without it, your production crash dumps are useless (§28).

```
g++ -O2 -g -o app main.cpp
objcopy --only-keep-debug app app.debug
objcopy --strip-debug --add-gnu-debuglink=app.debug app
```

-march and **-mtune**

```
-march=x86-64-v2      # SSE4.2, popcnt — ~2009+
-march=x86-64-v3      # ★ AVX2, BMI, FMA — ~2015+, the common modern baseline
-march=x86-64-v4      # AVX-512 — narrower hardware support
-march=native         # ⚠ tuned for THIS machine — do not ship this
-mtune=generic         # schedule for a target without requiring its instructions
```

-march sets the **instruction set you're allowed to emit** (running on older hardware = **SIGILL**). **-mtune** only affects scheduling and is always safe. The common shipping choice: **-march=x86-64-v2 -mtune=generic**, or runtime dispatch (**__builtin_cpu_supports**, function multiversioning) for hot kernels.

A production build

```
g++ -std=c++20 -O2 -g -DNDEBUG \
    -march=x86-64-v2 -mtune=generic \
    -flto=auto -ffunction-sections -fdata-sections -Wl,--gc-sections \
    -fstack-protector-strong -D_FORTIFY_SOURCE=3 -D_GLIBCXX_ASSERTIONS \
    -fvisibility=hidden -fvisibility-inlines-hidden \
    -Wl,-z,relro,-z,now -fPIE -pie
```

(The hardening flags are §23; the visibility flags are §20.)

19.5 Reading generated assembly, and Godbolt

Why this is non-negotiable in C++

Every performance claim in Part 1 — "moves are $O(1)$ ", "the lambda inlines", "RVO means no copy", "**unique_ptr** is zero-overhead", "**final** devirtualizes" — is a claim about generated code. In a language built on the zero-overhead principle, **checking the output is how you verify the abstraction actually cost nothing**. No other mainstream language requires this as routinely.

Compiler Explorer (godbolt.org)

The single most useful C++ tool that exists. Learn it properly:

- **Multiple compilers side by side**, every version back a decade

- **Colour-mapped source↔assembly** — click a line, see its instructions
- **Diff view** — compare two flag sets or two code versions
- **Execution** — actually run it
- **Library support** — fmt, Boost, Google Benchmark, ranges
- **Opt Pipeline / LLVM IR views** — see what each pass did
- **cppinsights integration** (§10) — see the *source* the compiler generated
- Shareable links, which makes it the standard way to ask a C++ question

What to look for

```
// 1. Did the lambda inline?           -> is there a `call` at all?
// 2. Did RVO fire?                   -> is there a copy/move constructor call?
 (§5.7)
// 3. Did the virtual call devirtualize?-> `call [rax]` vs `call Circle::area`
 (§8.2)
// 4. Did it vectorize?                -> xmm/ymm/zmm registers, `vpaddq` etc.
// 5. Did the allocation get elided?   -> a `call operator new` you didn't want
 (§16.8)
// 6. Did UB delete your check?       -> a missing branch (§14.3)
```

⚠ **Prevent dead-code elimination** or you'll measure nothing:

```
static void escape(void* p) { asm volatile("" : : "g"(p) : "memory"); }
static void clobber()      { asm volatile("" : : : "memory"); }
// or, in a benchmark: benchmark::DoNotOptimize(x); benchmark::ClobberMemory();
```

Optimization remarks — the compiler explaining itself

```
# Clang
-Rpass=inline -Rpass-missed=inline
-Rpass-missed=loop-vectorize -Rpass-analysis=loop-vectorize
-fsave-optimization-record      # YAML -> use with opt-viewer.py

# GCC
-fopt-info-vec -fopt-info-vec-missed
-fopt-info-inline-missed
```

This tells you *why* the vectorizer bailed ("loop not vectorized: unsafe dependent memory operations") which is far more actionable than staring at scalar assembly.

Readability helpers

```
-masm=intel          # Intel syntax instead of AT&T
-fverbose-asm        # annotate with variable names
```

```
llvm-mca          # static throughput/latency analysis of a code block
```

19.6 -E, -S, and dumps

```
g++ -E main.cpp -o main.i          # preprocess only – see macro expansion,
include bloat
g++ -S main.cpp -o main.s          # assembly
g++ -c main.cpp                    # object file
g++ -H main.cpp                    # ★ print the include TREE (depth by dots)
g++ -MMD -MP -c main.cpp           # generate .d dependency files for your build
system

# Clang AST and template debugging
clang++ -Xclang -ast-dump -fsyntax-only main.cpp
clang++ -ftime-trace main.cpp      # ★ per-template compile time
($10.10)
-ftemplate-backtrace-limit=0      # full instantiation chain in
errors

# GCC internals
g++ -fdump-tree-all -fdump-rtl-all
g++ -ftime-report
```

-E is the tool for "why is this macro doing that" and for measuring how much text a header actually costs. **-H** is the tool for "why does this TU take 8 seconds."

19.7 LTO and section garbage collection

Link-Time Optimization

```
-flto=auto          # GCC: parallel LTO
-flto=thin          # ★ Clang ThinLTO – near-full benefit, much faster than
full LTO
-flto               # full/monolithic LTO – best results, slowest, memory-
hungry
```

What it buys you: **cross-TU inlining**, **cross-TU devirtualization** (§8.2 — often the biggest win), whole-program dead code elimination, and better constant propagation. Typical result: 5–20% faster, meaningfully smaller.

⚠ Practicalities:

- Static libraries need the LTO-aware wrappers: **gcc-ar**, **gcc-ranlib**, **gcc-nm**
- **Link time and memory go up substantially** — full LTO on a large C++ project can need many GB

- **LTO surfaces ODR violations** (§6.4) that were previously silent. `-Wodr` fires here. This is a *feature*, though it feels like a regression the first time
- Use ThinLTO by default; it's designed for exactly this trade-off

Section GC — dropping unused code

```
-ffunction-sections -fdata-sections    # one section per function/object
-Wl,--gc-sections                      # linker discards unreferenced sections
-Wl,--print-gc-sections                # see what went
-Wl,--icf=all                          # identical code folding (gold, lld) -
merges                                # identical functions, e.g. template
instantiations
```

`--icf=all` is particularly effective against template code bloat (§10.10): `vector<int*>` and `vector<char*>` often generate byte-identical code that gets folded into one.

Linkers

Linker	Notes
bfd	GNU default, slowest
gold	faster, ELF-only, now largely superseded
lld	LLVM's, very fast, <code>-fuse-ld=lld</code>
mold	★ fastest available, <code>-fuse-ld=mold</code> , drop-in

Switching to mold or lld is a one-flag change that can cut incremental build times noticeably on large projects, because link time dominates the edit-build-test loop.

19.8 Compile-time management

C++ builds are slow for structural reasons (§10.10, §17.1). In rough order of **value per effort**:

1. `ccache` / `sccache` — do this first

```
export CMAKE_CXX_COMPILER_LAUNCHER=ccache
ccache -s                                # hit rate statistics
ccache -M 50G                            # cache size
```

Near-free, no code changes, and it makes rebuilds after a branch switch nearly instant. `sccache` adds distributed/cloud caching.

2. Measure before optimizing

```
clang++ -ftime-trace ... # per-TU JSON, open in chrome://tracing
ClangBuildAnalyzer --all . out && ClangBuildAnalyzer --analyze out # ★
aggregate the build
g++ -ftime-report
g++ -H # the include tree for one TU
```

ClangBuildAnalyzer's output ("these 10 headers cost 40% of your build; these 20 templates cost 30%") turns build-time work from guesswork into a targeted fix.

3. Reduce what gets included

- **include-what-you-use** — mechanical, high yield
- **Forward declarations** instead of includes in headers
- **Pimpl** (§4.4) for heavy implementation dependencies
- **extern template** (§10.10) for hot instantiations
- Move definitions out of headers where they don't need to be inline

4. Precompiled headers

```
target_precompile_headers(mytarget PRIVATE <vector> <string> <memory> "common.h")
```

Effective for stable, universally-included headers. Caveats: must be rebuilt when *anything* in the PCH changes, they're compiler-specific, and they interact badly with **ccache** unless configured (**CCACHE_SLOPPINESS=pch_defines,time_macros**).

5. Unity / jumbo builds

Concatenate N source files into one TU. Real speedups (2–5×) because headers parse once per batch instead of once per file.

⚠ Costs: **worse incremental builds** (change one file, rebuild the whole batch), and they expose ODR-adjacent problems — anonymous-namespace symbols and **static** functions from different files now collide, and a missing **#include** in one file compiles because a neighbour had it.

```
set(CMAKE_UNITY_BUILD ON)
set(CMAKE_UNITY_BUILD_BATCH_SIZE 8)
```

6. Distributed builds

distcc, **icecc**, FASTBuild, or a build system with remote execution (Bazel, Buck2). Worth it above a certain team/codebase size.

7. Ninja over Make

```
cmake -G Ninja ..
```

Faster dependency checking, better parallelism, required for modules (§17.5). There's no reason to use Make generators for new projects.

19.9 Name mangling

C++ needs unique linker symbols for overloads, namespaces, templates, and const-qualified members — so names get **mangled**.

```
namespace foo { void bar(int); }
```

```
Itanium ABI (GCC/Clang):  _ZN3foo3barEi
MSVC:                    ?bar@foo@@YAXH@Z
```

Reading them

```
nm -C libfoo.a                # ★ -C = demangle
c++filt _ZN3foo3barEi        # demangle one symbol
objdump -TC libfoo.so        # dynamic symbols, demangled
readelf -Ws libfoo.so
llvm-nm --demangle
nm -D --undefined-only app    # what does this binary still need?

# MSVC
dumpbin /SYMBOLS obj.obj
undname "?bar@foo@@YAXH@Z"
```

Why you'll actually need this

"undefined reference to `foo::bar(int)`" — the linker demangled it for you; the question is which object file should have provided it. Common causes: a definition left in a header without `inline`, a missing library, a template instantiated nowhere (§10.10), or a **signature mismatch** (const on the member function, a different namespace).

ABI mismatch symptoms — undefined references mentioning `__cxx11` mean the `_GLIBCXX_USE_CXX11_ABI` split (§25): one object was built against the old pre-C++11 `std::string` and one against the new one.

```
-D_GLIBCXX_USE_CXX11_ABI=0    # force the old ABI to match an old prebuilt
library
```

extern "C" disables mangling entirely (§25) — which is why it's the universal interop boundary and why you can't overload an **extern "C"** function.

19.10 Cross-compilation and sysroots

Target triples

```
aarch64-linux-gnu          # arch - os - abi
x86_64-w64-mingw32
armv7-none-eabi           # bare metal
riscv64-unknown-elf
wasm32-unknown-emscripen
```

Clang is natively cross; GCC is not

```
# Clang: one binary, any target
clang++ --target=aarch64-linux-gnu --sysroot=/path/to/aarch64-sysroot main.cpp

# GCC: a separate toolchain per target
aarch64-linux-gnu-g++ --sysroot=/path/to/sysroot main.cpp
```

A **sysroot** is a directory tree containing the target's headers and libraries (**usr/include**, **usr/lib**), so the compiler resolves **#include <stdio.h>** against the *target*'s libc rather than the host's.

CMake toolchain files (§20)

```
# aarch64.cmake
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR aarch64)
set(CMAKE_C_COMPILER aarch64-linux-gnu-gcc)
set(CMAKE_CXX_COMPILER aarch64-linux-gnu-g++)
set(CMAKE_SYSROOT /path/to/sysroot)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER) # find programs on the HOST
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)  # find libs on the TARGET
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
```

```
cmake -DCMAKE_TOOLCHAIN_FILE=aarch64.cmake ..
```

⚠ Pitfalls

- **Host vs target tools.** Code generators and test binaries built during the build must target the *host*. This usually means two CMake configurations.

- **try_compile fails** because it tries to *run* the produced binary. Set `CMAKE_TRY_COMPILE_TARGET_TYPE` `STATIC_LIBRARY`.
 - **Testing:** `qemu-user` (`qemu-aarch64 ./app`) lets you run target binaries on the host, and CMake can use it via `CMAKE_CROSSCOMPILING_EMULATOR`.
 - **Latent assumptions** surface here: endianness, `sizeof(long)`, `char` signedness (unsigned on ARM!), alignment requirements (§14.2), and `long double` differences.
 - **zig cc** is a surprisingly good drop-in cross-compiler that bundles sysroots for many targets — useful when you don't want to assemble a toolchain.
-

Gotcha checklist for this section

- ☐ MSVC needs `/permissive-` and `/Zc:__cplusplus` or it isn't standard C++
- ☐ Use `-isystem` for third-party headers or `-Werror` is impossible
- ☐ `-Werror` belongs in CI with a **pinned compiler**, not in released tarballs
- ☐ `-Ofast` / `-ffast-math` **breaks IEEE 754** and can affect other libraries in the process
- ☐ `-O3` is sometimes slower than `-O2` — measure
- ☐ `-march=native` is **not shippable**
- ☐ `-g` is independent of `-O` — **ship release builds with debug info**, stripped to a separate file
- ☐ Benchmarks and Godbolt snippets need a **DCE barrier** or you measure nothing
- ☐ LTO **surfaces existing ODR violations** — that's a feature
- ☐ LTO with static libraries needs `gcc-ar` / `gcc-ranlib`
- ☐ Unity builds break anonymous-namespace/`static` isolation and hide missing includes
- ☐ PCH + ccache needs `CCACHE_SLOPPINESS` configured
- ☐ `__cxx11` in an undefined-reference message means the `_GLIBCXX_USE_CXX11_ABI` split
- ☐ `char` is **unsigned** on ARM
- ☐ Cross-compiling breaks `try_compile` unless you set `CMAKE_TRY_COMPILE_TARGET_TYPE`

Exercises worth doing

1. Put a small function on Compiler Explorer. Compare `-O0`, `-O2`, `-O3` output. Then compare GCC vs Clang vs MSVC.
 2. Write a `unique_ptr` factory and confirm at `-O2` that the assembly is identical to raw `new/delete`.
 3. Write a virtual call in a loop. Look at the assembly. Add `final` to the class. Look again.
 4. Compile with `-Rpass-missed=loop-vectorize` on a loop that won't vectorize and read the reason. Fix it.
 5. Run `-H` on your largest TU and find the header costing the most. Then run `-ftime-trace` + ClangBuildAnalyzer over the whole build.
 6. Enable `ccache` on a real project. Measure a clean build, then a rebuild after `touching` one file, before and after.
 7. Turn on `-flto` on a project that's never used it. Note any new `-Wodr` warnings — those are real bugs.
 8. Deliberately create an undefined-reference error, then use `nm -C` to find which object was supposed to define the symbol.
 9. Cross-compile a hello-world for aarch64 with a CMake toolchain file and run it under `qemu-aarch64`.
 10. Compile a float-heavy computation with and without `-ffast-math` and diff the numeric results.
-

Chapter 20 — Build Systems & Libraries

What this chapter covers

- **CMake** — the industry default; **modern target-based CMake** (`target_link_libraries` with `PUBLIC/PRIVATE/INTERFACE`)
- Generator expressions, `FetchContent`, `find_package`, config packages, `install(EXPORT)`
- Ninja, Meson, Bazel, Buck2; autotools legacy
- **Package managers**: `vcpkg`, `Conan`, CPM, system packages
- Static vs shared libraries; **ABI stability is brutal in C++** (templates, inline, name mangling, `std::string` layout)
- `-fvisibility=hidden` and export macros
- Header-only vs compiled; `extern template`
- `compile_commands.json` (powers clangd, clang-tidy)

Two things dominate this section. First: **modern CMake is target-based**, and almost every CMake tutorial you'll find online teaches the pre-2014 variable-based style that you should not use. Second: **C++ ABI stability is brutal** — far worse than C's — and understanding why is the difference between shipping a library people can upgrade and shipping one that crashes mysteriously after a patch release.

20.1 Modern CMake: targets, not variables

The mental model

Everything is a target. Targets carry properties. Properties propagate along `target_link_libraries` edges according to `PUBLIC / PRIVATE / INTERFACE`.

Once that clicks, CMake stops being a pile of magic variables and becomes a dependency graph with usage requirements.

Old style — don't write this

```
# X 2010-era CMake. Global, order-dependent, leaks into every target.
include_directories(include/)
add_definitions(-DFOO)
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -std=c++20")
link_libraries(foo)
file(GLOB SOURCES src/*.cpp)           # ⚠ new files don't trigger a re-configure
```

Modern style

```
cmake_minimum_required(VERSION 3.23)  # 3.23 for FILE_SET; 3.28 for C++20 modules (§17)
project(mylib VERSION 1.2.0 LANGUAGES CXX)

add_library(mylib
```

```
    src/widget.cpp
    src/engine.cpp                # list sources explicitly, not with GLOB
)
add_library(My::mylib ALIAS mylib) # ★ namespaced alias

target_compile_features(mylib PUBLIC cxx_std_20) # not CMAKE_CXX_STANDARD

target_include_directories(mylib
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
    PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}/src
)

target_link_libraries(mylib
    PUBLIC fmt::fmt                # appears in mylib's public headers
    PRIVATE nlohmann_json::nlohmann_json # implementation detail only
)

target_compile_definitions(mylib PRIVATE MYLIB_BUILDING)
```

PUBLIC / PRIVATE / INTERFACE

Keyword	Used when building this target	Propagated to consumers
PRIVATE	✓	✗
INTERFACE	✗	✓
PUBLIC	✓	✓

The decision rule:

Does it appear in my public headers? → PUBLIC. Only in my .cpp files? → PRIVATE.

Getting this right is what makes transitive dependencies work automatically. Getting it wrong either leaks your implementation dependencies onto every consumer (over-PUBLIC) or gives consumers compile errors about missing headers (over-PRIVATE).

Namespaced ALIAS targets

```
add_library(My::mylib ALIAS mylib)
```

Two reasons this matters:

- 1. **My::mylib** works identically whether the consumer got you via `find_package` or `add_subdirectory` — so a project can vendor you or find you with no code change.
- 2. **A name with :: that doesn't exist is a configure-time error.** Without the namespace, `target_link_libraries(app myLib)` with a typo is silently treated as a raw `-lmyLib` link flag and

fails much later, confusingly.

Warning sets: use an INTERFACE target, not global flags

```
add_library(project_warnings INTERFACE)
target_compile_options(project_warnings INTERFACE
    $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall;-Wextra;-Wshadow;-Wconversion;-Wnon-
virtual-dtor>
    $<$<CXX_COMPILER_ID:MSVC>:/W4;/permissive-;/Zc:__cplusplus>
)
target_link_libraries(mylib PRIVATE project_warnings) # PRIVATE – don't inflict
on consumers
```

Same trick for sanitizers, C++ standard settings, and hardening flags: package them as INTERFACE targets and link them where you want them. Never touch `CMAKE_CXX_FLAGS`.

Presets

```
// CMakePresets.json
{
  "version": 6,
  "configurePresets": [
    { "name": "dev", "generator": "Ninja", "binaryDir": "build/dev",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_EXPORT_COMPILE_COMMANDS": "ON",
        "CMAKE_CXX_COMPILER_LAUNCHER": "ccache"
      }
    },
    { "name": "asan", "inherits": "dev", "binaryDir": "build/asan",
      "cacheVariables": { "CMAKE_CXX_FLAGS": "-fsanitize=address,undefined -fno-
sanitize-recover=all" }
    },
    { "name": "release", "generator": "Ninja", "binaryDir": "build/release",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "RelWithDebInfo" }
    }
  ]
}
```

```
cmake --preset dev && cmake --build --preset dev
```

Presets replace the shell wrapper scripts every project used to have, and IDEs read them directly.

20.2 Generator expressions

```
$<CONFIG:Debug> # 1 if the config is Debug
$<CXX_COMPILER_ID:GNU,Clang> # compiler check
```

```

$<COMPILE_LANGUAGE:CXX>           # for mixed C/C++ or CUDA targets
$<BUILD_INTERFACE:...>           # only when building from the source tree
$<INSTALL_INTERFACE:...>        # only when consumed from an install
$<TARGET_PROPERTY:tgt,PROP>
$<IF:$<CONFIG:Debug>,valA,valB>
$<$<BOOL:${SOME_VAR}>:flag>      # conditional inclusion
$<TARGET_FILE:mytarget>         # the built artifact's path

```

⚠ **Generator expressions are evaluated at *generate* time, not *configure* time.** So they are useless in `if()`, `message()`, or `string()`:

```

if($<CONFIG:Debug>)               # ✗ never works – the string isn't evaluated yet
message("${CMAKE_BUILD_TYPE}")   # ✓ but empty for multi-config generators
(Ninja Multi-Config, VS)

```

The `BUILD_INTERFACE/INSTALL_INTERFACE` pair is the one you'll use constantly: include directories differ between "someone is building me in-tree" and "someone installed me and is including from `/usr/include`".

20.3 Getting dependencies

`find_package` — two modes

```

find_package(fmt 10.0 REQUIRED)      # CONFIG mode if fmt-config.cmake
exists
find_package(Threads REQUIRED)      # MODULE mode – CMake ships
FindThreads.cmake
target_link_libraries(app PRIVATE fmt::fmt Threads::Threads)

```

- **CONFIG mode:** the package ships its own `<Name>Config.cmake` with real targets. This is what you should both consume and produce (§20.4).
- **MODULE mode:** CMake (or you) provides a `Find<Name>.cmake` that hunts for headers and libs. Necessary for old libraries that predate CMake config packages.

`FetchContent` — source-level dependencies

```

include(FetchContent)
FetchContent_Declare(fmt
  GIT_REPOSITORY https://github.com/fmtlib/fmt.git
  GIT_TAG        10.2.1           # ★ pin a TAG or SHA, never a
branch
  FIND_PACKAGE_ARGS 10.0          # CMake 3.24+: use a system copy if
present
)
FetchContent_MakeAvailable(fmt)

```

Good for: small dependencies, reproducible CI, projects where "clone and build" must just work. Bad for: large dependencies (you now compile Boost yourself), and version conflicts between transitive dependencies.

CPM.cmake is a thin wrapper over **FetchContent** that adds a source cache across projects — a meaningful quality-of-life improvement.

Package managers

	vcpkg	Conan	System (apt/brew)
Model	source builds, curated ports	binary + source, recipes	prebuilt, distro-controlled
Integration	toolchain file	generator files, conan install	find_package
Versioning	manifest + baseline	full version ranges, lockfiles	whatever the distro has
Binary caching	✓	✓ (strong)	n/a
Cross/embedded	triplets	profiles (better)	X

```
// vcpkg.json – manifest mode
{ "name": "myapp", "version": "1.0.0",
  "dependencies": ["fmt", "spdlog", { "name": "boost-asio", "version>=": "1.83.0"
}] }
```

```
cmake -DCMAKE_TOOLCHAIN_FILE=$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake ..
```

```
# conanfile.txt
[requires]
fmt/10.2.1
spdlog/1.13.0
[generators]
CMakeDeps
CMakeToolchain
```

Rough guidance: **vcpkg** if you're Microsoft-adjacent or want the simplest onboarding; **Conan** if you need real version resolution, multiple ABIs/profiles, or a private artifact server; **FetchContent/CPM** for small projects with few dependencies; **system packages** only for stable, slow-moving system libraries.

20.4 Shipping an installable config package

This is the boilerplate that makes **find_package(MyLib)** work for your consumers. Worth having once, correctly.

```

include(GNUInstallDirs)
include(CMakePackageConfigHelpers)

# Declare headers as a FILE_SET (CMake 3.23+) – handles install + include dirs
# together
target_sources(mylib
    PUBLIC FILE_SET HEADERS
        BASE_DIRS include
        FILES include/mylib/widget.hpp include/mylib/engine.hpp
)

install(TARGETS mylib
    EXPORT mylibTargets
    FILE_SET HEADERS
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
)

install(EXPORT mylibTargets
    FILE mylibTargets.cmake
    NAMESPACE My::
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mylib
    # ★ consumers get My::mylib
)

write_basic_package_version_file(
    "${CMAKE_CURRENT_BINARY_DIR}/mylibConfigVersion.cmake"
    VERSION ${PROJECT_VERSION}
    COMPATIBILITY SameMajorVersion
    # semver
)

configure_package_config_file(
    cmake/mylibConfig.cmake.in
    "${CMAKE_CURRENT_BINARY_DIR}/mylibConfig.cmake"
    INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mylib
)

install(FILES
    "${CMAKE_CURRENT_BINARY_DIR}/mylibConfig.cmake"
    "${CMAKE_CURRENT_BINARY_DIR}/mylibConfigVersion.cmake"
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/mylib
)

```

```

# cmake/mylibConfig.cmake.in
@PACKAGE_INIT@
include(CMakeFindDependencyMacro)
find_dependency(fmt 10.0)
# ★ your PUBLIC dependencies, or
# consumers break
include("${CMAKE_CURRENT_LIST_DIR}/mylibTargets.cmake")
check_required_components(mylib)

```

⚠ **Forgetting `find_dependency` for your `PUBLIC` dependencies** is the most common packaging bug: it works in your build tree and fails for everyone else.

20.5 Other build systems

System	Character
Ninja	not a build system — a fast <i>backend</i> . Use it under CMake or Meson, always
Meson	cleaner syntax, fast, excellent cross-compilation and <code>pkg-config</code> integration. Strong in the GNOME/freedesktop world; smaller C++ ecosystem
Bazel / Buck2	hermetic, reproducible, remote caching and execution, monorepo-scale. Steep learning curve, awkward third-party story, mostly worth it at large-company scale
Autotools	<code>./configure && make</code> . Legacy, but you <i>will</i> meet it. Slow, arcane, still ubiquitous in older C/C++ projects
build2, xmake, premake, SCons	exist, have adherents, low ecosystem gravity

Why CMake won despite everything: libraries ship CMake config packages, IDEs read CMake, package managers integrate with CMake. It's the lingua franca, so it's the default even when another tool is technically nicer.

20.6 Static vs shared libraries

	Static (<code>.a</code> / <code>.lib</code>)	Shared (<code>.so</code> / <code>.dll</code> / <code>.dylib</code>)
Linked	at build time, copied in	at load time
Binary size	larger per binary	smaller; shared across processes
Deployment	one file, no runtime deps	must ship/locate the library
Update without relink	✗	✓
ABI risk	low	high (\$20.7)
LTO / whole-program opt	✓ works well	limited across the boundary
Call overhead	none	PLT/GOT indirection
Requires <code>-fPIC</code>	no	yes

⚠ Static libraries drop unreferenced objects

A static library is an *archive*; the linker pulls in only the object files that resolve a needed symbol. This silently breaks the **self-registration idiom**:


```
// plugin.cpp – nothing references this TU
static bool registered = registerPlugin("myplugin", &create); // ⚠ never
linked in
```

Fixes: `-Wl,--whole-archive lib.a -Wl,--no-whole-archive`, CMake's

`$<LINK_LIBRARY:WHOLE_ARCHIVE,mylib>`, MSVC `/WHOLEARCHIVE`, or an explicit reference from linked-in code.

Windows DLLs export nothing by default

Unlike ELF (where everything is exported unless hidden), Windows requires explicit `__declspec(dllexport)` — see §20.8. CMake's `WINDOWS_EXPORT_ALL_SYMBOLS` property automates it for simple cases, but not for data symbols.

CMake library types

```
add_library(a STATIC ...)
add_library(b SHARED ...)
add_library(c ...) # follows BUILD_SHARED_LIBS
add_library(d OBJECT ...) # object files, reusable in several targets
add_library(e INTERFACE) # ★ no sources – header-only libs, flag
bundles
```

20.7 ⚠ ABI stability in C++

What "ABI" covers

- Name mangling (§19.9)
- **Class layout**: member offsets, size, alignment, padding
- **Vtable layout**: which slot each virtual function occupies (§8.2)
- Calling conventions, register usage
- Exception handling tables, RTTI representation
- The layout of every standard library type you expose
- **The bodies of inline functions and templates** — because those get compiled into the *consumer*

Why C++ ABI is far worse than C's

In C, a struct and a function signature are the contract, and it's small. In C++:

```
// v1.0 – shipped
class Widget {
    int x_;
public:
    int get() const { return x_; } // INLINE – the body is compiled into
every caller
```

```
virtual void draw();  
};
```

Every one of these is an **ABI break**:

Change	Why it breaks
Add a data member	<code>sizeof(Widget)</code> changes; callers allocate the wrong size
Reorder data members	offsets shift; old callers read the wrong field
Change an inline function's body	callers have the old body compiled in — you get two behaviours
Add a virtual function	the vtable grows and slots may shift; old callers call the wrong one
Reorder virtual functions	slot indices shift — callers dispatch to the wrong function
Add a base class	layout and vtable change
Change a default argument	it's baked into the <i>caller</i> (§6.2)
Change a template's definition	consumers instantiated the old one
Add/remove <code>const</code> on a member function	mangled name changes
Change an <code>enum</code> 's underlying type	size changes

Anything visible in a header is part of your ABI. In C++, that's almost everything.

The three ways to have a stable ABI

1. Pimpl (§4.4) — one opaque pointer; the real class is invisible:

```
class Widget {  
    class Impl;  
    std::unique_ptr<Impl> impl_;           // sizeof(Widget) never changes  
public:  
    Widget(); ~Widget();                 // out-of-line – no inline bodies in the  
    header  
    int get() const;                     // declared only  
};
```

2. Pure-virtual interface + factory — the C++ COM model:

```
class IWidget {  
public:  
    virtual ~IWidget() = default;  
    virtual int get() const = 0;
```

```
};
extern "C" IWidget* createWidget();      // only ADD virtuals at the END, never
reorder
```

3. A C ABI (§25) — opaque handles and free functions. The only *genuinely* stable option, which is why every cross-language boundary uses it.

⚠ The standard library is part of your ABI

- **Never expose `std::string`, `std::vector`, or any standard type across a shared-library boundary you don't control both sides of.** `sizeof(std::string)` is 32 on libstdc++ and MSVC, 24 on libc++ (§11.9). Mixing them corrupts memory.
- **libstdc++ and libc++ are not interchangeable.**
- **`_GLIBCXX_USE_CXX11_ABI`:** GCC 5 changed `std::string`'s layout (COW → SSO) and kept both ABIs alive. Undefined references containing `__cxx11` mean a mismatch (§19.9). - `_D_GLIBCXX_USE_CXX11_ABI=0` forces the old one.
- **MSVC:** `/MD` vs `/MT` (dynamic vs static CRT), and **debug vs release CRT must match across the whole process**. Mixing them is a classic heap-corruption source. MSVC's ABI has been stable since VS2015.

The standard library has its own version of this problem: `std::regex` is known to be slow and `std::unordered_map` is slow by mandate (§11.3), and **neither can be fixed** without breaking every compiled binary in existence.

Tools

```
abidiff old.so new.so                # libabigail - ★ tells you exactly what broke
abi-compliance-checker
```

Pair with SONAME versioning:

```
set_target_properties(mylib PROPERTIES VERSION 1.2.0 SOVERSION 1)
```

Bump SOVERSION on any ABI break; bump only VERSION for compatible changes.

20.8 `-fvisibility=hidden` and export macros

```
-fvisibility=hidden -fvisibility-inlines-hidden
```

Default everything to hidden, then export explicitly. Four benefits: smaller symbol tables and **much faster dynamic loading**, more aggressive optimization (the compiler knows a hidden symbol can't be interposed), no accidental symbol collisions between libraries, and **the same semantics as Windows**, so you find export bugs on Linux instead of at Windows port time.

The portable export macro

```
// mylib/export.hpp
#ifdef _WIN32
    #if defined(MYLIB_BUILDING)
        #define MYLIB_API __declspec(dllexport)
    #else
        #define MYLIB_API __declspec(dllimport)
    #endif
#else
    #define MYLIB_API __attribute__((visibility("default")))
#endif

// usage
class MYLIB_API Widget { ... };
MYLIB_API void doThing();
```

CMake generates this for you:

```
include(GenerateExportHeader)
generate_export_header(mylib)          # produces mylib_export.h with MYLIB_EXPORT
set_target_properties(mylib PROPERTIES
    CXX_VISIBILITY_PRESET hidden
    VISIBILITY_INLINES_HIDDEN ON
)
```

⚠ Exceptions and RTTI across shared-library boundaries

This is the trap that costs people a full day:

If a polymorphic or exception type's `typeinfo` is hidden, `dynamic_cast` and `catch` silently fail across a `.so` boundary. The type comparison is by `type_info` address (or name string), and two hidden copies aren't the same object.

So exception types and types you `dynamic_cast` **must** be exported:

```
class MYLIB_API MyError : public std::runtime_error { ... }; // ★ MYLIB_API is
required
```

The symptom is an exception sailing past a handler that visibly matches it, or a `dynamic_cast` returning null for an object that is obviously of that type.

20.9 Header-only vs compiled

Header-only

Compiled

	Header-only	Compiled
Consumer setup	trivial — just include	link step, install, packaging
ABI issues	none (recompiled with the consumer)	real (§20.7)
Consumer compile time	bad — parsed in every TU	good
Code bloat	duplicate instantiations, mitigated by <code>--icf</code> (§19.7)	contained
Iteration speed for the library author	edit-and-everything-rebuilds	fast

The middle ground, if you have hot instantiations:

```
// widget.hpp
extern template class Widget<int>;           // §10.10 – don't instantiate here
// widget.cpp
template class Widget<int>;                  // instantiate once
```

And the split-mode pattern several libraries use (fmt, Asio, Catch2):

```
#ifdef MYLIB_HEADER_ONLY
    #define MYLIB_INLINE inline
    #include "mylib/impl.ipp"
#else
    #define MYLIB_INLINE
#endif
```

Consumers pick. `#define FMT_HEADER_ONLY` and `#define ASIO_HEADER_ONLY` are the well-known instances.

Modules (§17) are meant to eventually dissolve this trade-off — a module interface is parsed once *and* has no linkage burden — but not yet.

20.10 `compile_commands.json`

A JSON array recording the exact compile command for every source file. It is what makes the entire Clang tooling ecosystem work on your project.

```
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
```



```
cmake -G Ninja -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -B build
ln -sf build/compile_commands.json .          # ★ tools look in the project root
```

What consumes it:

Tool	Purpose
clangd	IDE intelligence — completion, go-to-definition, inline diagnostics
clang-tidy	static analysis (\$21)
include-what-you-use	include hygiene (\$19.8)
clang-format	(not really — it uses <code>.clang-format</code>)
cppcheck, various sanitizer and refactoring tools	

⚠️ **Only the Makefile and Ninja generators produce it.** Visual Studio and Xcode generators don't — use Ninja, or generate it another way:

```
bear -- make          # intercept an arbitrary build (Make, autotools, anything)
compdb -p build list  # add header entries so clangd handles headers properly
```

For non-CMake builds, `bear` is the universal answer. Without a `compile_commands.json`, clangd guesses at flags and your IDE experience degrades to roughly grep.

Gotcha checklist for this section

- ☐ Never set `CMAKE_CXX_FLAGS` / `include_directories` globally — use `target_*` commands
- ☐ `file(GLOB)` doesn't re-run when files are added — list sources explicitly
- ☐ `PUBLIC` if it's in your public headers, `PRIVATE` otherwise
- ☐ Always provide a **namespaced ALIAS** — typos in `My::lib` fail at configure time
- ☐ Generator expressions don't work in `if()` or `message()`
- ☐ `${CMAKE_BUILD_TYPE}` is empty for multi-config generators
- ☐ Pin `FetchContent` to a **tag or SHA**, never a branch
- ☐ Config packages must `find_dependency()` your `PUBLIC` dependencies
- ☐ Static libraries **drop unreferenced object files** — breaks self-registration
- ☐ Windows DLLs export nothing by default
- ☐ Anything in a header is ABI: adding a member, a virtual, or editing an **inline body** all break it
- ☐ Never expose `std::string/std::vector` across an uncontrolled shared-library boundary
- ☐ `libstdc++` and `libc++` are not ABI-compatible; `__cxx11` in a link error means the GCC 5 ABI split
- ☐ MSVC debug and release CRTs must not be mixed in one process

-   Hidden `typeinfo` makes `dynamic_cast` and `catch` **silently fail** across `.so` boundaries — export exception and polymorphic types
-  `compile_commands.json` requires the Ninja or Makefile generator

Exercises worth doing

1. Take an existing CMakeLists.txt using `include_directories` and global flags and convert it fully to target-based, including an INTERFACE warnings target.
2. Build a library with a `PUBLIC` and a `PRIVATE` dependency. Verify that a consumer sees one and not the other.
3. Write the full install/export boilerplate, `cmake --install` to a staging directory, and consume it from a separate project with `find_package`.
4. Deliberately omit `find_dependency` from your Config file and watch the consumer fail.
5. Build a shared library, add a data member to a class in a public header, rebuild **only** the library, and run the old consumer. Then run `abidiff` on the two `.sos`.
6. Throw a custom exception type from a shared library, catch it in the executable, with and without exporting the type under `-fvisibility=hidden`.
7. Put a self-registering static initializer in a static library and watch it not run. Fix it with `--whole-archive`.
8. Set up `compile_commands.json` and clangd on a project that doesn't have it. Note the difference in your editor.

Chapter 21 — Debugging & Analysis

What this chapter covers

- **GDB / LLDB / Visual Studio debugger:** breakpoints, conditional breakpoints, watchpoints, pretty-printers for STL types
- Core dumps, post-mortem, `std::stacktrace` (C++23)
- **Sanitizers:** ASan, UBSan, TSan, MSan, LeakSanitizer — the single biggest quality lever
- Valgrind (memcheck, callgrind, helgrind, massif)
- `rr` record/replay
- `perf`, VTune, flame graphs
- Static analysis: **clang-tidy** (with `modernize-*`, `bugprone-*`, `cppcoreguidelines-*`), cppcheck, PVS-Studio, Coverity, MSVC `/analyze`
- **Deciphering template error messages** — a real skill

C++ gives you no runtime safety net, so the tooling *is* the safety net. The practitioners who look fastest at debugging C++ are not smarter — they've internalized which tool answers which question, and they reach for it in seconds instead of adding `printfs` for an hour.

The decision table, roughly:

Question	Tool
Why did it crash?	GDB/LLDB on a core dump

Question	Tool
Did I corrupt memory?	ASan
Is this UB?	UBSan
Is this a race?	TSan
Am I reading uninitialized memory?	MSan or Valgrind memcheck
Where does the time go?	perf + flame graph
Where does the memory go?	massif / heaptrack
When did this variable change?	rr + watchpoint + reverse-continue
What bugs haven't I hit yet?	clang-tidy + fuzzing

21.1 GDB, LLDB, and the Visual Studio debugger

Build for it

```
-g3                # full debug info, including macros
-Og                # ★ optimized but debuggable – better than -O0 for
realistic behaviour
-fno-omit-frame-pointer # usable backtraces
-fno-inline         # when you specifically need to step into small
functions
```

⚠ At **-O2**, expect **<optimized out>** for variables and inlined frames that don't appear in backtraces. That's not a broken debugger — it's the optimizer doing its job. Use **-Og** for interactive debugging and keep **-O2 -g** for production (§19.4).

Command equivalences

Task	GDB	LLDB
Run	run / r	run / r
Breakpoint	break file.cpp:42 / b	breakpoint set -f file.cpp -l 42 / b file.cpp:42
Conditional	break f if x > 10	b f -c "x > 10"
Watchpoint	watch myVar	watchpoint set variable myVar
Backtrace	bt / bt full	bt / bt all
Frame	frame 3, up, down	frame select 3, up, down
Print	p expr, p/x, ptype	p expr, expression, type lookup
Locals	info locals, info args	frame variable
Step	n, s, finish, until	n, s, finish

Task	GDB	LLDB
Memory	<code>x/16xb ptr</code>	<code>memory read -c16 -fx -s1 ptr</code>
Threads	<code>info threads, thread 2</code>	<code>thread list, thread select 2</code>
All threads	<code>thread apply all bt</code>	<code>bt all</code>
Attach	<code>gdb -p PID</code>	<code>lldb -p PID</code>

The commands that earn their keep

```

catch throw                # ★ break at the THROW, not the catch – where the
info still is
catch catch
catch throw std::runtime_error

break f if strcmp(name, "x") == 0    # conditional breakpoints
break f
    ignore 1 999                    # skip the first 999 hits
tbreak f                            # one-shot
rbreak ^Widget::.*                 # regex – break on every method of a class

watch myVar                      # ★ break when a value CHANGES – hardware-
assisted,                          # but x86 gives you only 4. Software watchpoints
are ~1000x slower.
rwatch myVar                      # on read
watch -l ptr->field                # watch the LOCATION, not the expression

info frame  info registers  info sharedlibrary  info line
p *this
p *vec@10                          # print 10 elements starting at a pointer
set var x = 5                       # modify state and continue
call myFunction(3)                  # ⚠ runs real code in the inferior – can have
side effects

```

Pretty-printers

Without them, printing a `std::map` shows you red-black tree nodes. libstdc++ ships Python printers (usually auto-loaded); check with `info pretty-printer`, and if they're missing:

```

python
import sys; sys.path.insert(0, '/usr/share/gcc/python')
from libstdcxx.v6.printers import register_libstdcxx_printers
register_libstdcxx_printers(None)
end

```

libc++ has printers in the LLVM tree; LLDB has built-in formatters for most standard types. For your own types, write a GDB Python printer or — on Windows — a **.natvis** file, which is genuinely pleasant and worth doing for any custom container.

A **.gdbinit** worth having

```
set print pretty on
set print object on           # show the DYNAMIC type of polymorphic pointers
set print static-members off
set print vtbl on
set pagination off
set history save on
set confirm off
```

Visual Studio / WinDbg

- **Data breakpoints** (the equivalent of watchpoints), and they're well-integrated
- **Edit and Continue** — genuinely useful, no Linux equivalent
- **.natvis** custom visualizers
- **Time Travel Debugging** in WinDbg — record and replay, the Windows analogue of **rr**
- Mixed-mode (managed/native) debugging

21.2 Core dumps and post-mortem

Getting a core

```
ulimit -c unlimited
cat /proc/sys/kernel/core_pattern
sysctl -w kernel.core_pattern=/tmp/core.%.%.%.%t

# systemd systems
coredumpctl list
coredumpctl gdb 12345          # ★ pulls the right binary and symbols
                                automatically
coredumpctl dump PID > core
```

```
gdb ./myapp core.12345
(gdb) bt full
(gdb) thread apply all bt
(gdb) info registers
(gdb) frame 3
(gdb) p *this
```

What you need for it to be useful

1. **The exact same binary** that crashed
2. **Debug symbols** — either in the binary or in a separate `.debug` file linked by build-id (§19.4)
3. The same **shared libraries**, or a `set sysroot` pointing at copies

This is why §19 insisted on shipping release builds with debug info stripped to a side file. A production core dump against a stripped binary with no symbol store is nearly worthless.

In-process capture: `std::stacktrace` (C++23)

```
#include <stacktrace>
std::println("{", std::stacktrace::current());
```

Pre-C++23 alternatives: `boost::stacktrace`, `backward-cpp`, `libunwind`, or `backtrace()/backtrace_symbols()` from glibc.

⚠ Signal handlers must be async-signal-safe

```
void handler(int sig) {
    // ⚠ You may NOT: allocate, use std::cout, use most of the standard library,
    //   take a lock, or call anything not async-signal-safe.
    write(STDERR_FILENO, msg, len);           // ✓ write() is safe
    signal(sig, SIG_DFL); raise(sig);         // ✓ re-raise so a core dump is
    produced
}
```

`std::stacktrace::current()` allocates, so it is **not** signal-safe — capturing it in a handler is a common but technically unsound pattern. The robust approach is a **separate crash-handler process** (Breakpad/Crashpad), which is what Chrome and most shipped desktop software do (§28).

21.3 Sanitizers in practice

The what-each-catches table is in §14.5. This is the operational half.

Runtime options

```
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1:strict_string_checks=1
:\
check_initialization_order=1:abort_on_error=1:symbolize=1:log_path=/tmp/asan

UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1

TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1:history_size=7

LSAN_OPTIONS=suppressions=lsan.supp
ASAN_SYMBOLIZER_PATH=/usr/bin/llvm-symbolizer # ⚠ without this you get raw
addresses
```

`-fno-sanitize-recover=all` at compile time makes UBSan abort rather than print-and-continue — essential in CI, where printed warnings are ignored (§14.5).

Suppressions

```
# lsan.supp
leak:third_party_library
leak:dlopen
```

```
__attribute__((no_sanitize("address")))
void intentionallyWeirdFunction() { ... }

__attribute__((no_sanitize("undefined"))) // e.g. a deliberate unaligned
read
```

Use these sparingly and comment every one. A suppression file that grows unreviewed becomes a place where real bugs hide.

Practical notes

- **ASan + TSan cannot be combined.** Separate builds, separate CI jobs.
- **ASan needs `-fno-omit-frame-pointer -g`** for readable reports.
- **MSan requires an instrumented standard library** (an MSan build of libc++), which is why most projects use Valgrind memcheck for uninitialized reads instead.
- **HWASan** (`-fsanitize=hwaddress`) on AArch64 has much lower memory overhead than ASan and is viable for on-device testing.
- Sanitizers interact badly with `LD_PRELOAD` tricks, custom allocators, and some container/`ptrace` restrictions.
- **Sanitizers are not production tools.** For production, use the hardening flags in §23.
- **★ Sanitizers + fuzzing** (§22) is the highest-yield bug-finding setup in C++. Sanitizers only see executed paths (§14.5); fuzzing generates the paths.

21.4 Valgrind

No recompilation needed — it runs an unmodified binary under a synthetic CPU.

Tool	Purpose	Cost
memcheck (default)	invalid reads/writes, uninitialized reads , leaks, bad frees	20–50×
callgrind	instruction-level profiling with call graphs, deterministic	50–100×
cachegrind	cache and branch-predictor simulation	20–100×
helgrind / DRD	race detection	20–100×

Tool	Purpose	Cost
massif	heap usage over time	20×
DHAT	heap access patterns — what's allocated and never read	20×

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./app
valgrind --tool=callgrind ./app && kcache-grind callgrind.out.*
valgrind --tool=massif ./app && massif-visualizer massif.out.*
```

When Valgrind beats sanitizers

1. **You can't recompile** — a third-party binary, or a build you don't control
2. **Uninitialized reads without MSan's build burden** — `--track-origins=yes` tells you *where the uninitialized value came from*, which MSan does too but at far greater setup cost
3. **Deterministic instruction counts** — callgrind gives you a repeatable number, which makes it excellent for detecting performance *regressions* in CI where wall-clock timing is too noisy

Otherwise, sanitizers are faster and catch more (notably stack and global overflows, which memcheck misses).

21.5 `rr` — record and replay

The tool that changes how you debug hard bugs.

```
rr record ./myapp --args
rr replay                                # a normal GDB session, but deterministic
rr replay -p PID
rr pack                                  # bundle the trace so you can share it
rr record --chaos ./myapp                # ★ randomize scheduling to provoke rare races
```

Inside the replay session you get **reverse execution**:

```
reverse-continue    # rc
reverse-step        # rs
reverse-next
reverse-finish
```

The killer workflow

```
# "This pointer is null and I have no idea who nulled it."
(rr) watch -l obj->ptr
(rr) reverse-continue    # ★ runs BACKWARD to the last write. There's your culprit.
```

That combination — hardware watchpoint plus backward execution — answers "when did this become wrong?" directly, instead of by bisecting `printfs`.

The second thing it changes: **a rare failure only has to happen once**. Record your flaky test in a loop, and when it fails you have a trace you can replay identically as many times as you like.

Requirements and limits: x86-64 Intel/AMD with PMU access (`kernel.perf_event_paranoid <= 1`), often unavailable in containers and VMs, ~1.2–2× recording overhead, and no AVX-512 on some configurations. Windows' equivalent is WinDbg **Time Travel Debugging**.

21.6 `perf`, VTune, flame graphs

Interpretation is §24; these are the mechanics.

```
# Counters first — is it a CPU problem, a memory problem, or a branch problem?
perf stat -d ./app
#   cycles, instructions, IPC, cache-misses, branch-misses, page-faults

# Sampling profile
perf record -g --call-graph dwarf ./app           # DWARF unwinding — works without
frame pointers
perf record -g ./app                             # frame-pointer unwinding — needs
-fno-omit-frame-pointer
perf report                                       # interactive
perf annotate                                    # source+assembly with sample
attribution
perf top                                         # live, system-wide

# Specialized
perf c2c    # ★ cache-line contention / FALSE SHARING (§15.10)
perf mem    # memory access sampling
perf lock   # lock contention
perf sched  # scheduler latency
```

Flame graphs

```
perf record -F 99 -g ./app
perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
```

Width = time spent, stacked vertically by call depth. **Differential flame graphs** (`difffolded.pl`) compare before/after and are the fastest way to see what a change did.

⚠ Bad stacks are the usual failure: build with `-fno-omit-frame-pointer`, or use `--call-graph dwarf`, and make sure symbols are present (`perf` needs the unstripped binary or a matching debug file).

Other profilers

Tool	Notes
Intel VTune	microarchitecture analysis, memory access, threading — the deepest on Intel hardware
AMD µProf	the AMD equivalent
Superluminal	Windows, excellent UI, sampling
Tracy	★ frame-oriented instrumented profiler; the standard in game dev
heaptrack	allocation profiling, far friendlier than massif
Pyroscope / Parca	continuous profiling in production
WPA / ETW	Windows system-wide tracing

21.7 Static analysis

Static analysis finds bugs on paths your tests never execute — the complement to sanitizers, which only see what runs.

clang-tidy — the primary tool

```
# .clang-tidy — a reasonable starting point
Checks: >
  bugprone-*,
  cert-*,
  clang-analyzer-*,
  concurrency-*,
  cppcoreguidelines-*,
  misc-*,
  modernize-*,
  performance-*,
  portability-*,
  readability-*,
  -modernize-use-trailing-return-type,
  -readability-magic-numbers,
  -cppcoreguidelines-avoid-magic-numbers,
  -readability-identifier-length
WarningsAsErrors: 'bugprone-*,clang-analyzer-*'
HeaderFilterRegex: '^(src|include)/'
```

```
run-clang-tidy -p build -j$(nproc)           # whole project, parallel
clang-tidy --fix --fix-errors src/foo.cpp     # ★ many checks auto-fix
git diff -U0 HEAD~1 | clang-tidy-diff.py -p1 -path build # ★ only changed
lines — CI-friendly
```

```
// NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
auto* p = reinterpret_cast<Header*>(buf);
// NOLINTBEGIN(...) ... // NOLINTEND(...)
```

The check families, mapped to Part 1:

Family	Catches
bugprone-*	use-after-move (§5.6), exception escape (§13.10), forwarding-reference overload (§5.4), spurious wakeups (§15.5)
cppcoreguidelines-*	special member functions (§4.9), slicing (§8.6), owning memory (§3.6), reference coroutine params (§16.7)
modernize-*	make_unique, override, nullptr, ranges, = default
performance-*	unnecessary copies, shared_ptr by value (§4.7), missing reserve (§11.1)
concurrency-*	thread-unsafe library calls
clang-analyzer-*	path-sensitive analysis — null derefs, leaks, uninitialized use

clang-analyzer-* is the Clang Static Analyzer, which is genuinely different from the rest: it simulates execution paths rather than pattern-matching the AST. Slower, deeper, and worth running even if you skip other families. Standalone: scan-build make.

The others

Tool	Role
cppcheck	no compilation database needed, different check set — a real complement, not a substitute
PVS-Studio	commercial, excellent signal-to-noise, free for open source
Coverity	commercial, deep interprocedural, free tier for open source (Coverity Scan)
SonarQube	quality gates, trend tracking
MSVC /analyze	built in; plus the C++ Core Guidelines checker
Infer (Meta)	interprocedural, good at null/leak/race

Rolling it out on an existing codebase

Do **not** turn on everything and stare at 8,000 findings. Either:

1. clang-tidy-diff on changed lines only — new code is clean, old code decays away naturally, or
2. A ratcheting baseline — record today's count, fail CI if it increases

21.8 ⚠ Deciphering template error messages

A real, learnable skill. The errors are long because instantiation is recursive and every failed overload candidate gets reported.

The reading algorithm

- 1. **Read the first error only.** Everything after it is usually cascade.
- 2. **Find the required from here / in instantiation of chain** and read it **bottom-up** until you hit a file that is *yours*. That's the line you actually wrote wrong.
- 3. **Read the innermost complaint** — the last line of the chain is the real problem.
- 4. **Look for a type mismatch in the details** — nine times out of ten it's a stray `const`, a missing `&`, a missing member function, or a comparator issue.

```
main.cpp:42:5: required from here                                     ← ★ YOUR line
/usr/include/c++/13/bits/stl_algo.h:1234: required from ...
/usr/include/c++/13/bits/predefined_ops.h:43: error:
    no match for 'operator<' (operand types are 'const Widget' and 'const Widget')
                                     ↑ the actual problem: Widget has
no operator<
```

Common signatures, decoded

Message	Usually means
no match for 'operator<'	missing comparator for <code>sort/map</code> (§9, §12.6)
invalid use of incomplete type	forward declaration where a definition is needed — often pimpl's destructor (§4.4)
use of deleted function 'T::T(const T&)'	a move-only member, or a suppressed special member (§7.1)
no type named 'type' in 'std::enable_if<false, ...>'	SFINAE rejected every candidate (§10.4)
constraints not satisfied	★ a concept failure — modern and readable (§10.5)
passing 'const X' as 'this' argument discards qualifiers	calling a non- <code>const</code> method on a <code>const</code> object (§7.5)
undefined reference to 'X<int>::f()'	template defined in a <code>.cpp</code> and not instantiated (§10.2)
ambiguous overload	see §6.1 — or the C++20 reversed-candidate trap (§9.6)

Making them better

```
clang++ ...                                     # ★ Clang's template diagnostics beat
GCC's                                           #
-fconcepts-diagnostics-depth=3                 # GCC: explain WHICH sub-constraint
failed                                         #
-ftemplate-backtrace-limit=0                  # full chain (or =1 to cut the noise)
```

```
-fmacro-backtrace-limit=1
2>&1 | head -60                                # the first error is the one that matters
```

The structural fix: constrain your own templates. A concept (§10.5) or an early `static_assert` moves the error from 40 frames deep in `<algorithm>` to the line where the caller made the mistake:

```
template <class T>
    requires std::totally_ordered<T>
void mySort(std::vector<T>& v);           // "T does not model totally_ordered" –
at the call site

static_assert(std::forward_iterator<MyIter>);    // §12.1 – fail at the
definition
static_assert(std::is_nothrow_move_constructible_v<T>);    // §5.5
```

This is the practical argument for concepts that goes beyond overload resolution: **they relocate errors to where the human is.**

Gotcha checklist for this section

- ☐ `<optimized out>` at `-O2` is normal — use `-Og` for interactive debugging
- ☐ `catch throw` breaks where the exception is *thrown*, which is where the context still exists
- ☐ x86 gives you only **4 hardware watchpoints**; software watchpoints are ~1000× slower
- ☐ `set print object on` — otherwise GDB shows static types for polymorphic pointers
- ☐ A core dump needs the **exact binary** plus symbols
- ☐ `std::stacktrace::current()` allocates and is **not signal-safe**
- ☐ Signal handlers may not allocate, lock, or use most of the standard library
- ☐ Sanitizers need `ASAN_SYMBOLIZER_PATH` or you get raw addresses
- ☐ ASan and TSan can't be combined
- ☐ MSan needs an instrumented standard library — Valgrind memcheck is often the pragmatic choice
- ☐ `perf` needs `-fno-omit-frame-pointer` or `--call-graph dwarf` for usable stacks
- ☐ `rr` needs PMU access — usually unavailable in containers
- ☐ Read the **first** template error, and find *your* file in the instantiation chain
- ☐ Don't enable all clang-tidy checks on a legacy codebase at once — diff-only or ratchet

Exercises worth doing

1. Write a program with a null dereference. Debug it four ways: GDB on a core, ASan, Valgrind, and clang-analyzer. Note what each tells you that the others don't.
2. Use `catch throw` to break at the origin of an exception thrown five frames deep.
3. Set a watchpoint on a member that gets corrupted and use `rr + reverse-continue` to find the write. (If this is the only exercise you do here, do this one.)
4. Profile a program with `perf record -g` and generate a flame graph. Then make a change and generate a *differential* flame graph.
5. Reproduce the false-sharing case from §15.10 and confirm it with `perf c2c`.

6. Write a `.natvis` or GDB Python pretty-printer for one of your own container types.

7. Take a 300-line template error and reduce it to the one line you actually got wrong. Then add a concept and see the error move to the call site.

8. Run clang-tidy over an existing project with the config above. Fix everything in `bugprone-*` and `performance-*`.

9. Set up `clang-tidy-diff` in CI so only changed lines are checked.

Chapter 22 — Testing

What this chapter covers

- **GoogleTest / GoogleMock, Catch2, doctest, Boost.Test**
- Fixtures, parameterized and typed tests, death tests
- Mocking: interfaces vs templates; `gmock`, `Fakelt`, `Trompeloel`
- **Property-based testing:** `RapidCheck`
- Coverage: `gcov/lcov`, `llvm-cov`
- **Fuzzing:** `libFuzzer`, `AFL++`, `OSS-Fuzz` — essential for anything parsing input
- **Benchmarking:** `Google Benchmark`, `nanobench`, and correct methodology
- `CTest`, CI matrices across compilers/standards/platforms

Testing C++ has one dimension other languages don't: **the test suite is the delivery vehicle for the sanitizers**. A well-covered suite run under `ASan+UBSan` finds memory bugs; the same suite run under `TSan` finds races; the same code fuzzed finds the inputs your tests never thought of. Coverage isn't a vanity metric here — it's the fraction of your program that the dynamic analysis tools can actually see (§14.5).

If you do one thing from this section: **fuzz anything that parses untrusted input, under ASan+UBSan**. It is the highest bug-yield practice in the language.

22.1 Choosing a framework

	GoogleTest	Catch2 v3	doctest	Boost.Test
Adoption	highest	high	moderate	legacy/Boost shops
Compile time	moderate	slow	★ fastest	slow
Mocking	★ GoogleMock, integrated	none (use Trompeloel)	none	none
Expression decomposition	EXPECT_THAT + matchers	★ REQUIRE(a == b) prints both sides	same as Catch2	yes
Sub-tests	parameterized/typed	★ SECTION	SUBCASE	datasets
Header-only	X	X (v3 dropped it)	✓	X
In-source tests	X	X	★ ✓	X

Rough guidance: GoogleTest if you need mocking or you're on a large team (the ecosystem, CI integrations, and hiring pool are all biggest here). doctest if compile time matters or you want tests next to the code. Catch2 if you like **SECTION** and matchers and can afford the build cost. Boost.Test only if you're already deep in Boost.

22.2 GoogleTest

```
#include <gtest/gtest.h>

TEST(WidgetTest, DefaultIsEmpty) {
    Widget w;
    EXPECT_TRUE(w.empty());
    EXPECT_EQ(w.size(), 0);
}
```

ASSERT_* vs EXPECT_*

ASSERT_* aborts the current test function on failure; **EXPECT_*** records and continues. Default to **EXPECT_*** so one run tells you everything that's wrong; use **ASSERT_*** when continuing would crash (e.g. after checking a pointer isn't null).

⚠ **ASSERT_* only works in a function returning void** — it's implemented as **return**. In a helper that returns a value it won't compile, and in a helper that returns **void** it aborts *the helper*, not the test:

```
void checkInvariants(const Widget& w) { ASSERT_TRUE(w.valid()); }

TEST(WidgetTest, X) {
    checkInvariants(w);
    ASSERT_NO_FATAL_FAILURE(checkInvariants(w)); // ✓ propagate the failure properly
}
```

Fixtures

```
class DatabaseTest : public ::testing::Test {
protected:
    void SetUp() override { db_ = openTestDb(); }
    void TearDown() override { db_.reset(); }

    static void SetUpTestSuite() { startServerOnce(); } // once for the whole suite
    static void TearDownTestSuite() { stopServer(); }

    std::unique_ptr<Database> db_;
};
```

```
TEST_F(DatabaseTest, InsertThenQuery) { ... }
```

Matchers — where GoogleTest gets expressive

```
#include <gmock/gmock.h>
using namespace ::testing;

EXPECT_THAT(v, ElementsAre(1, 2, 3));
EXPECT_THAT(v, UnorderedElementsAre(3, 1, 2));
EXPECT_THAT(v, Contains(Gt(5)));
EXPECT_THAT(v, Each(AllOf(Ge(0), Le(100))));
EXPECT_THAT(v, SizeIs(3));
EXPECT_THAT(name, StartsWith("pre"));
EXPECT_THAT(opt, Optional(Eq(42)));
EXPECT_THAT(var, VariantWith<int>(5));
EXPECT_THAT(p, Pointee(Field(&Widget::id, Eq(7))));
EXPECT_THAT(w, Property(&Widget::size, Gt(0)));
EXPECT_THAT(actual, Pointwise(DoubleNear(1e-9), expected));
```

`EXPECT_THAT` gives far better failure messages than a hand-written loop — it prints the whole container and says exactly which element mismatched.

Parameterized tests

```
class ParseTest : public TestWithParam<std::tuple<std::string, int>> {};

TEST_P(ParseTest, ParsesCorrectly) {
    auto [input, expected] = GetParam();
    EXPECT_EQ(parse(input), expected);
}

INSTANTIATE_TEST_SUITE_P(Valid, ParseTest, Values(
    std::tuple{"1", 1}, std::tuple{"42", 42}, std::tuple{"-7", -7}
));
// also: ValuesIn(container), Combine(a, b), Range(0, 10)
```

Typed tests — test a concept across types

```
template <class T> class ContainerTest : public Test {};
using MyTypes = ::testing::Types<std::vector<int>, std::deque<int>,
std::list<int>>;
TYPED_TEST_SUITE(ContainerTest, MyTypes);

TYPED_TEST(ContainerTest, EmptyOnConstruction) {
    TypeParam c;
```

```
EXPECT_TRUE(c.empty());
}
```

This is how you verify that every implementation of an interface — or every type satisfying a concept (§10.5) — honours the same contract. Underused.

Death tests

```
EXPECT_DEATH(divide(1, 0), "division by zero");
EXPECT_DEATH_IF_SUPPORTED(f(), "");
EXPECT_EXIT(f(), ExitedWithCode(2), "fatal");
```

⚠ Death tests **fork**. They're slow, they don't compose well with threads (set `::testing::FLAGS_gtest_death_test_style = "threadsafe"`), and they're unavailable on some platforms. Use them for assertion/contract testing (§13.9), not routinely.

Running

```
./tests --gtest_filter=WidgetTest.*:*Slow*
./tests --gtest_repeat=100 --gtest_shuffle      # ★ flush out order
dependencies
./tests --gtest_break_on_failure                # drop into the debugger
./tests --gtest_output=xml:results.xml
```

`--gtest_shuffle --gtest_repeat=N` finds hidden inter-test coupling, which is one of the more common sources of "passes locally, fails in CI."

22.3 Catch2 and doctest

```
#include <catch2/catch_test_macros.hpp>

TEST_CASE("vector operations", "[vector]") {
    std::vector<int> v{1, 2, 3};

    REQUIRE(v.size() == 3);          // REQUIRE aborts; CHECK continues

    SECTION("push_back grows") {
        v.push_back(4);
        REQUIRE(v.size() == 4);
    }
    SECTION("clear empties") {      // ★ the whole TEST_CASE re-runs from the
top                                // for each SECTION – fresh state, no
    v.clear();                      // fixture class
    REQUIRE(v.empty());
}
```

```
    }  
}
```

SECTION is Catch2's best idea: each leaf section gets a completely fresh run of the enclosing code, so setup is just... code. Sections nest.

Expression decomposition means `REQUIRE(a == b)` prints both values without you writing `REQUIRE_EQ`. It's implemented with expression templates and it's why Catch2 compiles slowly.

```
auto n = GENERATE(1, 2, 3, 5, 8);           // parameterization, re-runs per value  
TEMPLATE_TEST_CASE("works for all", "[t]", int, long, double) { TestType x{}; }  
REQUIRE_THROWS_AS(f(), std::runtime_error);  
REQUIRE_THAT(str, Catch::Matchers::StartsWith("pre"));
```

doctest uses nearly the same API (`SUBCASE` instead of `SECTION`) and compiles several times faster. Its distinguishing feature is **in-source tests**:

```
// widget.cpp  
int add(int a, int b) { return a + b; }  
  
#ifndef DOCTEST_CONFIG_DISABLE  
TEST_CASE("add") { CHECK(add(2, 2) == 4); }           // right next to the  
implementation  
#endif
```

Compile the release build with `-DDOCTEST_CONFIG_DISABLE` and the tests vanish entirely.

22.4 Mocking: interfaces vs templates

The real question is: where's the seam?

To substitute a dependency, you need a **seam** — a place where the production implementation can be swapped. C++ gives you three, at three different costs:

Seam	Mechanism	Production cost	Mockable with
Runtime	pure-virtual interface + dependency injection (\$8.5)	vtable + no inlining (\$8.2)	gmock, Trompeloeil
Compile-time	template parameter / policy (\$8.9)	★ zero	hand-written mocks, Trompeloeil
Link-time	swap a <code>.cpp</code> or use weak symbols	zero	a separate test implementation

```
// Runtime seam
class IClock { public: virtual ~IClock() = default; virtual TimePoint now() const
= 0; };
class Service { IClock& clock_; public: explicit Service(IClock& c) : clock_(c) {}
};

// Compile-time seam – no virtual, fully inlined in production
template <class Clock = SystemClock>
class Service { [[no_unique_address]] Clock clock_; };

Service<FakeClock> svc;    // the test instantiation
```

The compile-time seam is the C++-specific answer and it's underused: you get testability with genuinely zero production overhead. The cost is that the class becomes a template (header-only, §20.9) and error messages get worse.

Link-time seams are standard in embedded work (§26), where you replace the HAL layer at link time.

GoogleMock

```
class MockClock : public IClock {
public:
    MOCK_METHOD(TimePoint, now, (), (const, override));
    MOCK_METHOD(void, sleep, (Duration d), (override));
};

TEST(ServiceTest, RetriesOnFailure) {
    MockClock clock;
    EXPECT_CALL(clock, now())
        .Times(3)
        .WillOnce(Return(t0))
        .WillRepeatedly(Return(t1));
    EXPECT_CALL(clock, sleep(Ge(Duration{100})))
        .Times(AtLeast(1));

    Service svc{clock};
    svc.doWork();
} // expectations are verified in the mock's DESTRUCTOR
```

```
// Cardinalities: Times(n), AtLeast(n), AtMost(n), Between(a,b), AnyNumber()
// Actions: Return, ReturnRef, Throw, Invoke, SetArgPointee, DoAll, SaveArg
// Ordering: InSequence seq; or .After(other) or Sequence objects
NiceMock<MockClock> nice;    // uninteresting calls are silent
StrictMock<MockClock> strict; // uninteresting calls FAIL
// default (NaggyMock): uninteresting calls warn
```


Prefer **NiceMock** by default and **StrictMock** only where the interaction protocol *is* the thing under test. The default naggy behaviour produces warning spam that people learn to ignore.

⚠ Over-mocking

A test that asserts *which methods were called in which order* is a **change detector**: it fails when you refactor, even though behaviour is unchanged. It tests your implementation, not your contract.

Prefer a fake — a real, simple, in-memory implementation — over a mock, wherever you can write one.

```
class InMemoryDatabase : public IDatabase {           // ★ a FAKE: real
behaviour, no I/O
    std::map<Key, Value> data_;
public:
    void put(Key k, Value v) override { data_[k] = v; }
    std::optional<Value> get(Key k) const override { ... }
};
```

A fake is written once, reused by every test, exercises real logic, and doesn't break when you refactor. Use mocks for interactions you genuinely need to assert on (was the retry attempted? was the error logged?).

Alternatives

	Notes
Trompeloeil	header-only, C++14+, works with templates and non-virtual types far better than gmock, cleaner syntax
FakeIt	⚠ zero-boilerplate but works by manipulating vtables — platform-specific and formally UB. Convenient, fragile
Hand-written fakes	often the right answer

22.5 Property-based testing

Instead of "for this input, expect this output", state a **property that must hold for all inputs**, and let the framework generate them and **shrink** failures to a minimal case.

```
#include <rapidcheck.h>

rc::check("sorting produces a sorted permutation", [] (std::vector<int> v) {
    auto sorted = v;
    std::sort(sorted.begin(), sorted.end());
    RC_ASSERT(std::is_sorted(sorted.begin(), sorted.end()));
    RC_ASSERT(std::is_permutation(sorted.begin(), sorted.end(), v.begin()));
});
```

```
nc::check("serialize round-trips", [] (const Config& c) {
    RC_ASSERT(deserialize(serialize(c)) == c);
});
```

Shrinking is the point. A failure on a 400-element vector gets automatically reduced to the 2-element case that actually breaks, which is usually enough to see the bug immediately.

The four property shapes worth knowing:

Property	Example
Round-trip	<code>decode(encode(x)) == x</code> — parsers, serializers, compression
Invariant	output is sorted; the tree is still balanced; size is conserved
Oracle	matches a slow, obviously-correct reference implementation
Metamorphic	<code>sort(shuffle(v)) == sort(v); f(x+y) == f(x)+f(y)</code>

RapidCheck integrates with GoogleTest (`RC_GTEST_PROP`) and Catch2. For simpler cases, Catch2's `GENERATE` covers a lot of ground without another dependency.

22.6 Coverage

```
# GCC — gcov/lcov
g++ --coverage -O0 -g test.cpp -o tests && ./tests
lcov --capture --directory . --output-file cov.info
lcov --remove cov.info '/usr/*' '*/third_party/*' --output-file cov.info
genhtml cov.info --output-directory coverage_html

# Clang — llvm-cov (better: region and branch coverage)
clang++ -fprofile-instr-generate -fcoverage-mapping -O0 -g test.cpp -o tests
LLVM_PROFILE_FILE="tests.profraw" ./tests
llvm-profdata merge -sparse tests.profraw -o tests.profdata
llvm-cov show ./tests -instr-profile=tests.profdata --format=html > cov.html
llvm-cov report ./tests -instr-profile=tests.profdata
```

`gcovr` produces Cobertura/HTML output and integrates with most CI systems.

⚠ Coverage is a floor, not a goal

Metric	Strength
Line coverage	weakest — <code>if (a && b)</code> counts as covered with one input
Branch coverage	better — both directions of every branch
Condition / MC/DC	required for safety-critical (DO-178C, ISO 26262 — §26)

100% line coverage with no assertions tests nothing. And in C++ specifically: **coverage measures which lines the sanitizers can see** (§14.5). Uncovered code is code ASan and UBSan will never examine. That's the real argument for it.

⚠ Template coverage is reported **per instantiation**, which makes numbers for heavily templated code confusing and often pessimistic.

22.7 Fuzzing ★

libFuzzer — in-process, coverage-guided

```
#include <cstdint>
#include <cstdint>

extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {
    parseConfig(std::string_view{reinterpret_cast<const char*>(data), size});
    return 0;
}
```

```
clang++ -g -O1 -fsanitize=fuzzer,address,undefined fuzz_parse.cpp parser.cpp -o
fuzz_parse
./fuzz_parse corpus/ -max_total_time=300 -jobs=8
./fuzz_parse crash-abc123 # ★ reproduce a specific crash
./fuzz_parse -merge=1 corpus/ new_inputs/ # minimize the corpus
```

Always combine with ASan and UBSan. The fuzzer generates the paths; the sanitizers detect that something went wrong on them. Fuzzing without sanitizers only finds crashes and hangs — a fraction of the real bugs.

Structured inputs

```
#include <fuzzer/FuzzedDataProvider.h>

extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {
    FuzzedDataProvider fdp{data, size};
    auto count = fdp.ConsumeIntegralInRange<int>(0, 100);
    auto flag = fdp.ConsumeBool();
    auto name = fdp.ConsumeRandomLengthString(50);
    auto rest = fdp.ConsumeRemainingBytesAsString();
    myApi(count, flag, name, rest);
    return 0;
}
```

For grammar-shaped inputs, **libprotobuf-mutator** fuzzes a protobuf message and converts it to your format, so the fuzzer spends its time on semantics rather than rediscovering syntax.

AFL++

Out-of-process, with **persistent mode** for speed. Better than libFuzzer when the target is a whole binary, has complex global state, or can't be linked in-process. Also has stronger instrumentation options (CmpLog, laf-intel) for getting past magic-value comparisons.

OSS-Fuzz

Google runs **continuous fuzzing for open-source projects for free**, with ClusterFuzz doing corpus management, deduplication, bisection, and automated bug filing. If you maintain a widely-used library that parses anything, getting onto OSS-Fuzz is one of the highest-leverage things you can do.

Practicalities

- **Fuzz targets must be deterministic** and free of global state that persists between runs
- Keep the target **fast** (microseconds) — throughput is everything
- **Commit a seed corpus** and a **.dict** of magic values/keywords
- **Commit every crash input as a regression test** once fixed
- In CI: run each target for a fixed budget (5–15 min) per PR, plus longer nightly runs

22.8 Benchmarking

Google Benchmark

```
#include <benchmark/benchmark.h>

static void BM_VectorPushBack(benchmark::State& state) {
    for (auto _ : state) {
        state.PauseTiming();
        std::vector<int> v;                                // setup outside the measurement
        state.ResumeTiming();

        for (int i = 0; i < state.range(0); ++i) v.push_back(i);
        benchmark::DoNotOptimize(v.data());               // ★ or it all disappears
        benchmark::ClobberMemory();
    }
    state.SetItemsProcessed(state.iterations() * state.range(0));
    state.SetComplexityN(state.range(0));
}
BENCHMARK(BM_VectorPushBack)->Range(8, 8<<12)->Complexity(benchmark::oN);
BENCHMARK_MAIN();
```

nanobench is a lighter alternative for microbenchmarks: single header, uses hardware performance counters, and reports an error percentage so you know whether a difference is real.

⚠ Methodology — where most benchmarks go wrong

1. Dead-code elimination. Without `DoNotOptimize/ClobberMemory` (or the asm barriers from §19.5), the compiler deletes the work and you measure an empty loop.

2. Machine noise. Control it:

```
sudo cpupower frequency-set -g performance      # disable frequency scaling
echo 0 | sudo tee /sys/devices/system/cpu/cpufreq/boost # disable turbo
taskset -c 3 ./bench                             # pin to a core
setarch -R ./bench                               # disable ASLR
# close everything else; run on a machine, not a shared CI runner
```

3. Report the median and spread, not the mean. One scheduler hiccup ruins a mean.

```
./bench --benchmark_repetitions=20 --benchmark_report_aggregates_only=true
compare.py benchmarks before.json after.json # ★ ships with Google Benchmark
```

4. Interleave A/B runs. Running all of A then all of B lets thermal drift masquerade as a result. Alternate them.

5. ⚠ Microbenchmarks lie. In a microbenchmark the cache is warm, the branch predictor is trained on your loop, there's no memory pressure, and the code is the only thing in the icache. A change that wins by 20% in isolation can lose in the real system. **Always validate at the system level.**

6. For CI regression detection, use instruction counts. `valgrind --tool=callgrind` (§21.4) gives a deterministic number, immune to the noise that makes wall-clock benchmarking useless on shared runners.

22.9 CTest and CI matrices

```
enable_testing()
include(GoogleTest)

add_executable(unit_tests test_widget.cpp test_engine.cpp)
target_link_libraries(unit_tests PRIVATE mylib GTest::gtest_main GTest::gmock)

gtest_discover_tests(unit_tests                                # ★ registers each TEST
individually
    PROPERTIES TIMEOUT 30 LABELS "unit"
)
add_test(NAME integration COMMAND integration_tests)
set_tests_properties(integration PROPERTIES LABELS "integration" TIMEOUT 300)
```

```
ctest -j$(nproc) --output-on-failure
ctest -L unit                # by label
ctest -R "Widget.*"          # by regex
```

```
ctest --rerun-failed
ctest --repeat until-fail:50    # ★ hunt flaky tests
```

`gtest_discover_tests` runs the binary at build time to enumerate tests, so CTest can parallelize and report them individually — much better than `add_test` on the whole binary.

The matrix

The full cross product (compiler × version × standard × build type × sanitizer × platform × arch × stdlib) is far too large. Pick a representative sample per PR and run the full matrix nightly.

A good per-PR set:

```
- { os: ubuntu, cc: gcc-14,   std: 20, build: Release }
- { os: ubuntu, cc: clang-18, std: 20, build: Debug, sanitizers:
  "address,undefined" }
- { os: ubuntu, cc: clang-18, std: 20, sanitizers: "thread" }
- { os: ubuntu, cc: gcc-14,   std: 20, flags: "-D_GLIBCXX_DEBUG" }
- { os: macos,   cc: clang,   std: 20, stdlib: libc++ }
- { os: windows, cc: msvc,    std: 20, flags: "/permissive- /W4 /WX" }
- clang-tidy (diff-only) + clang-format check
- fuzz targets, 10 min each
```

Nightly additions: the newest and oldest supported compiler versions, C++23, `-fllto`, ARM64, a coverage run, and longer fuzzing.

Why the breadth pays (§19.1): different front ends and different standard libraries catch genuinely different bugs, and `libc++` vs `libstdc++` will surface unspecified behaviour you accidentally depended on.

Gotcha checklist for this section

- ☐ `ASSERT_*` only works in `void` functions — use `ASSERT_NO_FATAL_FAILURE` around helpers
- ☐ Prefer `EXPECT_*` so one run reports everything
- ☐ `--gtest_shuffle --gtest_repeat` exposes inter-test coupling
- ☐ Death tests fork — slow, thread-hostile, not universally supported
- ☐ `NiceMock` by default; `StrictMock` only when the protocol is under test
- ☐ **Over-mocking creates change detectors** — prefer fakes
- ☐ `Fakelt` manipulates vtables and is formally UB
- ☐ 100% line coverage with no assertions tests nothing; branch coverage is the meaningful metric
- ☐ Template coverage is reported per instantiation
- ☐ **Always fuzz under ASan+UBSan** — the fuzzer finds paths, the sanitizers find bugs
- ☐ Fuzz targets must be deterministic and free of persistent global state
- ☐ Commit every fixed crash input as a regression test
- ☐ Benchmarks need `DoNotOptimize/ClobberMemory` or you measure nothing
- ☐ Pin frequency, pin the core, disable ASLR; report medians; interleave A/B
- ☐ **Microbenchmarks lie** — validate at the system level

- ☐ For CI regression detection, callgrind instruction counts beat wall-clock

Exercises worth doing

1. Write a typed test that runs the same contract across `vector`, `deque`, and `list`.
2. Take a class with a hard dependency (clock, filesystem, network) and introduce all three seam types — virtual interface, template parameter, link-time — and write a test for each.
3. Replace a heavily-mocked test with a fake. Then refactor the implementation and see which version breaks.
4. Write a RapidCheck round-trip property for a serializer. Introduce a bug and watch the shrinker minimize the failing case.
5. Write a libFuzzer target for any parser you have. Build with `-fsanitize=fuzzer,address,undefined` and run for 10 minutes. (Something will turn up.)
6. Benchmark a function with and without `DoNotOptimize` and compare the numbers.
7. Benchmark the same change twice: all-A-then-all-B, versus interleaved. Compare the conclusions.
8. Set up a CI matrix with GCC, Clang+ASan, Clang+TSan, and MSVC. Note which job catches what.
9. Run your test suite with `ctest --repeat until-fail:100` and find your flaky test.

Chapter 23 — Secure & Safe Coding

What this chapter covers

- Memory safety: buffer overflows, UAF, double-free, dangling `string_view`/iterators/references
- Integer overflow, signed/unsigned confusion, truncation
- **Where C++ is safer than C** (RAII, containers, `span`, `string`) and where it isn't
- **C++ Core Guidelines + GSL** (`gsl::span`, `not_null`, `narrow`)
- MISRA C++ / AUTOSAR C++14 / High Integrity C++
- CERT C++, CWE Top 25
- Hardening: `-D_GLIBCXX_ASSERTIONS`, `-D_FORTIFY_SOURCE=3`, `-fstack-protector-strong`, PIE, RELRO, CFI, hardened STL modes
- Constant-time code, secure erase (`memset_s` / `SecureZeroMemory`)
- The **memory-safety debate** — Safe C++ proposals, profiles, and the Rust pressure

The context for this section changed in the last few years. C++ went from "a language with some sharp edges" to **a language named in government advisories**, with US agencies recommending memory-safety roadmaps and the widely-cited figure that *memory safety issues account for roughly 70% of vulnerabilities at both Microsoft and Google*.

Two practical things to take from here. First: **there is a set of hardening flags you can turn on today** that costs almost nothing and eliminates whole bug classes — most projects haven't. Second: the memory-safety debate is genuinely contested, the committee has made a decision, and it's worth understanding both positions rather than either slogan.

23.1 The bug classes

Class	Mechanism	Mitigation
Buffer overflow	<code>v[i]</code> past the end, <code>memcpy</code> with a wrong size, off-by-one	<code>span</code> , <code>at()</code> , hardened STL, ASan, <code>_FORTIFY_SOURCE</code>
Use-after-free	dangling <code>unique_ptr::get()</code> , freed then used, callbacks outliving objects	RAII, <code>weak_ptr</code> , ASan
Double free	two owners of one raw pointer	<code>unique_ptr</code> , Rule of Zero (§4.9)
Dangling references/views	<code>string_view</code> , <code>span</code> , iterators, <code>const&</code> (§3.4, §11.9, §16.7)	discipline, <code>-Wdangling-*</code> , ASan use-after-scope
Iterator invalidation	mutate a container while holding an iterator (§11.5)	<code>_GLIBCXX_DEBUG</code> , <code>erase_if</code>
Uninitialized read	forgotten member, uninitialized local (§14.2)	NSDMI, <code>-ftrivial-auto-var-init</code> , MSan
Integer overflow / truncation	§23.2	UBSan, <code>gsl::narrow</code> , <code>std::cmp_*</code>
Type confusion	<code>reinterpret_cast</code> , unions, bad downcasts	<code>bit_cast</code> , <code>variant</code> , avoid casts
Data race	§15.1	TSan, mutexes
Format string	<code>printf(userInput)</code>	<code>std::format/std::print</code> (§18.1)
Injection / path traversal	untrusted input reaching a shell, SQL, or a path	validate at the boundary; parameterize

These map directly onto the CWE Top 25 — CWE-787 (out-of-bounds write), CWE-125 (out-of-bounds read), CWE-416 (use-after-free), CWE-476 (null deref), CWE-190 (integer overflow), CWE-119, CWE-20 are all perennial top-ten entries and all live in this table.

23.2 Integer issues

```
int x = INT_MAX; x++; // ⚠ UB (§14.2) – but UBSan catches it
unsigned u = 0; u--; // ✓ defined, wraps to UINT_MAX – and SILENT

for (size_t i = v.size() - 1; i >= 0; --i) // ⚠ infinite loop on an empty vector
for (int i = std::ssize(v) - 1; i >= 0; --i) // ✓ C++20

if (someInt < v.size()) // ⚠ -Wsign-compare: int converted to size_t
if (std::cmp_less(someInt, v.size())) // ✓ C++20 – correct across signedness

size_t total = count * itemSize; // ⚠ overflow -> small allocation -> heap overflow
```



```
                                // the classic allocation-size
vulnerability
```

The tools

```
#include <utility>
std::cmp_less, cmp_greater, cmp_equal, cmp_less_equal, ... // C++20, sign-
safe comparison
std::in_range<int>(someSizeT); // C++20, does it
fit?

gsl::narrow<int>(bigValue); // throws if the value changes – a CHECKED
narrowing cast
gsl::narrow_cast<int>(bigValue); // documented, unchecked – "I know this narrows"

std::add_sat(a, b); std::mul_sat(a, b); // C++26 saturating arithmetic
__builtin_add_overflow(a, b, &r); // GCC/Clang: overflow-checked,
returns bool
```

⚠ **The signed/unsigned debate matters here** (§14.6): signed overflow is UB, which means **UBSan catches it**. Unsigned wraparound is *defined*, therefore silent, therefore undetectable by any sanitizer. That's the Core Guidelines' and Google's argument for signed indices despite `size()` returning unsigned.

```
-Wconversion -Wsign-conversion -Wsign-compare -Wshorten-64-to-32
-fsanitize=integer # Clang: includes unsigned-overflow
reporting
-ftrapv # trap on signed overflow (slower than -
fwrapv)
```

23.3 Where C++ is safer than C — and where it isn't

Genuinely safer

C hazard	C++ answer
<code>malloc/free</code> mismatches, leaks on early return	RAII (§4) — the single biggest win
Manual array + length, <code>realloc</code>	<code>std::vector</code> — bounds-checkable, self-managing
<code>strcpy/strcat</code> overflows, missing NUL	<code>std::string</code>
<code>(ptr, len)</code> pairs drifting apart	<code>std::span</code> , <code>std::string_view</code> (§3.8)
<code>void*</code> and unchecked casts	templates, <code>variant</code> , named casts
Macros	<code>constexpr</code> , templates, <code>inline</code>
<code>printf</code> format mismatches	<code>std::format</code> / <code>std::print</code> — compile-time checked

C hazard	C++ answer
Null-checking every parameter	references, <code>gsl::not_null</code>
Manual cleanup on every error path	destructors + exceptions (§13.3)

Not safer — and in some ways worse

- **The entire UB catalogue is inherited** (§14). `operator[]` is unchecked by convention, exactly like C arrays.
- **C++ adds new failure modes C doesn't have**: iterator invalidation (§11.5), use-after-move misuse (§5.6), slicing (§8.6), dangling `string_view`/`span`/lambda captures (§3, §6.9, §16.7), static initialization order (§7.6), and ODR violations (§6.4).
- **More non-local code runs implicitly** — destructors, implicit conversions, operator overloads — so "what does this line do" is harder to audit.
- **Exceptions add control-flow paths** through every call.
- **Templates multiply the code** you'd have to review (§10.10).

Honest verdict: C++ eliminates most of the *common* C bugs and introduces a smaller number of *subtler* ones. It is a large net improvement and it is not memory safety. A C++ program with no `new`, no raw owning pointers, no C arrays, and hardened containers is dramatically safer than the equivalent C — and still not sound.

23.4 C++ Core Guidelines and the GSL

The [Core Guidelines](#) (Stroustrup and Sutter) are the closest thing to an official style-and-safety standard. The rules that carry the most weight:

Rule	Statement
P.1–P.13	Express intent directly; prefer compile-time checking; use libraries
I.11	Never transfer ownership by a raw pointer or reference
I.13	Do not pass an array as a single pointer — use <code>span</code>
F.7	For general use, take <code>T*</code> or <code>T&</code> , not a smart pointer
R.1	★ Manage resources automatically using RAII
R.3–R.5	A raw pointer is non-owning; prefer scoped objects
C.35	A base class destructor should be public+virtual, or protected+non-virtual (§8.3)
C.20	Prefer the Rule of Zero (§4.9)
ES.20	★ Always initialize an object
ES.47	Use <code>nullptr</code> , not <code>0</code> or <code>NULL</code>
ES.107	Don't use <code>unsigned</code> for subscripts — prefer <code>gsl::index</code>
CP.20	Use RAII locks, never bare <code>lock/unlock</code> (§15.4)

GSL — the Guidelines Support Library

```
#include <gsl/gsl>

gsl::span<int> s = myVector;           // → std::span (C++20)
gsl::not_null<Widget*> p = &widget;    // never null, enforced
gsl::narrow<int>(x);                   // checked narrowing (throws)
gsl::narrow_cast<int>(x);              // documented unchecked narrowing
gsl::owner<Widget*> raw;               // marks ownership at a C boundary (§3.6)
auto guard = gsl::finally([&]{ cleanup(); }); // scope guard (§4.10)
Expects(index < size);                // precondition } superseded by
Ensures(result != nullptr);           // postcondition } contracts (§13.9)
```

Implementations: Microsoft **GSL** (the reference), **gsl-lite** (lighter, wider compiler support).

Much of GSL has been absorbed into the standard — `span`, and soon `contracts`. `not_null`, `narrow`, and `owner` remain useful.

Enforcement: `clang-tidy` with `cppcoreguidelines-*` (§21.7) checks a substantial subset mechanically. That's the practical path — the Guidelines are 500 pages, the checks are a config file.

23.5 MISRA, AUTOSAR, HIC++, CERT

Coding standards for regulated and safety-critical domains (§26).

Standard	Domain	Notes
MISRA C++:2023	automotive, industrial, safety-critical	★ the current one — supersedes MISRA C++:2008 and absorbs AUTOSAR C++14 . Targets C++17
AUTOSAR C++14	automotive	merged into MISRA C++:2023; you'll still meet it in existing projects
HIC++ 4.0	high-integrity	more permissive than MISRA
CERT C++	security	secure-coding rules, maps to CWE
JSF++	avionics (F-35)	historical, very restrictive

What they typically restrict

Exceptions, RTTI and `dynamic_cast`, dynamic allocation after initialization, recursion, `goto`, multiple inheritance, unions, macros, implicit conversions, `reinterpret_cast`, variadic functions, and much of the standard library.

The rationale is usually **analyzability and bounded resources**, not that the features are bad: you can't certify worst-case execution time through an unbounded unwinder, and you can't prove memory bounds with a heap.

The certification context

Standard	Domain
ISO 26262 (ASIL A–D)	automotive functional safety
DO-178C (DAL A–E)	avionics
IEC 62304	medical devices
IEC 61508 (SIL 1–4)	industrial
EN 50128	railway

These require MC/DC coverage (§22.6), traceability from requirements to tests, and **qualified tools** — you can't just use any compiler; it must have a qualification package.

Tooling: PC-lint Plus, Helix QAC, PVS-Studio, Parasoft C/C++test, Coverity, LDRA. Compliance is checked by tools, documented with deviation records for every intentional violation.

23.6 Hardening — what to turn on in production

★ **This is the highest value-per-effort part of the section.** Most of these are near-free and most projects don't enable them.

Flag	Protects against	Cost
-D_GLIBCXX_ASSERTIONS	★ <code>vector::operator[]</code> , front, back OOB; libstdc++ preconditions	~1%
- D_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_FAST	the libc++ equivalent	~1%
-D_FORTIFY_SOURCE=3	libc buffer overflows (<code>memcpy</code> , <code>sprintf</code> , ...); needs -O1+	~0–1%
-fstack-protector-strong	stack smashing	~1–3%
-fstack-clash-protection	stack-clash attacks	~0%
-fPIE -pie	ASLR for the executable	~0–2%
-Wl,-z,relro,-z,now	GOT overwrite (full RELRO)	slower startup
-Wl,-z,noexecstack	executable stack	0
-ftrivial-auto-var-init=zero	★ uninitialized reads (§14)	~0–2%
-fcf-protection=full	ROP/JOP via Intel CET	~0–1%
-mbranch-protection=standard	ARM PAC/BTI	~0–1%

Flag	Protects against	Cost
<code>-fsanitize=cfi</code> (needs LTO + visibility)	control-flow integrity, bad casts, indirect-call hijacking	~1–5%
<code>-fsanitize=safe-stack</code> / <code>-fsanitize=shadow-call-stack</code>	return-address overwrite	low
MSVC <code>/GS</code> <code>/guard:cf</code> <code>/guard:ehcont</code> <code>/DYNAMICBASE</code> GCC <code>/NXCOMPAT</code> <code>/sdl</code>	the Windows equivalents	low

A production build

```
g++ -O2 -g -DNDEBUG \
    -D_GLIBCXX_ASSERTIONS -D_FORTIFY_SOURCE=3 \
    -fstack-protector-strong -fstack-clash-protection \
    -ftrivial-auto-var-init=zero \
    -fcf-protection=full \
    -fPIE -pie -Wl,-z,relro,-z,now,-z,noexecstack
```

⚠ `-D_GLIBCXX_ASSERTIONS` is not `-D_GLIBCXX_DEBUG`. The former is cheap and **ABI-compatible**; the latter changes container layouts and requires every TU and library to match (§11).

Many Linux distributions already build their packages with most of this, and hardened standard-library modes are the direction of travel — *the hardened stdlib (P3471) is the first user of contracts, formalising bounds checks as contract violations* (§13.9).

Verify what you shipped:

```
checksec --file=./myapp          # RELRO, canary, NX, PIE, fortify status
hardening-check ./myapp
```

23.7 Constant-time code and secure erase

Timing side channels

```
// ⚠ Early exit leaks the position of the first differing byte
bool compare(const uint8_t* a, const uint8_t* b, size_t n) {
    for (size_t i = 0; i < n; ++i) if (a[i] != b[i]) return false;
    return true;
}

// Constant-time: no early exit, no secret-dependent branch
bool ctCompare(const uint8_t* a, const uint8_t* b, size_t n) {
    uint8_t diff = 0;
    for (size_t i = 0; i < n; ++i) diff |= a[i] ^ b[i];
}
```

```
    return diff == 0;
}
```

The three rules: **no branch on a secret**, **no memory access indexed by a secret** (cache timing), **no variable-latency instruction on a secret** (division).

⚠ **The compiler can undo all of it.** It may reintroduce a branch, vectorize with an early exit, or optimize the accumulation. There is no portable way to forbid this. In practice: use a vetted library (`sodium_memcmp`, `CRYPTO_memcmp`), or write it in assembly, and **verify the generated code** (§19.5) — and re-verify after every compiler upgrade. Tools like `dudect` and `ct-verify` exist for validation.

Secure erase

```
void wipe(char* p, size_t n) { std::memset(p, 0, n); } // ⚠ DEAD STORE –
deleted by the optimizer
```

The memory is never read afterwards, so the write is provably dead and the compiler removes it. The portable-ish options:

```
::explicit_bzero(p, n);           // glibc, BSD
SecureZeroMemory(p, n);           // Windows
memset_s(p, n, 0, n);             // C11 Annex K – ⚠ not widely available
sodium_memzero(p, n);             // libsodium
OPENSSL_cleanse(p, n);

// Manual fallback
volatile char* vp = p;
while (n--) *vp++ = 0;
std::atomic_signal_fence(std::memory_order_seq_cst);
```

⚠ **And value semantics fights you** (§3.1): a secret in a `std::string` may already exist in a reallocated old buffer, a moved-from source, a copy made by a container, or a page swapped to disk. Serious handling needs a custom allocator with `mlock/VirtualLock`, no reallocation, and wiping in the deallocate path.

`std::secure_clear` has been proposed but is not standard.

23.8 ⚠ The memory-safety debate

This is a live, contested disagreement with strong views on both sides. Here is the state of it and the case each side makes.

The pressure

US agencies (CISA, NSA, FBI) have published guidance naming C and C++ as memory-unsafe and *stating that for existing products written in memory-unsafe languages, not having a published memory-safety roadmap by 1*

January 2026 elevates risk to national security. It is non-binding guidance, but it shapes procurement and industry roadmaps.

The two proposals

Profiles (P3081, Sutter). *Define standard enforced profiles — notably bounds, type, and lifetime — that a conforming implementation must enforce when enabled. Each rule must be decidable at compile time (even if it injects a runtime check), efficient enough to implement in the compiler itself, and enforced by the C++ implementation rather than a separate static-analysis tool.* The emphasis is on *silently fixing serious bugs in existing code by recompiling, or by mechanical source updates, without manual code changes.*

Safe C++ (P3390, Sean Baxter, prototyped in the Circle compiler). A rigorously safe *subset* with borrow checking and lifetime annotations — the Rust model, brought into C++, with provable soundness for code written in the safe subset.

What happened

The Safety and Security study group voted to prioritize Profiles over Safe C++, and Safe C++ is not being continued — in its author's words. He identifies the root cause as an EWG-adopted language-evolution principle stating that C++ should avoid requiring a safe-or-pure function annotation with the semantics that such a function may only call other such functions, and calls this an irreconcilable design disagreement, since safe function colouring is the core of the Rust safety model.

The case for Profiles

- Guaranteed safety requires rewriting code; there are **billions of lines** of existing C++ that will not be rewritten. An incremental improvement applied to all of it may prevent more vulnerabilities than perfect safety applied only to new code.
- *The proponents' stated goal is an overall reduction in vulnerabilities, not perfection — and any mitigation is worth considering.*
- No function colouring means no bifurcated ecosystem and no viral annotation burden.
- Enforcement lives in the compiler, so it can't be skipped the way an optional linter can.

The case against

- *Baxter's analysis "Why Safety Profiles Failed" argues that C++ lacks the aliasing and lifetime annotations necessary for static enforcement without a borrow checker.*
- *A related critique: we cannot achieve guaranteed memory safety without annotating most references — and even that would be insufficient; and lifetime annotations would have to be part of the ABI to offer safety over time.*
- *And the confidence argument: there is a giant gap between 99% and 100% memory safety, and it is reasonable to treat code that may have false negatives as memory-unsafe, so profiles could help with safety without helping with confidence.*

Both positions are held by serious people with deep knowledge of the language. The disagreement is real and not resolved by either side's talking points.

Where it landed for C++26 and C++29

C++26 shipped **contracts** and a **hardened standard library** (P3471). *The profile attribute itself was deferred, with SG23 expected to refine the type-safety profile against P3589's framework, and the May 2026 pre-Brno mailing carrying three load-bearing papers that sharpen the C++29 direction — signal that the C++29 timeline is being defended rather than slipping. The committee has adopted a three-year C++29 cycle in which memory safety is a major focus.*

What to do today, regardless of your position

The recommendations are unchanged by the standardization argument: turn on the hardened-stdlib macro, run clang-tidy with the safety check sets, and use the available mechanisms — real, workable ones exist on shipping toolchains for the bug classes the deferred profile attribute would have caught.

Concretely: §23.6's flags, §14.5's sanitizers in CI, §22.7's fuzzing, `cppcoreguidelines-*` in clang-tidy, `span/string_view/not_null` at every boundary, and no raw `new/delete`.

And the pragmatic industry answer that is happening in parallel: **mixed-language codebases**, with new security-sensitive components written in Rust behind a C ABI (§25) while the existing C++ is hardened rather than rewritten.

Gotcha checklist for this section

- ☐ Unsigned wraparound is **defined and therefore silent** — no sanitizer catches it
- ☐ `someInt < v.size()` converts the int — use `std::cmp_less`
- ☐ `count * itemSize` overflowing is the classic allocation-size vulnerability
- ☐ `_GLIBCXX_ASSERTIONS` (cheap, ABI-safe) $\neq$ `_GLIBCXX_DEBUG` (heavy, ABI-breaking)
- ☐ `_FORTIFY_SOURCE` requires `-O1` or higher to do anything
- ☐ `memset` for secure erase is a **dead store** and gets deleted
- ☐ Value semantics leaves secrets in moved-from buffers, container copies, and reallocated storage
- ☐ The compiler can reintroduce branches into constant-time code — verify the assembly, every upgrade
- ☐ C++ removes most common C bugs and **adds** iterator invalidation, use-after-move, slicing, and dangling views
- ☐ MISRA C++:2023 supersedes MISRA C++:2008 **and** AUTOSAR C++14
- ☐ Certification requires *qualified* tools, not just any compiler

Exercises worth doing

1. Run `checksec` on a binary you built with default flags, then with the §23.6 block. Compare.
2. Benchmark your test suite with and without `-D_GLIBCXX_ASSERTIONS`. Then write an out-of-bounds `operator[]` and confirm it's caught.
3. Write `std::memset` secure-erase and confirm at `-O2` that the store is gone from the assembly. Then fix it three ways and check each.
4. Write the early-exit comparison and the constant-time one; look at both at `-O2` and `-O3`. Check whether the compiler reintroduced a branch.
5. Write a function that computes an allocation size by multiplication, feed it values that overflow, and watch the heap overflow under ASan.
6. Turn on `cppcoreguidelines-*` in clang-tidy on a real file. Categorize the findings into real bugs, style, and noise.

7. Read P3081 (Profiles) and Baxter's "Why Safety Profiles Failed", then write down which specific technical claim you find you can't yet evaluate. That's the thing to go learn.

Chapter 24 — Performance Engineering

What this chapter covers

- **Profile first** — `perf`, VTune, flame graphs
- **Cache**: lines, locality, prefetching, **false sharing**
- **Data-oriented design**, AoS vs SoA, ECS
- Branch prediction, branchless code
- **Allocation is the usual culprit** — arenas, pools, PMR, `reserve()`, small-buffer optimization
- Move semantics and copy elision as performance tools
- `std::function` and virtual-call overhead; devirtualization
- SIMD: intrinsics, `std::simd` (C++26), auto-vectorization, `-march`
- **Templates vs runtime polymorphism** — the monomorphization trade-off
- Compile-time computation (`constexpr`) as a runtime optimization
- Benchmark methodology: warmup, variance, `DoNotOptimize`, avoiding elision
- PGO, BOLT

C++ is chosen when performance matters, so this is the section where the language's whole value proposition gets cashed in. The recurring finding, across almost every codebase: **the bottleneck is not the algorithm you were worried about**. It's memory access patterns, and it's allocation. People spend a day micro-optimizing a hot function and get 3%; they change an `AoS` to `SoA` or add a `reserve()` and get 3×.

Mechanics for `perf` are in §21.6 and benchmark hygiene is in §22.8. This section is about **interpretation and technique**.

24.1 Profile first

The discipline

1. **Measure** at the system level. Know your actual latency/throughput target.
2. **Find** the bottleneck with a profiler — not by reading code.
3. **Understand why** it's slow: is it instructions, memory, branches, or waiting?
4. **Fix**, then **re-measure** on a real workload.
5. Stop when you hit the target.

Amdahl's law is the reason step 2 comes before step 4: making a function that's 5% of runtime twice as fast buys you 2.5%. It doesn't matter how clever the fix was.

⚠ **Your intuition about where time goes is unreliable.** Everyone's is. This isn't a failure of skill — modern CPUs are too complex to model in your head, and the costs that dominate (cache misses, allocator behaviour, branch misprediction) are invisible in source code.

Reading `perf stat` — the first question

```
perf stat -d ./app
```

Metric	What it tells you
IPC (instructions/cycle)	> 2: compute-bound and doing fine. < 1: stalled — go look at memory or branches
cache-misses / cache-references	high ratio → memory-bound (§24.2)
branch-misses	> 1–2% of branches → misprediction is costing you (§24.4)
stalled-cycles-frontend	instruction fetch — icache pressure, often code bloat (§24.9)
stalled-cycles-backend	data, usually memory latency
page-faults	allocation churn or a cold working set

Intel's **Top-Down Methodology** (via VTune or [toplev](#)) formalizes this into four buckets — Retiring, Bad Speculation, Frontend Bound, Backend Bound — and it's the fastest way to know which of the sections below applies.

The numbers that shape every decision

Operation	Approx. latency	In "cycles" terms
L1 cache hit	~1 ns	~4 cycles
L2 hit	~4 ns	~14 cycles
L3 hit	~15–20 ns	~50 cycles
DRAM	~60–100 ns	~200–300 cycles
Branch misprediction	~5 ns	~15–20 cycles
malloc/free (uncontended)	~20–100 ns	
Mutex lock (uncontended)	~20 ns	one atomic RMW
Thread context switch	~1–5 μs	
NVMe read	~50–100 μs	
Same-datacenter RTT	~0.1–0.5 ms	

A DRAM access costs as much as ~200 arithmetic instructions. That single fact explains most of this section: the machine is starved for data, not for compute. Optimizing instruction count while ignoring memory layout is optimizing the wrong resource.

Priority order

1. **Algorithms and data structures** — $O(n^2) \rightarrow O(n \log n)$ beats everything

2. **Memory layout** — locality, AoS→SoA (§24.2, §24.3)
 3. **Allocation** — remove it (§24.5)
 4. **Parallelism** — if the work is parallelizable (§15)
 5. **Micro-optimization** — SIMD, branchless, intrinsics (§24.4, §24.8)
-

24.2 Cache

The hardware

- **Cache line = 64 bytes** on x86-64 and most ARM (**128 bytes** on Apple Silicon's L2). All transfers are line-granular: touching one byte loads 64.
- L1D ~32–48 KB per core, L2 ~0.5–2 MB per core, L3 ~8–64 MB shared.
- **Hardware prefetchers** detect sequential and simple strided patterns and fetch ahead. They cannot follow pointers.

The consequences

```
// Sequential – prefetcher-friendly, ~1 miss per 16 ints
for (int x : vec) sum += x;

// Strided – every access a new line; the prefetcher may cope, may not
for (size_t i = 0; i < n; i += 16) sum += vec[i];

// Pointer chasing – a full DRAM latency per node, unpredictable, no prefetch
for (Node* p = head; p; p = p->next) sum += p->value;
```

This is the whole **vector** vs **list** argument (§11.1) in hardware terms.

The demo everyone should run once

```
// Row-major: sequential
for (int i = 0; i < N; ++i) for (int j = 0; j < N; ++j) sum += m[i][j];
// Column-major: a cache miss on nearly every access
for (int j = 0; j < N; ++j) for (int i = 0; i < N; ++i) sum += m[i][j];
```

Identical instruction counts. Typically **5–10× different** in wall time for a large **N**. Nothing about the source suggests it.

Structure layout

```
struct Bad { char a; double b; char c; int d; }; // 24 bytes – padding everywhere
struct Good { double b; int d; char a; char c; }; // 16 bytes – largest first
```

Ordering members largest-to-smallest minimizes padding. `-Wpadded` reports it. And beyond padding, **hot/cold splitting**:

```
struct Entity {
    Vec3 pos, vel;                // touched every frame
    std::unique_ptr<ColdData> rarelyUsed; // debug name, metadata, creation time
};
```

If a hot loop touches 12 bytes of a 200-byte struct, you're loading 200 bytes of DRAM bandwidth to use 12.

Measuring

```
perf stat -e cache-references,cache-misses,LLC-load-misses ./app
perf c2c          # ★ false sharing ($15.10)
valgrind --tool=cachegrind ./app
```

Prefetch hints

```
__builtin_prefetch(&data[i + 16], 0, 3);
```

⚠ Rarely helps — the hardware prefetcher is good and manual hints can evict useful lines. The one case where it does pay is a pointer-chasing loop where you can compute the next address early. Measure both ways.

24.3 Data-oriented design: AoS vs SoA

The transformation

```
// Array of Structs – the object-oriented default
struct Particle { Vec3 pos, vel; Color color; float life; bool active; }; // 44 bytes
std::vector<Particle> particles;

for (auto& p : particles) p.pos += p.vel * dt; // ⚠ loads 44 bytes to use 24
```

```
// Struct of Arrays
struct Particles {
    std::vector<Vec3> pos, vel;
    std::vector<Color> color;
    std::vector<float> life;
};
```

```
for (size_t i = 0; i < n; ++i) pos[i] += vel[i] * dt;    // ✓ only pos and vel
are touched
```

Two wins, and the second is often bigger than the first:

1. **Bandwidth**: you load only the fields you use — here roughly half the bytes.
2. **Vectorization**: contiguous same-type data is what the auto-vectorizer needs (§24.8). AoS with mixed field types usually can't be vectorized at all.

When AoS is right

- You touch most fields together
- Access is random per-entity rather than a sweep
- Entities are inserted/removed constantly (SoA makes that awkward)
- The struct already fits in a cache line

Hot/cold splitting is the pragmatic middle: keep the hot fields in an AoS that fits a line, and move the cold ones out.

ECS

The generalization used in games and simulation: **entities are IDs, components live in SoA arrays, systems iterate over component arrays**. Two implementation families — *archetype* (entities with identical component sets stored together, fast iteration) and *sparse set* (fast add/remove).

The handle/generation pattern from §4.8 is what makes references into these arrays safe.

C++26 reflection (§18.2) makes generating an SoA view from an AoS struct declaration mechanical, which may make this a lot less manual.

24.4 Branch prediction and branchless code

A mispredicted branch costs ~15–20 cycles — the pipeline is flushed. A *correctly predicted* branch is essentially free. Modern predictors are very good, so the cost only shows up on **genuinely unpredictable** data.

The famous demonstration

```
for (int i = 0; i < n; ++i)
    if (data[i] >= 128) sum += data[i];
```

Sort `data` first and this loop runs several times faster — same instructions, same memory pattern. Sorted data makes the branch perfectly predictable.

Branchless techniques

```
int m = (a > b) ? a : b;           // usually compiles to CMOV – no branch
sum += (x >= 128) * x;           // arithmetic instead of control flow
```

```
int mask = -(x >= 128); sum += x & mask;
std::clamp(v, lo, hi); std::min/std::max;
```

⚠ **Branchless is often slower when the branch is predictable**, because you unconditionally execute both sides and CMOV creates a data dependency the out-of-order engine can't speculate past. It only wins on unpredictable data. This is squarely a measure-it situation.

`[[likely]]/[[unlikely]]` (§6.7) affect code layout, not prediction. **PGO does this better than you can** (§24.12) — it has actual profile data.

```
perf stat -e branches,branch-misses ./app
```

24.5 Allocation — the usual culprit

Every `malloc` is 20–100 ns, may take a lock, returns memory that is cold in cache, and fragments the heap over time. In most real C++ programs, allocation shows up in the top few entries of a profile.

The audit

```
// Drop this in temporarily – it's the fastest allocation diagnostic there is
static std::atomic<size_t> gAllocCount{0}, gAllocBytes{0};
void* operator new(std::size_t n) {
    gAllocCount++; gAllocBytes += n;
    if (void* p = std::malloc(n)) return p;
    throw std::bad_alloc{};
}
void operator delete(void* p) noexcept { std::free(p); }
```

Print the count around a hot section. The number is usually much larger than expected.

The fixes, roughly in order of yield

```
v.reserve(n); // ★ §11.1 – often the single biggest
win
```

Technique	Where
<code>reserve()</code> before a fill loop	§11.1
Arena / <code>monotonic_buffer_resource</code> — allocate freely, free all at once	§11.12
Object pools for fixed-size churn	

Technique	Where
Small-buffer optimization — <code>std::string</code> 's SSO, a <code>small_vector</code> , <code>function_ref</code> instead of <code>std::function</code>	\$11.9, \$6.10
<code>string_view</code> / <code>span</code> instead of copies	\$3.8, \$11.9
<code>flat_map</code> instead of <code>map</code> — no node per element	\$11.4
<code>emplace</code> instead of construct-then-copy	\$11.6
Reuse buffers across iterations instead of reallocating	
Replace the allocator entirely	below

Swap the allocator — often free performance

```
LD_PRELOAD=/usr/lib/libtcmalloc.so ./app
LD_PRELOAD=/usr/lib/libjemalloc.so ./app
```

tcmalloc, **jemalloc**, **mimalloc**, **snmalloc** typically beat glibc's `malloc` by 5–20% on allocation-heavy multithreaded workloads, with per-thread caches and better fragmentation behaviour. It costs one link flag. Try it before you refactor anything.

Measuring

```
heaptrack ./app && heaptrack_gui heaptrack.app.*.gz # ★ friendliest
valgrind --tool=massif ./app
valgrind --tool=dhat ./app # what's allocated and
never read
perf record -g -e "syscalls:sys_enter_brk,syscalls:sys_enter_mmap"
```

24.6 Moves, elision, and the cheap wins

Full treatment in §5. As performance tools:

```
std::vector<T> make(); auto v = make(); // guaranteed elision – free (§5.7)
void sink(std::string s) { member_ = std::move(s); } // one move for lvalues and
rvalues
v.push_back(std::move(x)); // don't copy what you're done with
```

⚠ And the two silent killers already covered, because they belong on any performance checklist:

- **A non-`noexcept` move makes `vector` reallocation copy** (§5.5)

- **Declaring a destructor suppresses your move operations** (§4.9) — every "move" becomes a deep copy

Both are invisible. `static_assert(std::is_nothrow_move_constructible_v<T>)` is how you find them.

24.7 `std::function` and virtual-call overhead

The cost of both is **lost inlining**, not the indirect jump (§8.2, §6.10). An opaque call boundary kills constant propagation, vectorization across it, and dead-code elimination.

Fix	Notes
<code>final</code> on classes/methods	lets the compiler prove the target (§8.4)
LTO	cross-TU devirtualization (§19.7) — often the biggest single win here
PGO	speculative devirtualization: <code>if (vptr == known) direct();</code> <code>else indirect();</code>
<code>std::variant</code> + <code>visit</code>	closed type set, no allocation, inlinable (§8.9)
Template parameter instead of <code>std::function</code>	zero overhead (§6.10)
<code>function_ref</code> for non-storing callbacks	no allocation

⚠ And the one from §4.7: **`shared_ptr` by value in a hot function** is two atomic RMWs per call. Pass `const T&`.

24.8 SIMD

Auto-vectorization first

The compiler will vectorize a loop when it can prove it's safe. What it needs:

- A **countable** trip count known before the loop
- **Contiguous** access (stride 1)
- **No aliasing** between input and output (§3.9)
- No early exit, no function calls it can't inline
- No loop-carried dependencies

```
clang++ -O2 -Rpass=loop-vectorize -Rpass-missed=loop-vectorize -Rpass-analysis=loop-vectorize
g++ -O3 -fopt-info-vec -fopt-info-vec-missed
```

Read the "not vectorized because..." messages — they're specific and usually actionable (§19.5).

⚠ **Floating-point reductions don't vectorize by default**, because vectorizing changes the summation order and FP addition isn't associative. Rather than reaching for `-ffast-math` globally (§19.4), scope it:

```
#pragma omp simd reduction(+:sum)
for (...) sum += a[i] * b[i];
```

Explicit SIMD

```
#include <immintrin.h> // intrinsics: fastest, ugliest, least portable
__m256 va = _mm256_loadu_ps(a);
__m256 vc = _mm256_fmadd_ps(va, vb, vc);
```

Approach	Trade-off
Intrinsics	maximum control, per-ISA rewrite, unreadable
Highway (Google)	★ portable, runtime dispatch, actively maintained
xsimd, Vc, EVE	portable wrapper libraries
std::simd (C++26)	standard, portable (§18.2)

Runtime dispatch

You can't ship `-march=x86-64-v4` (§19.4), so dispatch per-function:

```
__attribute__((target_clones("default,avx2,avx512f"))))
void hotKernel(float* out, const float* a, const float* b, size_t n);
```

Or manually with `__builtin_cpu_supports("avx2")`. ⚠ Note that heavy AVX-512 use causes frequency downclocking on some Intel generations — which can make the *rest* of your program slower. Measure at the application level.

24.9 Templates vs runtime polymorphism — the monomorphization trade-off

Templates inline and specialize, so each instantiation is optimal in isolation. But every instantiation is a **separate copy of the code** (§10.10), and code has a cost:

- **icache pressure** — the L1 instruction cache is ~32 KB
- **iTLB pressure**
- Longer link times, larger binaries

```
perf stat -e L1-icache-load-misses,iTLB-load-misses ./app
```

Situation	Better
A handful of types, hot inner loop	templates — inlining wins
Hundreds of instantiations of a large function	type erasure / virtual — one copy fits in icache
Large function, small hot core	thin template over a type-erased core (§10.10)

`-Wl,--icf=all` (§19.7) folds identical instantiations automatically and is worth turning on regardless.

This is the case where the "zero-cost abstraction" framing misleads: monomorphization is zero-cost *per call site* and not free *in aggregate*.

24.10 `constexpr` as a runtime optimization

Move work to compile time and it costs nothing at runtime (§6.5, §10.9):

```
constexpr auto buildTable() {
    std::array<uint8_t, 256> t{};
    for (size_t i = 0; i < 256; ++i) t[i] = computeCrcByte(i);
    return t;
}
constexpr auto kCrcTable = buildTable(); // in .rodata; zero startup cost
```

Also removes static-initialization-order risk (§7.6) and startup time. ⚠ The cost is compile time — the constant evaluator is an interpreter (§10.9). Watch `-ftime-trace`.

24.11 Methodology reminders

Full detail in §22.8. The four that cause the most wrong conclusions in perf work specifically:

1. **DoNotOptimize/ClobberMemory**, or you measure an empty loop.
2. **Interleave A/B runs**. All-A-then-all-B lets thermal drift look like a result.
3. **Microbenchmarks lie** — warm cache, trained predictor, no memory pressure, whole icache to yourself. A 20% microbenchmark win can be a system-level loss. **Always validate end-to-end.**
4. **Use callgrind instruction counts for CI regression detection** — deterministic, immune to shared-runner noise.

24.12 PGO and BOLT

Profile-Guided Optimization

```
# 1. Instrumented build
g++ -O2 -fprofile-generate -o app_gen main.cpp
# 2. Run a REPRESENTATIVE workload
./app_gen typical-input
# 3. Rebuild with the profile
g++ -O2 -fprofile-use -fprofile-correction -o app main.cpp
```

```
# Clang
clang++ -fprofile-instr-generate ... && LLVM_PROFILE_FILE=x.profraw ./app
llvm-profdata merge -output=x.profdata x.profraw
clang++ -fprofile-use=x.profdata ...
```

What PGO improves: **inlining decisions** (it knows which calls are hot), **basic-block layout** (hot paths contiguous, cold paths moved away → icache), **branch layout**, **speculative devirtualization** (§24.7), and register allocation.

Typical gain: **10–20% on branchy code** — compilers, interpreters, browsers, databases. Much less on tight numeric kernels, which were already optimal.

⚠ **The profile must be representative.** A profile from an unrepresentative workload can make things *worse*, and stale profiles decay silently.

AutoFDO avoids the instrumented build entirely: collect with **perf** from production, convert, feed to **-fauto-profile**. Lower fidelity, dramatically lower operational cost, and what large deployments actually use.

BOLT

A **post-link binary optimizer** (from Meta, now in LLVM). It rewrites the *final binary*, reordering basic blocks and functions to optimize instruction-cache and iTLB behaviour — using layout information that isn't fully available at compile time.

```
perf record -e cycles:u -j any,u -o perf.data -- ./app
perf2bolt -p perf.data -o app.fdata app
llvm-bolt app -o app.bolt -data=app.fdata -reorder-blocks=ext-tsp \
    -reorder-functions=hfsort -split-functions -split-all-cold -icf=1
```

Typical extra **5–15% on large binaries**, and it **stacks with PGO** rather than overlapping. LLVM's Propeller is a related approach.

The stacking order

```
-O2 → LTO → PGO → BOLT
```

Each layer sees information the previous one didn't have. On a large server binary, the cumulative effect can be 30%+ over plain `-O2` — with no source changes at all, which makes it the best effort/reward ratio available once your algorithms are right.

Gotcha checklist for this section

- ☐ **A DRAM access $\approx$ 200 instructions** — you are usually memory-bound, not compute-bound
- ☐ IPC below 1 means stalled, not slow code — look at memory and branches
- ☐ Row-major vs column-major traversal: same instructions, 5–10 $\times$ different
- ☐ A hot loop touching 12 bytes of a 200-byte struct wastes 94% of its bandwidth
- ☐ **Branchless is slower when the branch is predictable**
- ☐ `[[likely]]`/`[[unlikely]]` are layout hints — PGO does it better
- ☐ Manual `__builtin_prefetch` usually doesn't help
- ☐ A non-`noexcept` move makes `vector` growth copy (§5.5)
- ☐ A declared destructor suppresses your moves (§4.9)
- ☐ `shared_ptr` by value = two atomic RMWs per call (§4.7)
- ☐ FP reductions don't auto-vectorize — scope the relaxation, don't use global `-ffast-math`
- ☐ AVX-512 can downclock the core and slow down everything else
- ☐ Template monomorphization is zero-cost per call site and **not free in aggregate** (icache)
- ☐ An unrepresentative PGO profile can make things worse
- ☐ Microbenchmarks lie — validate at the system level

Exercises worth doing

1. Run the row-major vs column-major matrix sum at $N=4096$. Measure. Then confirm with `perf stat -e cache-misses`.
2. Run the sorted-vs-unsorted branch benchmark. Then make it branchless and re-run **both** cases — note that branchless loses on sorted data.
3. Convert a particle system from AoS to SoA. Measure, then check whether the SoA version vectorized with `-Rpass=loop-vectorize`.
4. Drop the counting `operator new` into a real program and print allocations for one typical operation. Be surprised.
5. `LD_PRELOAD` jemalloc or mimalloc onto an existing allocation-heavy program. Measure.
6. Take a `vector` fill loop, measure with and without `reserve`, and count reallocations with §5's Tracker.
7. Benchmark a virtual call in a loop; add `final`; add `-flto`; add PGO. Record the four numbers.
8. Build a real project with `-O2`, then `+LT0`, then `+PGO`, then `+BOLT`. Chart the cumulative gain.
9. Find a struct in your codebase with bad padding using `-Wpadded`, reorder it, and measure `sizeof` and cache misses.
10. Take one function where you're *certain* you know the bottleneck. Profile it. Compare with what you predicted.

Chapter 25 — Interoperability

What this chapter covers

- **extern "C"**, name mangling, C ABI as the universal boundary
- Consuming C libraries; wrapping them in RAII types
- Exposing C++ to C — the opaque handle + free-function pattern
- ⚠ **Never let exceptions cross a C boundary**
- **pybind11** / **nanobind** (Python), **PyO3-style Rust interop**, JNI, SWIG
- WebAssembly via Emscripten
- ⚠ **ABI fragility**: `std::string`/`std::vector` layout, compiler and standard-library mismatches, `-D_GLIBCXX_USE_CXX11_ABI`

C++ is rarely the only language in a system. It's the engine under a Python API, the native core behind a Java or C# application, the thing a Rust component is gradually replacing, or the code compiled to WebAssembly and called from JavaScript.

Every one of those boundaries has the same answer: **the C ABI**. Not because it's pleasant, but because C++'s own ABI is unstable across compilers, across standard libraries, and sometimes across compiler *versions* (§20.7) — while C's has been stable for decades and every language on earth can call it.

25.1 extern "C" and the C ABI

```
extern "C" void myFunction(int x);

extern "C" {
    void a(int);
    int b(const char*);
}
```

What **extern "C"** does

- **Disables name mangling** (§19.9) — the symbol is `myFunction`, not `_Z10myFunctioni`
- **Uses the C calling convention**
- **Forbids overloading** — one symbol, one function

What it does *not* do ⚠

- It does **not** change the language inside the function. You can use templates, RAII, and the STL freely in the body.
- It does **not** disable exceptions. An exception can still escape — and doing so is UB (§25.4).
- It does **not** make your parameter types C-compatible. `extern "C" void f(std::string s);` compiles and is meaningless to a C caller.

The dual-language header

```
/* mylib.h — includable from both C and C++ */
#ifdef MYLIB_H
#define MYLIB_H
```

```
#include <stddef.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef struct MyHandle MyHandle;           /* opaque */

MyHandle* mylib_create(void);
void      mylib_destroy(MyHandle*);
int32_t   mylib_process(MyHandle*, const uint8_t* data, size_t len);

#ifdef __cplusplus
}
#endif
#endif
```

Types that are safe at the boundary

Safe	Not safe
fixed-width integers (<code>int32_t</code> , <code>uint64_t</code>)	<code>std::string</code> , <code>std::vector</code> , any STL type
<code>float</code> , <code>double</code>	classes with virtual functions
pointers, opaque handles	references (name-mangleable, but not C)
C structs of the above	templates, overloads
function pointers	<code>bool</code> — ⚠ 1 byte in C++, <code>BOOL</code> is 4 bytes on Windows
enums with an explicit underlying type	plain <code>enum</code> (size is implementation-defined)
<code>char*</code> (null-terminated)	<code>long</code> (⚠ 8 bytes on Linux LP64, 4 on Windows LLP64)

```
static_assert(sizeof(MyStruct) == 24);           // ★ pin the layout
static_assert(offsetof(MyStruct, field) == 8);
static_assert(std::is_standard_layout_v<MyStruct>); // §7.1
```

25.2 Consuming C libraries: wrap them in RAII

The single highest-value habit at a C boundary. Full pattern in §4.6:

```
struct SqliteDeleter { void operator()(sqlite3* p) const noexcept {
    sqlite3_close(p); } };
using DbPtr = std::unique_ptr<sqlite3, SqliteDeleter>;
static_assert(sizeof(DbPtr) == sizeof(sqlite3*)); // zero overhead
```

Out-parameters: `std::out_ptr` (C++23)

C APIs return handles through `T**`, which used to require a raw temporary:

```
// Pre-C++23
sqlite3* raw = nullptr;
if (sqlite3_open(path, &raw) != SQLITE_OK) { sqlite3_close(raw); throw ...; }
DbPtr db{raw};

// C++23 – no raw pointer ever exists unowned
DbPtr db;
if (sqlite3_open(path, std::out_ptr(db)) != SQLITE_OK)
    throw DbError{sqlite3_errmsg(db.get())};

std::inout_ptr(ptr);      // for APIs that read AND replace the handle
```

Translating error codes

```
class Database {
    DbPtr db_;
public:
    explicit Database(const std::string& path) {
        if (int rc = sqlite3_open(path.c_str(), std::out_ptr(db_)); rc !=
        SQLITE_OK)
            throw DbError{sqlite3_errstr(rc)};      // or return std::expected
    }
};
```

Do this **once**, at the wrapper, so the rest of your code never sees an error code.

Callbacks into C

C callbacks take a `void*` context. A **captureless** lambda converts to a function pointer (§6.8):

```
extern "C" int rowCallback(void* ctx, int argc, char** argv, char** names)
noexcept;

sqlite3_exec(db.get(), sql,
    [](void* ctx, int argc, char** argv, char** cols) noexcept -> int {
        try {
            auto* self = static_cast<Query*>(ctx);
            self->onRow(std::span{argv, static_cast<size_t>(argc)});
            return 0;
        } catch (...) { return 1; }      // ★ §25.4 – never let it escape
    },
    &query, nullptr);
```

Strings ⚠

```
void cApi(const char*);

cApi(myString.c_str());           // ✓ guaranteed null-terminated
cApi(stringView.data());         // ⚠ UB – string_view is NOT null-terminated (§11.9)
cApi(std::string{sv}.c_str());    // ✓ allocates, but correct
```

Also: the C API may return a pointer you must free, a pointer you must *not* free, or a pointer into a static buffer valid until the next call. Read the documentation and encode the answer in a wrapper type — this is exactly the ownership-in-the-type-system argument from §4.8.

25.3 Exposing C++ to C: opaque handle + free functions

```
// engine.h – pure C, no C++ visible
#ifdef __cplusplus
extern "C" {
#endif

typedef struct Engine Engine;

typedef enum { ENGINE_OK = 0, ENGINE_ERR_INVALID = 1, ENGINE_ERR_OOM = 2 }
EngineStatus;

Engine*      engine_create(const char* config);
void         engine_destroy(Engine*);
EngineStatus engine_process(Engine*, const uint8_t* in, size_t inLen,
                           uint8_t* out, size_t* outLen);
const char*  engine_last_error(void);          /* thread-local, valid until the
next call */

#ifdef __cplusplus
}
#endif
```

```
// engine.cpp – C++ inside
#include "engine.h"
#include "EngineImpl.hpp"

namespace {
    thread_local std::string tlsError;

    template <class F>
    EngineStatus guard(F&& f) noexcept {           // ★ one exception
        firewall
```



```

        try { f(); return ENGINE_OK; }
        catch (const std::bad_alloc&)      { tlsError = "out of memory"; return
ENGINE_ERR_OOM; }
        catch (const std::exception& e)    { tlsError = e.what();          return
ENGINE_ERR_INVALID; }
        catch (...)                        { tlsError = "unknown error"; return
ENGINE_ERR_INVALID; }
    }
}

struct Engine { EngineImpl impl; }; // the opaque type,
defined here

extern "C" Engine* engine_create(const char* config) {
    try { return new Engine{EngineImpl{config}}; }
    catch (...) { return nullptr; }
}

extern "C" void engine_destroy(Engine* e) { delete e; } // ★ freed by the SAME
module

extern "C" EngineStatus engine_process(Engine* e, const uint8_t* in, size_t inLen,
                                     uint8_t* out, size_t* outLen) {
    if (!e || !in || !outLen) return ENGINE_ERR_INVALID;
    return guard([&]{ e->impl.process(std::span{in, inLen}, out, *outLen); });
}

extern "C" const char* engine_last_error(void) { return tlsError.c_str(); }

```

Design rules for a C-facing API

1. **Opaque handles only** — never expose a C++ type's layout
2. **Create/destroy pairs**, and **free in the same module that allocated** (§25.9)
3. **Error codes**, never exceptions
4. **Document lifetime and ownership** for every pointer in and out
5. **noexcept on every exported function**
6. **Version it**: either a `size` field as the first struct member, or an explicit `engine_abi_version()`
7. Prefer caller-provided buffers (`out`, `outLen`) over returning allocations

25.4 ⚠ Never let an exception cross a C boundary

This is **undefined behavior**, not a clean `std::terminate` (§13.10). C frames have no unwind information, so the unwinder walks into stack it cannot interpret. Behaviour ranges from a crash to silent corruption, and it varies by platform.

The boundaries where this applies:

```

extern "C" int cb(void*) noexcept { try { ... } catch (...) { return -1; } } //
C callbacks

```

```
extern "C" void api(void) noexcept { ... } //
your exports
std::qsort(..., comparator); //
C stdlib callbacks
pthread_create(&t, nullptr, entry, arg); //
C thread entry
signal(SIGINT, handler); //
signal handlers
```

Mark every one `noexcept` and put a `catch (...)` inside. The `noexcept` turns a mistake into a deterministic `terminate` (bad, but debuggable) instead of UB; the `catch (...)` avoids the mistake entirely.

The **thread-local last-error** pattern from §25.3 is the standard way to carry the message across without carrying the exception.

`clang-tidy: bugprone-exception-escape` finds these mechanically (§21.7).

25.5 Python: pybind11 and nanobind

```
#include <pybind11/pybind11.h>
#include <pybind11/stl.h> // automatic vector/map/optional conversion
namespace py = pybind11;

PYBIND11_MODULE(mymodule, m) {
    m.doc() = "example";
    m.def("add", &add, "Add two numbers", py::arg("a"), py::arg("b") = 0);

    py::class_<Widget>(m, "Widget")
        .def(py::init<std::string>())
        .def("process", &Widget::process)
        .def_property("name", &Widget::getName, &Widget::setName)
        .def("__repr__", [](const Widget& w) { return "<Widget " + w.getName() +
">"; });

    py::register_exception<MyError>(m, "MyError");
}
```

The things that actually bite

1. The GIL. Release it for any long-running C++ work, or you block the whole interpreter:

```
m.def("heavyWork", [](const Data& d) {
    py::gil_scoped_release release; // ★
    return compute(d);
});
```

⚠ And never touch a Python object while the GIL is released. Re-acquire with `py::gil_scoped_acquire` if a callback needs to.

2. Ownership. `return_value_policy` decides who owns what, and the wrong choice is a use-after-free or a leak:

```
py::return_value_policy::take_ownership      // Python frees it
py::return_value_policy::copy
py::return_value_policy::reference           // ⚠ C++ owns it; Python must not
outlive it
py::return_value_policy::reference_internal // tied to the parent's lifetime
py::keep_alive<0, 1>()                     // keep argument 1 alive while the
result lives
```

3. Zero-copy numpy. For arrays, use the buffer protocol rather than converting element-by-element:

```
m.def("process", [](py::array_t<double, py::array::c_style | py::array::forcecast>
arr) {
    auto buf = arr.request();
    std::span<double> data{static_cast<double*>(buf.ptr), static_cast<size_t>
(buf.size)};
    // operates on numpy's memory directly – no copy
});
```

4. Exceptions translate automatically — `std::runtime_error` → `RuntimeError`, `std::out_of_range` → `IndexError`, and so on.

nanobind

Same author, same shape of API, built for C++17 and modern Python. The pitch is **much smaller binaries and much faster compile times** (often several times better on both) at the cost of dropping some pybind11 legacy features. For a new, large binding layer, start here.

Alternatives

	Notes
Cython	Python-like syntax compiled to C; good when the glue itself is complex
ctypes / cffi	pure Python, C ABI only , no build step — great for a small C-shaped API
SWIG	multi-language from one interface file; aging, generated code is opaque
Boost.Python	pybind11's predecessor; legacy

Packaging: **scikit-build-core** is the modern CMake↔Python-wheel bridge.

25.6 Rust interop

Newly relevant because of the memory-safety pressure in §23.8 — the common industry pattern is *not* rewriting, it's **new security-sensitive components in Rust alongside hardened C++**.

Tool	Direction
cxx (dtolnay)	★ safe bidirectional C++↔Rust; you declare a shared bridge and it generates both sides
autocxx	cxx + bindgen — generates the bridge from existing C++ headers
bindgen	C headers → Rust declarations
cbindgen	Rust → C headers
PyO3	the Rust analogue of pybind11 (Rust→Python), for comparison

```
#[cxx::bridge]
mod ffi {
    unsafe extern "C++" {
        include!("mylib/engine.h");
        type Engine;
        fn create_engine(config: &str) -> UniquePtr<Engine>;
        fn process(self: Pin<&mut Engine>, data: &[u8]) -> Result<u32>;
    }
    extern "Rust" {
        fn rust_callback(value: u32);
    }
}
```

cxx handles `std::string↔String`, `std::vector↔Vec`, `std::unique_ptr↔UniquePtr`, and turns C++ exceptions into Rust `Result` — which is what makes it much more usable than a hand-written C shim. The alternative, always available, is a plain C ABI in the middle (§25.1).

25.7 JNI

```
#include <jni.h>

extern "C" JNIEXPORT jint JNICALL
Java_com_example_Engine_process(JNIEnv* env, jobject obj, jbyteArray data) {
    jsize len = env->GetArrayLength(data);
    jbyte* buf = env->GetByteArrayElements(data, nullptr);
    // ... RAII wrapper strongly recommended for the Release call ...
    int result = 0;
    try {
        result = processData(reinterpret_cast<const uint8_t*>(buf), len);
    } catch (const std::exception& e) {
        env->ReleaseByteArrayElements(data, buf, JNI_ABORT);
        env->ThrowNew(env->FindClass("java/lang/RuntimeException"), e.what());
        return -1;
    }
    env->ReleaseByteArrayElements(data, buf, 0);
}
```

```
    return result;
}
```

The pain points: verbose signature encoding, **manual local/global reference management** (local refs leak if you create many in a loop — `PushLocalFrame/PopLocalFrame`), and **you must call `ExceptionCheck()` after almost every JNI call** because a pending Java exception doesn't stop C++ execution. Threads created in C++ must `AttachCurrentThread` before touching `JNIEnv`.

△ Worth knowing: **Java's Foreign Function & Memory API (Project Panama)**, stable from Java 22, is the modern replacement for JNI — it calls C ABI functions directly from Java with no C++ shim at all. For new work, that's usually the better boundary.

25.8 WebAssembly via Emscripten

```
#include <emscripten/bind.h>
using namespace emscripten;

EMSCRIPTEN_BINDINGS(mymodule) {
    function("add", &add);
    class_<Widget>("Widget")
        .constructor<std::string>()
        .function("process", &Widget::process)
        .property("name", &Widget::getName, &Widget::setName);
    register_vector<int>("VectorInt");
}
```

```
emcc -O3 -std=c++20 --bind -o out.js main.cpp \
    -s MODULARIZE=1 -s ALLOW_MEMORY_GROWTH=1 -fwasm-exceptions
```

What's different about the target

- **wasm32**: pointers and `size_t` are **32-bit**. Code that assumed 64-bit pointers breaks. (`wasm64` exists but is not widely deployed.)
- **Linear memory** — one contiguous array, growable, shared with JavaScript.
- **You cannot block**. No blocking main loop, no synchronous I/O. Use `emscripten_set_main_loop`, or `Asyncify` (which costs size and speed).
- **Exceptions**: `-fwasm-exceptions` uses native Wasm EH (fast, needs a modern runtime). The older JS-based emulation is dramatically slower.
- **Threads** need `pthread` support plus `SharedArrayBuffer`, which requires COOP/COEP HTTP headers on the serving site.
- **Filesystem** is emulated (MEMFS/IDBFS).
- **Size matters**: `-Os/-Oz`, `--closure 1`, `wasm-opt`, and `-s FILESYSTEM=0` if you don't need it.

For non-browser targets, **WASI** is the alternative ABI (`wasi-sdk`), used by server-side Wasm runtimes.

25.9 ⚠️ ABI fragility

The full mechanics are in §20.7. The interop-specific rules:

The rules

- 1. **Never pass standard-library types across a boundary you don't control both sides of.**
`sizeof(std::string)` is 32 on libstdc++/MSVC and 24 on libc++ (§11.9). Mixing corrupts memory silently.
- 2. **Never free memory in a different module than allocated it.** On Windows especially, each DLL can have its own CRT heap; a `free` in the wrong module is a crash. Always pair `x_create`/`x_destroy`.
- 3. **Don't cross a boundary with a C++ exception** (§25.4).
- 4. **Don't rely on class layout** — opaque handles only.

The specific landmines

Issue	Detail
<code>_GLIBCXX_USE_CXX11_ABI</code>	GCC 5 changed <code>std::string</code> (COW→SSO) and kept both. <code>__cxx11</code> in a link error means a mismatch. <code>-D_GLIBCXX_USE_CXX11_ABI=0</code> forces the old one
libstdc++ vs libc++	not interchangeable, at all
MSVC <code>/MD</code> vs <code>/MT</code>	dynamic vs static CRT — must match process-wide
MSVC debug vs release CRT	must match; mixing is a classic heap corruption
<code>long</code>	8 bytes on Linux LP64, 4 on Windows LLP64 — use <code>int64_t</code>
<code>bool</code>	1 byte in C++; Windows <code>BOOL</code> is a 4-byte <code>int</code>
plain <code>enum</code> size	implementation-defined — specify the underlying type
<code>#pragma pack</code>	must match on both sides or every offset is wrong
<code>char</code> signedness	signed on x86, unsigned on ARM

Defensive practice

```
static_assert(sizeof(WireStruct) == 32);
static_assert(offsetof(WireStruct, len) == 8);
static_assert(std::is_standard_layout_v<WireStruct>);
static_assert(alignof(WireStruct) == 8);
```

```
abidiff old.so new.so          # libabigail – what changed, precisely (§20.7)
```

Gotcha checklist for this section

- ☐ `extern "C"` disables mangling — it does **not** disable exceptions or make types C-compatible
- ☐ An exception crossing a C boundary is **UB**, not `terminate`
- ☐ Mark every `extern "C"` function **`noexcept`** and put a `catch (...)` inside
- ☐ `string_view::data()` is not null-terminated — never hand it to a C API
- ☐ Free memory in the **same module** that allocated it
- ☐ Never pass `std::string/std::vector` across an uncontrolled ABI boundary
- ☐ `long` is 4 bytes on Windows, 8 on Linux; `bool` is 1 byte but `BOOL` is 4
- ☐ `char` is unsigned on ARM
- ☐ pybind11: release the GIL for long C++ work, and never touch Python objects while it's released
- ☐ pybind11 `return_value_policy` — the wrong one is a leak or a use-after-free
- ☐ JNI: check `ExceptionCheck()` after nearly every call; local refs leak in loops
- ☐ Wasm is **32-bit** and **cannot block**
- ☐ `__cxx11` in a link error = the GCC 5 `std::string` ABI split

Exercises worth doing

1. Wrap a real C library (sqlite3, libcurl, zlib) in RAII types. `static_assert` that your handle is pointer-sized. Use `std::out_ptr` for the create call.
2. Write a C API over a C++ class using the opaque-handle pattern, with a `guard()` exception firewall and a thread-local error string. Call it from a real `.c` file.
3. Deliberately let an exception escape an `extern "C"` function called from C. Run it under a debugger and see where control goes.
4. Build a pybind11 or nanobind module exposing one class. Then add a long-running method and measure throughput with and without `gil_scoped_release` from multiple Python threads.
5. Pass a numpy array to C++ zero-copy via the buffer protocol; mutate it in place and verify Python sees the change.
6. Build the same small library with libstdc++ and libc++ and try to link them with a `std::string` in the interface. Read the failure.
7. Compile something to WebAssembly with Emscripten and call it from a web page. Note what breaks because pointers are 32-bit.
8. Set up a `cxx` bridge between a small Rust module and a C++ one, passing a string in each direction.

Chapter 26 — Domain Tracks

What this chapter covers

- **Game dev:** Unreal/Unity native, ECS, custom allocators, no exceptions/RTTI, SIMD, fixed timesteps
- **Finance / HFT:** latency, cache, lock-free, kernel bypass, no allocation in the hot path
- **Embedded / automotive:** MISRA/AUTOSAR, no heap, no exceptions, freestanding, safety certification (ISO 26262)
- **Graphics:** Vulkan/DirectX/OpenGL, shaders, math libraries (GLM, Eigen)
- **Scientific/HPC:** Eigen, BLAS, OpenMP, MPI, CUDA/SYCL
- **Systems/infra:** databases, browsers, compilers, OS components

- **Qt** for desktop applications

C++ is not one language. It is a family of dialects that share a syntax and disagree about almost everything else. A game engine and a trading system are both "C++" and have close to nothing in common in style, in which features are permitted, or in what "good code" means.

This matters practically: most C++ jobs are domain jobs. The Part 1 fundamentals transfer everywhere; the dialect does not. Knowing *which* subset a domain uses, and **why**, is a large part of being hireable in it.

26.0 The map

Domain	Dominant constraint	Exceptions	RTTI	Heap	STL
Games	frame budget, consistency	✗	✗	pooled/arena	often replaced (EASTL)
HFT	tail latency (p99.9)	✗ hot path	✗	pre-allocated only	selectively
Embedded/auto	RAM, WCET, certification	✗	✗	✗ after init	freestanding subset
Graphics	GPU sync, throughput	~	~	pooled (VMA)	✓
HPC	throughput, scaling	~	✓	✓	✓
Systems/infra	correctness at scale	✓ mostly	✓	✓ custom allocators	✓
Qt/desktop	UX, portability	✓	✓	✓	Qt's own types

26.1 Game development

The constraint

A **frame budget**: 16.7 ms at 60 fps, 8.3 ms at 120, 11.1 ms for VR — and **consistency matters more than average throughput**. A frame that takes 30 ms is a visible stutter even if the average is 10 ms. This makes anything with unbounded latency unacceptable: exceptions (§13.1), general-purpose `malloc`, and page faults.

The dialect

```
-fno-exceptions -fno-rtti
```

- **Custom allocators everywhere** (§11.12): a **frame arena** reset to zero at the start of every frame, pool allocators per object type, stack allocators for scoped work, and explicit per-subsystem memory budgets.
- **Data-oriented design and ECS** (§24.3) — the defining architectural idea. Libraries: **EnTT**, **flecs**; engine-native: Unity DOTS, Unreal Mass.

- **EASTL or a custom STL** (§27.3) — allocator-first design, `fixed_vector`/`fixed_string` with inline storage, intrusive containers, and *much* better debug-build performance.
- **Handles, not pointers** (§4.8) — index + generation counter, serializable and stale-detecting.
- **SIMD** (§24.8) for physics, animation, culling.
- **Fixed timestep** for simulation with interpolated rendering; **determinism** where lockstep multiplayer or replays require it (which rules out `-ffast-math` variation across machines, §19.4).

⚠ **Debug-build performance is a first-class requirement.** A game must run at interactive rates in a debug build or designers can't work. This is why the standard library's debug modes (§11) are often unusable here and why EASTL exists.

Unreal Engine — a dialect within a dialect

Unreal is arguably the largest non-standard C++ dialect in production:

```
UCLASS()
class MYGAME_API AMyActor : public AActor {
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    float Speed = 100.f;

    UFUNCTION(BlueprintCallable)
    void DoThing();
};
```

- **UHT (Unreal Header Tool)** parses those macros and generates reflection code — property enumeration, serialization, Blueprint binding, network replication. It is a **pre-C++26 reflection system** (§18.2), like Godot's `ClassDB` (§29.12) and Qt's MOC (§26.7).
- **Garbage collection for UObjects** — Unreal has its own GC, so ownership rules differ from §4 entirely.
- `TArray`, `FString`, `TMap`, `TSharedPtr` instead of the STL.
- Its own module system, build tool (UBT), and coding standard.

Working in Unreal means learning *Unreal*, not just C++. The same is true of Godot's GDExtension and Unity's native plugin layer (where C# is the primary language and C++ appears in performance-critical plugins and the Burst/DOTS layer).

Tooling

Tracy (§21.6) is the standard frame profiler. **RenderDoc**, **PIX**, and platform vendor tools for graphics. **Unity builds** (§19.8) because compile times are brutal at engine scale.

Study: Godot and O3DE (§29.12).

26.2 Finance / HFT

The constraint

Tail latency, not throughput. Nobody cares about your mean; they care about p99.9. **Jitter is the enemy** — a predictable 5 μ s beats an average of 2 μ s with occasional 200 μ s spikes.

The rules of the hot path

- **No allocation.** Everything pre-allocated at startup; arenas and object pools for anything dynamic (§24.5).
- **No exceptions** — unbounded latency, and the unwinder's process-global lock (§13.1) is catastrophic under load.
- **No virtual calls** in the inner loop — templates and CRTP instead (§8.10, §24.9).
- **No syscalls, no locks, no logging** on the critical path. Log by writing binary records to a pre-allocated ring buffer and formatting off-thread (NanoLog-style, §27.2).
- **Busy-poll rather than block** — a blocking wait costs a context switch (~1–5 μ s), which is an eternity.

The techniques

Area	Technique
Cache	keep the hot path in L1i/L1d; cache warming; avoid false sharing (§15.10)
Branches	PGO and BOLT (§24.12) — layout matters more here than almost anywhere
Queues	lock-free SPSC ring buffers (§15.11); <code>rigtorp::SPSCQueue</code> is the canonical small implementation
Memory	huge pages, NUMA-local allocation, pinned memory
OS	core isolation (<code>isolcpus</code> , <code>nohz_full</code>), IRQ affinity, disabling C-states and turbo for <i>determinism</i> rather than speed
Network	kernel bypass : DPDK, Solarflare/Onload, <code>ef_vi</code> , RDMA. <code>io_uring</code> as a middle ground. FPGAs for the fastest paths
Timing	<code>rdtscp</code> , hardware NIC timestamps, HdrHistogram for latency distributions
Data structures	flat/open-addressed maps (§11.3), intrusive containers, seqlocks

The culture

Measure everything, report **distributions not means**, and benchmark reproducibly (§22.8) — pinned cores, disabled frequency scaling, interleaved A/B. Determinism also matters for replay-based testing against recorded market data.

Domain knowledge (FIX, binary exchange protocols like ITCH/OUCH, order book mechanics, risk checks) is as much of the job as the C++.

26.3 Embedded and automotive

The constraint

Kilobytes of RAM, no MMU on many parts, hard real-time deadlines with **WCET (worst-case execution time)** analysis, and — in automotive, aerospace, and medical — **certification**.

Freestanding C++

```
// Available: <type_traits>, <utility>, <array>, <bit>, <limits>,
//           <initializer_list>, <cstdint>, <concepts>, <span>
// Not available / not usable: <iostream>, <string>, <vector>, <memory>,
//           anything that allocates or throws
```

- **No heap after initialization.** Static allocation, fixed-capacity containers, memory pools.
- **No exceptions** — unbounded time and space, no unwinder on many targets, and impossible to bound for WCET.
- **No RTTI**, often no virtual dispatch in critical paths.
- **No recursion** — stack depth must be statically provable.

What C++ still buys you over C

This is the interesting part, because the "banned" list looks like it removes everything:

- **constexpr** (§6.5) — compute lookup tables, validate configuration, at zero runtime and zero flash cost beyond the data
- **Templates** — zero-cost abstraction over hardware peripherals, with no function-pointer indirection
- **RAII** (§4) — interrupt guards, peripheral locks, DMA transaction scopes. `~Guard()` re-enabling interrupts is exactly the right shape
- **Strong types** — units, register widths, pin identities as distinct types, so a mismatch is a compile error
- **Type-safe register maps** — a **constexpr** struct of bitfield accessors instead of `#define REG (*volatile uint32_t*)0x40021000`

```
class InterruptGuard {                                // RAI in a context with no heap and
no exceptions
    uint32_t saved_;
public:
    InterruptGuard() : saved_(disableInterrupts()) {}
    ~InterruptGuard() { restoreInterrupts(saved_); }
    InterruptGuard(const InterruptGuard&) = delete;
};
```

Standards and certification

MISRA C++:2023 (which now absorbs AUTOSAR C++14), plus ISO 26262 (ASIL A–D), DO-178C, IEC 62304, IEC 61508 — all covered in §23.5. Practically this means **qualified tools**, **MC/DC coverage** (§22.6), requirement-to-test traceability, and a documented deviation record for every intentional rule violation.

AUTOSAR **Classic** is mostly C; AUTOSAR **Adaptive** is C++14 on a POSIX-like platform, and is where most new automotive C++ lives.

Toolchain

`arm-none-eabi-gcc`, IAR, Keil, Green Hills. Cross-compilation and sysroots (§19.10). Size discipline: `-Os`, LTO, `-ffunction-sections -Wl,--gc-sections` (§19.7), map-file analysis, and **static stack-depth analysis**.

⚠ **volatile is for memory-mapped I/O, not for threading**. It gives you no atomicity and no ordering (§15.6). This confusion is endemic in embedded code.

Libraries: **ETL** (Embedded Template Library — fixed-capacity STL-alikes with no heap), fixed-size Eigen, fmt in its no-allocation mode.

Debugging: JTAG/SWD, OpenOCD, Segger J-Link, semihosting, ITM/SWO trace, and a logic analyzer — the debugging chapter (§21) mostly doesn't apply.

26.4 Graphics

The shift to explicit APIs

Vulkan, DirectX 12, and Metal moved memory management, synchronization, command recording, and descriptor management from the driver into **your** code. That's more C++ to write, and it's where the bugs are.

RAII with a twist ⚠

GPU objects are natural RAII candidates (§4.6) — but **you cannot destroy a resource the GPU may still be reading**. So the pattern is RAII plus **deferred destruction**:

```
class DeletionQueue {                                // destroy after N frames-in-flight
have retired
    std::array<std::vector<VkBuffer>, kFramesInFlight> pending_;
public:
    void enqueue(VkBuffer b) { pending_[currentFrame_].push_back(b); }
    void flush(uint32_t frame) { for (auto b : pending_[frame])
vkDestroyBuffer(dev_, b, nullptr);
                                pending_[frame].clear(); }
};
```

Ownership here is a function of *time and GPU progress*, not just scope — a genuinely different model from §4.

The stack

Need	Tool
Memory	VMA (Vulkan Memory Allocator) — sub-allocation, defragmentation
Math	GLM (GLSL-like, header-only), DirectXMath , Eigen for heavier work
Shaders	HLSL / GLSL / WGSL / Slang , compiled to SPIR-V; hot reload is table stakes
Bindings	vulkan-hpp — C++ bindings with RAII variants (⚠ notoriously heavy to include, §17.3)

Need	Tool
Debugging	validation layers (non-negotiable), RenderDoc, NSight, PIX, NVIDIA Aftermath for GPU crash dumps

⚠ **Synchronization is where most Vulkan bugs live** — pipeline barriers, semaphores, fences, timeline semaphores. The failure mode is a race between CPU and GPU, or between GPU queues, which looks exactly like the concurrency bugs in §15 but with no TSan to find them. The validation layers are your sanitizer.

Architecturally, **render graphs / frame graphs** (declare passes and resource dependencies; let the system derive barriers and aliasing) are the standard answer to managing this complexity.

26.5 Scientific computing and HPC

The stack, by scale

Scale	Tool
Single core	Eigen (§27.4), BLAS/LAPACK (OpenBLAS, MKL, BLIS), FFTW, SIMD (§24.8), cache blocking (§24.2)
One node	OpenMP (mature, pragma-based), TBB, <code>std::execution::par</code> (§12.9)
Cluster	MPI — the standard. ⚠ Its C++ bindings were deprecated in MPI-2.2 and removed in MPI-3.0 ; use the C API or Boost.MPI
GPU	CUDA (nvcc, <code>__device__</code> , unified memory), HIP (AMD), SYCL (oneAPI, AdaptiveCpp — single-source standard C++), OpenMP target offload, <code>nvc++ -stdpar</code> (runs <code>std::execution::par</code> on the GPU)
Portability layers	Kokkos , RAJA — write once, run on CPU/CUDA/HIP/SYCL
Frameworks	Trilinos, PETSc, deal.II, AMReX

⚠ Floating point is a correctness domain here

- **-ffast-math changes results** (§19.4) and breaks NaN handling. In HPC it is sometimes used deliberately and always deliberately.
- **Reproducibility across thread counts**: `std::reduce` may reorder, `std::accumulate` may not (§12.5). Parallel reductions give different low-order bits at different thread counts — which breaks bitwise-reproducible regression tests.
- **Kahan / compensated summation** for long accumulations; mixed precision (FP16/BF16/TF32) where the science tolerates it.

The rest

- **Data**: HDF5, NetCDF, ADIOS
- **Build/deployment**: Spack, EasyBuild, environment modules — cluster software stacks, not `apt`
- **Profiling**: NSight Systems/Compute, Intel VTune and Advisor (roofline analysis), TAU, Score-P, Vampir

- **Culture:** strong- and weak-scaling studies, numerical verification against analytic solutions, reproducibility as a publication requirement

26.6 Systems and infrastructure

Databases (**ClickHouse**, **ScyllaDB**, **RocksDB**, **DuckDB**, Postgres/MySQL extensions), browsers (**Chromium**, **WebKit**), compilers and runtimes (**LLVM**, **V8**), OS components, storage, and networking.

Characteristics

- **Enormous, long-lived codebases** — 1M to 30M lines, decades old
- **Compatibility pressure:** on-disk formats, wire protocols, and ABIs (§20.7) that cannot break
- **Conservative standard adoption** — a compiler-baseline bump means a full rebuild of everything and a supported-platform decision
- Extensive in-house test infrastructure

The techniques

- Custom allocators and arenas everywhere (§11.12)
- `io_uring` / `epoll` / `kqueue`, memory-mapped I/O
- **Thread-per-core, shared-nothing** (Seastar, ScyllaDB) — no locks because no sharing
- **Vectorized execution** (ClickHouse, DuckDB) — process columns in batches, which is data-oriented design (§24.3) applied to query execution
- JIT compilation for query plans or bytecode
- Lock-free structures where genuinely justified (§15.11)

Testing at this scale

- **Fuzzing** (§22.7) is mandatory — every one of these parses untrusted input
- **Deterministic simulation testing** — FoundationDB's approach: run the whole distributed system in a single-threaded deterministic simulator with injected faults, and replay any failure exactly. One of the most influential testing ideas in the field
- Jepsen-style consistency testing for distributed systems
- Sanitizer and coverage builds in CI (§14.5, §22.9)

Study: LLVM, Chromium `base/`, RocksDB, ClickHouse (§29).

26.7 Qt for desktop applications

Qt is a **full application framework**, not a GUI library: object model, signals/slots, containers, networking, SQL, threading, i18n, and two UI systems (Widgets and QML/Qt Quick).

Its own dialect

```
class Worker : public QObject {
    Q_OBJECT // ★ the MOC marker
public:
```

```
    explicit Worker(QObject* parent = nullptr) : QObject(parent) {}
signals:
    void progress(int percent);
public slots:
    void doWork();
};

connect(worker, &Worker::progress, bar, &QProgressBar::setValue); // type-safe,
Qt5+ syntax
```

- **MOC (Meta-Object Compiler)** generates the introspection code that makes signals/slots, properties, and QML binding work — **another pre-C++26 reflection system** (§18.2), alongside Unreal's UHT and Godot's `ClassDB`. Three major C++ ecosystems each built a code generator for the same missing language feature.
- **QObject parent/child ownership**: a parent deletes its children in its destructor. Raw pointers with a well-defined owning parent — a different, valid ownership model from §4, and one you must not mix carelessly with `unique_ptr`.
- **Implicit sharing (COW)** in `QString`, `QVector`, etc. — copies are cheap, which cuts against §3.1's value-semantics defaults and surprises people coming from standard C++.
- **QString is UTF-16**, so every boundary with `std::string` needs an explicit conversion.

⚠️ Licensing

Qt is **GPLv2/GPLv3/LGPLv3 or commercial**. LGPLv3 requires that users can relink against a modified Qt — which in practice means **dynamic linking**, and that's a genuine constraint for static builds, embedded targets, and some app stores (§27.10, §28.7). This is a business decision, not just a technical one, and it should be made before the code is written.

Alternatives

	Use for
Dear ImGui	★ debug UIs, tools, game editors — immediate-mode, trivial to embed (§27.9)
wxWidgets	native look, permissive licence
Slint	modern, declarative, embedded-friendly
JUCE	audio applications (its own strong ecosystem)
GTK	C, but usable; Linux-native
FLTK	tiny and old-fashioned

26.8 Moving between domains

What transfers: everything in Part 1 — value semantics, RAII, ownership, move semantics, templates, UB — plus the tooling from Part 2. A good C++ engineer is portable.

What doesn't:

- The **banned-feature list** and its rationale
- The **libraries** (EASTL vs Abseil vs ETL vs Qt's containers)
- The **cost model you carry in your head** — an HFT engineer flinches at a heap allocation; an HPC engineer flinches at a cache-unfriendly loop; an embedded engineer flinches at recursion
- **Domain knowledge**, which is often half the job: order books, rendering pipelines, functional-safety processes, query planners

Practical advice: go deep in one domain. The fundamentals make you portable; the depth makes you hireable. And when you do switch, the fastest way in is to read that domain's flagship open-source codebase (§29) and pay attention to what it *doesn't* use.

Gotcha checklist for this section

- ☐ Games: **debug-build performance** is a hard requirement, not a nicety
- ☐ Games: determinism rules out `-ffast-math` and cross-machine FP variation
- ☐ HFT: **tail latency, not mean** — and exceptions' global unwinder lock is disqualifying on the hot path
- ☐ HFT: blocking costs a context switch; busy-poll on isolated cores
- ☐ Embedded: **volatile is for MMIO, not threading** — no atomicity, no ordering
- ☐ Embedded: no exceptions means no failing constructors — two-phase init or factories (§13.8)
- ☐ Embedded: certification needs **qualified tools** and MC/DC coverage
- ☐ Graphics: you cannot destroy a resource the GPU may still be using — deferred deletion queues
- ☐ Graphics: validation layers are your only race detector
- ☐ HPC: parallel reductions are **not bitwise reproducible** across thread counts
- ☐ HPC: MPI's C++ bindings were **removed** in MPI-3.0
- ☐ Qt: LGPLv3 effectively requires dynamic linking
- ☐ Qt: `QObject` parent/child ownership does not mix casually with `unique_ptr`
- ☐ Unreal, Godot, and Qt each built their own reflection system — all obsoleted by C++26 (§18.2)

Exercises worth doing

1. Build a frame-arena allocator that resets each frame, and a `std::pmr` adapter over it (§11.12). Measure against `new/delete` in a loop.
 2. Write an SPSC ring buffer with correct acquire/release ordering (§15.6). Measure round-trip latency and plot the **distribution**, not the mean.
 3. Take a small C++ program and build it freestanding for an ARM Cortex-M target with no heap and `-fno-exceptions`. See exactly what you lose.
 4. Write an RAII interrupt guard and a `constexpr` type-safe register map for a real peripheral.
 5. Write a Vulkan triangle with a deletion queue and correct frames-in-flight handling. Then deliberately destroy a buffer in use and watch the validation layer catch it.
 6. Sum ten million floats with `std::accumulate`, `std::reduce(par)`, and Kahan summation. Compare the results bit for bit at 1, 4, and 16 threads.
 7. Build the same small app in Qt Widgets and in Dear ImGui. Compare development time, binary size, and licensing implications.
 8. Pick a domain flagship from §29 and spend a week reading it with a list of "what does this codebase refuse to use, and why."
-

Chapter 27 — Libraries Worth Knowing

What this chapter covers

- *Boost** (the incubator) · **fmt** / **std::format** · **spdlog** · **Abseil** · **folly** · **EASTL** · **Eigen** · **{nlohmann,simdjson,glaze}** JSON · **Asio** · **gRPC** / **protobuf** · **range-v3** · **Catch2/GoogleTest** · **Google Benchmark** · **magic_enum** · **CLI11**

C++ has no standard package manager and no **crates.io**, so its ecosystem is discovered by word of mouth and inherited from whatever your team chose in 2016. That makes a survey like this more useful in C++ than in most languages — and it makes **dependency evaluation** (§27.10) a real skill, because adding a library here costs build time, ABI surface, and packaging work in a way it doesn't elsewhere.

27.1 Boost — the incubator

Boost is where much of the standard library was prototyped. Things that graduated:

shared_ptr/unique_ptr → **<memory>** · **thread/mutex** → **<thread>** · **chrono** · **regex** · **filesystem** · **optional** · **variant** · **any** · **string_view** · **array** · **tuple** · **bind/function** · **random** · **unordered** · **math** special functions

Boost libraries still worth reaching for

Library	Why
Asio	★ the de facto async I/O library (§27.6)
Beast	HTTP and WebSocket on Asio
Boost.Unordered	★ unordered_flat_map — a fast open-addressing hash map (§11.3)
Boost.Container	flat_map/flat_set (§11.4), small_vector , static_vector , stable_vector
Boost.PFR	★ struct field reflection without macros , pre-C++26 (§18.2)
Boost.Describe	annotated reflection for enums and classes
Boost.Multi_index	one container, several independent indices — genuinely hard to replicate
Boost.Graph	mature generic graph algorithms
Boost.Geometry	computational geometry, spatial indexes (R-tree)

Library	Why
Boost.Interprocess	shared memory, memory-mapped files, interprocess sync
Boost.Multiprecision	arbitrary-precision arithmetic
Boost.Outcome	expected -style error handling with more policy control (§13.6)
Boost.Hana	heterogeneous compile-time metaprogramming
Boost.Spirit X3	parser combinators (⚠ legendary compile times)
Boost.Process , Boost.JSON , Boost.URL , Boost.Charconv , Boost.Cobalt (coroutines), Boost.Redis , Boost.MySQL	newer, actively developed

⚠ **The costs:** a large source tree, heavy headers, long compile times, and a version-upgrade treadmill. Use only the libraries you need (many are header-only; **system**, **filesystem**, **program_options**, **thread** are compiled). Boost's CMake support has improved substantially — modern usage is **find_package(Boost CONFIG)** and **Boost::asio**-style targets.

27.2 Formatting and logging

{fmt} and **std::format**

std::format (C++20) and **std::print** (C++23) are **derived from fmt** (§18.1). **fmt** remains worth using when:

- Your toolchain predates the standard versions
- You want features that haven't landed yet
- You want **FMT_COMPILE** — compile-time format-string parsing, which removes the remaining runtime cost

```
#include <fmt/format.h>
#include <fmt/ranges.h>
#include <fmt/chrono.h>

fmt::format("{:>10.3f}", value);
fmt::format!("{}", std::vector{1,2,3});           // [1, 2, 3]
fmt::format!("{}", fmt::join(v, ", "));           // custom separator
fmt::format_to(std::back_inserter(buf), "{}", x);  // ★ no allocation of a
result string
fmt::format(FMT_COMPILE("{}-{}"), a, b);           // format string parsed
at compile time
```

fmt is faster than both **printf** and **iostream**, and it's type-safe with **compile-time** format checking.

spdlog

```
#include <spdlog/spdlog.h>
#include <spdlog/sinks/rotating_file_sink.h>

auto logger = spdlog::rotating_logger_mt("app", "logs/app.log", 1048576*5, 3);
spdlog::set_level(spdlog::level::debug);
SPDLOG_INFO("processed {} items in {}", count, elapsed); // macro form: compiled
out by level
```

Built on fmt. Sinks, async mode (a background thread + ring buffer), rotation, per-logger levels.

⚠ **Use the compiled library** (`SPDLOG_COMPILED_LIB`, and `SPDLOG_FMT_EXTERNAL` if you already use fmt) — the header-only mode is a significant compile-time tax across a large codebase.

Alternatives: **quill** (lower latency, does formatting on the background thread), **NanoLog** (extreme low-latency, binary log format), **Glog**, **Boost.Log**.

27.3 The "second standard library" tier: Abseil, folly, EASTL

	Abseil (Google)	folly (Meta)	EASTL (EA)
Purpose	portable utilities + faster containers	performance-first, Meta's needs	STL replacement for games
Build friendliness	good (Bazel + CMake)	⚠ hard standalone, many deps	easy, self-contained
Stability	⚠ "live at head"	churns	stable

Abseil — the pieces people actually pull in:

```
absl::flat_hash_map / node_hash_map / btree_map // ★ much faster than std::
($11.3)
absl::StatusOr<T>, absl::Status // error handling ($13.6)
absl::StrCat, StrSplit, StrJoin, StrFormat // string utilities
absl::Mutex // with deadlock detection
and condition support
absl::Cleanup // scope guard ($4.10)
absl::Span, absl::FunctionRef, absl::InlinedVector
absl::Time, absl::Duration
absl::Flags, absl::LOG
```

⚠ Abseil's **"live at head"** policy means no long-term API/ABI stability promises — the assumption is you build from source with a pinned commit. Don't mix an Abseil-linked prebuilt binary with a different Abseil version.

folly — F14 hash maps, `fbstring`, `small_vector`, `Future/SemiFuture`, `folly::coro` (§16.6), `Synchronized<T>` (data + its mutex in one type), `MPMCQueue`, `ProducerConsumerQueue`, `dynamic`, `IOBuf`. Excellent engineering; a heavy dependency with aggressive platform assumptions. Read it for the techniques (§29) even if you don't link it.

EASTL — allocator-first design, no exceptions or RTTI, `fixed_vector/fixed_string` (stack-allocated), intrusive containers, and much better **debug-build** performance than a typical STL (which matters enormously in games, where debug builds must still run at interactive framerates).

27.4 Eigen — linear algebra

```
#include <Eigen/Dense>

Eigen::Matrix3d m = Eigen::Matrix3d::Identity();
Eigen::VectorXd v = Eigen::VectorXd::Random(1000);
Eigen::MatrixXd result = A * B + C;           // ONE loop, no temporaries
auto x = A.colPivHouseholderQr().solve(b);
```

Eigen is the canonical **expression template** library: `A * B + C` builds a compile-time expression tree and evaluates it in a single fused pass, with no intermediate matrices. Header-only, vectorized, mature.

⚠ The three Eigen gotchas

1. **auto is dangerous**. It captures the *expression*, not the result:

```
auto expr = A * B;           // ⚠ an expression object referencing A and B
someFunctionThatModifies(A);
std::cout << expr;           // evaluated NOW, with the modified A — or dangling
Eigen::MatrixXd result = A * B; // ✓ force evaluation
auto r2 = (A * B).eval();      // ✓ explicit
```

This is the single most common Eigen bug and it's a direct consequence of §3's lifetime rules meeting lazy evaluation.

2. Aliasing.

```
A = A * B;           // ⚠ A is read while being written
A = (A * B).eval();   // ✓
A.noalias() = B * C;  // ✓ assert no aliasing, skip the temporary
```

3. **Alignment in containers** (pre-C++17). Fixed-size vectorizable types need over-aligned allocation; you needed `EIGEN_MAKE_ALIGNED_OPERATOR_NEW` and `Eigen::aligned_allocator`. C++17's aligned `new` largely fixed this — but you'll still meet the macro in older code.

Alternatives: **Blaze**, **Armadillo** (MATLAB-like), **xtensor** (numpy-like), raw **BLAS/LAPACK** (OpenBLAS, MKL) for large dense work, **Kokkos** for portable HPC, and `std::mdspan` (§11.10) for plain multidimensional views.

27.5 JSON

Library	Parse speed	Ergonomics	Serialize	Notes
nlohmann/json	slow	★ best	✓	single header; STL-like; allocation-heavy
simdjson	★ GB/s	moderate	✗	SIMD, on-demand API, read-only
glaze	very fast	★ excellent	✓	★ compile-time struct binding, header-only
RapidJSON	fast	dated	✓	DOM + SAX; maintenance has slowed
Boost.JSON	fast	good	✓	excellent allocator support (monotonic, §11.12)

```
// nlohmann – when developer time matters more than throughput
nlohmann::json j = {{"name", "x"}, {"count", 3}};
auto name = j["name"].get<std::string>();
NLOHMANN_DEFINE_TYPE_INTRUSIVE(Config, host, port, tls) // struct <-> JSON

// simdjson – when you're parsing gigabytes
simdjson::ondemand::parser parser;
auto doc = parser.iterate(padded_string::load("big.json"));
for (auto tweet : doc["statuses"]) { std::string_view t = tweet["text"]; }

// glaze – struct binding with no macros, fast both directions
struct Config { std::string host; int port; bool tls; };
auto cfg = glz::read_json<Config>(jsonText);
std::string out = glz::write_json(cfg).value_or("");
```

Choosing: nlohmann for config files and tooling; simdjson for high-throughput read-only ingestion; glaze when you want speed *and* struct binding *and* serialization. (C++26 reflection, §18.2, is likely to reshape this whole category.)

27.6 Networking and RPC

Asio

The de facto async I/O library, available standalone or as Boost.Asio. `io_context`, executors, strands, timers, TCP/UDP/serial/pipes, SSL via OpenSSL, and **coroutine support** that's the most production-ready in the ecosystem (§16.6).

```
asio::awaitable<void> session(tcp::socket sock) {
    char data[1024];
    for (;;) {
        auto n = co_await sock.async_read_some(asio::buffer(data),
asio::use_awaitable);
        co_await asio::async_write(sock, asio::buffer(data, n),
asio::use_awaitable);
    }
}
asio::co_spawn(ctx, session(std::move(socket)), asio::detached);
```

Asio was the basis of the Networking TS, which was **not** adopted; `std::execution` (§15.9) is where the standard went instead. Asio itself is unaffected and remains the answer today.

- **Beast** — HTTP/1.1 and WebSocket on Asio
- **cpp-httplib** — a single header, blocking, trivially easy; perfect for tools and tests
- **libcurl** / **cpr** — HTTP client (cpr is a pleasant C++ wrapper)
- **Crow**, **Drogon**, **oatpp** — web frameworks
- **Seastar** — shared-nothing, thread-per-core, extreme throughput (ScyllaDB's foundation)
- **libuv** — the C library behind Node; used when you want C portability

protobuf and gRPC

```
message Request { string query = 1; int32 limit = 2; }
service Search { rpc Find(Request) returns (Response); }
```

protobuf gives you a schema, generated C++, a compact wire format, and **arena allocation** (§11.12) for high-throughput servers. gRPC adds RPC over HTTP/2 with streaming, deadlines, and interceptors.

⚠ Both are **heavy**: large dependencies, a code-generation step in your build, substantial generated-code size, and protobuf has a history of ABI churn (pin your version, and make sure everything in the process uses the same one). gRPC's async API is genuinely difficult — prefer the **callback API** over the older completion-queue API.

Lighter alternatives: **Cap'n Proto** (zero-copy, no parse step), **FlatBuffers** (zero-copy, popular in games), **MessagePack**, **Thrift**.

27.7 range-v3

Eric Niebler's library, the basis for C++20 ranges (§12). Still useful for pre-C++20 toolchains, and for things the standard doesn't have yet:

```
ranges::views::group_by, chunk, cycle, cache1, sliding, concat
ranges::actions::sort, unique, remove_if // ★ EAGER, in-place – no std
equivalent
ranges::to<std::vector>() // std got this in C++23
```

`actions::` is the notable gap: `std::ranges` has only lazy views, while `range-v3` also offers eager in-place container operations.

27.8 Testing and benchmarking

Covered in §22. In one line each: **GoogleTest/GoogleMock** (the default, mocking included), **Catch2** (**SECTIONS**, expression decomposition), **doctest** (fastest compiles, in-source tests), **Google Benchmark** (the standard), **nanobench** (better statistics for micro-benchmarks), **RapidCheck** (property-based), **Trompeloeil** (mocking that works with templates).

27.9 Small, high-leverage utilities

`magic_enum`

```
#include <magic_enum.hpp>
magic_enum::enum_name(Color::Red);           // "Red"
magic_enum::enum_cast<Color>("Red");
magic_enum::enum_values<Color>();
```

Enum reflection with no macros and no code generation, implemented via a compiler-specific trick on `__PRETTY_FUNCTION__`.

⚠ Limits: only enumerator values in a bounded range (default –128..128, configurable), a real compile-time cost that scales with that range, and no support for very large sparse enums. **C++26 reflection makes it obsolete** (§18.2) — it was always a workaround for a missing language feature.

CLI11

```
CLI::App app{"My tool"};
std::string file; int count = 10; bool verbose = false;
app.add_option("-f,--file", file, "Input file")->required()-
>check(CLI::ExistingFile);
app.add_option("-c,--count", count)->check(CLI::Range(1, 100));
app.add_flag("-v,--verbose", verbose);
CLI11_PARSE(app, argc, argv);
```

Subcommands, validators, config files, environment variables, generated help. Alternatives: **cxxopts** (lighter), **argparse** (Python-like), **absl::Flags**, **Boost.Program_options** (heavier), **structopt** (struct-driven).

The rest, by category

Need	Reach for
------	-----------

Need	Reach for
Regex	⚠ not <code>std::regex</code> — it's slow and ABI-frozen (§20.7). Use CTRE (compile-time, header-only, very fast) or RE2 (linear time, safe on untrusted input)
Date/time	<code>std::chrono</code> (C++20 calendar); Howard Hinnant's date for older toolchains, plus its tz database
Scanning/parsing input	scnlib (the <code>scanf</code> counterpart to <code>fmt</code>), <code>std::from_chars</code>
Hashing	xxHash , wyhash , absl::Hash , BLAKE3
Compression	zstd (the current default), lz4 (speed), zlib (compatibility)
Crypto	libsodium (★ hard to misuse), BoringSSL/OpenSSL , Botan
Unicode	ICU (complete, heavy), utf8cpp (light)
Containers	ankerl::unordered_dense , tsl::robin_map , plf::colony (→ <code>std::hive</code> , §18.2)
Error handling	tl::expected , tl::optional , tl::function_ref (§13.6, §6.10)
GUI	Qt (the industrial answer), Dear ImGui (★ immediate-mode, debug tools), wxWidgets , Slint
Graphics/math	GLM (GLSL-like), SDL3 , bgfx
Profiling in-app	Tracy (§21.6)
DB	SQLiteCpp , sqlite_orm , libpqxx , Boost.MySQL , soci
Embedding a script VM	sol2 (Lua), pybind11 (§25.5)
Terminal UI	FTXUI , indicators , tabulate

27.10 Choosing a dependency

⚠ **A dependency costs more in C++ than in most languages.** No standard package manager, real compile-time impact, ABI surface (§20.7), and packaging work for every platform you ship to. Evaluate accordingly.

The checklist

Question	Why
Maintained? Recent commits, responsive issues, a release cadence	abandoned C++ libraries become permanent forks
CMake config package (§20.4), or <code>vcpkg</code> /Conan port?	otherwise you're writing the integration
Header weight / compile-time cost? Measure with <code>-ftime-trace</code> (§19.8)	some libraries cost more than the feature is worth

Question	Why
Transitive dependencies?	one library can drag in five
Header-only or compiled? (§20.9)	affects both build time and ABI exposure
ABI surface — does it appear in <i>your</i> public headers?	if so, your consumers inherit the constraint
License	see below
Security history — CVEs, is it fuzzed, is it on OSS-Fuzz (§22.7)?	especially for anything parsing untrusted input
Does the standard already do this?	<code>std::format</code> , <code>std::span</code> , <code>std::expected</code> , <code>std::generator</code> replaced a lot of dependencies

Licences

Class	Examples	Constraint
Permissive	MIT, BSD, Boost Software License (no attribution needed for binaries)	~none
Permissive + patent grant	Apache-2.0	⚠ known incompatibility with GPLv2
Weak copyleft	LGPL , MPL	LGPL generally requires <i>dynamic</i> linking or relinkable objects — a real constraint for static builds and app stores
Strong copyleft	GPL , AGPL	viral for distributed binaries

⚠ LGPL + static linking is the trap that catches C++ projects specifically, because static linking is so common here.

Vendor or package-manage?

- **Package manager** (vcpkg/Conan) — scales, keeps your tree small, standard tooling
- **FetchContent/CPM with a pinned tag** — reproducible, no external tooling, good for a handful of deps
- **Vendoring** (a copy in-tree) — full control and reproducibility, at the cost of manual security updates. Justified for small libraries and for regulated builds

Gotcha checklist for this section

- `std::regex` is slow and **permanently ABI-frozen** — use CTRE or RE2
- `auto expr = A * B;` in Eigen captures an **expression**, not a result

- ☐ Eigen aliasing: `A = A * B`; needs `.eval()` or `.noalias()`
- ☐ spdlog header-only mode is a significant compile-time cost — use the compiled library
- ☐ Abseil **lives at head** — no ABI stability; pin and build from source
- ☐ protobuf has ABI churn; everything in one process must use the same version
- ☐ gRPC's completion-queue async API is much harder than the callback API
- ☐ magic_enum only sees a bounded enumerator range, and costs compile time
- ☐ nlohmann/json is ergonomic and slow — not for hot paths
- ☐ simdjson is **read-only** — no serialization
- ☐ LGPL + static linking is a real constraint, and static linking is the C++ norm
- ☐ Check whether the standard library already has it before adding a dependency

Exercises worth doing

1. Replace every `iostream` formatting call in a small project with `std::format/fmt`. Measure binary size and runtime.
2. Parse the same 100 MB JSON file with nlohmann, simdjson, and glaze. Record throughput and peak memory for each.
3. Reproduce the Eigen `auto` bug: store `auto expr = A * B`, modify `A`, then evaluate. Then fix it two ways.
4. Swap `std::unordered_map` for `absl::flat_hash_map` in a lookup-heavy path and benchmark (§11.3).
5. Write a small TCP echo server with Asio coroutines, then again with `cpp-httpplib` for HTTP. Compare the effort.
6. Benchmark `std::regex` against CTRE and RE2 on the same pattern and input.
7. Run `-ftime-trace` + ClangBuildAnalyzer (§19.8) on a TU before and after adding a heavy header-only dependency.
8. Audit one project's dependencies against the §27.10 checklist. Find the one you'd drop.

Chapter 28 — Professional Practice

What this chapter covers

- Git, `git bisect`, code review culture in C++ (correctness + ownership + exception safety)
- `clang-format`, `.clang-tidy`, `.editorconfig`, pre-commit
- Style guides: Google, LLVM, **C++ Core Guidelines**
- Documentation: Doxygen, Sphinx+Breathe, ADRs
- **ABI/API versioning**, semantic versioning, `abidiff`
- Licensing implications
- Reading large C++ codebases (clangd, cscope, `-ftime-trace`)
- Production debugging, logging, crash reporting (Breakpad/Crashpad, Sentry)

The material that separates "can write C++" from "can be trusted with a C++ codebase." Most of it isn't language knowledge — it's knowing what a reviewer must check that the compiler won't, how to find the commit that broke something in a codebase that takes twenty minutes to build, and how to get a usable stack trace out of a binary that crashed on a customer's machine six weeks ago.

28.1 Git and `git bisect`

`git bisect run` — the highest-value git feature for C++

Because C++ builds are slow and bugs are often non-local, "which commit broke this" is genuinely hard. Automate it:

```
#!/bin/bash
# bisect.sh
cmake --build build -j$(nproc) 2>/dev/null || exit 125 # 125 = SKIP this commit
./build/tests --gtest_filter=Broken.Test                # 0 = good, 1 = bad
```

```
git bisect start
git bisect bad HEAD
git bisect good v1.2.0
git bisect run ./bisect.sh # ★ walk away; come back to the answer
git bisect reset
```

Exit codes: 0 good, 1–124/126–127 bad, 125 **skip** (won't build — very common when bisecting through a refactor), >127 abort.

⚠ **Bisect cost is build cost.** Make it viable with `ccache` (§19.8), a build directory reused across checkouts, and a *minimal* reproducer rather than the whole suite. `git worktree add` lets you bisect in a separate directory without disturbing your working tree.

Bisect also works for performance regressions — have the script compare a benchmark against a threshold (or use callgrind instruction counts, §22.8, since they're deterministic).

Other git things that matter here

```
git log -S "someSymbol" # ★ pickaxe: when did this string
                        appear/disappear?
git log -G "regex"
git blame -w -C -C      # ignore whitespace, follow code moved between
                        files
git log --stat --since=1.year -- src/ | ... # churn analysis: where the bugs
live
```

- **Commit hygiene is a bisect prerequisite.** Small, individually-building commits make bisect useful; a 4,000-line "refactor" commit makes it useless.
- `.gitattributes` for line endings — CRLF/LF mismatches break builds and pollute diffs on mixed-platform teams.
- **Submodules vs package manager** (§20.3): submodules are reproducible and painful (detached HEADs, forgotten `--recursive`, no version resolution). Prefer `vcpkg`/`Conan`/`FetchContent` unless you need to patch the dependency.

28.2 Code review culture in C++

What makes C++ review different: **the compiler checks less than in most languages**, so the reviewer is the type system for ownership, lifetime, and exception safety.

The C++ review checklist

Ownership and lifetime — the two that cause the worst bugs:

- ☐ Who owns this? Is every raw pointer non-owning (§3.6, §4.8)?
- ☐ Any `new/delete` outside a resource class (§4.4)?
- ☐ Does this reference, `string_view`, `span`, iterator, or lambda capture outlive its referent (§3.4, §6.9, §11.9)?
- ☐ Coroutine parameters taken **by value** (§16.7)?
- ☐ Container mutation while iterators are held (§11.5)?

Special members and performance:

- ☐ Rule of Zero, or all five declared together (§4.9)?
- ☐ Did adding a destructor silently kill the move operations (§4.9)?
- ☐ Are moves `noexcept` (§5.5)?
- ☐ `shared_ptr` by value where a `const&` would do (§4.7)?
- ☐ Allocations in a hot path; missing `reserve` (§24.5)?

Correctness:

- ☐ What exception guarantee does this offer? Is the destructor no-throw (§13.2, §13.4)?
- ☐ `const` correct — and are `const` methods actually thread-safe (§7.5)?
- ☐ Signedness and overflow in size/index arithmetic (§23.2)?
- ☐ `virtual` destructor on a polymorphic base (§8.3)?

Interfaces:

- ☐ Does this change a public header — i.e. is it an **ABI break** (§20.7)?
- ☐ Is the ownership/lifetime contract documented for every pointer in the signature?

Automate everything that can be automated

Reviewer attention is scarce and should go to design and lifetime, not to brace placement or a missing `override`. **clang-format**, **clang-tidy**, and **sanitizer CI** should have removed all the mechanical findings before a human looks (§21.7, §22.9). A review culture where people argue about formatting is one where the real issues get less attention.

28.3 clang-format, clang-tidy, editorconfig, hooks

```
# .clang-format
BasedOnStyle: Google
IndentWidth: 4
```

```
ColumnLimit: 100
PointerAlignment: Left          # int* p, not int *p
AllowShortFunctionsOnASingleLine: Empty
IncludeBlocks: Regroup
IncludeCategories:
- { Regex: '^<.*\\.h>', Priority: 1 } # C headers
- { Regex: '^<.*>', Priority: 2 }   # C++ standard
- { Regex: '^\".*\"', Priority: 3 }   # project
```

```
git clang-format          # ★ format only what YOU changed
clang-format -i $(git ls-files '*.cpp' '*.hpp')
clang-format --dry-run --Werror ... # CI check
```

⚠ The big-bang reformat

Do it **once**, in a single commit that changes nothing else, then:

```
# .git-blame-ignore-revs
a1b2c3d4... # repo-wide clang-format
```

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

Otherwise every `git blame` for the rest of the project's life points at the reformat commit.

```
# .editorconfig – for editors that aren't running clang-format
root = true
[*]
charset = utf-8
insert_final_newline = true
trim_trailing_whitespace = true
[*.{cpp,hpp,h,cc}]
indent_style = space
indent_size = 4
```

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/pre-commit/mirrors-clang-format
  rev: v18.1.0
  hooks: [{ id: clang-format }]
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.5.0
```

```
hooks: [{id: trailing-whitespace}, {id: end-of-file-fixer}, {id: check-merge-conflict}]
```

Keep hooks **fast** — anything slower than a couple of seconds gets bypassed with `--no-verify`, and then it isn't a hook.

28.4 Style guides

Guide	Character
Google	widely known, opinionated; historically banned exceptions and RTTI; specific naming; 80 columns
LLVM	pragmatic, <code>camelCase</code> functions, minimal ceremony
Mozilla, WebKit, Chromium	project-specific, all available as clang-format presets
C++ Core Guidelines	★ not a formatting guide — it's about correctness and safety (§23.4)

The Core Guidelines are complementary to the others, not competing: Google tells you where the brace goes, the Core Guidelines tell you not to return a raw owning pointer. Adopt one formatting style *and* the Core Guidelines.

The meta-point: pick one, encode it in `.clang-format`, and stop discussing it. The specific choice matters far less than consistency and automation.

Common naming conventions (pick one, be consistent):

```
class WidgetFactory;           // PascalCase types – near-universal
void processData();           // camelCase (LLVM/Qt) or PascalCase (Google)
functions
int itemCount;                // camelCase locals
int member_; int m_member;    // trailing or leading underscore for members
constexpr int kMaxSize;       // Google-style constants
#define MY_MACRO               // SCREAMING_CASE macros, always
```

⚠ Never use a leading underscore followed by a capital, or a double underscore anywhere — those identifiers are **reserved for the implementation**, always, in every scope.

28.5 Documentation

Doxygen

```
/// @brief Processes a batch of records.
///
/// @param records Non-owning view; must remain valid for the call's duration.
```

```

/// @param opts    Processing options.
/// @return The number of records successfully processed.
/// @throws ParseError if a record is malformed. Strong exception guarantee.
/// @note Thread-safe: may be called concurrently on distinct Processor instances.
/// @complexity O(n log n) in the number of records.
std::size_t process(std::span<const Record> records, const Options& opts);

```

⚠ **A doc comment that restates the signature is noise.** `@param count` The `count`. costs a line and adds nothing. Document what the *types don't say*:

- **Ownership and lifetime** — who frees it, how long must it stay valid
- **Thread safety** — safe to call concurrently? on the same object?
- **Exception guarantee** (§13.2)
- **Preconditions** — until contracts (§13.9) let you state them in code
- **Complexity**
- **Invariants** the class maintains

Turn on `WARN_IF_UNDOCUMENTED` and `WARN_AS_ERROR` for public headers if you want docs to actually stay current.

Doxygen + Sphinx via Breathe/Exhale is the standard way to get prose documentation with generated API reference. `doxygen-awesome-css` makes plain Doxygen look modern with no other changes.

ADRs

Architecture Decision Records — one short markdown file per significant decision, in the repo:

```

# ADR 0007: Use std::expected for parser errors

## Status
Accepted (2026-03-14)

## Context
The parser is on the hot path and throws for ~5% of inputs...

## Decision
Return std::expected<Ast, ParseError>; keep exceptions for programmer errors.

## Consequences
+ No unwinder cost on the common failure path (§13.1)
- Callers must check; mitigated by [[nodiscard]]
- Requires C++23 or tl::expected on our older targets

```

The value isn't the decision, it's the **context** — six months later nobody remembers why, and the alternative is re-litigating it.

28.6 API and ABI versioning

Semantic versioning covers your API. ABI is a separate, stricter contract (§20.7).

```
set_target_properties(mylib PROPERTIES
    VERSION    ${PROJECT_VERSION}    # 1.2.3
    SOVERSION  1                      # ★ bump ONLY on an ABI break
)
```

Gate ABI changes in CI

```
abidiff --headers-dir1 old/include --headers-dir2 new/include old.so new.so
```

Run it against the previous release on every PR that touches a public header. It's the only reliable way to catch "someone added a member to a class in a shipped header."

Deprecation

```
[[deprecated("use processV2() instead; removed in 3.0")]]
void process();
```

A deprecation *period* — a release where the old thing warns and the new thing exists — is the difference between a migration and a broken downstream.

Inline-namespace versioning

Lets two ABI versions coexist in one binary:

```
namespace mylib {
    namespace v1 { class Widget { int a; }; }
    inline namespace v2 { class Widget { int a, b; }; } // mylib::Widget
    resolves to v2
}
```

Old objects link against the mangled **v1** symbols, new ones against **v2**, and both can be loaded. This is how libstdc++ handles the **__cxx11** ABI split (§19.9, §25.9).

28.7 Licensing

From the shipping side (evaluation is §27.10):

- **SPDX identifiers** in every source file — `// SPDX-License-Identifier: Apache-2.0` — machine-readable and increasingly expected

- **Attribution:** MIT/BSD/Apache require the licence text in your distribution. Generate a `THIRD_PARTY_LICENSES` file from your dependency manifest rather than maintaining it by hand
- **SBOM** — a Software Bill of Materials (CycloneDX or SPDX format) is now a procurement requirement in many contexts; vcpkg and Conan can generate one
- **Scanning:** FOSSA, ScanCode, `licensee`, or your package manager's metadata
- ⚠ **LGPL + static linking** — the C++-specific trap, because static linking is normal here (§27.10)
- ⚠ **Apache-2.0 is incompatible with GPLv2** (the patent clause)

28.8 Reading large C++ codebases

Set up the tools first

```
cmake -G Ninja -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -B build
ln -sf build/compile_commands.json .
```

clangd with a `compile_commands.json` (§20.10) is the single biggest lever. Without it, you're reading C++ with `grep` — and `grep` loses to templates, macros, overloads, and ADL. With it you get real go-to-definition, find-references, call hierarchy, and type-on-hover.

A strategy that works

1. **Read the build files first.** What are the targets? What depends on what? That's the architecture, more honestly than any diagram.
2. **Find the entry points** — `main`, library init, the plugin registry, the request handler.
3. **Read the tests.** They're executable documentation of intended behaviour, and they show you how to construct the objects.
4. **Follow one path end to end** — one request, one frame, one transaction. Depth beats breadth at first.
5. **Use the debugger as a reading tool.** Set a breakpoint somewhere interesting, run, and read the stack. Ten minutes of this beats an hour of static reading, and `rr` (§21.5) lets you scrub backwards through the trace.
6. **Find the hot spots by churn:** `git log --stat` over the last year shows where the work — and the bugs — actually are.

```
rg "class Widget"                # ripgrep for text
-ftime-trace                     # ★ also reveals structure: what
includes what (§19.8)
doxygen with CALL_GRAPH=YES     # generated call graphs
```

⚠ **Templates and macros defeat text search.** A `grep` for a function name will miss every call through a template parameter, a macro, or an operator. Use `clangd`'s find-references.

28.9 Production debugging, logging, crash reporting

Logging

```
SPDLOG_INFO("request {} completed in {}ms", requestId, elapsed); // macro form
```

- **Structured** fields, not prose, so you can query them
- **Correlation IDs** through every layer — the single most useful field
- **Levels used consistently**, and the macro form so disabled levels compile out (§27.2)
- **Async logging** so I/O latency doesn't land on the request path
- **Sampling** for high-frequency events
- ⚠ **Never log secrets, credentials, or PII.** Redact at the logging layer, not at every call site
- ⚠ Formatting is not free — `SPDLOG_DEBUG` with an expensive argument still evaluates the argument unless the macro guards it

Beyond logs: **OpenTelemetry C++** for traces and metrics, **prometheus-cpp**, and continuous profiling (§21.6).

Crash reporting

★ **The correct architecture is out-of-process**, because an in-process handler runs in a corrupted address space and may not be able to allocate or even unwind (§21.2).

Tool	Notes
Crashpad	★ the current Chromium approach; a separate handler process captures a minidump
Breakpad	Crashpad's predecessor, still widely deployed
Sentry (native SDK)	wraps Crashpad/Breakpad, adds symbolication and a UI
Backtrace.io, Bugsnag	commercial equivalents
<code>systemd-coredump</code> / <code>coredumpctl</code>	server-side, no client integration needed

⚠ Keep your symbols, or none of this works

For every build you ship, archive the unstripped binary or its separate debug file, indexed by **build-id** (ELF) or **PDB GUID** (Windows), in a **symbol server**. Without it, a minidump is a list of hexadecimal addresses.

```
# §19.4 – split, ship stripped, keep the .debug
objcopy --only-keep-debug app app.debug
objcopy --strip-debug --add-gnu-debuglink=app.debug app
readelf -n app | grep "Build ID"          # ★ the index key
```

```
minidump_stackwalk crash.dmp ./symbols/      # Breakpad/Crashpad
coredumpctl gdb PID                          # systemd
```

⚠️ **Core dumps contain user data** — buffers, keys, PII. Treat them as sensitive: restrict access, set retention limits, and consider whether you're allowed to collect them at all in your jurisdiction.

Operational practices

- **Feature flags / kill switches** so a bad code path can be disabled without a redeploy
- **Canary deploys** — new binary to 1% first, watch the crash rate
- **Watchdogs** and graceful degradation for native services
- ⚠️ **Reproduce in the release configuration.** A bug that vanishes at `-O0` is usually UB (§14.3), and debugging the wrong build wastes days. Build the release config *with symbols* in CI so you can always attach to what you shipped.

Gotcha checklist for this section

- ☐ `git bisect run` exit **125** means "skip" — essential when commits don't build
- ☐ Large commits make bisect useless
- ☐ Add repo-wide reformat to `.git-blame-ignore-revs`
- ☐ Slow pre-commit hooks get bypassed
- ☐ Identifiers with a double underscore, or a leading underscore + capital, are **reserved**
- ☐ Doc comments should say what the types don't: ownership, lifetime, thread safety, exception guarantee
- ☐ SOVERSION bumps on ABI breaks, not API changes — run `abidiff` in CI
- ☐ LGPL + static linking; Apache-2.0 + GPLv2 incompatibility
- ☐ `grep` misses template, macro, and ADL call sites — use `clangd`
- ☐ In-process crash handlers run in a corrupted address space — capture out-of-process
- ☐ **Without archived symbols, crash reports are worthless**
- ☐ Core dumps contain user data
- ☐ A bug that disappears at `-O0` is usually UB

Exercises worth doing

1. Write a `bisect.sh` for your project handling the build-failure case with exit 125, and run a real `git bisect run`.
2. Bisect a *performance* regression using a callgrind instruction count as the pass/fail criterion.
3. Do a repo-wide `clang-format` in one commit and set up `.git-blame-ignore-revs`. Verify `git blame` still points at real authors.
4. Take a recent PR from your project and review it against the §28.2 checklist. Note how many items the compiler couldn't have caught.
5. Add an `abidiff` job to CI comparing against the last release tag. Then add a member to a class in a public header and confirm it fires.
6. Set up `clangd` on a codebase you don't know. Pick a feature and trace it from entry point to implementation using only go-to-definition.
7. Configure Crashpad or Sentry on a small application, archive the symbols, trigger a crash, and symbolicate the report end to end.
8. Write an ADR for a real decision your team made in the last six months. See how much context you've already forgotten.

Chapter 29 — Study These Real Codebases

What this chapter covers

- *fmt** (clean modern C++) · **spdlog** · **nlohmann/json** (header-only design) · **Catch2** · **abseil-cpp** (industrial-grade) · **folly** (advanced techniques) · **LLVM** (large-scale architecture) · **Chromium** (*base/*) · **libstdc++** / **libc++** (the STL itself) · **Godot** / **o3de** (games) · **Eigen** (expression templates)

The last gap between knowing C++ and being good at it is **reading a lot of it**. Not tutorials — production code, written by people solving problems at a scale where the trade-offs in Part 1 stop being theoretical.

This section is a reading *guide*, not a list. For each codebase: what it uniquely teaches, where to start, and what specifically to look for. Read with a **question**, not from page one.

29.1 How to read someone else's C++

Set up the tools first (§28.8) — without clangd and a *compile_commands.json*, you're reading C++ with grep, and grep loses to templates, macros, overloads, and ADL:

```
cmake -G Ninja -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -B build
ln -sf build/compile_commands.json .
```

Then:

1. **Read with a question**. "How does *fmt* check format strings at compile time?" beats "read *fmt*."
2. **Read the tests** — they show intended usage and how to construct things.
3. **Read the git history** of one interesting file. The *why* is in the commit messages.
4. **Use the debugger as a reading tool** (§28.8) — breakpoint, run, read the stack.
5. **Diff across versions** to see how a design evolved (SFINAE → concepts, C++11 → C++20).

The ladder

	Codebase	Size	Difficulty	Read it for
1	fmt	~15k	★★☆☆☆	clean modern C++, compile-time work
2	spdlog	~10k	★★☆☆☆	composition, policy design
3	nlohmann/json	~25k	★★★★☆	header-only design, API ergonomics
4	Catch2	~20k	★★★★☆	macro/template interplay, self-registration
5	Eigen	~200k	★★★★☆	expression templates
6	Abseil	~200k	★★★★☆	industrial-grade utilities, Swiss tables
7	libc++	~300k	★★★★★	the STL from the inside

	Codebase	Size	Difficulty	Read it for
8	folly	~500k	★★★★★	advanced performance techniques
9	Godot / O3DE	1M+	★★★★☆	game architecture, ECS, custom memory
10	LLVM	3M+	★★★★★	large-scale architecture
11	Chromium base/	~100k of 30M	★★★★☆	cross-platform abstraction at scale

29.2 fmt — clean modern C++

Start here. Small, exceptionally well-written, and you already know what it does (§18.1, §27.2).

Where to start: the base/core header (the public API and the type-erased argument store), then the main `format.h`, then `compile.h`.

What to look for:

- **Type erasure done well** (§8.11) — `format_args` stores heterogeneous arguments in a small inline array with a packed type-tag word, and falls back to dynamic storage only when needed. Compare it to `std::function`'s approach.
- **constexpr format-string checking** (§6.5) — how a compile-time parse turns `std::format("{ }", 1, 2)` into a compile error rather than a runtime throw. This is the cleanest real-world use of `constexpr` you'll find.
- **FMT_COMPILE** — parsing the format string into a type-level representation so the runtime work disappears entirely.
- **The extension point** — `formatter<T>` specialization (§10.3), and how it composes for ranges and chrono types.
- **Grisu/Dragonbox float formatting** — a genuinely hard algorithm, well documented in the source.
- Compile-time/binary-size discipline throughout: the whole library is designed to keep the *instantiated* code small (§10.10).

29.3 spdlog — composition and policy

Where to start: `logger.h`, then `sinks/` (start with `base_sink.h` and `rotating_file_sink.h`), then `async_logger.h`.

What to look for:

- **Sinks as a policy layer** — how one logger fans out to many destinations, and how the locking policy (`std::mutex` vs a null mutex) is a template parameter rather than a runtime branch (§8.9 compile-time seam).
- **Async logging** — a ring buffer plus a background thread, and the overflow policies (block vs drop). A clean, readable producer/consumer (§15).
- **Level checks as macros** — how `SPDLOG_DEBUG` compiles out entirely so a disabled level costs nothing, including the argument evaluation (§28.9).
- **Building on fmt** rather than reimplementing formatting — a good example of dependency layering.

29.4 nlohmann/json — header-only design and API ergonomics

Where to start: the single-header amalgamation for the shape, but read the `include/nlohmann/` tree for the real structure — `json.hpp`, `detail/input/`, `detail/output/`, `detail/conversions/`.

What to look for:

- **Ergonomics as a design goal.** `j["key"] = value` working for every type is a lot of machinery — ADL-based `to_json/from_json` customization points (§10.11), SFINAE constraints, and implicit conversion operators. Note the trade-offs: §9.4's warning about implicit conversions is visible here.
 - **How the single header is generated** from a multi-file source — the amalgamation script and the discipline that makes it possible.
 - **Where the performance goes** (§27.5) — count the allocations in the parse path. This is a useful concrete lesson in what ergonomics can cost.
 - **The macro-based struct binding** (`NLOHMANN_DEFINE_TYPE_INTRUSIVE`) — the pre-reflection workaround, and worth comparing to glaze's approach and to what C++26 reflection makes possible (§18.2).
-

29.5 Catch2 — macros meeting templates

Where to start: `catch_test_macros.hpp`, `catch_assertion_handler`, and the decomposer.

What to look for:

- **Expression decomposition** (§22.3) — how `REQUIRE(a == b)` captures both operands without you writing `REQUIRE_EQ`. It works by exploiting operator precedence: a `Decomposer` object binds `a` with `<=` (lower precedence than `==`), then the resulting proxy's `operator==` captures `b`. Genuinely clever, and directly relevant to §9's operator overloading.
 - **Self-registration** — how a `TEST_CASE` at namespace scope registers itself before `main`, and why this breaks in static libraries (§20.6's `--whole-archive` problem).
 - **SECTION re-entry** — how the same test body is re-executed once per leaf section, tracking the path through a tree.
 - **The v2 → v3 migration:** Catch2 abandoned header-only in v3 for compile-time reasons. Read the rationale — it's a real, documented instance of the §20.9 trade-off.
-

29.6 Eigen — expression templates

The reference implementation of the technique, and the reason to read it even if you never do linear algebra.

Where to start: `Eigen/src/Core/` — `CwiseBinaryOp.h`, `Product.h`, `AssignEvaluator.h`, `DenseBase.h`.

What to look for:

- **How `A * B + C` becomes one loop** (§27.4) — the expression tree built at compile time, and the evaluator that fuses it into a single pass with no temporaries.
- **The cost model in the type system** — Eigen encodes flags (vectorizable, aliasing, linear access) as compile-time traits and dispatches on them.

- **Why `auto` breaks** (§27.4) — you'll see exactly why once you've read `CwiseBinaryOp`: the returned object *references* its operands (§3.4).
- **Explicit vectorization** (§24.8) — `Eigen/src/Core/arch/` has per-ISA packet implementations behind one abstraction.
- **Template metaprogramming at production scale** (§10) — dense, but it's real code solving a real problem, not a toy.

29.7 Abseil — industrial-grade utilities

Where to start: `absl/container/internal/raw_hash_set.h` (Swiss tables), `absl/strings/`, `absl/synchronization/mutex.h`, `absl/base/internal/`.

What to look for:

- ★ **`raw_hash_set` — the Swiss table** (§11.3). Open addressing with a separate array of one-byte control words, probed **16 at a time with SIMD**. Read the header comment; it's one of the best-documented data structures in open source, and it explains exactly why `std::unordered_map` can't be made competitive.
- **`absl::Mutex`** — deadlock detection built into the mutex, plus `Condition` (predicate-based waiting instead of condition variables). Compare with §15.5.
- **String utilities** — `StrCat`'s single-allocation concatenation, `StrSplit`'s lazy view design.
- **Inline-namespace ABI versioning** (§28.6) — how Abseil isolates its ABI so two versions can coexist.
- **`absl::Status/StatusOr`** (§13.6) — a mature error-handling design predating `std::expected`, with attached payloads and error-space conventions.
- **Portability layers** in `absl/base/` — how you actually abstract compiler and platform differences (§19.1).

29.8 folly — advanced performance techniques

Meta's library. Hard to build, hard to read, and full of techniques you won't find elsewhere.

Where to start: `folly/container/F14*.h`, `folly/FBString.h`, `folly/small_vector.h`, `folly/Synchronized.h`, `folly/concurrency/`, `folly/experimental/coro/`.

What to look for:

- **F14 hash maps** — a different SIMD-probing design from Swiss tables, with excellent documentation of the trade-offs.
- **`fbstring`** — three storage modes (small/medium/large) with a COW-ish large mode. Compare with `libc++`'s SSO (§11.9).
- ★ **`Synchronized<T>`** — data and its mutex in one type, accessible only through a locked proxy. The design makes "forgot to take the lock" unrepresentable, and it's an excellent example of §9.7's `operator->` proxy pattern applied to a real safety problem.
- **`MPMCQueue`, `ProducerConsumerQueue`** — production lock-free queues with the reclamation problem actually solved (§15.11).
- **`folly::coro`** — a complete coroutine framework: `Task`, `AsyncGenerator`, cancellation, executor affinity (§16.5).

- **small_vector** — inline storage with configurable thresholds (§24.5).

29.9 libstdc++ / libc++ — the STL from the inside

Read libc++. It's dramatically more readable than libstdc++ (which carries decades of compatibility machinery) and it's a real implementation, not a teaching one.

Where to start: `libcxx/include/` — `vector`, `string`, `memory` (`shared_ptr`'s control block), `algorithm`, `__hash_table`, `optional`, `variant`.

What to look for:

- **vector's growth and exception safety** (§11.1, §13.2) — find `__split_buffer` and see how the strong guarantee is actually implemented, and where `move_if_noexcept` is used (§5.5).
- **string's SSO** (§11.9) — the union of a short and a long representation, and the bit tricks that distinguish them.
- **shared_ptr's control block** (§4.4) — the type-erased deleter, why `make_shared` is one allocation, and the aliasing constructor.
- **variant's implementation** — recursive union storage and the visit dispatch table (§8.9).
- **__attribute__((__abi_tag__))** and the **_LIBCPP_version namespace** — ABI management in practice (§20.7).
- ★ **The naming discipline:** every internal identifier is `__reserved` (§28.4). Reading this teaches you *why* the reserved-identifier rule exists — user macros must not be able to collide with anything here.

29.10 LLVM — large-scale architecture

3M+ lines that stay navigable. The lesson is organizational as much as technical.

Where to start: `llvm/include/llvm/ADT/` (the data structures — read these first, they're small and excellent), then `llvm/lib/IR/`, then one pass in `llvm/lib/Transforms/`.

What to look for:

- ★ **ADT/** — `SmallVector`, `DenseMap`, `StringRef`, `ArrayRef`, `Optional`, `PointerIntPair`, `TinyPtrVector`, `BitVector`. These predate and outperform their standard equivalents, and each header explains its trade-offs. `SmallVector` is probably the single most-copied C++ data structure in existence.
- **isa<> / cast<> / dyn_cast<>** — RTTI-free type discrimination (§8.8), because LLVM builds with `-fno-rtti`. The `Kind`-enum pattern generalized properly.
- **The pass manager** — a plugin architecture with analysis dependencies and invalidation, done at scale.
- **Error/Expected<T>** — LLVM's error handling, which predates `std::expected` and *forces* you to check (an unchecked `Error` aborts in debug builds — §13.6).
- **lit + FileCheck** — testing a compiler by matching against expected output, an entirely different testing model from §22.
- **Coding standards** — LLVM's `CodingStandards.rst` is one of the better-argued style documents in existence.

29.11 Chromium `base/` — cross-platform abstraction at scale

Don't try to read Chromium. Read `base/`, which is a self-contained ~100k-line portability and infrastructure layer.

Where to start: `base/memory/` (`raw_ptr`, `WeakPtr`, `scoped_refptr`), `base/task/`, `base/containers/`, `base/functional/`.

What to look for:

- ★ `raw_ptr<T>` (**MiraclePtr**) — a raw-pointer replacement that detects use-after-free in production with acceptable overhead. This is one of the most significant real answers to §23's memory-safety problem, deployed at browser scale. Read the design docs alongside the code.
- `WeakPtr` — a non-atomic, single-sequence weak reference, much cheaper than `std::weak_ptr` when you don't need thread safety (§4.4).
- The task/sequence model** — `SequencedTaskRunner`, sequence affinity, and `DCHECK_CALLED_ON_VALID_SEQUENCE`: enforcing threading contracts by convention plus assertions rather than locks (§15).
- `CheckedNumeric` — overflow-safe arithmetic (§23.2) in production use.
- Platform abstraction** — how one API covers Windows/POSIX/Mac/Android/Fuchsia without `#ifdef` sprawl in caller code.
- Chromium's C++ style and "banned features" list** — an opinionated, security-driven subset with reasons attached.

29.12 Godot and O3DE — game architecture

Godot (~1M lines, C++17) is the more readable of the two.

Where to start: `core/object/` (`Object`, `ClassDB`), `core/templates/`, `servers/rendering/`, `scene/main/`.

What to look for:

- ★ `ClassDB` — a hand-rolled runtime reflection system: property registration, method binding, signals, and script-language interop, all without language reflection. The clearest large-scale demonstration of what C++26 reflection (§18.2) will replace.
- Custom containers** in `core/templates/` — why a game engine writes its own `Vector`, `HashMap`, and `RID` allocator instead of using the STL (§27.3's EASTL rationale, made concrete).
- The RID handle system** — the generation-counter pattern from §4.8 in production.
- fno-exceptions in practice** (§13.8) — how a large codebase handles errors without them.
- Server/scene split** — a threading architecture where the render server owns its own state.

O3DE (formerly Amazon Lumberyard, from CryEngine) is bigger and messier, but it has a full **ECS** (§24.3) and a serialization/reflection context worth comparing to Godot's `ClassDB`.

29.13 Techniques index — what to read for what

To learn

Read

To learn	Read
RAII wrappers over C APIs (§4.6)	fmt, spdlog sinks, Godot's platform layer
Type erasure (§8.11)	fmt's <code>format_args</code> , <code>folly::Function</code> , libc++'s <code>std::function</code>
Expression templates (§10)	Eigen
Compile-time parsing (§6.5)	fmt (<code>FMT_COMPILE</code> , <code>consteval</code> checks), CTRE
Hash table design (§11.3)	Abseil <code>raw_hash_set</code> , folly F14
Small-buffer optimization (§11.9)	libc++ <code>string</code> , <code>folly::fbstring</code> , <code>SmallVector</code> , <code>absl::InlinedVector</code>
Allocators and arenas (§11.12)	Abseil, protobuf arenas, Godot's memory layer
Lock-free structures (§15.11)	folly <code>MPMCQueue</code> , LLVM
Coroutine frameworks (§16)	folly::coro , Asio, cppcoro
Concepts replacing SFINAE (§10.4→10.5)	diff libc++ or fmt across their C++20 migration
ABI versioning (§20.7, §28.6)	Abseil inline namespaces, libstdc++ <code>__cxx11</code> , libc++ ABI tags
Cross-platform abstraction (§19.1)	Chromium <code>base/</code> , <code>absl/base/</code>
Memory safety in production (§23)	Chromium <code>raw_ptr/MiraclePtr</code>
Reflection before C++26 (§18.2)	Godot <code>ClassDB</code> , Boost.PFR, glaze, magic_enum
Error handling without exceptions (§13.8)	LLVM <code>Error/Expected</code> , <code>absl::StatusOr</code> , Godot
RTTI-free polymorphism (§8.8)	LLVM <code>isa<>/dyn_cast<></code>
Header-only design (§20.9)	nlohmann/json, and Catch2's v3 rationale for abandoning it
Testing a compiler/tool (§22)	LLVM <code>lit</code> + <code>FileCheck</code>
Task scheduling (§15.10)	Chromium <code>base/task/</code> , folly executors, Asio <code>io_context</code>

29.14 A reading plan

Roughly a quarter of deliberate practice, if you want a structure:

Weeks	Do
1–2	Read fmt end to end. Small enough to actually finish. Then implement a mini-formatter with a <code>consteval</code> format check.
3	spdlog — trace one log call from macro to sink to file.
4–5	libc++ : <code>vector</code> , <code>string</code> , <code>shared_ptr</code> , <code>optional</code> . Then implement your own <code>vector</code> with the strong guarantee and correct <code>move_if_noexcept</code> behaviour.

Weeks	Do
6	Catch2's decomposer — then implement expression decomposition yourself.
7–8	Abseil <code>raw_hash_set</code> . Read the header comment, then implement a simplified Swiss table and benchmark it against <code>std::unordered_map</code> .
9	Eigen <code>CwiseBinaryOp</code> and the evaluator. Then build a tiny expression-template vector class.
10	LLVM <code>ADT/</code> — <code>SmallVector</code> , <code>StringRef</code> , <code>DenseMap</code> , <code>PointerIntPair</code> . Implement <code>SmallVector</code> .
11	Chromium <code>base/memory/</code> — <code>raw_ptr</code> and <code>WeakPtr</code> , plus the design docs.
12	folly <code>Synchronized<T></code> and one queue, or Godot <code>ClassDB</code> — pick by your domain.

The pattern that makes this work: **read, then reimplement a simplified version, then benchmark against the original**. Reading alone doesn't stick; the reimplementation is where you discover the cases the real one handles that you didn't think of.

Closing

If you want the compressed version of the whole thing:

1. **Value semantics and RAI** are the foundation; everything else is built on deterministic destruction (§3, §4).
2. **Ownership is a type-level decision**. Express it in the signature and most lifetime bugs become impossible to write (§3.6, §4.8).
3. **The compiler generates, suppresses, and deletes more than you think** — and it never warns you about the expensive cases (§4.9, §5.8, §7.1).
4. **UB is a compile-time licence to assume**. Sanitizers and fuzzing aren't optional tooling; they're how you get most of memory safety back (§14, §22.7).
5. **Templates are a code generator** with their own lookup and overload rules — and monomorphization is free per call site, not in aggregate (§10, §24.9).
6. **Every abstraction has a cost model** — vtables, `shared_ptr`, `std::function`, coroutines, exceptions. Knowing them, and checking the assembly, is the job (§8.2, §4.7, §6.10, §16.8, §13.1, §19.5).
7. **The bottleneck is memory and allocation**, not the algorithm you were worried about (§24).
8. **The ecosystem is half the language**. CMake, sanitizers, clang-tidy, fuzzing, and a multi-compiler CI matrix do more for correctness than any amount of careful typing (Part 2).

Appendix A — The Flag Reference

Everything the book scatters across §14.5, §19, §21.3, §23.6 and §24, collected in one place. Copy from here.

A.1 The everyday development build

```
# GCC / Clang — what you should be compiling with while you work
g++ -std=c++20 -g -O1 \
    -Wall -Wextra -Wpedantic \
    -Wshadow -Wconversion -Wsign-conversion \
    -Wnon-virtual-dtor -Woverloaded-virtual \
    -Wold-style-cast -Wcast-qual -Wdouble-promotion \
    -Wnull-dereference -Wformat=2 -Wimplicit-fallthrough \
    -fsanitize=address,undefined -fno-omit-frame-pointer \
    -D_GLIBCXX_ASSERTIONS \
    main.cpp -o app
```

```
:: MSVC equivalent
cl /std:c++20 /permissive- /W4 /Zi /Od /EHsc /fsanitize=address main.cpp
```

`/permissive-` is not optional on MSVC — without it you are not compiling C++, you are compiling a Microsoft dialect that will reject two-phase-lookup-correct code the moment you port it (§19.1).

The warnings, and what each actually catches

Flag	Catches
<code>-Wall -Wextra</code>	The baseline. Neither is "all".
<code>-Wpedantic</code>	Non-standard extensions you didn't know you were using (§1.3)
<code>-Wshadow</code>	A local hiding a member or outer local — a real bug source in long functions
<code>-Wconversion -Wsign-conversion</code>	Silent narrowing and signed/unsigned mixing (§2.2, §23.2). Noisy on legacy code; worth the noise.
<code>-Wnon-virtual-dtor</code>	Polymorphic base without a virtual destructor (§8.3)
<code>-Woverloaded-virtual</code>	A derived function hiding rather than overriding (§6.1)
<code>-Wold-style-cast</code>	C-style casts (§2.9)
<code>-Wcast-qual</code>	Casting away <code>const</code>
<code>-Wdouble-promotion</code>	Accidental <code>float</code> → <code>double</code> in hot loops
<code>-Wnull-dereference</code>	Provable null dereference
<code>-Wformat=2</code>	<code>printf</code> -family mismatches, including security-relevant ones
<code>-Wimplicit-fallthrough</code>	Missing <code>break</code> ; fix with <code>[[fallthrough]]</code>
<code>-Wuseless-cast</code> (GCC)	Casts that do nothing
<code>-Wlifetime</code> (Clang, partial)	Dangling-pointer heuristics

Flag	Catches
<code>-Wsuggest-override</code> (GCC)	Overrides missing <code>override</code>

Warnings worth *not* enabling globally

`-Weverything` (Clang) is a debugging tool for your warning configuration, not a build setting — it includes mutually contradictory warnings and every experimental check. Run it once, harvest what's useful, and pin that list (§19.2).

Third-party headers

```
-isystem /path/to/dep/include      # suppresses warnings from inside
```

Do this before you enable `-Werror`, or you inherit every dependency's warning debt (§19.2).

A.2 Sanitizers

They are the single biggest correctness lever available in C++, and they are cheap enough to run in CI on every commit (§14.5).

```
# The everyday pair – combine freely
-fsanitize=address,undefined -fno-omit-frame-pointer -g

# Threads: separate build, CANNOT be combined with ASan
-fsanitize=thread

# Uninitialized reads: Clang only, needs an instrumented libc++
-fsanitize=memory -fsanitize-memory-track-origins=2
```

Sanitizer	Finds	Slowdown	Memory
ASan	Heap/stack/global overflow, use-after-free, use-after-return, leaks	~2×	~3×
UBSan	Signed overflow, bad shifts, misaligned access, null deref, bad casts	~1.2×	~1×
TSan	Data races, lock-order inversion	5–15×	5–10×
MSan	Reads of uninitialized memory	~3×	~2×

Sub-checks not on by default

```
-fsanitize=undefined,float-divide-by-zero,unsigned-integer-overflow,implicit-
conversion,local-bounds,nullability
```

```
-fno-sanitize-recover=all           # make UBSan abort instead of printing and
continuing
```

`unsigned-integer-overflow` is *not* UB (§2.2) — it is defined wraparound. Enable it anyway when hunting a bug; disable it when the wraparound is intentional (hashing, PRNGs).

Runtime options

```
export
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1:strict_string_checks=1
:check_initialization_order=1:abort_on_error=1
export UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1
export TSAN_OPTIONS=second_deadlock_stack=1:history_size=7
export LSAN_OPTIONS=suppressions=lsan.supp
```

The CI matrix worth having

Job	Build
1	GCC, <code>-O2</code> , <code>-Werror</code> , all warnings
2	Clang, <code>-O2</code> , <code>-Werror</code> , all warnings
3	Clang, ASan + UBSan, full test suite
4	Clang, TSan, concurrency tests
5	MSVC <code>/W4</code> <code>/permissive-</code> <code>/WX</code>
6	Release build, no sanitizers, benchmarks
7	clang-tidy on the diff
8	Fuzz targets, short run per commit; long run nightly

The reason for jobs 1 and 2 together is that GCC and Clang diagnose genuinely different things; the reason for job 5 is that MSVC's two-phase lookup and ABI expose different bugs again (§19.1).

A.3 Optimization

Flag	Meaning
<code>-O0</code>	No optimization. Debug only. Abstractions are <i>not</i> free here (§1.5).
<code>-O1</code>	Cheap optimizations; still debuggable. Good default for a sanitizer build.
<code>-O2</code>	The production default.

Flag	Meaning
<code>-O3</code>	More inlining and vectorization. Sometimes slower. Measure.
<code>-Os</code>	Optimize for size. Frequently the fastest choice on embedded.
<code>-Ofast</code>	<code>-O3</code> plus <code>-ffast-math</code> . Breaks IEEE semantics — never in a numerics codebase (§26.5).
<code>-Og</code>	Optimize but keep debuggability (GCC).
<code>-g / -g3</code>	Debug info. Independent of <code>-O</code> . Always ship it <i>somewhere</i> (§28.9).
<code>-march=native</code>	Use every instruction this CPU has. Not portable to other machines.
<code>-mtune=native</code>	Schedule for this CPU, keep the baseline ISA. Safe.
<code>-flto / -flto=thin</code>	Link-time optimization; cross-TU inlining and devirtualization (§19.7)
<code>-ffunction-sections -fdata-sections -Wl,--gc-sections</code>	Drop unreferenced code from the binary
<code>-fno-exceptions -fno-rtti</code>	The bans you'll meet in games, embedded, HFT (§13.8, §8.8)
<code>-fno-semantic-interposition</code>	Big win for shared libraries built with <code>-fPIC</code>
<code>-fvisibility=hidden -fvisibility-inlines-hidden</code>	Shrink the exported symbol table (§20.8)

A production build

```
g++ -std=c++20 -O2 -DNDEBUG -g -flto=auto \
    -march=x86-64-v3 -mtune=native \
    -ffunction-sections -fdata-sections -Wl,--gc-sections \
    -fvisibility=hidden -fvisibility-inlines-hidden \
    -fstack-protector-strong -D_FORTIFY_SOURCE=3 \
    -fPIE -pie -Wl,-z,relro,-z,now,-z,noexecstack \
    -o app *.cpp

objcopy --only-keep-debug app app.debug      # keep these forever
objcopy --strip-debug --add-gnu-debuglink=app.debug app
```

`-march=x86-64-v3` (AVX2-era) is usually the right portability/performance compromise for server code; `native` ties the binary to the build machine.

A.4 Hardening

Turn these on in production. They are close to free (§23.6).

Flag	Effect
<code>-D_GLIBCXX_ASSERTIONS</code>	libstdc++ bounds checks on <code>operator[]</code> , iterators. Cheap.
<code>-D_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_FAST</code>	libc++ equivalent (modern spelling)
<code>-D_FORTIFY_SOURCE=3</code>	Compile-time and runtime checks on <code>str*/mem*</code> . Needs <code>-O1+</code> .
<code>-fstack-protector-strong</code>	Stack canaries
<code>-fstack-clash-protection</code>	Stack-clash attacks
<code>-fcf-protection=full</code>	Intel CET / indirect-branch tracking
<code>-fPIE -pie</code>	ASLR for the executable
<code>-Wl,-z,relro,-z,now</code>	Read-only GOT
<code>-Wl,-z,noexecstack</code>	Non-executable stack
<code>-fsanitize=cfi</code> (Clang, needs LTO)	Control-flow integrity
<code>/GS /guard:cf /DYNAMICBASE /NXCOMPAT</code>	MSVC equivalents

A.5 Inspection and diagnosis

```
g++ -E f.cpp           # preprocessor output – see what #include really
                        pulled in
g++ -S -masm=intel f.cpp # assembly
g++ -fverbose-asm -S f.cpp # assembly with source correspondence

nm -C --defined-only libfoo.a | less      # demangled symbols (§19.9)
c++filt _ZNSt6vectorIiSaIiEE9push_backERKi
readelf -d app | grep NEEDED             # shared-library dependencies
objdump -dC --no-show-raw-insn app | less

# Compile-time analysis (§19.8)
clang++ -ftime-trace f.cpp  # → f.json, open in chrome://tracing or Speedscope
g++ -ftime-report f.cpp
clang++ -Xclang -fdump-record-layouts f.cpp
g++ -fdump-lang-class f.cpp  # class layouts, vtables

# Optimization remarks – the compiler explaining its choices (§19.5)
clang++ -O2 -Rpass=inline -Rpass-missed=inline -Rpass-analysis=loop-vectorize
f.cpp
g++ -O3 -fopt-info-vec-missed f.cpp
```


Profiling

```
perf stat -d ./app                                # first question: CPU-bound, memory-
bound, or branch-bound?
perf record -g --call-graph=dwarf ./app && perf report
perf record -e cache-misses,branch-misses ./app
valgrind --tool=callgrind ./app                    # exact, ~50x slow
heaptrack ./app                                    # allocation profile – usually where the
time is (§24.5)
```

A.6 Feature-test macros, not version checks

Never branch on `__cplusplus` (§1.2). Branch on the feature:

```
#include <version>                                // C++20: every library feature-test macro

#if __cpp_lib_expected >= 202202L
#   include <expected>
    template <class T, class E> using Expected = std::expected<T, E>;
#else
#   include <tl/expected.hpp>
    template <class T, class E> using Expected = tl::expected<T, E>;
#endif

#if __cpp_if_consteval >= 202106L
    // ...
#endif
```

Macro	Feature
<code>__cpp_concepts</code>	Concepts
<code>__cpp_lib_ranges</code>	Ranges library
<code>__cpp_impl_coroutine</code> / <code>__cpp_lib_coroutine</code>	Coroutines (language / library)
<code>__cpp_modules</code>	Modules
<code>__cpp_lib_format</code> / <code>__cpp_lib_print</code>	<code>std::format</code> / <code>std::print</code>
<code>__cpp_lib_expected</code>	<code>std::expected</code>
<code>__cpp_lib_mdspan</code> / <code>__cpp_lib_flat_map</code>	C++23 containers
<code>__cpp_explicit_this_parameter</code>	Deducing <code>this</code>
<code>__cpp_lib_generator</code>	<code>std::generator</code>
<code>__cpp_static_assert</code> / <code>__cpp_constexpr</code>	Core-language levels

Appendix B — Feature Index by Standard

What arrived when, and where in this book it's covered. Useful when you inherit a codebase pinned to an old standard and need to know what you're missing — and when you're arguing for an upgrade.

B.1 C++98 / C++03 — the baseline

Classes, inheritance, virtual functions, templates, exceptions, namespaces, the STL (containers, iterators, algorithms), `std::string`, streams, RTTI, the four named casts.

What it lacks that shapes legacy code: no move semantics (so everything copies), no `auto`, no lambdas, no smart pointers except the broken `auto_ptr`, no threading in the standard, no `nullptr`.

If you meet a C++03 codebase, the tell is `std::vector<std::string> v = f();` being an expensive line, functors written as hand-rolled structs, and `NULL` everywhere.

B.2 C++11 — the reboot

The largest single change in the language's history. Everything modern C++ means starts here.

Feature	§
Move semantics , rvalue references, <code>std::move</code> , <code>std::forward</code>	\$5
Lambdas and closures	\$6.8
<code>auto</code> , <code>decltype</code> , trailing return types	\$2.6, \$6.3
Smart pointers : <code>unique_ptr</code> , <code>shared_ptr</code> , <code>weak_ptr</code>	\$4.4
Uniform initialization <code>{}</code> , <code>initializer_list</code>	\$2.4, \$2.5
<code>nullptr</code>	\$3.7
<code>enum class</code>	\$2.10
<code>constexpr</code> (single-return functions only)	\$2.8, \$6.5
<code>static_assert</code>	\$13.9
Variadic templates, parameter packs	\$6.12, \$10.6
<code>= default</code> , <code>= delete</code>	\$7.2
<code>override</code> , <code>final</code>	\$8.4
Delegating and inheriting constructors, NSDMI	\$7.3
Range-based <code>for</code>	\$2.6
The memory model , <code>std::thread</code> , <code>mutex</code> , <code>atomic</code> , <code>future</code>	\$15

Feature	§
<code>noexcept</code>	§6.6
<code>std::array</code> , <code>unordered_*</code> , <code>std::tuple</code>	§11
<code>std::function</code> , <code>std::bind</code>	§6.10
<code><type_traits></code> , <code><chrono></code> , <code><random></code> , <code><regex></code>	§10.8
Alias templates, <code>using</code> over <code>typedef</code>	§2.3
Raw string literals, user-defined literals	§9.8
<code>thread_local</code>	§15.2

If your codebase is stuck below C++11, that is the migration to fight for. Everything else on this list is a refinement.

B.3 C++14 — the completion patch

Feature	§
<code>std::make_unique</code> (the C++11 oversight)	§4.5
Generic lambdas (<code>auto</code> parameters)	§6.8
Init-capture (<code>[p = std::move(p)]</code>)	§6.8
Relaxed <code>constexpr</code> — loops and multiple statements	§6.5
Return-type deduction for normal functions	§6.3
Variable templates	§10.1
<code>std::shared_timed_mutex</code> , <code>std::exchange</code> , <code>std::integer_sequence</code>	§5.9, §15.3
Transparent comparators (<code>std::less<></code>)	§11.3
Digit separators <code>1'000'000</code> , binary literals	§2.1
<code>[[deprecated]]</code>	§6.7

C++14 is a small release that removes most of the friction from using C++11 properly. Adopting it is essentially free.

B.4 C++17 — the practical release

Feature	§
Guaranteed copy elision (RVO is no longer an optimization)	§5.7
Structured bindings	§2.11
<code>if constexpr</code>	§10.8

Feature	§
<code>if/switch</code> with initializer	§2.11
CTAD and deduction guides	§10.7
<code>std::optional</code> , <code>std::variant</code> , <code>std::any</code> , <code>std::string_view</code>	§3.7, §11.9, §8.11
Parallel algorithms (<code>std::execution::par</code>)	§12.9
<code>std::filesystem</code>	—
Inline variables	§6.4, §7.6
Fold expressions	§6.12
<code>[[nodiscard]]</code> , <code>[[maybe_unused]]</code> , <code>[[fallthrough]]</code>	§6.7
<code>std::invoke</code> , <code>std::apply</code>	§6.11
Node handles (<code>extract</code> , <code>merge</code>)	§11.3
<code>try_emplace</code> , <code>insert_or_assign</code>	§11.6
<code>noexcept</code> as part of the type system	§6.6
<code>std::byte</code> , <code>std::launder</code> , <code>std::clamp</code> , <code>std::gcd/lcm</code>	§14.4, §12.5
Class template <code>auto</code> non-type parameters	§10.1
PMR (<code><memory_resource></code>)	§11.12
Removed: <code>auto_ptr</code> , <code>std::random_shuffle</code> , trigraphs	—

C++17 is the practical floor for new work in 2026. Every mainstream compiler and every long-term-support distribution supports it fully.

B.5 C++20 — the second big one

Feature	§
Concepts and constraints	§10.5
Ranges and views	§12.7
Coroutines (<code>co_await</code> , <code>co_yield</code> , <code>co_return</code>)	§16
Modules	§17
<code>operator<=></code> and defaulted comparisons	§9.5, §9.6
<code>std::format</code>	§27.2
<code>std::span</code>	§3.8
<code>std::jthread</code> and <code>stop_token</code>	§15.2
<code>latch</code> , <code>barrier</code> , <code>counting_semaphore</code> , <code>atomic_ref</code> , <code>atomic<shared_ptr></code>	§15.6, §15.8

Feature	§
Designated initializers	§7.8
<code>constexpr</code> , <code>constexpr</code>	§2.8, §6.5
<code>constexpr new/delete</code> , <code>vector</code> , <code>string</code> , algorithms	§10.9
<code>[[no_unique_address]]</code> , <code>[[likely]]</code> / <code>[[unlikely]]</code>	§7.11, §6.7
<code>std::bit_cast</code> , <code><bit></code> (<code>popcount</code> , <code>countl_zero</code> , ...)	§2.9, §14.4
<code>std::erase</code> / <code>std::erase_if</code>	§12.4
Abbreviated function templates (<code>void f(auto x)</code>)	§10.5
Template lambdas, lambdas in unevaluated contexts	§6.8
<code><source_location></code> , <code><numbers></code> , <code>std::ssize</code> , <code>starts_with/ends_with</code>	—
<code>char8_t</code> , <code><version></code>	§2.1, §1.2
Implicit-lifetime types	§14.4

C++20 is where most well-run new codebases sit in 2026. The caveat is modules (§17.5), where the build-system story is still incomplete — everything else on this list is production-ready.

B.6 C++23 — the consolidation release

Feature	§
<code>std::expected<T, E></code>	§13.6
<code>std::print</code> / <code>std::println</code>	§18.1
<code>std::mdspan</code>	§11.10
<code>std::flat_map</code> / <code>flat_set</code>	§11.4
<code>std::generator</code>	§16.4
Deducing <code>this</code> (explicit object parameter)	§7.9
<code>import std;</code>	§17.4
<code>views::zip</code> , <code>enumerate</code> , <code>chunk</code> , <code>slide</code> , <code>join_with</code> , <code>adjacent</code> , <code>cartesian_product</code>	§12.7
Multidimensional <code>operator[]</code>	§9.4
<code>if constexpr</code>	§6.5
<code>std::stacktrace</code>	§18.1, §21.2
<code>std::out_ptr</code> / <code>inout_ptr</code>	§25.2
<code>std::start_lifetime_as</code>	§14.4
<code>std::byteswap</code> , <code>to_underlying</code> , <code>unreachable</code> , <code>move_only_function</code>	§18.1

Feature	§
<code>[[assume]]</code>	\$13.9
Ranges: <code>to<>()</code> , <code>fold_left/fold_right</code>	\$12.7
Static <code>operator()</code> and <code>operator[]</code> , <code>auto(x)</code> decay-copy	\$18.1
Fixed: range-for temporary lifetime	\$3.4

C++23 has no single headline feature; it is a hundred small removals of friction. It is also unusually cheap to adopt because almost nothing in it changes existing semantics.

B.7 C++26 — status and headline features

Feature	§
Static reflection (<code>^^</code> , <code>[: :]</code> , <code><meta></code>)	\$18.2
Contracts (<code>pre</code> , <code>post</code> , <code>contract_assert</code>)	\$13.9, \$18.2
<code>std::execution</code> — senders and receivers	\$15.9, \$18.2
<code>std::simd</code>	\$18.2, \$24.8
<code>std::hive</code>	\$18.2
Erroneous behaviour — reading an uninitialized value is no longer UB	\$18.2
Pack indexing <code>pack...[I]</code>	\$18.2
<code>std::inplace_vector</code> , <code>std::submdspan</code> , <code>std::text_encoding</code>	\$18.2
Structured bindings as condition, in packs, with attributes	\$18.2
<code>constexpr</code> exceptions and <code>constexpr</code> containers of more kinds	\$18.2
<code>= delete("reason")</code> , <code>@/\$/`</code> in the basic character set	\$18.2

Reflection is the change with the largest downstream effect: serialization, ORM layers, enum-to-string, test-case registration and dependency injection are all currently done with macros or a code generator, and all of them collapse into ordinary `constexpr` code once reflection is available (§18.2).

Compiler support is partial and moving; check the feature-test macros (§A.6), not the marketing.

B.8 Choosing a standard, professionally

Situation	Standard
New project, no legacy constraint	C++20 , moving to 23 as your compilers settle
Must support an older LTS distribution or vendor toolchain	C++17

Situation	Standard
Automotive / avionics under MISRA or AUTOSAR	Whatever the certified toolchain supports — often C++14 (§26.3)
Console games	Whatever the platform SDK ships (§26.1)
Library intended for wide consumption	C++17 in the public headers, newer internally behind feature-test macros

The compiler flag is the easy part. The real constraint is your dependency graph, your CI images, and — if you ship a library — the oldest standard your users compile with (§20.7).

Appendix C — Complexity, Invalidation, and Cost Models

The tables you should be able to recall without looking. Detail and caveats live in §11, §12, §24.

C.1 Container operation complexity

Container	Random access	Insert front	Insert back	Insert middle	Erase	Find	Memory overhead
<code>vector</code>	$O(1)$	$O(n)$	$O(1)$ amortized	$O(n)$	$O(n)$	$O(n)$	~0, one allocation
<code>array</code>	$O(1)$	—	—	—	—	$O(n)$	zero
<code>deque</code>	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	block table
<code>list</code>	—	$O(1)$	$O(1)$	$O(1)$ at iterator	$O(1)$	$O(n)$	2 pointers/element
<code>forward_list</code>	—	$O(1)$	—	$O(1)$ after iterator	$O(1)$	$O(n)$	1 pointer/element
<code>map / set</code>	—	—	—	$O(\log n)$	$O(\log n)$	$O(\log n)$, ordered	3 pointers + colour/element
<code>unordered_map / _set</code>	—	—	—	$O(1)$ average	$O(1)$ average	$O(1)$ average	bucket array + 1 pointer/node
<code>flat_map / flat_set</code> (C++23)	$O(1)$	—	—	$O(n)$	$O(n)$	$O(\log n)$	~0, two vectors

Container	Random access	Insert front	Insert back	Insert middle	Erase	Find	Memory overhead
<code>priority_queue</code>	top $O(1)$	—	push $O(\log n)$	—	pop $O(\log n)$	—	~ 0

The complexity table is the *least* useful thing to know about these containers. The useful thing is that `vector` beats `list` for insertion in the middle up to surprisingly large sizes, because the $O(n)$ memmove is contiguous and the $O(1)$ list insert costs a cache miss to reach the node (§24.2). **Measure before believing the asymptotics.**

`std::unordered_map` is $O(1)$ average but slow in absolute terms by mandate: the standard's reference-and-iterator-stability requirements force separate chaining with node allocation (§11.3). If lookup is hot, reach for `absl::flat_hash_map`, `boost::unordered_flat_map`, or `ankerl::unordered_dense`.

C.2 Iterator, pointer, and reference invalidation

The single table most worth memorizing (§11.5).

Container	Operation	Iterators	Pointers/references
<code>vector</code>	<code>push_back/emplace_back</code> with reallocation	all	all
<code>vector</code>	<code>push_back</code> without reallocation	<code>end()</code> only	none
<code>vector</code>	<code>insert/erase</code> at position <code>p</code>	at and after <code>p</code>	at and after <code>p</code>
<code>vector</code>	<code>reserve</code> , <code>shrink_to_fit</code> , <code>resize</code> up	all if capacity changes	same
<code>vector</code>	<code>clear</code>	all	all
<code>deque</code>	<code>push_front/push_back</code>	all iterators	none
<code>deque</code>	<code>insert/erase</code> in the middle	all	all
<code>deque</code>	<code>erase</code> at either end	erased only	erased only
<code>list/forward_list</code>	<code>insert</code>	none	none
<code>list/forward_list</code>	<code>erase</code>	erased only	erased only
<code>map/set/multi*</code>	<code>insert</code>	none	none
<code>map/set/multi*</code>	<code>erase</code>	erased only	erased only
<code>unordered_*</code>	<code>insert</code> with rehash	all iterators	none
<code>unordered_*</code>	<code>insert</code> without rehash	none	none
<code>unordered_*</code>	<code>erase</code>	erased only	erased only
<code>string</code>	anything that changes capacity	all	all

Container	Operation	Iterators	Pointers/references
<code>flat_map/flat_set</code>	any insert or erase	all	all

Two patterns fall out of this table:

- **deque invalidates iterators but not references** on end insertion. This is the exception people forget.
- **Node-based containers (`list`, `map`, `unordered_*`) never invalidate anything except what you erased.** That stability is what you are paying the allocation and cache-miss cost for; if you aren't relying on it, you're paying for nothing (§11.3).

Erasing while iterating, correctly:

```
for (auto it = m.begin(); it != m.end(); )
    if (pred(*it)) it = m.erase(it);    // erase returns the next iterator
    else          ++it;

std::erase_if(m, pred);                // C++20 – just do this instead (§12.4)
```

C.3 Algorithm complexity

Algorithm	Complexity	Notes
<code>sort</code>	$O(n \log n)$	introsort; not stable
<code>stable_sort</code>	$O(n \log n)$ with memory, else $O(n \log^2 n)$	allocates
<code>partial_sort</code>	$O(n \log k)$	top-k
<code>nth_element</code>	$O(n)$ average	the right tool for medians and top-k
<code>find</code> , <code>count</code> , <code>for_each</code> , <code>transform</code>	$O(n)$	
<code>lower_bound</code> , <code>upper_bound</code> , <code>binary_search</code> , <code>equal_range</code>	$O(\log n)$ on random access; $O(n)$ on <code>list/set</code> iterators	the hidden trap
<code>merge</code> , <code>inplace_merge</code>	$O(n)$ / $O(n \log n)$	
<code>unique</code>	$O(n)$	needs a sorted range to be meaningful; pair with <code>erase</code> (§12.4)
<code>rotate</code> , <code>reverse</code>	$O(n)$	<code>rotate</code> returns the new middle
<code>make_heap</code>	$O(n)$	
<code>push_heap</code> / <code>pop_heap</code>	$O(\log n)$	
<code>next_permutation</code>	$O(n)$ per call	

Algorithm	Complexity	Notes
<code>accumulate</code>	$O(n)$, sequential	<code>reduce</code> may reorder — different result for floats (§12.5)
Set operations (<code>set_union</code> etc.)	$O(n+m)$	require sorted inputs

⚠ `map::find` ($O(\log n)$) is not the same as `std::find(m.begin(), m.end(), ...)` ($O(n)$). Member algorithms exist because they can exploit the structure; always prefer the member version when one exists.

C.4 Cost model of C++ abstractions

Approximate, on a modern x86-64 server core. Measure yours (§24.1) — but these are the right order-of-magnitude priors.

Thing	Cost
Non-virtual call, inlined	0
Non-virtual call, not inlined	~1–2 ns
Virtual call	~1–3 ns; the real cost is the lost inlining and the mispredicted indirect branch (§8.2)
<code>std::function</code> call	~2–5 ns plus a possible allocation on construction (§6.10)
Coroutine frame allocation	~20–50 ns unless HALO elides it (§16.8)
<code>shared_ptr</code> copy	~10–20 ns — two atomic RMWs, contended (§4.7)
<code>unique_ptr</code> copy	doesn't compile; move is free
<code>new</code> / <code>delete</code> small object	~20–50 ns, plus fragmentation and a likely cache miss (§24.5)
L1 hit	~1 ns / ~4 cycles
L2 hit	~4 ns
L3 hit	~15–40 ns
Main memory	~80–100 ns — ~300 cycles of doing nothing
Branch mispredict	~15–20 cycles (§24.4)
Uncontended mutex lock/unlock	~20 ns
Contended mutex	microseconds — a syscall and a context switch
Atomic increment, uncontended	~5–10 ns
Atomic increment, contended across cores	100+ ns (§24.2, false sharing)
Thread creation	~10–50 μ s — pool them (§15.10)
Exception throw + catch	~1–10 μ s; zero when not thrown (§13.1)

Thing	Cost
<code>dynamic_cast</code>	~10–100 ns, depends on hierarchy depth (§8.8)
SSO string copy (≤ 15 chars)	no allocation (§11.9)
<code>std::format</code> of a short string	~50–100 ns
A syscall	~100 ns – 1 μ s

The two entries that should shape your designs: main memory is $\sim 100\times$ an L1 hit, and an allocation is $\sim 50\times$ a non-inlined call. That is why data layout (§24.3) and allocation elimination (§24.5) beat almost every other optimization you can make, and why the algorithmic complexity table above so often fails to predict which code is faster.

C.5 Parameter passing — the decision table

From §3.6, condensed:

You want	Signature
Read a cheap value (<code>int</code> , <code>double</code> , pointer, small trivially-copyable struct)	<code>void f(T x)</code>
Read a large object without copying	<code>void f(const T& x)</code>
Read a string you won't store	<code>void f(std::string_view s)</code> ⚠ never outlive the source
Read a contiguous sequence	<code>void f(std::span<const T> s)</code>
Take ownership	<code>void f(T x)</code> and <code>std::move</code> into place, or <code>void f(std::unique_ptr<T> p)</code>
Share ownership (genuinely)	<code>void f(std::shared_ptr<T> p)</code>
Observe without owning	<code>void f(T* p)</code> or <code>void f(T& r)</code> — <code>gsl::not_null<T*></code> when null is invalid
Modify in place	<code>void f(T& x)</code>
Forward to something else	<code>template <class T> void f(T&& x) + std::forward<T>(x)</code>
Optional value	<code>void f(std::optional<T> x)</code> or an overload
Return anything	<code>T f()</code> — by value, always. Copy elision makes it free (§5.7)

⚠ `void f(const std::shared_ptr<T>&)` is the wrong default: it says "I might share this" but costs a pointer chase and prevents the caller from passing a `unique_ptr`. If the function only *uses* the object, take `const T&` and let the caller keep ownership (§4.8).

Appendix D — The Master Gotcha Checklist

Every chapter ends with its own checklist. This is the consolidated version — the one to read before a code review, before shipping, or the night before an interview. Each item points at the section that explains it.

D.1 Lifetime and ownership

- ❑ A `std::string_view` or `std::span` outliving the buffer it views — including the classic `sv = returns_string()` (§11.9, §3.8)
- ❑ A reference or pointer to an element of a `vector` that later grew (§11.5)
- ❑ A lambda capturing by reference that outlives the enclosing scope — **the single most common C++ bug** (§6.9)
- ❑ A lambda capturing `this` when the object may die first; use `[*this]` or a `weak_ptr` (§6.9)
- ❑ A reference parameter to a coroutine — the argument dies at the first suspension (§16.7)
- ❑ A dangling iterator after `erase`, `insert`, or `rehash` (§11.5)
- ❑ Returning a reference or pointer to a local (§3.5)
- ❑ Lifetime extension **not** applying: through a function return, to a reference member, to a subobject bound in the wrong place (§3.4)
- ❑ A `shared_ptr` cycle leaking — break it with `weak_ptr` (§4.4)
- ❑ Two `shared_ptr` control blocks for one object, created by passing a raw pointer twice (§4.4)
- ❑ A view over a temporary in a range pipeline (`views::filter` over a returned container) (§12.8)

D.2 Initialization and types

- ❑ `T x;` at block scope leaves fundamental types **uninitialized** — `T x{}`; (§2.4)
- ❑ `std::vector<int> v(3, 1)` is three ones; `std::vector<int> v{3, 1}` is `{3, 1}` — the `initializer_list` hijack (§2.5)
- ❑ The most vexing parse: `Widget w();` declares a function (§2.5)
- ❑ Member initialization runs in **declaration order**, not the order you wrote in the init list (§7.3)
- ❑ Static initialization order fiasco across translation units — use a function-local static (§1.6, §7.6)
- ❑ Signed overflow is UB; unsigned wraps (§2.2)
- ❑ Mixed signed/unsigned comparison — use `std::cmp_less` (§2.2, §23.2)
- ❑ Narrowing in a C-style cast that `{}` would have caught (§2.5, §2.9)
- ❑ `auto` copying when you meant `auto&` — especially in range-for over a map (§2.6)
- ❑ `decltype(auto)` and the parenthesis trap: `return (x);` returns a reference (§6.3)

D.3 Copies, moves, and special members

- ❑ Declaring a destructor silently **suppresses the move operations** — every "move" becomes a copy, with no warning (§4.9)
- ❑ `std::move` on a `const` object silently copies (§5.2)
- ❑ A move constructor that isn't `noexcept` makes `vector` reallocation copy instead of move (§5.5)
- ❑ `return std::move(local)` disables NRVO — a pessimization, not an optimization (§5.8)
- ❑ Using a moved-from object for anything but assignment or destruction (§5.6)
- ❑ A polymorphic base class without a `virtual` destructor (§8.3)
- ❑ Rule of Zero violated: a class managing a resource *and* holding other members (§4.9)

- ☐ Slicing on assignment or pass-by-value of a derived object (§8.6)
- ☐ A perfect-forwarding constructor hijacking the copy constructor (§5.4)

D.4 Templates and overloads

- ☐ Missing `typename` / `template` disambiguators on dependent names (§10.2)
- ☐ A dependent base class hiding its members — needs `this->` or a `using` (§10.2)
- ☐ Function templates cannot be partially specialized; overload instead (§10.3)
- ☐ The Dimov/Abrahams trap: specialization chosen after overload resolution, not during (§10.3)
- ☐ `T&&` in a deduced context is a **forwarding** reference; in a non-deduced one it is an rvalue reference (§5.3)
- ☐ Forwarding the same parameter twice (§5.4)
- ☐ `static_assert(false)` in the `else` of an `if constexpr` fires unconditionally before C++23 (§10.8)
- ☐ Default arguments are bound **statically**, so a virtual override's default is ignored (§6.2)
- ☐ Hiding rather than overriding — always write `override` (§6.1, §8.4)
- ☐ CTAD picking a surprising specialization (§10.7)

D.5 The standard library

- ☐ `std::vector<bool>` is not a container of `bool` (§11.11)
- ☐ `unordered_map::operator[]` **inserts** on a missing key; `at/find` don't (§11.6)
- ☐ `map::operator[]` default-constructs — a silent mutation in what looks like a read (§11.6)
- ☐ The erase-remove idiom: `remove` alone doesn't erase (§12.4)
- ☐ `std::accumulate`'s init type determines the arithmetic — `0` makes it integer (§12.5)
- ☐ `reduce` may reorder, so floating-point results differ from `accumulate` (§12.5)
- ☐ A comparator that isn't a **strict weak ordering** is UB, not just wrong output — `<=` in a sort predicate will read out of bounds (§12.6)
- ☐ `std::sort` is not stable; `std::stable_sort` is (§12.3, §C.3)
- ☐ Binary search on a non-random-access iterator degrades to $O(n)$ (§C.3)
- ☐ Heterogeneous lookup needs a transparent comparator, or every `find(const char*)` allocates a `string` (§11.3)
- ☐ Views are lazy — side effects in a `transform` run at iteration time, and possibly more than once (§12.7)

D.6 Concurrency

- ☐ A data race is UB, not "an occasionally wrong value" (§15.1)
- ☐ `std::lock_guard(mtx);` with no variable name — an unnamed temporary that unlocks immediately (§15.4)
- ☐ `condition_variable::wait` without a predicate — spurious wakeups are real (§15.5)
- ☐ `std::thread` destroyed while joinable calls `std::terminate` — use `std::jthread` (§15.2)
- ☐ `std::async` with the default policy may run synchronously; the returned future's destructor blocks (§15.7)
- ☐ `volatile` is for MMIO, not threading — no atomicity, no ordering (§15.6, §26.3)
- ☐ `const` member functions must be safe to call concurrently — the standard library assumes it (§7.5)
- ☐ False sharing between adjacent atomics — pad to `hardware_destructive_interference_size` (§15.10, §24.2)

- ☐ `shared_ptr`'s **control block** is thread-safe; the **pointee** is not; and the `shared_ptr` object itself is not (§4.7)
- ☐ Lock ordering inversion — use `std::scoped_lock` with all mutexes at once (§15.4)

D.7 Errors and undefined behaviour

- ☐ An exception escaping a destructor calls `std::terminate` (§13.4)
- ☐ An exception escaping a `noexcept` function calls `std::terminate` (§13.4)
- ☐ An exception crossing a C callback boundary is UB — catch everything at the boundary (§13.10, §25.4)
- ☐ `catch (std::exception e)` slices; catch by `const&` (§13.1, §8.6)
- ☐ `assert` compiles away under `NDEBUG` — never put side effects inside one (§13.9)
- ☐ Strict aliasing violated by a `reinterpret_cast` type pun — use `std::bit_cast` or `memcpy` (§14.4, §2.9)
- ☐ Reading an uninitialized value (UB before C++26, erroneous behaviour after) (§14.2, §18.2)
- ☐ Signed overflow, out-of-bounds, null dereference — the optimizer assumes they cannot happen and deletes your checks (§14.3)
- ☐ "Time travel": UB later in a function can affect observable behaviour *before* it (§14.3)
- ☐ `const_cast`-ing away `const` on a genuinely `const` object, then writing (§14.2)

D.8 Build, ABI, and shipping

- ☐ Mixing standard-library versions or `_GLIBCXX_USE_CXX11_ABI` settings across a link (§20.7, §25.9)
- ☐ Passing `std::string` or `std::vector` across a shared-library boundary compiled with a different compiler (§25.9)
- ☐ Changing a class's private members and breaking ABI for every existing caller (§20.7)
- ☐ A static library dropping objects whose symbols are never referenced — self-registration silently stops working (§20.6)
- ☐ Windows DLLs export nothing by default; `-fvisibility=hidden` on ELF changes what's exported (§20.8)
- ☐ `-Ofast/-ffast-math` silently breaking IEEE semantics in numerical code (§A.3, §26.5)
- ☐ Shipping a stripped binary with no saved symbols — every future crash report is useless (§28.9)
- ☐ ODR violation from the same inline function compiled with different `NDEBUG` or different flags (§1.4, §6.4)

D.9 The five-minute pre-review pass

If you only have time for one sweep over a diff, look for these:

1. **Every raw `new/delete`, `malloc/free`.** Should be a container or a smart pointer (§4).
2. **Every reference-capturing lambda that is stored, queued, or returned** (§6.9).
3. **Every class with a destructor.** Does it need the other four? Did it just lose its moves? (§4.9)
4. **Every `string_view`, `span`, and iterator that crosses a function boundary or gets stored** (§3.8, §11.9).
5. **Every new `shared_ptr`.** Is the sharing real, or is this a `unique_ptr` that someone couldn't be bothered to reason about? (§4.8)

Appendix E — Glossary

Terms in the order you'll meet them in C++ conversation, not alphabetical order within categories. Section references point at the full treatment.

Core language

ODR (One Definition Rule) — every entity may have exactly one definition across the whole program; inline functions and templates may be defined in many translation units, but every definition must be token-for-token identical. Violations are IFNDR: no diagnostic required, and the linker picks one arbitrarily. §1.4

IFNDR — "ill-formed, no diagnostic required". The standard says your program is broken but does not require the compiler to tell you. Worse than UB in practice, because there is no sanitizer for it. §1.4

Translation unit (TU) — one source file after preprocessing. The compilation atom. §1.4

UB (undefined behaviour) — a construct on which the standard imposes no requirements. Not "it does something random"; it is a licence for the optimizer to assume the construct never happens, which is why UB deletes your null checks. §14

Unspecified behaviour — the implementation picks from several valid options and need not document which (e.g. argument evaluation order pre-C++17). **Implementation-defined behaviour** — same, but must be documented (e.g. `sizeof(int)`). §14.1

Erroneous behaviour (C++26) — a new category between defined and undefined: the program is wrong, the value is unspecified, implementations are encouraged to diagnose, but the optimizer may not assume it away. §18.2

Zero-overhead principle — you don't pay for what you don't use, and what you do use you couldn't hand-code better. §1.5

RAII — a resource's lifetime is tied to an object's lifetime: acquire in the constructor, release in the destructor. The single most important idea in C++. §4.1

Value semantics — objects behave like values: assignment copies, and there is no hidden sharing. The default in C++, unlike most managed languages. §3.1

Values, references, moves

Value category — every expression is an lvalue, xvalue, or prvalue. **glvalue** = lvalue or xvalue (has identity); **rvalue** = xvalue or prvalue (can be moved from). §3.2

Lvalue — has identity, can't be moved from implicitly. **Prvalue** — a pure value, no identity yet. **Xvalue** — has identity and is expiring; what `std::move` produces. §3.2

std::move — an unconditional cast to an rvalue reference. It moves nothing. §5.2

std::forward — a conditional cast used with forwarding references to preserve the caller's value category. §5.4

Forwarding (universal) reference — `T&&` where `T` is a template parameter deduced from that same argument. Binds to anything. Not the same as an rvalue reference. §5.3

Reference collapsing — `& & → &`, `& && → &`, `&& & → &`, `&& && → &&`. The rule that makes perfect forwarding work. §3.5

Copy elision / RVO / NRVO — the compiler constructing a returned object directly in the caller's storage. Guaranteed for prvalues since C++17; permitted but not required for named locals. §5.7

Moved-from state — valid but unspecified. You may destroy or assign to the object; you may not assume anything about its value. §5.6

Rule of Zero / Three / Five — design classes so the compiler generates all special members (Zero); if you write one of destructor/copy-ctor/copy-assign, you probably need all three; with moves, all five. §4.9

Classes and polymorphism

Special member functions — default constructor, destructor, copy constructor, copy assignment, move constructor, move assignment. The compiler generates, suppresses, and deletes these according to rules that surprise people. §7.1

Vtable / vptr — the per-class table of virtual function pointers, and the per-object pointer to it. The mechanism of dynamic dispatch. §8.2

Slicing — copying a derived object into a base object, losing everything derived. §8.6

CRTP — the Curiously Recurring Template Pattern: `class D : public Base<D>`. Static polymorphism with no vtable. §8.10

Type erasure — hiding a concrete type behind a uniform interface at runtime while keeping value semantics; how `std::function` and `std::any` work. §8.11

NVI (non-virtual interface) — public non-virtual functions calling private virtual ones, so the base controls the algorithm and derived classes fill in steps. §8.5

EBO / `[[no_unique_address]]` — empty base optimization: an empty base or member need occupy no storage. §7.11

Copy-and-swap — implement assignment by constructing a copy and swapping. Gives the strong exception guarantee and handles self-assignment for free. §7.10

Templates

Instantiation — generating concrete code from a template for specific arguments. Templates are code generators, not compiled once. §10.2

Two-phase lookup — non-dependent names are looked up at template definition time, dependent names at instantiation time. The source of most portability surprises between MSVC and GCC/Clang. §10.2, §1.3

ADL (argument-dependent lookup) — unqualified function names are also looked up in the namespaces of their arguments. Why `swap(a, b)` finds the right `swap`. §6.1

SFINAE — "substitution failure is not an error": a failed substitution removes an overload rather than failing the compile. Largely obsoleted by concepts. §10.4

Concept — a named compile-time predicate on types, used to constrain templates and produce readable errors. §10.5

Subsumption — the partial ordering by which the compiler decides one constrained overload is more specialized than another. §10.5

CTAD — class template argument deduction: `std::vector v{1,2,3}` deduces `vector<int>`. §10.7

Monomorphization — one instantiation of the generated code per distinct set of template arguments. The reason templates are fast and the reason they bloat binaries and compile times. §10.10, §24.9

Niebloid / CPO — a function *object* (rather than a function template) used for range algorithms, so ADL cannot hijack the call. Why `std::ranges::sort` is an object. §10.11

Library and runtime

SSO — small string optimization: short strings live inside the `std::string` object, with no allocation. Typically ≤ 15 chars on libstdc++. §11.9

PMR — polymorphic memory resources: runtime-polymorphic allocators that don't change the container's type. §11.12

View — a lazy, non-owning range adaptor. Cheap to copy, dangerous to outlive its source. §12.7

Borrowed range — a range whose iterators remain valid after the range object itself dies. §12.8

Projection — a callable applied to each element before a range algorithm's predicate sees it: `ranges::sort(v, {}, &Employee::name)`. §12.6

Strict weak ordering — the contract every comparator must satisfy (irreflexive, asymmetric, transitive, with transitive incomparability). Violating it is UB. §12.6

Exception safety guarantees — *no-throw*: never throws. *Strong*: the operation either succeeds or leaves everything as it was. *Basic*: no leaks and all invariants hold, but state may have changed. §13.2

Happens-before — the partial order that determines whether one thread's write is visible to another's read. If two conflicting accesses are not ordered by it, you have a data race. §15.1

Memory order — `relaxed`, `acquire`, `release`, `acq_rel`, `seq_cst`. The synchronization strength attached to an atomic operation. §15.6

False sharing — two threads writing to distinct variables in the same cache line, forcing the line to bounce between cores. §15.10, §24.2

ABA problem — a lock-free algorithm reading value A, being preempted while another thread changes it to B and back to A, and wrongly concluding nothing changed. §15.11

Coroutine — a function whose body contains `co_await`, `co_yield`, or `co_return`, and which can suspend and resume. Stackless in C++: the frame is heap-allocated unless the compiler elides it (HALO). §16

Module — a C++20 compilation unit with explicit exports, replacing textual `#include`. §17

Build and shipping

ABI (application binary interface) — the binary contract: name mangling, class layout, calling conventions, vtable layout, exception mechanics. Vastly more fragile in C++ than in C, because layout and inlining leak into it. §20.7

Name mangling — encoding a function's namespace, class, and parameter types into its linker symbol, so overloads can coexist. `c++filt` reverses it. §19.9

LTO — link-time optimization: whole-program inlining and devirtualization at link time. §19.7

extern "C" — suppress mangling and use the C calling convention. The universal interop boundary. §25.1

PGO / BOLT — profile-guided optimization at compile time, and binary layout optimization after linking. §24.12

Sanitizer — a compiler instrumentation pass that detects a bug class at runtime: ASan (memory errors), UBSan (undefined behaviour), TSan (races), MSan (uninitialized reads). §14.5

Fuzzing — feeding a program automatically mutated inputs under coverage guidance to find crashes. Essential for anything that parses untrusted input. §22.7

Freestanding — an implementation without a hosted runtime: no OS, a reduced standard library. The embedded world. §26.3

Appendix F — A Starter Project

Everything Part 2 argues for, wired together in one skeleton you can copy. This is the setup the book assumes you have when it says "run it under sanitizers" or "check the assembly".

F.1 Layout

```
myproject/
├── CMakeLists.txt
├── CMakePresets.json
├── .clang-format
├── .clang-tidy
├── .editorconfig
├── .gitignore
├── .pre-commit-config.yaml
├── include/myproject/      # public headers – namespaced directory, always
│   └── engine.hpp
├── src/
│   ├── engine.cpp
│   └── main.cpp
└── tests/
```

```

├── CMakeLists.txt
├── engine_test.cpp
├── bench/
│   └── engine_bench.cpp
├── fuzz/
│   └── parse_fuzzer.cpp
├── cmake/
│   └── CompilerWarnings.cmake
└── .github/workflows/ci.yml

```

The `include/myproject/` nesting is not decoration: it means consumers write `#include <myproject/engine.hpp>`, which cannot collide with anyone else's `engine.hpp` (§20.4).

F.2 CMakeLists.txt

```

cmake_minimum_required(VERSION 3.25)
project(myproject VERSION 0.1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)           # -std=c++20, not -std=gnu++20
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)   # powers clangd and clang-tidy (§20.10)

include(cmake/CompilerWarnings.cmake)

# ---- the library -----
add_library(engine)
add_library(myproject::engine ALIAS engine)      # namespaced alias (§20.1)

target_sources(engine
    PRIVATE src/engine.cpp
    PUBLIC FILE_SET HEADERS BASE_DIRS include FILES include/myproject/engine.hpp)

target_compile_features(engine PUBLIC cxx_std_20)
target_link_libraries(engine PRIVATE project_warnings)

# ---- the executable -----
add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE myproject::engine project_warnings)

# ---- tests -----
include(CTest)
if(BUILD_TESTING)
    add_subdirectory(tests)
endif()

# ---- install -----
include(GNUInstallDirs)
install(TARGETS engine EXPORT myprojectTargets FILE_SET HEADERS)
install(EXPORT myprojectTargets

```

```

NAMESPACE myproject::
DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/myproject)

```

Note what is *not* here: no `include_directories`, no `add_definitions`, no `set(CMAKE_CXX_FLAGS ...)`. Those are the 2010-era global commands that leak into every target (§20.1).

cmake/CompilerWarnings.cmake

```

add_library(project_warnings INTERFACE)

if(MSVC)
    target_compile_options(project_warnings INTERFACE /W4 /permissive- /w14640
/Zc:__cplusplus)
else()
    target_compile_options(project_warnings INTERFACE
        -Wall -Wextra -Wpedantic
        -Wshadow -Wconversion -Wsign-conversion
        -Wnon-virtual-dtor -Woverloaded-virtual -Wold-style-cast
        -Wcast-qual -Wdouble-promotion -Wnull-dereference
        -Wformat=2 -Wimplicit-fallthrough)
    if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
        target_compile_options(project_warnings INTERFACE -Wuseless-cast -Wsuggest-
override)
    endif()
endif()

option(WARNINGS_AS_ERRORS "Treat warnings as errors" OFF)
if(WARNINGS_AS_ERRORS)
    target_compile_options(project_warnings INTERFACE
        $<IF:$<CXX_COMPILER_ID:MSVC>, /WX, -Werror>)
endif()

```

Warnings as a linkable INTERFACE target, not global flags, so third-party code brought in with `FetchContent` doesn't inherit them (§20.1).

F.3 CMakePresets.json

```

{
  "version": 6,
  "configurePresets": [
    {
      "name": "base", "hidden": true,
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/${presetName}",
      "cacheVariables": { "CMAKE_EXPORT_COMPILE_COMMANDS": "ON" }
    },
    {
      "name": "debug", "inherits": "base",

```

```

    "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug" }
  },
  {
    "name": "asan", "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "RelWithDebInfo",
      "CMAKE_CXX_FLAGS": "-fsanitize=address,undefined -fno-omit-frame-pointer -fno-sanitize-recover=all -D_GLIBCXX_ASSERTIONS",
      "CMAKE_EXE_LINKER_FLAGS": "-fsanitize=address,undefined"
    }
  },
  {
    "name": "tsan", "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "RelWithDebInfo",
      "CMAKE_CXX_FLAGS": "-fsanitize=thread -fno-omit-frame-pointer",
      "CMAKE_EXE_LINKER_FLAGS": "-fsanitize=thread"
    }
  },
  {
    "name": "release", "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "RelWithDebInfo",
      "CMAKE_INTERPROCEDURAL_OPTIMIZATION": "ON",
      "WARNINGS_AS_ERRORS": "ON"
    }
  }
],
"buildPresets": [
  { "name": "debug", "configurePreset": "debug" },
  { "name": "asan", "configurePreset": "asan" },
  { "name": "release", "configurePreset": "release" }
],
"testPresets": [
  { "name": "asan", "configurePreset": "asan",
    "output": { "outputOnFailure": true },
    "execution": { "noTestsAction": "error" } }
]
}

```

```
cmake --preset asan && cmake --build --preset asan && ctest --preset asan
```

Presets are the fix for the "what were the seventeen flags again" problem, and they make the CI configuration identical to the local one (§20.1).

F.4 Tests

```
# tests/CMakeLists.txt
include(FetchContent)
FetchContent_Declare(googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG v1.15.2
  SYSTEM)          # SYSTEM: don't inherit our warnings (§19.2)
FetchContent_MakeAvailable(googletest)

add_executable(engine_test engine_test.cpp)
target_link_libraries(engine_test PRIVATE myproject::engine GTest::gtest_main
project_warnings)

include(GoogleTest)
gtest_discover_tests(engine_test)
```

Pin the tag. `GIT_TAG main` is a build that stops reproducing the moment upstream commits (§20.3).

F.5 .clang-format and .clang-tidy

```
# .clang-format
BasedOnStyle: LLVM
Standard: c++20
IndentWidth: 4
ColumnLimit: 100
PointerAlignment: Left
AlwaysBreakTemplateDeclarations: Yes
AllowShortFunctionsOnASingleLine: Empty
IncludeBlocks: Regroup
```

```
# .clang-tidy
Checks: >
  bugprone-*,
  cppcoreguidelines-*,
  modernize-*,
  performance-*,
  readability-*,
  concurrency-*,
  -modernize-use-trailing-return-type,
  -readability-magic-numbers,
  -cppcoreguidelines-avoid-magic-numbers,
  -readability-identifier-length
WarningsAsErrors: 'bugprone-*,concurrency-*'
HeaderFilterRegex: '(include|src)/.*'
```

Enable `WarningsAsErrors` narrowly at first — `bugprone-*` and `concurrency-*` almost never produce false positives worth arguing about, whereas `readability-*` will start a style war on day one (§21.7).

Adopting on an existing codebase: run it on the **diff only** (`clang-tidy-diff.py`), not the whole tree, or the first run produces ten thousand findings and everyone turns it off (§21.7).

F.6 CI

```
# .github/workflows/ci.yml
name: ci
on: [push, pull_request]

jobs:
  build:
    strategy:
      fail-fast: false
    matrix:
      include:
        - { os: ubuntu-latest, preset: release, cc: gcc-13, cxx: g++-13 }
        - { os: ubuntu-latest, preset: release, cc: clang-18, cxx: clang++-18 }
        - { os: ubuntu-latest, preset: asan, cc: clang-18, cxx: clang++-18 }
        - { os: ubuntu-latest, preset: tsan, cc: clang-18, cxx: clang++-18 }
        - { os: macos-latest, preset: release, cc: clang, cxx: clang++ }
        - { os: windows-latest, preset: release, cc: cl, cxx: cl }
    runs-on: ${ matrix.os }
    env:
      CC: ${ matrix.cc }
      CXX: ${ matrix.cxx }
    steps:
      - uses: actions/checkout@v4
      - uses: lukka/get-cmake@latest
      - run: cmake --preset ${ matrix.preset }
      - run: cmake --build --preset ${ matrix.preset }
      - run: ctest --preset ${ matrix.preset } --output-on-failure
```

Three compilers is the point (§19.1). Two of them will find things the third silently accepted.

F.7 A fuzz target

```
// fuzz/parse_fuzzer.cpp – build with: clang++ -fsanitize=fuzzer,address,undefined
#include <myproject/engine.hpp>
#include <cstdint>
#include <cstddef>

extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {
    myproject::parse(std::string_view(reinterpret_cast<const char*>(data), size));
    return 0;
}
```

If your code parses anything that came from outside the process — a file format, a network message, a config file — this file is the highest-value 8 lines in the whole project (§22.7).

F.8 .pre-commit-config.yaml

```
repos:
- repo: https://github.com/pre-commit/mirrors-clang-format
  rev: v18.1.8
  hooks:
    - id: clang-format
      types_or: [c++, c]
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-merge-conflict
```

Format in a hook, not in review. Nobody should ever spend a review comment on brace placement (§28.3).

Appendix G — Self-Assessment

Not trivia. Each of these is a question a working C++ engineer answers by reasoning, and each maps to a section you can go back to. Answer out loud; if the explanation comes out muddy, that's the section to reread.

G.1 Junior — you should be able to answer all of these

1. What does `std::move` actually do? §5.2
2. When would you use `unique_ptr` versus `shared_ptr`? §4.4
3. Why does a polymorphic base class need a virtual destructor? §8.3
4. What is the difference between `T x;` and `T x{}`? §2.4
5. What does `const` on a member function mean? §7.5
6. Why is `std::vector` the default container? §11.1
7. What is RAII, and what problem does it solve? §4.1
8. What happens to iterators when a `vector` grows? §11.5
9. What is the difference between `push_back` and `emplace_back`? §11.6
10. What does `explicit` prevent? §7.4
11. Why catch exceptions by `const&`? §13.1
12. What is a dangling reference, and name three ways to create one. §3.5

G.2 Mid-level

13. Explain the Rule of Five and why the Rule of Zero is better. §4.9
14. Why does declaring a destructor make your class slower? §4.9

15. What is a forwarding reference, and how does it differ from an rvalue reference? §5.3
16. Why must a move constructor be `noexcept` for `vector` to use it? §5.5
17. What's wrong with `return std::move(local);`? §5.8
18. What is the erase-remove idiom, and what replaced it? §12.4
19. When is `std::unordered_map` the wrong choice? §11.3
20. How does `std::function` work, and what does it cost? §6.10
21. Why is `std::vector<bool>` a problem? §11.11
22. What exactly makes `string_view` dangerous? §11.9
23. Describe the three exception safety guarantees. §13.2
24. What is a data race, and why is "it usually works" not a defence? §15.1
25. Why should `condition_variable::wait` always take a predicate? §15.5
26. What is the static initialization order fiasco, and how do you avoid it? §1.6
27. What does `if constexpr` do that a runtime `if` cannot? §10.8
28. What is slicing, and how do you prevent it? §8.6

G.3 Senior

29. Walk through the special-member generation and suppression rules. §7.1
30. Explain two-phase lookup and where MSVC historically differed. §10.2, §1.3
31. When would you reach for CRTP over a virtual function, and what does C++23 change? §8.10, §7.9
32. How does type erasure work — implement the concept/model idiom. §8.11
33. Explain acquire/release ordering and when `seq_cst` is overkill. §15.6
34. What is false sharing, and how do you detect and fix it? §15.10, §24.2
35. Why does UB let the optimizer delete a null check you wrote? §14.3
36. Describe the ABI hazards of exposing `std::string` across a shared-library boundary. §20.7, §25.9
37. How do concepts get ordered against each other during overload resolution? §10.5
38. When does a coroutine frame get heap-allocated, and when doesn't it? §16.8
39. What breaks when a defaulted `operator<=>` meets a member without one? §9.5
40. Explain the difference between `accumulate` and `reduce` for floating point. §12.5
41. How would you eliminate allocation from a hot path, in priority order? §24.5
42. What does LTO enable that per-TU optimization cannot? §19.7
43. Given a latency regression with no obvious algorithmic cause, what do you measure first? §24.1
44. Design an API where misuse is a compile error rather than a runtime bug. §3.6, §4.8

G.4 The design questions

These have no single right answer; they're what senior interviews and design reviews actually turn on.

45. You own a widely used library. A caller needs a new field in a public struct. What are your options and what does each cost? §20.7, §28.6
46. Your team bans exceptions. What do you use for error handling, and what do you lose? §13.8, §13.6
47. A codebase has 400 `shared_ptr`s and a suspected leak. How do you approach it? §4.7, §21.3
48. You must call a C library from modern C++ and expose the result to Python. Sketch both boundaries. §25.2, §25.5
49. Compile times have reached 40 minutes. What do you do, in order? §19.8, §10.10
50. When is the right answer "don't use C++ for this"? §23.8

Appendix H — Further Reading

Ordered by when it becomes useful, not by fame.

Books

- **Bjarne Stroustrup, *A Tour of C++ (3rd ed.)*** — the fastest accurate overview of modern C++. Read it in a weekend.
- **Scott Meyers, *Effective Modern C++*** — still the best treatment of C++11/14 move semantics, forwarding, and `auto`, and everything in it remains true. Dated only in what it omits.
- **Nicolai Josuttis, *C++ Move Semantics: The Complete Guide*** — go here when \$5 isn't enough.
- **Nicolai Josuttis, *C++20: The Complete Guide*** — the reference for concepts, ranges and coroutines in one place.
- **Rainer Grimm, *C++ Core Guidelines Explained*** — the reasoning behind the guidelines, which matters more than the rules.
- **Anthony Williams, *C++ Concurrency in Action (2nd ed.)*** — the standard reference for §15. Read before writing anything lock-free.
- **Fedor Pikus, *The Art of Writing Efficient Programs*** — the best modern treatment of the material in §24.
- **Bjarne Stroustrup, *The Design and Evolution of C++*** — why the language is shaped this way (§1.1). Old, and still the best answer to "why on earth...".
- **Davis, Gottschling et al., *C++ Templates: The Complete Guide (2nd ed.)*** — the reference for §10 when you're writing library code.
- **John Lakos, *Large-Scale C++, Vol. 1*** — physical design and dependency management at scale.

Standing references

- **cppreference.com** — the working reference. Not the standard, but far more usable, and accurate enough that disagreements with it are usually your bug.
- **The C++ Core Guidelines** (isocpp.github.io/CppCoreGuidelines) — searchable, and each rule states its rationale.
- **The draft standard** (eel.is/c++draft) — when cppreference is ambiguous. Learn to read it; it is less frightening than it looks.
- **Compiler Explorer** (godbolt.org) — not a reference, a laboratory. §19.5 argues this is non-negotiable in C++.
- **Quick C++ Benchmarks** (quick-bench.com) — for the "is A faster than B" arguments, though see §22.8 on methodology before trusting any result.

Talks worth the hour

- Herb Sutter, *Leak-Freedom in C++... By Default* — the argument of §4 in 90 minutes.
- Scott Meyers, *CPU Caches and Why You Care* — the argument of §24.2, and the talk that changes how people write structs.
- Chandler Carruth, *Efficiency with Algorithms, Performance with Data Structures* — §24.3 in talk form.
- Louis Brandy, *Curiously Recurring C++ Bugs at Facebook* — a tour of §D from a codebase where all of them happened.

- Sean Parent, *Better Code: Runtime Polymorphism* — the definitive case for type erasure over inheritance (§8.11).
- Timur Doumler, *C++ Lifetimes* — the material of §3.4 and §14.4, precisely.

Staying current

- **isocpp.org** and the monthly mailing — where papers land before they become features.
 - **CppCon, C++Now, Meeting C++** — all posted free on YouTube. C++Now is the deep-library-design one.
 - **cppreference's compiler support table** — the honest answer to "can I use this yet".
-

Colophon

This book was assembled from a working set of notes into a single reference: the twenty-nine chapters are the original material, reorganized and de-duplicated, with front matter, a full table of contents, and eight appendices added to make it usable as a desk reference rather than a sequence of lessons.

Where the text quotes compiler output, sizes, or timings, they came from GCC 13.3 and Clang 18 on x86-64 Linux. Reproduce them. In C++, the habit of checking is the skill.